

PROJETO OAUTH-LABS

OAuth 2.0

Tratado Completo por RFC

Teoria aprofundada, RFC a RFC, com exemplos de implementação em Java (JDK 21, sem frameworks)

Compilado a partir do guia de estudo do projeto — 20 RFCs, 6 partes temáticas

Sumário

Parte 1 — Núcleo

RFC 6749 — The OAuth 2.0 Authorization Framework

RFC 6750 — Bearer Token Usage

Parte 2 — Segurança: Modelo de Ameaças e Boas Práticas

RFC 6819 — OAuth 2.0 Threat Model and Security Considerations

RFC 9700 — Best Current Practice for OAuth 2.0 Security (BCP 240)

RFC 7009 — OAuth 2.0 Token Revocation

Parte 3 — Extensões de Grant Type

RFC 7636 — Proof Key for Code Exchange by OAuth Public Clients (PKCE)

RFC 8628 — OAuth 2.0 Device Authorization Grant

RFC 8693 — OAuth 2.0 Token Exchange

RFC 9126 — OAuth 2.0 Pushed Authorization Requests (PAR)

RFC 9101 — The OAuth 2.0 Authorization Framework: JWT-Secured Authorization Request (JAR)

Parte 4 — Metadata e Registro de Clientes

RFC 8414 — OAuth 2.0 Authorization Server Metadata

RFC 7591 — OAuth 2.0 Dynamic Client Registration Protocol

RFC 7592 — OAuth 2.0 Dynamic Client Registration Management Protocol

Parte 5 — Formato e Introspecção de Tokens

RFC 7523 — JSON Web Token (JWT) Profile for OAuth 2.0 Client Authentication and Authorization Grants

RFC 7662 — OAuth 2.0 Token Introspection

RFC 9068 — JSON Web Token (JWT) Profile for OAuth 2.0 Access Tokens

Parte 6 — Segurança Avançada

RFC 8705 — OAuth 2.0 Mutual-TLS Client Authentication and Certificate-Bound Access Tokens

RFC 9449 — OAuth 2.0 Demonstrating Proof of Possession (DPoP)

RFC 8707 — Resource Indicators for OAuth 2.0

RFC 9396 — OAuth 2.0 Rich Authorization Requests (RAR)

Parte 1 — Núcleo

RFC 6749 — The OAuth 2.0 Authorization Framework

1. Contexto e motivação

1.1 O anti-padrão de senha

Antes do OAuth existir como especificação formal, a forma corriqueira de um aplicativo de terceiros acessar dados de um usuário em outro serviço era o que a comunidade batizou de **anti-padrão de senha** (*password anti-pattern*): o usuário simplesmente entregava sua senha do serviço-alvo (webmail, rede social, banco) diretamente ao aplicativo terceiro, que a armazenava e a usava para se autenticar "como se fosse" o usuário.

Esse modelo tem uma lista longa de problemas estruturais, não incidentais:

- **Acesso irrestrito.** A senha não carrega noção de escopo. Um app que só precisa ler a lista de contatos ganha, de fato, os mesmos poderes de qualquer login legítimo do usuário — trocar a senha, ler mensagens privadas, excluir a conta.
- **Impossível revogar seletivamente.** Se o usuário quiser cortar o acesso de um app específico sem afetar os demais, a única ferramenta disponível é trocar a própria senha — o que derruba *todos* os apps de uma vez, inclusive os que o usuário queria manter.
- **A senha passa a existir em outro lugar.** Cada app terceiro que guarda a senha do usuário é mais um ponto de vazamento possível. Um incidente de segurança nesse app terceiro compromete diretamente a conta original — não apenas os dados que o app deveria acessar.
- **Nenhuma trilha de auditoria útil.** Do ponto de vista do serviço original, uma requisição autenticada com a senha do usuário é indistinguível de uma requisição feita pelo próprio usuário logado no navegador. Não há como saber "isso veio do app X" nem revogar apenas aquele agente.
- **Incentiva o usuário a normalizar o phishing.** Se a prática legítima é "digite sua senha do Google neste app de terceiros", o usuário perde a capacidade de distinguir isso de um golpe de phishing — o próprio ecossistema erode a defesa que deveria ensinar.

1.2 A ideia central: autorização delegada

O OAuth 2.0 resolve esse problema trocando o modelo mental de "autenticação emprestada" por **autorização delegada**: o resource owner (o usuário) concede ao client (o app terceiro) um **token de acesso** — uma credencial derivada, de escopo e validade limitados, emitida por um terceiro de confiança (o authorization server) — sem que o client jamais veja ou manipule a senha original do usuário.

Três propriedades decorrem diretamente dessa mudança de modelo:

1. **Escopo explícito.** O token carrega (ou está associado no AS a) um conjunto de permissões que o resource owner aprovou explicitamente — "ler perfil", "listar repositórios", "iniciar uma transferência até R\$ 500", etc. O client nunca herda mais poder do que o concedido.

2. **Revogação granular.** O resource owner pode revogar o acesso de um client específico (via painel do AS, por exemplo "Aplicativos conectados à sua conta") sem afetar a senha da conta nem os outros clients autorizados.
3. **Auditoria e atribuição.** O AS sabe, a cada token emitido, para qual client, com qual escopo, e por vezes até em nome de qual resource owner ele foi emitido — permitindo logs, métricas e políticas de segurança por client.

Vale enfatizar que o OAuth 2.0 é um framework de **autorização**, não de **autenticação**. Ele resolve "o client X pode acessar o recurso Y em nome do usuário Z, com este escopo?" — não "quem é o usuário Z?". Confundir os dois é um erro histórico recorrente (por exemplo, usar um access token como prova de identidade do usuário no client) que a própria comunidade precisou corrigir mais tarde com uma camada adicional — o OpenID Connect —, que fica deliberadamente fora do escopo deste livro (que trata apenas de OAuth puro).

1.3 De OAuth 1.0 para OAuth 2.0 — o que mudou e por quê

A RFC 6749 substituiu formalmente a especificação anterior, RFC 5849 (OAuth 1.0), e a mudança mais visível é filosófica: o OAuth 1.0 exigia que **cada requisição** ao resource server fosse assinada criptograficamente (HMAC-SHA1 sobre uma string canônica construída a partir do método HTTP, URL e parâmetros, com um `client_secret` e um `token_secret` como chaves). Isso tinha uma vantagem real — a requisição carregava sua própria prova de integridade e de posse, mesmo trafegando sobre HTTP puro — mas um custo de implementação alto: bibliotecas cliente precisavam replicar exatamente o algoritmo de canonicalização (uma fonte enorme de bugs de interoperabilidade), e cada linguagem/plataforma precisava de uma biblioteca de assinatura dedicada.

O OAuth 2.0 troca essa complexidade por uma aposta diferente: **TLS obrigatório** em todo endpoint que transporta credenciais ou tokens, combinado com **bearer tokens** simples (definidos separadamente na RFC 6750) — uma string opaca que, uma vez capturada, dá acesso a quem a possuir, sem assinatura por requisição. Essa troca é deliberada e está documentada na própria RFC 6749 (§ Introduction): simplicidade de implementação em troca de depender inteiramente do transporte (TLS) para proteção contra interceptação, e de tempo de vida curto para conter o dano de uma eventual captura.

Essa decisão de design é a raiz de praticamente toda a evolução posterior do ecossistema OAuth. A ausência de "prova de posse por requisição" é exatamente o que os mecanismos de *sender-constraining* — RFC 8705 (mTLS) e RFC 9449 (DPoP), tratados mais adiante neste livro — foram desenhados para reintroduzir, de forma mais moderna, para os cenários que exigem essa garantia extra.

1.4 Um framework, não um protocolo único

Um ponto de leitura importante, e que costuma gerar confusão em quem chega do mundo mais rígido de protocolos como Kerberos ou SAML: a RFC 6749 se autodenomina explicitamente um **framework** de autorização, não um protocolo fechado. Ela define uma família de fluxos (*grant types*) para cenários distintos, deixa portas abertas para extensão (novos `grant_type`, novos `response_type`, parâmetros adicionais), e delega deliberadamente uma quantidade grande de decisões de segurança concretas para

especificações companheiras — a RFC 6819 (Threat Model, contemporânea) e, mais de uma década depois, a RFC 9700 (Best Current Practice), que revisita e restringe boa parte do que a 6749 deixou como "opção válida".

Essa característica de "esqueleto extensível" é ao mesmo tempo a força do OAuth 2.0 (permitiu nascer PKCE, DPoP, Resource Indicators, RAR, todos como extensões aditivas sem quebrar a RFC original) e a fonte de boa parte das implementações inseguras encontradas em produção — porque a RFC, isolada, não impõe as mitigações que hoje são consideradas obrigatórias.

2. Terminologia e papéis

2.1 Os quatro papéis

- **Resource Owner** — a entidade capaz de conceder acesso a um recurso protegido. Tipicamente uma pessoa física (o usuário final), mas pode ser uma entidade não-humana em cenários de automação (por exemplo, um sistema que autoriza outro sistema a agir por ele — embora aí frequentemente se recorra ao Client Credentials Grant, onde não há resource owner distinto do client, como veremos).
- **Client** — a aplicação que solicita acesso a um recurso protegido em nome do resource owner (ou, no caso do Client Credentials Grant, em nome de si mesma). É importante notar que "client" na terminologia OAuth é um papel, não uma camada de arquitetura: o mesmo processo pode acumular, em fluxos diferentes, o papel de client em uma integração e de resource server em outra.
- **Authorization Server (AS)** — o servidor que emite access tokens ao client após autenticar com sucesso o resource owner e obter sua autorização. O AS é o único componente que precisa conhecer a credencial primária do resource owner (senha, segundo fator, etc.).
- **Resource Server (RS)** — o servidor que hospeda os recursos protegidos e é capaz de aceitar e validar access tokens para servir requisições. Note que a RFC 6749 não especifica *como* o RS valida o token contra o AS (introspecção via RFC 7662, JWT autocontido via RFC 9068, um cache local, etc.) — isso fica, propositalmente, fora do escopo desta RFC.

Em implementações pequenas (como o próprio laboratório deste projeto), AS e RS podem ser processos totalmente separados que compartilham apenas um armazenamento (Redis, no caso do lab) como fonte de verdade sobre tokens válidos. Em arquiteturas maiores, é comum o AS ser um produto dedicado (Keycloak, Auth0, Okta, um IdP corporativo) e o RS ser cada microsserviço de negócio que aceita tokens emitidos por ele — validando-os via introspecção remota ou localmente, se forem JWTs auto-verificáveis.

2.2 Tipos de client: confidential vs. public

Essa é provavelmente a dimensão mais consequente da terminologia da RFC 6749, porque determina diretamente qual grant type é apropriado e qual superfície de ataque o client apresenta:

- **Confidential client** — capaz de manter um `client_secret` (ou outra credencial de autenticação) em segredo, de forma persistente e fora do alcance de um atacante que só tenha acesso ao dispositivo/rede do resource owner. O exemplo canônico é um servidor backend tradicional: o `client_secret` fica em uma variável de ambiente ou vault, nunca chega ao navegador.
- **Public client** — não tem como manter nenhum segredo de forma confiável, porque o binário/bundle do client é distribuído para o próprio dispositivo do resource owner (e, portanto, pode ser decompilado, inspecionado com um proxy, ou lido em texto claro). Exemplos: uma SPA rodando inteiramente no navegador, um app mobile nativo, um script de linha de comando (CLI) instalado localmente.

A RFC 6749 já reconhece essa distinção no corpo do texto (§2.1), mas é importante entender a implicação prática: **um `client_secret` "embutido" em um app público não é um segredo** — é, na melhor das hipóteses, um identificador ofuscado. Historicamente, muitas SPAs e apps mobile "confidenciais na configuração, públicos na realidade" embutiram client secrets em JavaScript ou em APKs, dando a falsa sensação de terem um client autenticado. A RFC 9700 (item 6 deste livro) formaliza essa lição e recomenda tratar qualquer client sem armazenamento seguro de segredo como público, período — com PKCE como mitigação obrigatória (e não apenas recomendada) do Authorization Code Grant nesse caso.

2.3 Perfis de client

A RFC 6749 (§2.1) também descreve três perfis de implantação, que na prática mapeiam para a distinção confidential/public acima:

- **Web application** — roda inteiramente em um servidor web; o resource owner acessa via navegador, mas o `client_secret` e os tokens ficam no backend. Confidential por natureza.
- **User-agent-based application** — o código do client é baixado para dentro de um user-agent (navegador) e roda lá (o caso clássico de SPA). Public por natureza, já que todo o código e estado é visível/manipulável no dispositivo do usuário.
- **Native application** — instalado e executado diretamente no dispositivo do resource owner (app mobile, desktop). Também public, pelo mesmo motivo — mesmo que o binário seja "compilado", engenharia reversa e interceptação de tráfego local (proxy no próprio device) tornam qualquer segredo embutido inseguro.

2.4 Client identifier e client secret

O `client_id` é um identificador único do client perante o AS — não é secreto (pode aparecer em URLs, no código-fonte de uma SPA, em logs) e serve apenas para o AS saber "qual configuração de client é essa" (redirect_uri registrada, grants permitidos, escopos permitidos). O `client_secret`, quando existe, é a credencial de autenticação do client junto ao AS — só faz sentido para clients confidenciais, e a RFC 6749 (§2.3.1) recomenda que ele seja transmitido preferencialmente via HTTP Basic Authentication no token endpoint (o que o laboratório deste projeto implementa, além de aceitar as credenciais no corpo como alternativa — mais adiante detalhado na seção 4).

3. Fluxos (Grant Types)

A RFC 6749 define quatro grant types nativos, mais um mecanismo de extensão para grants customizados (§4.5, via o parâmetro `grant_type` com um valor de URI absoluta definido por outra especificação — é assim, por exemplo, que a RFC 7523/7521 define grants baseados em JWT/SAML assertions, e como a própria RFC 8693 Token Exchange se encaixa no framework).

Nota de atualização, antecipando o item 7 deste capítulo: a RFC 9700 (2025) deprecia formalmente o Implicit Grant e o Resource Owner Password Credentials Grant. Ambos são descritos abaixo por completude histórica e porque ainda aparecem em sistemas legados que um arquiteto pode encontrar em produção — mas não devem ser adotados em projetos novos.

3.1 Authorization Code Grant (§4.1) — o fluxo central

Este é o fluxo mais importante do OAuth 2.0, desenhado especificamente para clients confidenciais (e, com a adição posterior de PKCE — RFC 7636 —, também seguro para clients públicos). A sequência completa:

```
(1) Client → User-Agent: redireciona o navegador para
    GET /authorize?response_type=code&client_id=...&redirect_uri=...&scope=...&state=...

(2) AS ↔ User-Agent: autentica o resource owner (login) e apresenta
    a tela de consentimento (quais escopos o client está pedindo)

(3) AS → User-Agent → Client: redireciona de volta para
    redirect_uri?code=AUTH_CODE&state=...

(4) Client → AS (back-channel): POST /token
    grant_type=authorization_code&code=AUTH_CODE&redirect_uri=...&client_id=...&client_secret=...

(5) AS → Client: { access_token, token_type, expires_in, refresh_token?, scope }
```

A propriedade de segurança central deste desenho é que **o access token nunca passa pelo user-agent/navegador** — apenas o `code`, um valor intermediário de vida curtíssima (a RFC recomenda no máximo 10 minutos; o laboratório deste projeto usa 60 segundos) e de **uso único** (o AS deve invalidá-lo assim que trocado por tokens, mesmo que a troca tenha falhado — porque um `code` que aparece duas vezes é evidência forte de replay ou vazamento). Como o `code` sozinho não vale nada sem o `client_secret` correspondente (no passo 4), mesmo que um atacante o intercepte no passo 3 (por exemplo, através de um `redirect_uri` mal validado, um app de terceiros malicioso no mesmo dispositivo, ou logs de navegação), ele não consegue completar a troca — a menos que também tenha comprometido o client confidencial, o que é um cenário de ameaça bem mais severo.

É exatamente essa dependência de "o atacante não tem o `client_secret`" que quebra para clients públicos — e é o motivo de existir o PKCE (RFC 7636, próximo capítulo deste livro): ele substitui a necessidade de um segredo estático de longo prazo por um segredo efêmero gerado a cada tentativa de autorização.

Um detalhe frequentemente ignorado: o `redirect_uri` deve ser enviado tanto no passo 1 (authorization request) quanto no passo 4 (token request), e o AS deve conferir que os dois **coincidem exatamente**. Essa checagem cruzada é uma segunda linha de defesa contra a interceptação do code — mesmo que o atacante consiga capturar o `code` retornado a um `redirect_uri` diferente do originalmente registrado, a troca no passo 4 falha se ele não souber reproduzir o mesmo `redirect_uri` usado na autorização.

3.2 Implicit Grant (§4.2) — *deprecado pela RFC 9700*

O Implicit Grant existia para resolver um problema de uma época específica: SPAs rodando inteiramente no navegador não tinham como executar o passo 4 do Authorization Code Grant sem expor o `client_secret` em JavaScript (visível a qualquer um que abrisse o DevTools). A "solução" foi eliminar essa troca: o AS devolvia o `access_token` **diretamente no fragmento da URL de redirect** (a parte depois de `#`, que por design nunca é enviada ao servidor em uma requisição HTTP subsequente, permanecendo só no navegador):

```
GET /authorize?response_type=token&client_id=...&redirect_uri=...&scope=...&state=...  
→ redirect_uri#access_token=...&token_type=Bearer&expires_in=3600&state=...
```

Os problemas deste desenho, hoje bem documentados (RFC 6819 já os antecipava; RFC 9700 os usa como justificativa formal de depreciação):

- O access token trafega pela **URL do navegador**, ficando exposto a: histórico de navegação, extensões de browser, qualquer script de terceiros rodando na mesma página, e ao header `Referer` se a página seguinte não tomar cuidado.
- Não há passo de troca back-channel, então **não há autenticação do client** nesse fluxo — qualquer app que conheça o `client_id` (não secreto, por definição) pode iniciar o fluxo.
- Como o token vem embutido na URL redirecionada por um user-agent, é mais fácil de um atacante induzir o navegador a entregar o token a um `redirect_uri` diferente do esperado, se a validação do AS for frouxa.
- Não há emissão de `refresh_token` nesse fluxo original (o navegador precisaria repetir a interação completa do usuário quando o token expirasse) — o que na prática levava implementações a usar tokens de vida mais longa, piorando o impacto de um vazamento.

A solução moderna e definitiva é Authorization Code + PKCE, mesmo em SPAs: o navegador não precisa mais de um `client_secret` porque o PKCE usa um segredo efêmero gerado por sessão de autorização (o `code_verifier`), eliminando a razão de existir do Implicit Grant.

3.3 Resource Owner Password Credentials — ROPC (§4.3) — *deprecado pela RFC 9700*

Neste fluxo, o resource owner entrega usuário e senha **diretamente ao client**, que os envia ao token endpoint em troca de tokens:

```
Client → AS: POST /token  
grant_type=password&username=...&password=...&client_id=...&client_secret=...&scope=...
```

O problema é evidente e literal: este é exatamente o anti-padrão de senha descrito na seção 1.1, apenas formalizado dentro do OAuth. O client vê e manipula a senha primária do usuário — reintroduzindo todos os riscos que o framework nasceu para eliminar. A própria RFC 6749 já o cercava de ressalvas ("SHOULD only be used when there is a high degree of trust between the resource owner and the client"), reservando-o essencialmente para clients de primeira parte (first-party) — por exemplo, o próprio app oficial de um banco autenticando o usuário em seu backend.

Mesmo nesse cenário restrito, o ROPC tem uma limitação estrutural que o BCP posterior aponta: ele é incompatível por design com qualquer mecanismo de autenticação que não seja usuário/senha simples — MFA interativo, biometria, WebAuthn, redirecionamento a um IdP corporativo — porque não há passo de redirecionamento ao AS onde essas interações poderiam acontecer. Isso, sozinho, já o tornava inadequado à medida que MFA se tornou padrão de mercado, independentemente da questão de confiança.

3.4 Client Credentials Grant (§4.4)

Este é o único grant nativo sem resource owner distinto: o próprio client se autentica com suas credenciais para obter um token que representa a si mesmo, não um usuário. É o padrão para comunicação **machine-to-machine** (M2M) — por exemplo, um serviço de backend que chama uma API de outro serviço, um job em batch, uma integração server-to-server:

```
Client → AS: POST /token
  grant_type=client_credentials&client_id=...&client_secret=...&scope=...

AS → Client: { access_token, token_type, expires_in, scope }
```

Note na RFC (§4.4.3) uma regra fácil de esquecer: um `refresh_token` **não deve** ser emitido neste grant. A justificativa é direta — como não há resource owner nem sessão de usuário para "renovar", o client sempre pode simplesmente chamar o token endpoint novamente com as mesmas credenciais quando precisar de um novo access token. Emitir um refresh_token aqui só adicionaria um segredo de longa duração extra para proteger, sem benefício real. O laboratório deste projeto respeita essa regra (`withRefresh: false` no grant `client_credentials`, ver seção final deste capítulo).

Client Credentials é, por natureza, restrito a clients confidenciais — um client público não tem como manter um `client_secret`, e portanto não tem uma identidade própria e verificável a apresentar neste grant. Em arquiteturas mais modernas (ver RFC 9700 e as extensões do item 6 deste livro), esse grant também é o alvo natural de reforço via mTLS (RFC 8705) para autenticação de client baseada em certificado, em vez de segredo compartilhado.

3.5 Refresh Token (§6) — mecanismo transversal

O refresh token não é um grant type por si só — é um mecanismo que atravessa os grants que envolvem resource owner (Authorization Code e, historicamente, ROPC): o AS pode, opcionalmente, emitir um `refresh_token` junto com o `access_token`. Esse refresh_token é usado depois para obter novos access tokens **sem repetir a interação com o resource owner** (sem novo login, sem nova tela de consentimento):

```
Client → AS: POST /token
grant_type=refresh_token&refresh_token=...&client_id=...&client_secret=...
```

Rotacionar ou manter o refresh token na renovação? A própria RFC 6749 (§6) já trata disso: é facultativo do AS. Se a resposta da renovação incluir um novo `refresh_token`, o client deve descartar o antigo e passar a usar apenas o novo a partir dali; se o campo vier ausente na resposta, o client mantém o `refresh_token` que já tinha — a ausência do campo já é, por si, o sinal de "nada mudou, continue usando o mesmo". Isso é importante porque um client mal implementado que *sempre* espera um novo `refresh_token` e descarta o campo quando ausente perderia a capacidade de se renovar novamente.

A RFC 9700, mais de uma década depois, aperta essa recomendação: rotação a cada uso passa a ser fortemente recomendada (especialmente para clients públicos), combinada com **detecção de reuso** — se um `refresh_token` já rotacionado (portanto inválido) for apresentado novamente, isso é evidência de que ele foi roubado e clonado, e o AS deveria revogar toda a família de tokens derivada dele, não apenas rejeitar a tentativa isolada. O laboratório deste projeto já adota rotação incondicional a cada uso (o handler do grant `refresh_token` sempre deleta o token antigo do Redis e emite um novo — ver seção final).

Também vale registrar o escopo do `refresh_token` na renovação: a RFC 6749 (§6) permite que o client solicite um `scope` mais restrito do que o originalmente concedido (nunca mais amplo) — um mecanismo de "downscoping" que muitas implementações client sequer exploram, mas que é útil para aderir ao princípio de menor privilégio quando o client sabe, no momento da renovação, que só precisará de um subconjunto das permissões originais.

3.6 Extensibilidade: grants customizados

A RFC 6749 (§4.5 e §8.3) reserva formalmente o espaço de valores de `grant_type` para extensão: qualquer valor que seja uma URI absoluta é tratado como um grant customizado definido por outra especificação, permitindo que o ecossistema cresça sem modificar o núcleo. Exemplos notáveis que um arquiteto financeiro encontra em produção: RFC 7523 (JWT Bearer Grant, `urn:ietf:params:oauth:grant-type:jwt-bearer` — comum em integrações de assertions SAML/JWT emitidas por um IdP corporativo) e RFC 8693 (Token Exchange, `urn:ietf:params:oauth:grant-type:token-exchange` — usado para troca de tokens entre domínios de confiança, delegação e "on-behalf-of" em arquiteturas de microsserviços). Ambas ficam fora do escopo direto deste livro, mas é útil saber que o mecanismo de extensão existe desde a RFC original.

4. Parâmetros e endpoints normativos

4.1 Authorization endpoint (§3.1)

É o endpoint que interage com o resource owner através do user-agent (tipicamente um navegador). É aqui, e somente aqui, que credenciais primárias do usuário (senha, segundo fator) devem ser inseridas — nunca no client. A RFC exige que este endpoint suporte o método HTTP GET (e pode, opcionalmente, suportar POST) e que os parâmetros sejam serializados na query string, mesmo que a

implementação renderize a interação subsequente (tela de login, tela de consentimento) via POST de formulário — que é exatamente como o laboratório deste projeto implementa (GET /authorize monta a tela de login; POST /authorize processa a decisão do usuário).

4.2 Token endpoint (§3.2)

É o canal back-channel (client-para-AS, direto, sem o navegador no meio) usado para trocar um `code`, `refresh_token` ou credenciais de client por tokens. A RFC exige TLS e o método HTTP POST, com o corpo serializado como `application/x-www-form-urlencoded` (§4.1.3, §4.3.2, §4.4.2, §6). É neste endpoint que o client se autentica, quando aplicável — a RFC 6749 (§2.3.1) recomenda HTTP Basic Authentication (`client_id` como username, `client_secret` como password, ambos devidamente `percent-encoded` antes do Base64) como método preferencial, mas também permite `client_id` / `client_secret` como parâmetros do corpo, para clients que não conseguem manipular o header `Authorization`.

4.3 Redirection endpoint (§3.1.2)

É a URI no próprio client para onde o AS devolve o resultado da autorização (o `code`, no fluxo padrão, ou um erro). Deve ser **registrada previamente** junto ao AS como parte do cadastro do client, e a RFC exige comparação **exata** (não apenas prefixo ou domínio) entre o `redirect_uri` enviado na requisição e o valor registrado — esse ponto é retomado com mais profundidade na seção de segurança (5.1) porque é uma fonte histórica recorrente de vulnerabilidades de open redirect.

4.4 Parâmetros-chave da authorization request (§4.1.1)

Parâmetro	Obrigatório?	Descrição
<code>response_type</code>	Sim	<code>code</code> (Authorization Code Grant) ou <code>token</code> (Implicit, deprecado)
<code>client_id</code>	Sim	Identificador público do client, registrado no AS
<code>redirect_uri</code>	Condicional	Obrigatório se o client tiver múltiplos <code>redirect_uris</code> registrados; recomendado sempre enviar
<code>scope</code>	Opcional	Lista de escopos separados por espaço; se omitido, o AS decide o padrão (frequentemente o escopo completo registrado, ou nenhum)
<code>state</code>	Recomendado (na prática, obrigatório)	Valor opaco gerado pelo client, ecoado sem alteração pelo AS no retorno — mecanismo anti-CSRF (ver 5.2)

4.5 Parâmetros-chave da token response (§4.1.4, §5.1)

Parâmetro	Obrigatório?	Descrição
<code>access_token</code>	Sim	O token propriamente dito
<code>token_type</code>	Sim	Tipicamente <code>Bearer</code> (RFC 6750)

Parâmetro	Obrigatório?	Descrição
<code>expires_in</code>	Recomendado	Tempo de vida em segundos, a partir do momento de emissão
<code>refresh_token</code>	Opcional	Presente conforme política do AS e do grant (ver 3.5)
<code>scope</code>	Condicional	Obrigatório apenas se o escopo efetivamente concedido difere do solicitado — mas é boa prática sempre incluir, para o client não ter que assumir

A resposta do token endpoint deve incluir os headers `Cache-Control: no-store` e `Pragma: no-cache` (§5.1) — porque o corpo contém segredos (tokens), e caches HTTP intermediários (proxies corporativos, CDNs) não devem persisti-los.

4.6 Erros padronizados (§4.1.2.1, §5.2)

`invalid_request`, `invalid_client`, `invalid_grant`, `unauthorized_client`, `unsupported_grant_type`, `unsupported_response_type`, `invalid_scope`, `access_denied`, `server_error`, `temporarily_unavailable` — devolvidos como parâmetros de **redirect** (erros do authorization endpoint, exceto quando o próprio `redirect_uri` é o problema — nesse caso, o AS deve exibir a mensagem diretamente ao usuário, nunca redirecionar a um destino não verificado) ou como **corpo JSON** com status HTTP apropriado (erros do token endpoint, normalmente `400` ou `401`).

5. Segurança já tratada na própria RFC 6749

5.1 Open redirector

O `redirect_uri` é, por definição, uma URL para onde o AS envia dados sensíveis (o `code`, no mínimo) de forma automatizada, disparada por uma resposta HTTP de redirecionamento. Se o AS aceitasse qualquer `redirect_uri` fornecido na requisição (ou validasse apenas por prefixo/domínio), um atacante poderia registrar um client legítimo, mas fornecer em tempo de requisição um `redirect_uri` apontando para um domínio próprio — o AS então enviaria o `code` de qualquer resource owner que autorizasse a requisição diretamente ao atacante. Pior ainda: o próprio AS se tornaria um "open redirector" reaproveitável para phishing genérico, mesmo sem relação alguma com OAuth.

A mitigação normativa (§3.1.2.3) é comparação **exata** contra a lista de `redirect_uri`s pré-registrados no cadastro do client — não prefixo, não apenas o domínio, string completa idêntica (incluindo path e query string, se houver). O laboratório deste projeto implementa exatamente essa checagem:

```
client.redirect_uri !== redirect_uri
```

rejeita qualquer valor que não seja idêntico byte a byte ao registrado.

5.2 CSRF no callback: o parâmetro `state`

Sem alguma proteção, um atacante poderia iniciar o próprio fluxo de autorização (obtendo um `code` seu, legítimo, para sua própria conta) e induzir a vítima a completar o passo de troca desse code no client da vítima — efetivamente "plantando" a própria conta do atacante na sessão da vítima (um ataque de CSRF na etapa de callback, às vezes chamado de "login CSRF" ou "session fixation via

OAuth"). O parâmetro `state` mitiga isso: o client gera um valor opaco e imprevisível antes do passo 1, guarda-o associado à sessão local do navegador (cookie de sessão, por exemplo), e confere que o valor devolvido no passo 3 é **exatamente** o mesmo antes de prosseguir com a troca do code. Se um atacante tentar reaproveitar seu próprio `code` /callback contra a sessão da vítima, o `state` que ele apresenta não vai bater com o que está associado à sessão dela.

5.3 Intercepção do authorization code

Mesmo com `redirect_uri` e `state` corretos, o `code` ainda trafega por um canal menos controlado que o back-channel (o navegador, possivelmente por redes não confiáveis, apps concorrentes no mesmo dispositivo em plataformas mobile mais antigas que não isolam esquemas de URL customizados). A RFC 6749 mitiga isso com vida útil curtíssima e uso único (o AS invalida o code assim que ele é trocado — ou, mais rigorosamente, assim que uma tentativa de troca acontece, válida ou não). A mitigação mais robusta e específica para esse vetor — especialmente relevante em apps nativos que dependem de custom URI schemes para o redirect — veio depois, com a RFC 7636 (PKCE), tratada no próximo capítulo deste livro.

5.4 TLS obrigatório

Todo endpoint que transporta credenciais (`client_secret` , senha do resource owner) ou tokens (`code` , `access_token` , `refresh_token`) deve operar sobre TLS. Isso vale tanto para o authorization endpoint (onde a senha do usuário é digitada) quanto para o token endpoint (onde o `client_secret` e os tokens trafegam). Sem TLS, tanto o modelo bearer (RFC 6750) quanto a autenticação básica de client ficam completamente expostos a qualquer um capaz de observar a rede.

Nota de laboratório: conforme já registrado no capítulo anterior e reiterado aqui, os labs práticos deste projeto rodam deliberadamente em HTTP puro (`http://localhost`), simplificação mantida até o capítulo da RFC 8705 (mTLS), onde TLS real passa a ser o próprio mecanismo em estudo. Até lá, o foco pedagógico é a mecânica de grants e tokens, não a camada de transporte.

5.5 Autenticação de client no token endpoint

A RFC 6749 (§2.3) deixa o método de autenticação de client parcialmente aberto, mas recomenda HTTP Basic (`client_id/client_secret` no header `Authorization`) como preferencial, permitindo alternativamente os mesmos valores no corpo da requisição. Um detalhe de nuance que aparece na prática (e que o laboratório deste projeto trata corretamente): quando a autenticação falha e foi tentada via Basic Auth, a resposta de erro 401 deve incluir o header `WWW-Authenticate: Basic realm="..."` — sinalizando ao client qual esquema de autenticação era esperado, consistente com a semântica geral de HTTP Authentication (RFC 7235).

5.6 O que a RFC 6749 deliberadamente deixa em aberto

A RFC 6749 é, por design, um framework "esqueleto": ela documenta as opções e os mecanismos básicos, mas deixa deliberadamente de fora recomendações mais assertivas sobre, por exemplo, obrigatoriedade de PKCE, rotação de refresh tokens, sender-constraining de access tokens, ou validação

de escopo em runtime. Essas lacunas foram primeiro catalogadas pela RFC 6819 (Threat Model, contemporânea à 6749) e, mais de uma década depois, endereçadas de forma prescritiva pela RFC 9700 (Best Current Practice) — que efetivamente promove várias "opções" da 6749 a "obrigações" à luz de uma década de exploração em produção.

6. Relação com outras RFCs

- **RFC 6750** define como o `access_token` do tipo Bearer, obtido através dos mecanismos descritos nesta RFC, é efetivamente apresentado ao resource server — este documento (6749) só define como o token é *obtido*, nunca como é *usado*. É o assunto do próximo capítulo deste livro.
- **RFC 6819** (Threat Model) cataloga sistematicamente as ameaças específicas de cada grant type descrito aqui, servindo de base analítica para tudo que viria depois.
- **RFC 7636 (PKCE)** foi criada especificamente para eliminar a dependência de `client_secret` no Authorization Code Grant para clients públicos, mitigando a interceptação de code (seção 5.3) sem exigir um segredo estático.
- **RFC 7523 / RFC 8693** exploram o mecanismo de extensão de `grant_type` (seção 3.6) para assertions e token exchange, respectivamente.
- **RFC 8705 (mTLS)** e **RFC 9449 (DPoP)** reforçam tanto a autenticação de client (seção 5.5) quanto o token em si, amarrando-o criptograficamente a quem o solicitou.
- **RFC 8707 (Resource Indicators)** e **RFC 9396 (RAR)** refinam a granularidade do `scope`, respondendo à limitação de que escopos são apenas strings planas sem estrutura.
- **RFC 9700** revisita e restringe as escolhas desta RFC à luz de mais de uma década de exploração em produção — depreciando Implicit e ROPC, e tornando PKCE obrigatório universalmente.
- **OAuth 2.1** (draft, não uma RFC formal até o momento deste livro) efetivamente reescreve este documento incorporando essas correções como obrigatórias em vez de opcionais, consolidando o que a RFC 9700 recomenda em uma única especificação coesa.

7. O que mudou desde a publicação (2012)

- **Implicit e ROPC**: de "opções válidas para cenários específicos" para "não usar" (RFC 9700) — a experiência de produção mostrou que os riscos superavam os benefícios em praticamente todos os casos reais.
- **PKCE**: nasceu como mitigação "só para clients públicos" (RFC 7636, 2015) e se tornou "obrigatório para todos os clients, inclusive confidenciais" (RFC 9700, 2025) — defesa em profundidade contra interceptação de code mesmo quando o client tem um segredo.
- `redirect_uri` : de comparação flexível (permitida por alguns AS na prática, embora nunca formalmente recomendada) para **exact string match** obrigatório, sem ambiguidade.
- **Bearer tokens simples** (sem sender-constraining): considerados adequados em 2012, hoje vistos como piso mínimo insuficiente para cenários de alta sensibilidade (sistemas financeiros, dados de saúde) — ver RFC 8705 (mTLS) e RFC 9449 (DPoP), tratadas no item 6 deste livro.
- **Refresh token rotation**: de "facultativo, sem orientação forte" para "fortemente recomendado, com detecção de reuso" (RFC 9700).

Exemplo em Java (JDK 21, sem frameworks)

Os exemplos a seguir usam exclusivamente a biblioteca padrão do JDK 21 — `java.net.http.HttpClient` para chamadas HTTP e um utilitário mínimo de JSON escrito à mão para este livro (sem Jackson, Gson, nem qualquer outra dependência externa). Este utilitário — `SimpleJson` — é definido aqui, no primeiro capítulo, e os capítulos seguintes deste livro apenas o referenciam, sem redefini-lo.

Nota importante: este é um parser deliberadamente ingênuo, feito só para fins didáticos deste livro. Ele não trata aninhamento, arrays, nem escaping completo de JSON. **Em código de produção, use sempre uma biblioteca JSON real** (Jackson, Gson, ou a que já for padrão no seu stack) — nunca um parser artesanal como este.

```
// SimpleJson.java – utilitário mínimo para os exemplos deste livro (JDK puro, sem bibliotecas exte
// Uso didático apenas: em produção, use uma biblioteca JSON real (ex: Jackson, Gson).
final class SimpleJson {
    private SimpleJson() {}

    static String object(Map<String, ?> fields) {
        StringBuilder sb = new StringBuilder("{");
        var it = fields.entrySet().iterator();
        while (it.hasNext()) {
            var e = it.next();
            sb.append('').append(e.getKey()).append("\":");
            Object v = e.getValue();
            if (v instanceof Number || v instanceof Boolean) sb.append(v);
            else sb.append('').append(String.valueOf(v).replace("\\", "\\").append(''));
            if (it.hasNext()) sb.append(',');
        }
        return sb.append('}').toString();
    }

    // Parser deliberadamente ingênuo: só serve para os exemplos deste livro,
    // assume JSON plano (sem aninhamento) com chaves/valores string, número ou booleano.
    static Map<String, String> parseFlat(String json) {
        Map<String, String> map = new LinkedHashMap<>();
        Matcher m = Pattern.compile("\"(.*)\"\\s*:\\s*\"(.*)\"|\"(.*)\"|[0-9.]+|true|false").matcher(json);
        while (m.find()) {
            String key = m.group(1);
            String value = m.group(3) != null ? m.group(3) : m.group(2);
            map.put(key, value);
        }
        return map;
    }
}
```


Exemplo 1 — Montando a URL da authorization request (§4.1.1)

O primeiro passo do Authorization Code Grant é redirecionar o navegador do resource owner para o authorization endpoint do AS, com os parâmetros corretamente codificados como query string. Note o uso de `URLEncoder` (obrigatório: `client_id`, `redirect_uri` e `scope` podem conter caracteres que precisam de percent-encoding) e a geração de um `state` opaco e imprevisível — aqui, via `SecureRandom`, nunca via `Random` comum, porque o `state` é um mecanismo de segurança (anti-CSRF, seção 5.2 deste capítulo), não um identificador qualquer.

```

import java.net.URLEncoder;
import java.nio.charset.StandardCharsets;
import java.security.SecureRandom;
import java.util.Base64;

final class AuthorizationRequestBuilder {

    private static final SecureRandom RNG = new SecureRandom();

    // Gera um valor opaco, imprevisível, para o parametro state (RFC 6749 5.2 / 4.1.1).
    // Por que SecureRandom e nao Random: state precisa ser imprevisivel para um atacante,
    // senao a protecao anti-CSRF pode ser burlada por forza bruta ou adivinhacao.
    static String newState() {
        byte[] raw = new byte[24];
        RNG.nextBytes(raw);
        return Base64.getUrlEncoder().withoutPadding().encodeToString(raw);
    }

    static String build(String authorizeEndpoint, String clientId, String redirectUri,
        String scope, String state) {
        String enc = "UTF-8";
        return authorizeEndpoint
            + "?response_type=code"
            + "&client_id=" + urlEncode(clientId)
            + "&redirect_uri=" + urlEncode(redirectUri)
            + "&scope=" + urlEncode(scope)
            + "&state=" + urlEncode(state);
    }

    private static String urlEncode(String value) {
        return URLEncoder.encode(value, StandardCharsets.UTF_8);
    }

    public static void main(String[] args) {
        String state = newState();
        String url = build(
            "http://localhost:3001/authorize",
            "client-demo",
            "http://localhost:3000/callback",
            "profile",
            state);

        // Em uma aplicacao web real, o client guardaria `state` associado a sessao
        // do navegador (ex: cookie de sessao no servidor) ANTES de redirecionar,
        // para poder conferir a igualdade quando o AS chamar de volta o redirect_uri.
        System.out.println("Redirecionar o navegador para: " + url);
        System.out.println("State a guardar na sessao local: " + state);
    }
}

```

Exemplo 2 — Trocando o authorization code por tokens (§4.1.3)

Depois que o AS redireciona de volta ao `redirect_uri` com `?code=...&state=...`, o client (no backend, fora do alcance do navegador) confere o `state` e faz a troca back-channel no token endpoint. Note o `Content-Type: application/x-www-form-urlencoded` (exigido pela RFC, §4.1.3) e a autenticação do client via HTTP Basic — o método preferencial segundo a RFC 6749 §2.3.1.

```

import java.net.URI;
import java.net.URLEncoder;
import java.net.http.HttpClient;
import java.net.http.HttpRequest;
import java.net.http.HttpResponse;
import java.nio.charset.StandardCharsets;
import java.util.Base64;
import java.util.Map;

final class TokenExchange {

    private static final HttpClient HTTP = HttpClient.newHttpClient();

    // Troca um authorization code por access_token (+ refresh_token, se emitido).
    // grant_type=authorization_code -- RFC 6749 4.1.3.
    static Map<String, String> exchangeAuthorizationCode(
        String tokenEndpoint, String clientId, String clientSecret,
        String code, String redirectUri) throws Exception {

        String body = "grant_type=authorization_code"
            + "&code=" + enc(code)
            + "&redirect_uri=" + enc(redirectUri);

        return post(tokenEndpoint, clientId, clientSecret, body);
    }

    // Uso do refresh_token para obter um novo access_token sem repetir o login
    // do resource owner -- RFC 6749 6. O client deve estar preparado para
    // receber (ou nao) um novo refresh_token na resposta: se vier, descarta o
    // antigo; se nao vier, continua usando o que ja tinha.
    static Map<String, String> refresh(
        String tokenEndpoint, String clientId, String clientSecret,
        String refreshToken) throws Exception {

        String body = "grant_type=refresh_token"
            + "&refresh_token=" + enc(refreshToken);

        return post(tokenEndpoint, clientId, clientSecret, body);
    }

    private static Map<String, String> post(String tokenEndpoint, String clientId,
        String clientSecret, String formBody) throws Exception {

        // Autenticacao de client via HTTP Basic (RFC 6749 2.3.1) -- metodo
        // preferencial. client_id e client_secret sao concatenados com ':' e
        // codificados em Base64; o RFC exige que cada um seja percent-encoded
        // ANTES da concatenacao, para evitar ambiguidade se algum contiver ':'.
        String basicAuth = Base64.getEncoder().encodeToString(
            (enc(clientId) + ":" + enc(clientSecret)).getBytes(StandardCharsets.UTF_8));

        HttpRequest request = HttpRequest.newBuilder()
            .uri(URI.create(tokenEndpoint))

```

```

        .header("Authorization", "Basic " + basicAuth)
        .header("Content-Type", "application/x-www-form-urlencoded")
        .POST(HttpRequest.BodyPublishers.ofString(formBody, StandardCharsets.UTF_8))
        .build();

    HttpResponse<String> response = HTTP.send(request, HttpResponse.BodyHandlers.ofString());

    if (response.statusCode() != 200) {
        // Corpo de erro do token endpoint e sempre JSON: {"error":"...", "error_description":"..."}
        Map<String, String> error = SimpleJson.parseFlat(response.body());
        throw new IllegalStateException("token endpoint respondeu "
            + response.statusCode() + ": " + error.get("error")
            + " -- " + error.get("error_description"));
    }

    return SimpleJson.parseFlat(response.body());
}

private static String enc(String value) {
    return URLEncoder.encode(value, StandardCharsets.UTF_8);
}

public static void main(String[] args) throws Exception {
    // Exemplo de uso apos o navegador retornar ?code=...&state=... para o
    // redirect_uri do client (conferencia do state omitida aqui por brevidade
    // -- ver Exemplo 1 sobre por que ela e obrigatoria).
    Map<String, String> tokens = exchangeAuthorizationCode(
        "http://localhost:3001/token",
        "client-demo",
        "s3cr3t",
        "AUTH_CODE_RECEBIDO_NO_CALLBACK",
        "http://localhost:3000/callback");

    System.out.println("access_token: " + tokens.get("access_token"));
    System.out.println("expires_in:    " + tokens.get("expires_in"));
    System.out.println("refresh_token: " + tokens.get("refresh_token"));

    // Mais tarde, quando o access_token expirar:
    Map<String, String> renewed = refresh(
        "http://localhost:3001/token", "client-demo", "s3cr3t",
        tokens.get("refresh_token"));
    System.out.println("novo access_token: " + renewed.get("access_token"));
}
}

```

Exemplo 3 — Client Credentials Grant (§4.4): comunicação machine-to-machine

Sem resource owner, sem redirect, sem code — apenas o client se autenticando diretamente no token endpoint. Este é o padrão típico para um serviço backend chamar outro serviço backend, sem usuário humano na equação.

```

import java.net.URI;
import java.net.URLEncoder;
import java.net.http.HttpClient;
import java.net.http.HttpRequest;
import java.net.http.HttpResponse;
import java.nio.charset.StandardCharsets;
import java.util.Base64;
import java.util.Map;

final class ClientCredentialsExample {

    private static final HttpClient HTTP = HttpClient.newHttpClient();

    static Map<String, String> requestServiceToken(
        String tokenEndpoint, String clientId, String clientSecret, String scope)
        throws Exception {

        String body = "grant_type=client_credentials&scope="
            + URLEncoder.encode(scope, StandardCharsets.UTF_8);

        String basicAuth = Base64.getEncoder().encodeToString(
            (clientId + ":" + clientSecret).getBytes(StandardCharsets.UTF_8));

        HttpRequest request = HttpRequest.newBuilder()
            .uri(URI.create(tokenEndpoint))
            .header("Authorization", "Basic " + basicAuth)
            .header("Content-Type", "application/x-www-form-urlencoded")
            .POST(HttpRequest.BodyPublishers.ofString(body, StandardCharsets.UTF_8))
            .build();

        HttpResponse<String> response = HTTP.send(request, HttpResponse.BodyHandlers.ofString());

        Map<String, String> parsed = SimpleJson.parseFlat(response.body());
        if (response.statusCode() != 200) {
            throw new IllegalStateException("falha ao obter token de servico: " + parsed);
        }

        // Nota (RFC 6749 4.4.3): este grant nao emite refresh_token -- se o
        // access_token expirar, o client simplesmente repete esta chamada com
        // as mesmas credenciais. Nao ha sessao de resource owner para "renovar".
        return parsed;
    }

    public static void main(String[] args) throws Exception {
        Map<String, String> tokens = requestServiceToken(
            "http://localhost:3001/token", "servico-backend", "outro-segredo", "service");
        System.out.println("access_token (M2M): " + tokens.get("access_token"));
    }
}

```

Relação com o laboratório deste projeto

O laboratório `rfc6749/` deste projeto (em Node.js/Express) implementa exatamente os mecanismos descritos neste capítulo, em três processos separados que se comunicam apenas via HTTP e um Redis compartilhado como fonte de verdade dos tokens: o `authorization-server` expõe `/authorize` (GET renderiza a tela de login, POST processa a decisão do resource owner e emite o `code` de uso único) e `/token` (que trata `authorization_code`, `refresh_token` e `client_credentials` no mesmo handler, validando `grant_types` permitidos por client); o `resource-server` valida o bearer token recebido; e o `client-demo` desempenha o papel de client confidencial, guardando o `client_secret` inteiramente no backend. A rotação incondicional de refresh token a cada uso (seção 3.5 deste capítulo) já está implementada — `redis.del(refresh_token:...)` seguido de emissão de um novo — assim como a validação exata de `redirect_uri` (seção 5.1) e a checagem cruzada `client_id / redirect_uri` entre o passo de autorização e o de troca do code (seção 3.1).

RFC 6750 — Bearer Token Usage

1. Contexto e motivação

1.1 A lacuna deixada pela RFC 6749

A RFC 6749 define, com detalhe considerável, como um `access_token` é **obtido** — os grant types, os endpoints, os parâmetros. Mas ela deixa deliberadamente de fora uma pergunta igualmente central: uma vez em posse do token, como o client o **usa** para efetivamente acessar um recurso protegido? Essa lacuna é o escopo exclusivo da RFC 6750, publicada no mesmo dia (outubro de 2012) que a 6749, como uma companion spec dedicada.

A RFC 6750 formaliza o tipo de token mais comum do OAuth 2.0 — na prática, o único que a RFC 6749 realmente exercita em seus exemplos: o **bearer token**.

1.2 A definição central: posse é acesso

Um bearer token é uma credencial na qual **qualquer parte que o possua pode usá-la**, sem precisar provar posse de nenhuma chave criptográfica associada, sem assinar a requisição, sem qualquer prova adicional de que é o legítimo destinatário original do token. Na prática: "quem tem o token, tem o acesso" — a analogia clássica na literatura é a de um **cheque ao portador** (*bearer check*): quem apresenta o papel ao banco recebe o valor, independentemente de ser ou não o titular original da conta.

Essa definição é a contrapartida direta de uma escolha de design já discutida no capítulo anterior deste livro: o OAuth 1.0 (RFC 5849) exigia que cada requisição fosse assinada criptograficamente com HMAC-SHA1, amarrando cada chamada a uma chave que o atacante precisaria roubar separadamente da simples observação do tráfego. O OAuth 2.0 removeu essa exigência em favor de simplicidade de implementação — e a RFC 6750 é, em certo sentido, **o preço dessa simplicidade**: como não há prova de posse por requisição, **toda a segurança do modelo recai sobre como o token é transportado, armazenado e sobre seu tempo de vida** — nunca sobre o protocolo de uso em si.

Esse é o ponto mais importante a reter deste capítulo, e vale repetir com todas as letras: bearer = simples de implementar, simples de interoperar, mas **roubável e livremente reutilizável por qualquer um que o capture**, até que expire ou seja revogado. Não há "assinatura errada" que um resource server possa detectar — um token roubado é, do ponto de vista do RS, indistinguível de um token usado pelo seu dono legítimo.

1.3 Por que esse modelo foi aceitável em 2012 (e por que hoje é visto como piso mínimo)

Em 2012, a aposta era razoável: TLS já era amplamente disponível e cada vez mais padrão, os tempos de vida recomendados eram curtos, e a maior parte dos cenários de uso não envolvia os níveis de sensibilidade que hoje justificam controles adicionais (transações financeiras de alto valor, dados de saúde, acesso a infraestrutura crítica). Anos de exploração em produção, no entanto, revelaram vetores

de roubo de token que TLS por si só não neutraliza completamente — vazamento por logs mal configurados, XSS em SPAs que armazenam tokens acessíveis a JavaScript, proxies corporativos mal configurados fazendo TLS termination e logando corpos de requisição, tokens capturados em ambientes de desenvolvimento/homologação com configuração mais frouxa que vazam para produção, entre outros.

É exatamente essa lacuna estrutural — "posse = acesso, sem mais nada" — que, uma década depois, motivou o desenvolvimento de mecanismos de **sender-constraining**: a RFC 8705 (OAuth 2.0 Mutual-TLS Client Authentication and Certificate-Bound Access Tokens) e a RFC 9449 (DPoP — Demonstrating Proof of Possession), ambas tratadas no item 6 deste livro. Essas especificações não substituem o bearer token da RFC 6750 — elas o **complementam**, amarrando o token a um material criptográfico que só o legítimo detentor possui (um certificado de cliente mTLS, ou um par de chaves DPoP), de forma que um token capturado por interceptação ou vazamento de log deixa de ser diretamente utilizável por quem o capturou.

1.4 O nome importa: por que "Bearer" e não apenas "token"

A RFC 6750 formaliza `Bearer` como o valor literal do campo `token_type` na resposta do token endpoint (RFC 6749 §5.1) e como o *authentication scheme* HTTP usado no header `Authorization`. Esse nome não é cosmético — ele sinaliza explicitamente, para qualquer implementador que leia a especificação, que aquele tipo particular de token não carrega nenhuma prova de posse embutida. Um esquema alternativo (como o `DPoP` introduzido pela RFC 9449) usa um nome diferente precisamente para deixar claro, já no header, que a semântica de segurança é outra — um resource server que só sabe validar `Bearer` não deve aceitar silenciosamente um esquema diferente como se fosse equivalente.

2. Terminologia e papéis

Nenhum papel novo é introduzido em relação à RFC 6749 — client, resource server, authorization server e resource owner permanecem exatamente os mesmos, com as mesmas responsabilidades já descritas no capítulo anterior. A RFC 6750 se limita a definir com precisão dois conceitos que a 6749 apenas menciona de passagem:

- **Bearer Token** — o tipo específico de credencial abordado por esta especificação (o valor de `token_type` no token endpoint é literalmente a string `"Bearer"`, case-insensitive por uma errata posterior — ver seção 7).
- **Bearer Token Usage** — o conjunto de regras normativas sobre como um client deve apresentar esse token e como um resource server deve validá-lo (ou rejeitá-lo) em uma requisição.

Um detalhe de nuance frequentemente perdido: a RFC 6750 é inteiramente **agnóstica quanto ao formato interno do token**. Ela não especifica se o `access_token` é um identificador opaco (uma string aleatória que só faz sentido em conjunto com uma tabela de estado no AS/RS — o modelo que o laboratório deste projeto usa, com o token como chave no Redis) ou um token auto-contido e verificável (como um JWT assinado, formalizado mais tarde pela RFC 9068). Ambos são "bearer tokens" do ponto de vista desta RFC — a diferença está inteiramente em como o resource server valida o token, o que fica fora do escopo tanto da 6749 quanto da 6750.

3. Fluxo(s) — as 3 formas de transmitir o token (§2)

A RFC define exatamente três métodos pelos quais o client pode apresentar o `access_token` ao resource server. Antes de detalhar cada um, uma regra transversal — e muitas vezes negligenciada em implementações reais — merece destaque: **o client não deve usar mais de um método na mesma requisição**. Se um resource server detectar o token informado em mais de um lugar simultaneamente (por exemplo, tanto no header `Authorization` quanto na query string), a resposta correta é `400 Bad Request` com `error=invalid_request` — nunca aceitar silenciosamente um dos dois e ignorar o outro, e nunca tratar isso como ambiguidade "inofensiva". A razão de ser dessa regra é reduzir a superfície de ataque de confusão: se dois locais pudessem carregar tokens diferentes, um sistema com múltiplas camadas (proxy, gateway, aplicação) poderia inspecionar um e a aplicação final usar outro, abrindo brechas de bypass de autorização.

3.1 Authorization Request Header Field (§2.1) — o método recomendado

Este é o método que a RFC explicitamente recomenda e que **todo resource server compatível deve suportar**:

```
GET /recurso HTTP/1.1
Host: api.exemplo.com
Authorization: Bearer mF_9.B5f-4.1JqM
```

A vantagem estrutural deste método é que o header `Authorization` não é, por padrão, persistido em logs de acesso comuns de servidores web (diferente da URL completa, que quase sempre é), não aparece no header `Referer` enviado a terceiros, e não fica no histórico de navegação — reduzindo consideravelmente a superfície de vazamento acidental em comparação aos outros dois métodos.

3.2 Form-Encoded Body Parameter (§2.2) — restrito e não recomendado

Este método só é permitido quando **todas** as condições a seguir são satisfeitas: o método HTTP é `POST` (a RFC também permite outros métodos que aceitem corpo, mas na prática é quase sempre `POST`), o `Content-Type` é integralmente `application/x-www-form-urlencoded`, e o corpo da requisição não contém nenhum outro conteúdo além dos parâmetros form-encoded (ou seja, não pode coexistir com um corpo multipart, um payload JSON, ou qualquer outro formato):

```
POST /recurso HTTP/1.1
Host: api.exemplo.com
Content-Type: application/x-www-form-urlencoded

access_token=mF_9.B5f-4.1JqM&outro_campo=valor
```

A RFC explicitamente qualifica este método como não recomendado, exceto quando o client não tem como manipular headers HTTP diretamente (algumas plataformas/bibliotecas legadas de formulário). Além da limitação evidente (só funciona com `POST/PUT` e corpo form-encoded), há um risco de log

adicional: muitos frameworks de aplicação registram corpos de requisição inteiros em logs de depuração — um hábito bem mais comum do que logar o header `Authorization` — o que amplia a chance de o token vaziar por essa via.

3.3 URI Query Parameter (§2.3) — desaconselhado explicitamente

Este é o método que a própria RFC trata com a linguagem mais forte de desaconselhamento, embora ainda o defina formalmente (para interoperabilidade com clients legados que não têm alternativa):

```
GET /recurso?access_token=mF_9.B5f-4.1JqM HTTP/1.1
Host: api.exemplo.com
```

Os riscos, todos bem documentados na literatura de segurança web e reiterados na própria RFC (§5): URLs completas (incluindo query string) são rotineiramente gravadas em logs de acesso de servidores web, proxies reversos e balanceadores de carga — muitas vezes por padrão, sem qualquer configuração adicional. URLs também ficam no histórico de navegação do dispositivo do usuário, indefinidamente, acessível a qualquer um com acesso físico ou a um backup do dispositivo. E, criticamente, se a página que recebe essa URL contiver qualquer link para um recurso de terceiros (uma imagem, um script, um link clicável), o header `Referer` da requisição subsequente pode incluir a URL completa da página atual — vazando o token para um domínio completamente alheio ao OAuth. A recomendação prática, reforçada por implementações modernas (ver seção 7), é tratar este método como **inaceitável** em vez de meramente desaconselhado, a menos que exista uma razão de interoperabilidade muito específica e documentada.

(Os 3 métodos — e a rejeição por uso simultâneo em mais de um local — já estão implementados e testados no laboratório `rfc6749/resource-server` deste projeto, na função `requireBearer`, que extrai o token do header, do corpo ou da query string e rejeita explicitamente quando mais de uma fonte está presente.)

4. Parâmetros e endpoints normativos

A RFC 6750 não define nenhum endpoint novo — ela reaproveita inteiramente os endpoints já definidos pela RFC 6749 (authorization, token, redirection). O que ela define são parâmetros e headers específicos para o momento de **uso** do token contra o resource server:

4.1 `token_type = "Bearer"` na resposta do token endpoint

Este é o único ponto de contato direto entre a RFC 6750 e a resposta do token endpoint definida pela RFC 6749 (§5.1): o campo `token_type` deve valer `"Bearer"` (com essa capitalização na emissão, embora a comparação na recepção deva ser case-insensitive — ver seção 7) para sinalizar ao client que ele deve seguir as regras desta RFC ao apresentar o token.

4.2 O header de desafio/erro `WWW-Authenticate` (§3)

Este é o mecanismo central pelo qual um resource server comunica, de forma padronizada, tanto um "desafio" inicial (quando nenhum token foi apresentado) quanto um erro específico (quando um token foi apresentado, mas é inválido, expirado, ou insuficiente):

```
WWW-Authenticate: Bearer realm="exemplo", error="invalid_token", error_description="o token expirou"
```

O parâmetro `realm` segue a semântica geral de HTTP Authentication (RFC 7235) — identifica o espaço de proteção ao qual a credencial se aplica, útil quando um mesmo host serve múltiplos domínios de autorização distintos. Os parâmetros `error` e `error_description` são específicos desta RFC e só devem ser incluídos quando há de fato um erro a reportar — nunca em um desafio inicial simples (ver a regra sutil da seção 4.4 abaixo).

4.3 Os 3 códigos de erro normativos (§3.1)

Código	Status HTTP	Significado
<code>invalid_request</code>	400	Requisição malformada — por exemplo, token informado em mais de um local (seção 3), ou parâmetro obrigatório ausente/duplicado
<code>invalid_token</code>	401	Token ausente de significado válido: expirado, revogado, malformado, ou emitido por um AS que o RS não reconhece
<code>insufficient_scope</code>	403	O token é válido (autenticação bem-sucedida), mas não carrega o escopo necessário para a operação solicitada (falha de autorização, não de autenticação) — o header de resposta deve incluir um parâmetro <code>scope</code> listando o(s) escopo(s) exigido(s)

A distinção entre `invalid_token` (401) e `insufficient_scope` (403) é semanticamente importante e corresponde exatamente à distinção clássica entre autenticação e autorização: 401 significa "eu não sei quem você é" (ou "sua credencial não é válida"), 403 significa "eu sei quem você é, e você não tem permissão para isso". Um resource server que devolve 401 para uma falha de escopo (ou vice-versa) está entregando uma informação de diagnóstico incorreta ao client — o que pode levar a lógicas de retry equivocadas (por exemplo, tentar renovar o token via `refresh_token` quando na verdade o problema é que o escopo concedido nunca incluiria a operação, e nenhuma renovação vai resolver isso).

4.4 A regra sutil e fácil de esquecer: desafio sem erro

Quando **nenhum** token foi enviado na requisição (o cliente sequer tentou se autenticar), o resource server deve responder `401` com um `WWW-Authenticate` **sem** os parâmetros `error` e `error_description` — apenas `Bearer realm="..."`. A justificativa, embora sutil, é uma boa prática de segurança por design: incluir o motivo detalhado do erro para uma parte que ainda nem tentou se autenticar vaza informação

desnecessária (por exemplo, detalhes sobre o esquema de validação interno) a um ator potencialmente hostil que está apenas sondando o endpoint. O erro detalhado (`invalid_token` , `insufficient_scope`) só faz sentido depois que uma tentativa real de autenticação foi feita e falhou por um motivo específico.

5. Segurança já tratada na própria RFC 6750 (§5)

A seção de considerações de segurança da RFC 6750 é desproporcionalmente extensa para uma especificação que, à primeira vista, parece "apenas" descrever como colocar um token em um header — e isso é deliberado: como bearer = posse é igual a acesso (seção 1.2), praticamente toda garantia de segurança do sistema depende de como essas recomendações são seguidas.

5.1 TLS obrigatório em toda requisição que carrega o token

Diferente da RFC 6749, onde TLS protege primariamente a emissão do token, aqui TLS deve proteger **toda e qualquer requisição subsequente** que carregue o bearer token — não apenas o momento de obtenção. Um bearer token trafegando em texto claro, ainda que apenas uma vez, é funcionalmente equivalente a vazar uma senha: qualquer observador passivo da rede (um Wi-Fi público não criptografado, um proxy corporativo mal configurado, um roteador comprometido) captura uma credencial diretamente utilizável até a expiração do token.

5.2 Tempo de vida curto como mitigação central

Como não há mecanismo de revogação instantânea universal embutido no bearer token em si (revogação depende de infraestrutura adicional — uma blocklist consultada pelo RS, ou tokens auto-contidos de vida curta o suficiente para que a janela de exposição seja pequena), a RFC recomenda tempos de vida curtos como a principal ferramenta de contenção de dano: se um token vaza, o estrago está limitado ao que pode ser feito antes da expiração. É por isso que o laboratório deste projeto usa um `ACCESS_TOKEN_TTL` de apenas 5 minutos — deliberadamente curto para tornar visível, na prática, o papel do refresh token (RFC 6749 §6) como o mecanismo que permite sessões longas sem exigir access tokens de vida longa.

5.3 Armazenamento: tratar bearer tokens como segredos

Tanto no client quanto no resource server, um bearer token deve ser tratado com o mesmo rigor que uma senha: nunca em texto claro em logs de aplicação, nunca em caches não protegidos, nunca em armazenamento local sem controle de acesso adequado (em uma SPA, por exemplo, `localStorage` é acessível a qualquer script rodando na mesma origem — incluindo um script malicioso injetado via XSS — o que motiva, em arquiteturas mais cuidadosas, guardar tokens em cookies `HttpOnly` geridos pelo backend, fora do alcance de JavaScript).

5.4 Nunca colocar o token na URL de página

Esta recomendação reforça diretamente o desaconselhamento do método de query parameter (seção 3.3): além dos riscos já descritos de log e histórico, colocar o token na URL da própria página (não apenas em uma chamada de API) o expõe a qualquer extensão de navegador com permissão de leitura

de URL, a ferramentas de analytics que capturam a URL corrente automaticamente, e a qualquer captura de tela ou compartilhamento de tela que inclua a barra de endereço.

5.5 Escopo reduzido como limitação de dano

Tokens devem carregar o menor escopo possível para a operação em questão — um princípio de menor privilégio aplicado diretamente ao design do token. Se um token de leitura de perfil vaza, o dano potencial deve se limitar à leitura de perfil, nunca se estender a operações mais sensíveis que o client nem sequer precisava solicitar. Essa recomendação, ainda relativamente genérica na RFC 6750, prenuncia diretamente desenvolvimentos posteriores tratados no item 6 deste livro: a RFC 8707 (Resource Indicators), que amarra o token a um recurso-alvo específico, e a RFC 9396 (Rich Authorization Requests), que permite escopos estruturados e muito mais granulares do que a simples lista de strings da RFC 6749.

Nota de laboratório: como já registrado no capítulo anterior, os labs práticos deste projeto rodam em HTTP puro (`http://localhost`), a despeito da exigência formal de TLS descrita em 5.1 — uma simplificação deliberada, mantida até o capítulo da RFC 8705 (mTLS, item 6 deste livro), onde TLS real passa a ser o próprio mecanismo em estudo. Até lá, o foco pedagógico permanece na mecânica de emissão/uso de tokens, não na camada de transporte.

6. Relação com outras RFCs

- A RFC 6750 depende inteiramente da **RFC 6749** para a emissão do token — ela cobre exclusivamente o uso, nunca a obtenção.
- O formato de erro que ela introduz (`WWW-Authenticate` com parâmetros `error / error_description`) estabeleceu um padrão que outras extensões do ecossistema OAuth reaproveitaram por analogia, mesmo fora do escopo estrito de bearer tokens.
- **RFC 8705 (mTLS)** e **RFC 9449 (DPoP)** existem justamente para resolver a fraqueza estrutural que a própria RFC 6750 documenta explicitamente em sua seção 5: um bearer token roubado equivale a acesso total até a expiração. Ambas adicionam uma camada de prova de posse por cima do que seria, de outra forma, um bearer token puro — sem alterar o formato básico definido aqui.
- **RFC 9068 (JWT Profile for Access Tokens)** formaliza um formato específico e auto-contido de bearer token (um JWT assinado com claims padronizadas) — a RFC 6750, como já notado na seção 2, é inteiramente agnóstica quanto a esse formato: um token opaco de tabela hash e um JWT assinado são, ambos, "bearer tokens" do ponto de vista desta especificação.
- **RFC 9700 (BCP)** reforça a recomendação de tokens sender-constrained para cenários sensíveis, elevando a uma quase-obrigação prática o que a RFC 6750 já sinalizava como uma limitação conhecida e aceita em 2012.
- **RFC 7662 (Token Introspection)** define um protocolo padronizado para o resource server consultar o authorization server sobre a validade e os metadados de um bearer token opaco — uma peça de infraestrutura complementar que a RFC 6750 pressupõe existir de alguma forma, mas não especifica.

7. O que mudou desde a publicação (2012)

- A RFC em si não foi revisada formalmente, mas a prática de mercado evoluiu de forma perceptível: hoje é comum tratar o método de query string (seção 3.3) como **inaceitável** — não apenas desaconselhado — em implementações modernas; muitas APIs financeiras e de alto valor nem sequer o implementam, forçando o client a usar o header `Authorization`.
- Uma errata publicada posteriormente esclareceu que o nome do esquema de autenticação (`Bearer`) deve ser tratado como **case-insensitive** na recepção, seguindo a regra geral de esquemas de autenticação HTTP definida pela RFC 7235 — um resource server rigoroso demais que exigisse exatamente `"Bearer "` (com maiúscula exata) estaria, tecnicamente, fora de conformidade. O resource server do laboratório deste projeto já trata isso corretamente, usando o regex `/^Bearer /i` (a flag `i` torna a checagem case-insensitive) para extrair o token do header.
- O padrão de "considerado seguro o suficiente" mudou substancialmente: em 2012, bearer token puro + TLS era o estado da arte aceitável para praticamente qualquer cenário. Hoje, para APIs financeiras ou de alto valor, a expectativa de mercado (refletida na RFC 9700 e em perfis de segurança setoriais como o FAPI — Financial-grade API, do OpenID Foundation) é sender-constraining via mTLS ou DPoP como padrão, não como exceção — bearer puro passou a ser tratado como piso mínimo aceitável apenas para cenários de baixa sensibilidade.

Exemplo em Java (JDK 21, sem frameworks)

Os exemplos a seguir reutilizam o utilitário `SimpleJson` já definido no Capítulo 1 (RFC 6749) deste livro — não o redefinimos aqui. Como antes, tudo usa exclusivamente `java.net.http.HttpClient` da biblioteca padrão do JDK 21, sem qualquer framework ou biblioteca externa.

Exemplo 1 — Enviando o bearer token via header (§2.1, o método recomendado)

```
import java.net.URI;
import java.net.http.HttpClient;
import java.net.http.HttpRequest;
import java.net.http.HttpResponse;
import java.util.Map;

final class BearerViaHeader {

    private static final HttpClient HTTP = HttpClient.newHttpClient();

    // Metodo recomendado pela RFC 6750 2.1 -- todo resource server compatível
    // DEVE suportar este metodo. O token nao aparece na URL nem no corpo,
    // reduzindo a chance de vazamento por Logs de acesso, historico ou Referer.
    static HttpResponse<String> callProtectedResource(String resourceUrl, String accessToken)
        throws Exception {

        HttpRequest request = HttpRequest.newBuilder()
            .uri(URI.create(resourceUrl))
            .header("Authorization", "Bearer " + accessToken)
            .GET()
            .build();

        return HTTP.send(request, HttpResponse.BodyHandlers.ofString());
    }

    public static void main(String[] args) throws Exception {
        HttpResponse<String> response = callProtectedResource(
            "http://localhost:3002/api/profile", "mF_9.B5f-4.1JqM");

        if (response.statusCode() == 200) {
            Map<String, String> body = SimpleJson.parseFlat(response.body());
            System.out.println("Perfil: " + body);
        } else {
            System.out.println("Falhou com status " + response.statusCode());
            // Ver Exemplo 3 para como interpretar o header WWW-Authenticate aqui.
        }
    }
}
```

Exemplo 2 — Enviando o bearer token via body e via query (§2.2 e §2.3, não recomendados)

Estes dois métodos são apresentados por completude — a RFC os define, mas desaconselha ambos (o segundo, de forma bem mais enfática). Note que os dois exemplos abaixo **nunca** devem ser combinados com o header `Authorization` na mesma requisição (RFC 6750 §2 proíbe mais de um método simultâneo — ver seção 3 deste capítulo).


```

import java.net.URI;
import java.net.URLEncoder;
import java.net.http.HttpClient;
import java.net.http.HttpRequest;
import java.net.http.HttpResponse;
import java.nio.charset.StandardCharsets;

final class BearerViaBodyOrQuery {

    private static final HttpClient HTTP = HttpClient.newHttpClient();

    // RFC 6750 2.2 -- permitido SOMENTE em POST/PUT com corpo inteiramente
    // application/x-www-form-urlencoded, e sem nenhum outro conteudo no corpo.
    // Nao recomendado: existe para clients que nao conseguem manipular headers.
    static HttpResponse<String> viaFormBody(String resourceUrl, String accessToken)
        throws Exception {

        String body = "access_token=" + URLEncoder.encode(accessToken, StandardCharsets.UTF_8);

        HttpRequest request = HttpRequest.newBuilder()
            .uri(URI.create(resourceUrl))
            .header("Content-Type", "application/x-www-form-urlencoded")
            .POST(HttpRequest.BodyPublishers.ofString(body, StandardCharsets.UTF_8))
            .build();

        return HTTP.send(request, HttpResponse.BodyHandlers.ofString());
    }

    // RFC 6750 2.3 -- desaconselhado explicitamente: a URL completa (com o
    // token) fica sujeita a Logs de proxy/servidor, historico do navegador e
    // vazamento via header Referer para terceiros. Usar apenas como ultimo recurso.
    static HttpResponse<String> viaQueryParam(String resourceUrlBase, String accessToken)
        throws Exception {

        String url = resourceUrlBase + "?access_token="
            + URLEncoder.encode(accessToken, StandardCharsets.UTF_8);

        HttpRequest request = HttpRequest.newBuilder()
            .uri(URI.create(url))
            .GET()
            .build();

        return HTTP.send(request, HttpResponse.BodyHandlers.ofString());
    }

    public static void main(String[] args) throws Exception {
        String token = "mF_9.B5f-4.1JqM";

        HttpResponse<String> r1 = viaFormBody("http://localhost:3002/api/profile", token);
        System.out.println("via body: " + r1.statusCode());

        HttpResponse<String> r2 = viaQueryParam("http://localhost:3002/api/profile", token);
    }
}

```

```
        System.out.println("via query: " + r2.statusCode());  
    }  
}
```

Exemplo 3 — Interpretando o header `WWW-Authenticate` de erro (§3)

Quando o resource server rejeita a requisição, ele sinaliza o motivo através do header `WWW-Authenticate`. Um client bem-comportado deveria distinguir os três códigos de erro normativos (seção 4.3 deste capítulo) para decidir a ação correta: `invalid_token` sugere tentar obter um novo token (via `refresh_token`, RFC 6749 §6); `insufficient_scope` sugere que uma renovação simples não resolve — seria necessário um novo fluxo de autorização pedindo o escopo adequado; `invalid_request` sinaliza um bug no próprio client (token enviado em mais de um lugar, por exemplo).

```

import java.util.LinkedHashMap;
import java.util.Map;
import java.util.regex.Matcher;
import java.util.regex.Pattern;

final class WwwAuthenticateParser {

    // Parser deliberadamente simples para o cabecalho:
    // WWW-Authenticate: Bearer realm="exemplo", error="invalid_token", error_description="o token expirou"
    // RFC 6750 3 / 3.1. Assume o esquema "Bearer" e parametros key="value" separados por virgula.
    static Map<String, String> parse(String header) {
        Map<String, String> params = new LinkedHashMap<>();
        if (header == null || !header.regionMatches(true, 0, "Bearer", 0, 6)) {
            return params; // nao e um desafio Bearer -- outro esquema, ou ausente
        }

        Matcher m = Pattern.compile("(\\w+=\\\"(.*)\\\")").matcher(header);
        while (m.find()) {
            params.put(m.group(1), m.group(2));
        }
        return params;
    }

    // Decide a proxima acao do client com base no error retornado (se houver).
    // Ausencia de 'error' com apenas 'realm' = desafio inicial (RFC 6750 3, ver
    // secao 4.4 deste capitulo) -- nenhum token foi enviado, nao ha o que reinterpretar.
    static String suggestAction(Map<String, String> challenge) {
        String error = challenge.get("error");
        if (error == null) {
            return "Nenhum token foi enviado (ou o desafio nao traz detalhes) -- autenticar normalmente";
        }
        return switch (error) {
            case "invalid_token" -> "Token invalido/expirado -- tentar renovar via refresh_token (RFC 6750 3.1)";
            case "insufficient_scope" -> "Escopo insuficiente (scope exigido: "
                + challenge.getOrDefault("scope", "")
                + ") -- e necessario um novo consentimento, refresh nao resolve.";
            case "invalid_request" -> "Requisicao malformada -- provavel bug no client (ex: token e invalido)";
            default -> "Erro desconhecido: " + error;
        };
    }

    public static void main(String[] args) {
        String header = "Bearer realm=\"oauth-rfc6749-lab\", error=\"invalid_token\", "
            + "error_description=\"o token expirou\"";

        Map<String, String> challenge = parse(header);
        System.out.println("Parametros: " + challenge);
        System.out.println("Acao sugerida: " + suggestAction(challenge));

        // Caso do desafio "puro" (nenhum token enviado -- secao 4.4 deste capitulo):
        // sem error/error_description, apenas realm.
        Map<String, String> bareChallenge = parse("Bearer realm=\"oauth-rfc6749-lab\"");
    }
}

```

```
        System.out.println("Desafio simples: " + bareChallenge);
        System.out.println("Acao sugerida: " + suggestAction(bareChallenge));
    }
}
```

Relação com o laboratório deste projeto

O laboratório `rfc6749/resource-server` deste projeto (Node.js/Express) implementa exatamente os três métodos de transmissão descritos neste capítulo dentro da função `requireBearer` : ela extrai o token do header `Authorization` via regex case-insensitive (`/^Bearer (.+)$/i` , coerente com a errata discutida na seção 7), do corpo form-encoded quando aplicável, e da query string — e rejeita explicitamente com `400 invalid_request` se mais de uma fonte estiver presente na mesma requisição, exatamente como a regra transversal da seção 3 exige. Ele também monta o header `WWW-Authenticate` com os três códigos de erro normativos (`invalid_token` , `insufficient_scope` , e o `invalid_request` do caso de múltiplas fontes), incluindo o parâmetro `scope` na resposta de `insufficient_scope` — e omite `error / error_description` no caso de nenhum token ter sido enviado, seguindo à risca a regra sutil da seção 4.4.

Parte 2 — Segurança: Modelo de Ameaças e Boas Práticas

RFC 6819 — OAuth 2.0 Threat Model and Security Considerations

1. Contexto e motivação

A RFC 6749 nasceu deliberadamente enxuta. Ela descreve um *framework* de autorização — quatro concessões, um punhado de endpoints, um vocabulário mínimo — e empurra para fora do documento normativo praticamente toda decisão que separa uma implementação segura de uma implementação ingênua e explorável. Isso não é um defeito de redação: é uma escolha de design consciente dos autores, que preferiram um núcleo estável e extensível a uma spec monolítica que tentasse antecipar todo ataque possível. O preço dessa escolha é que alguém precisa, em algum lugar, escrever o documento complementar que enumera "e se isto der errado?". Esse documento é a RFC 6819, publicada em janeiro de 2013, poucos meses depois da RFC 6749/6750.

É importante posicionar corretamente o que ela é e o que ela não é. A RFC 6819 tem status **Informational** — não é uma especificação de protocolo, não define uma única mensagem nova, não introduz um parâmetro sequer. Ela é, em essência, um exercício de *threat modeling* formal aplicado retroativamente a um protocolo que já existia: descreve a arquitetura OAuth 2.0 sob a ótica de "quais são os ativos valiosos aqui, quem pode atacá-los, por onde, e o que mitiga cada ataque". O gênero não é exclusivo do OAuth — é o mesmo tipo de exercício que se faz com STRIDE, com DREAD, com qualquer metodologia de análise de ameaças de sistemas distribuídos — mas aplicado com granularidade fina à arquitetura específica de authorization code, tokens portáteis e redirecionamentos via user-agent que o OAuth 2.0 introduziu.

Por que isso importa tanto hoje, mais de uma década depois? Porque a RFC 6819 se tornou, na prática, **o catálogo de ameaças mais citado e mais influente de todo o ecossistema OAuth**. Não é exagero dizer que quase toda extensão de segurança publicada depois dela resolve uma ameaça específica que já estava documentada aqui, em prosa, anos antes de existir uma RFC dedicada à mitigação formal. PKCE (RFC 7636) resolve a interceptação de authorization code que a 6819 já descrevia informalmente. Resource Indicators (RFC 8707) resolve a confusão de audience/"confused deputy" que a 6819 já apontava como risco estrutural. A própria RFC 9700 — o assunto do próximo capítulo — é, essencialmente, esta RFC reescrita em tom normativo: ela pega dezenas das recomendações "deveria fazer" daqui e as transforma em "deve fazer obrigatoriamente", à luz de mais de uma década de exploração real em produção.

Vale também entender por que a RFC 6819 nunca foi atualizada, e por que isso não a torna obsoleta. Um catálogo de ameaças é, por natureza, um retrato de um momento — mas a arquitetura fundamental que ele analisa (redirecionamento via navegador, troca de code por token, tokens bearer portáteis) não mudou desde 2013. As ameaças estruturais continuam as mesmas; o que mudou foi o consenso sobre qual mitigação é obrigatória versus opcional, e isso é exatamente o papel que a RFC 9700 assumiu treze anos depois.

2. Terminologia e papéis

A RFC 6819 não introduz nenhum papel novo — ela reaproveita integralmente o vocabulário da RFC 6749: *resource owner*, *client*, *authorization server*, *resource server*. O que ela introduz é vocabulário de *threat modeling* propriamente dito, que vale a pena destrinchar porque estrutura toda a leitura do documento:

- **Ativo (*asset*)** — algo de valor que precisa ser protegido. No OAuth, os ativos centrais são: as credenciais do resource owner (senha, sessão de login no AS), o `client_secret` (segredo do client confidencial), o *authorization code* (curta duração, mas trocável por token), o *access token* e o *refresh token* (o ativo de maior duração e maior "poder de compra" — um refresh token comprometido pode gerar access tokens indefinidamente até ser revogado), e os próprios dados do resource owner expostos pelo resource server.
- **Ameaça (*threat*)** — um evento ou ação que pode comprometer um ativo. A RFC organiza as ameaças por *onde* elas ocorrem na arquitetura, não por *que tipo* de ataque são — uma escolha metodológica que faz sentido porque o OAuth é, antes de tudo, uma coreografia entre partes distribuídas, e cada ponto da coreografia (o navegador do usuário, o canal de redirecionamento, o armazenamento do client, o armazenamento do AS) tem uma superfície de ataque distinta.
- **Contramedida (*countermeasure*)** — a mitigação recomendada para cada ameaça, normalmente expressa como "SHOULD" ou como recomendação de design, quase nunca como "MUST" (lembre-se: é um documento Informational, não tem poder normativo de spec).
- **Nível de impacto e probabilidade** — embora a RFC não use uma matriz de risco formal (tipo DREAD com pontuação numérica), ela qualifica cada ameaça em termos de quão fácil é executá-la e quão grave é o dano — um vocabulário implícito de risco que aparece na profundidade da discussão dedicada a cada item.

Um ponto sutil que vale reter: a RFC 6819 trata o **navegador do usuário (*user-agent*)** quase como um quinto papel implícito na análise de ameaças, mesmo que a RFC 6749 não o eleve à categoria de papel formal. Isso faz sentido porque boa parte das ameaças mais graves do catálogo — CSRF, vazamento via `Referer`, comprometimento por malware no dispositivo, phishing na tela de consentimento — dependem inteiramente do comportamento e das garantias (ou falta delas) do navegador como intermediário do redirecionamento.

3. Estrutura do documento — não há fluxo, há um mapa de superfície de ataque

Como já adiantado, não existe um "fluxo de protocolo" para descrever aqui — a contribuição da RFC é estrutural, não procedural. Ela segue a arquitetura clássica de um documento de threat modeling:

- **§3 — Arquitetura recapitulada.** Redesenha o diagrama de sequência do Authorization Code Grant (e dos demais grants da RFC 6749) especificamente para servir de mapa de referência à análise que segue. Cada seta do diagrama — requisição de autorização, redirecionamento com code, troca de code por token, chamada ao resource server com o token — vira um ponto de ancoragem para as ameaças listadas depois.

- **§4 — Proteções já embutidas na RFC 6749/6750.** Antes de listar o que pode dar errado, o documento resume o que a spec-base já exige ou recomenda: TLS obrigatório nos endpoints que carregam credenciais e tokens, expiração de token, validação de `redirect_uri` (ainda sem especificar *como* validar — esse detalhe fica para depois), escopo como mecanismo de limitação de dano. Essa seção existe para deixar claro que o catálogo que segue não está reinventando a segurança do zero — está identificando as lacunas que sobram *depois* dessas proteções de base.
- **§5 — O catálogo de ameaças propriamente dito**, organizado por alvo:
- **§5.1 — Ameaças ao client.** Falsificação de client, roubo de `client_secret`, malware no dispositivo do usuário capturando tokens em trânsito ou em repouso.
- **§5.2 — Ameaças ao authorization server.** Comprometimento do endpoint de autorização, phishing da tela de login/consentimento, ataques ao armazenamento de tokens e segredos.
- **§5.3 — Ameaças ao resource owner.** Engenharia social para induzir consentimento indevido, clickjacking na UI de autorização, vazamento de credenciais de login.
- **§5.4 — Ameaças ao resource server.** Replay de token, aceitação de token emitido para outro público (confused deputy), validação insuficiente de escopo.
- **§6 — Recomendações consolidadas**, que voltam a percorrer o mesmo território de §5 mas já na forma de "o que fazer a respeito", servindo de ponte para o que, treze anos depois, se tornaria a RFC 9700.

Essa arquitetura em duas passadas (primeiro o ativo/ameaça, depois a mitigação) é o motivo pelo qual a RFC 6819, apesar de longa, funciona bem como referência de consulta: dado um sintoma ou uma dúvida de design ("meu client precisa validar X?"), é possível ir direto à seção que trata do ativo em risco.

4. Parâmetros e endpoints normativos

Nenhum. Vale repetir com ênfase porque é fácil, ao ler um documento tão detalhado, presumir que ele define algo protocolar — não define. A RFC 6819 não cria nenhum endpoint, não adiciona nenhum parâmetro à requisição de autorização ou à requisição de token, não define um único código de erro novo. Toda a análise é sobre a arquitetura *já existente* da RFC 6749/6750. Isso contrasta diretamente com o próximo capítulo (RFC 9700), que também não define protocolo novo, mas que — por ser uma BCP e não um documento puramente informativo — assume tom prescritivo com força de "MUST"/"MUST NOT" mesmo sem inventar mensagens novas.

5. Ameaças e mitigações mais relevantes do catálogo (§5) — análise aprofundada

O catálogo completo tem dezenas de itens numerados. O que segue é uma seleção comentada das ameaças que mais moldaram o resto do ecossistema — não um resumo raso, mas uma discussão de cada uma com o cenário de exploração completo e o porquê estrutural da mitigação.

5.1 Intercepção do authorization code

O cenário. O authorization code trafega pelo canal menos controlado de todo o fluxo: o redirecionamento via navegador, tipicamente uma URL completa (`redirect_uri?code=...&state=...`) que passa por, no mínimo, o histórico do navegador e potencialmente por proxies, logs de servidor web e — em mobile, o caso mais grave — pelo próprio sistema operacional na forma de um *custom URI scheme* (`meuapp://callback?code=...`).

Em Android e iOS anteriores a mecanismos mais rígidos de verificação de domínio (App Links / Universal Links), múltiplos aplicativos podiam se registrar para o *mesmo* esquema de URI customizado. O sistema operacional, ao receber o redirecionamento, não tem garantia de entregá-lo ao app legítimo — um app malicioso instalado no mesmo dispositivo, registrado com o esquema idêntico, pode receber a intercepção e capturar o code antes (ou em vez) do client verdadeiro. Uma vez de posse do code, o atacante só precisa completar a troca por token — e se o client for público (sem `client_secret`, caso comum em mobile), não há nenhuma segunda barreira.

Por que isso é grave especificamente para clients públicos. Um client confidencial que perde um code interceptado ainda está parcialmente protegido, porque a troca final exige o `client_secret` que só ele possui. Um client público não tem esse escudo — o code sozinho é suficiente para completar o roubo. É exatamente esta assimetria que a RFC 6819 identifica como estrutural, sem ainda ter uma solução formal para propor.

A mitigação que a própria RFC já insinua. O texto da 6819 já sugere, em prosa, vincular a requisição de autorização inicial à troca final de code por token — de forma que um atacante que só intercepte o code (sem ter iniciado o fluxo) não consiga completá-lo. Essa ideia, ainda informal aqui, é **literalmente** a espinha dorsal do PKCE (RFC 7636), publicado três anos depois: o client gera um segredo local (`code_verifier`), envia um hash dele (`code_challenge`) na requisição de autorização, e só o `code_verifier` original — nunca exposto no canal de redirecionamento — permite trocar o code por token. Um atacante que capture o code no meio do caminho não tem o verifier e a troca falha.

5.2 CSRF no endpoint de redirecionamento (login CSRF via authorization code)

O cenário. Sem um mecanismo de correlação entre a requisição de autorização que o client iniciou e a resposta que ele recebe, um atacante pode montar o seguinte ataque: ele mesmo inicia um fluxo de autorização legítimo (autenticando-se com a *sua própria* conta), captura o authorization code resultante, e induz a vítima — via um link malicioso, um `<img>` oculto, um formulário auto-submit — a acessar o `redirect_uri` do client com esse code anexado. Se o client aceitar o code sem validar que ele corresponde à sessão que a vítima realmente iniciou, o resultado é que a conta da vítima no client fica vinculada à conta do atacante no resource server — um ataque de fixação de sessão/conta que pode ser usado, por exemplo, para plantar dados sob controle do atacante que depois a vítima acredita serem seus.

A mitigação. O parâmetro `state`, opaco e gerado pelo client antes de iniciar o fluxo, vinculado à sessão do navegador da vítima (tipicamente guardado num cookie de sessão do client). Ao receber o retorno no `redirect_uri`, o client compara o `state` recebido com o que ele próprio gravou para aquela sessão — se não bater, a requisição é rejeitada. O laboratório deste projeto já implementa essa validação de `state`.

Vale marcar uma nuance que a RFC 6819 já antecipa e que a RFC 9700 viria a detalhar melhor: o `state` protege contra CSRF **somente se for validado contra algo vinculado à sessão do navegador** — gerar um `state` aleatório mas não guardá-lo em lugar nenhum para comparar depois é uma implementação incompleta que dá falsa sensação de segurança.

5.3 Open redirector via `redirect_uri` mal validado

O cenário. Se o authorization server aceita qualquer `redirect_uri` que "comece com" ou "contenha" o valor registrado pelo client — validação por prefixo, por substring, ou pior, por wildcard —, um atacante pode manipular o parâmetro `redirect_uri` na requisição de autorização para apontar para um destino sob seu controle que ainda "bate" com a regra frouxa. Exemplos clássicos de bypass de validação por prefixo: registrar `https://client.exemplo.com/callback` e o atacante conseguir usar `https://client.exemplo.com/callback.atacante.com/` ou `https://client.exemplo.com@atacante.com/callback` (abuso de sintaxe de URI com userinfo) ou ainda `https://client.exemplo.com/callback/../../atacante`. O efeito prático: o authorization code (ou, em fluxos mais antigos, até o access token via Implicit Grant) é entregue a um endpoint do atacante, não ao client legítimo.

Por que a validação "flexível" é tentadora e por que ela é sempre um erro. Times de desenvolvimento frequentemente querem suportar múltiplos subcaminhos de callback (`/callback`, `/callback/mobile`, `/callback/web`) sem cadastrar cada um explicitamente, e por isso implementam validação por prefixo "por conveniência". A RFC 6819 já sinaliza isso como recomendação (não obrigação) de usar comparação estrita. É só treze anos depois, na RFC 9700, que o **exact match** se torna mandato universal (`MUST`) — o mesmo raciocínio de "conveniência abre brecha" se repete tantas vezes na história da segurança do OAuth que vale a pena grifar aqui: qualquer flexibilização de uma regra de comparação de URI tem, historicamente, se revelado explorável.

5.4 Vazamento de token via `Referer`, logs ou histórico do navegador

O cenário. Se o access token (ou o authorization code) aparece na query string de uma URL, ele fica exposto a três vetores de vazamento que nada têm a ver com o protocolo OAuth em si: (1) o cabeçalho `Referer` enviado pelo navegador para qualquer recurso de terceiro embutido na página seguinte (imagens, scripts, iframes) carrega a URL completa, incluindo a query string; (2) logs de servidor web tipicamente registram a URL completa da requisição, inclusive parâmetros de query, em texto plano, muitas vezes retidos por meses; (3) o histórico do navegador retém a URL indefinidamente, acessível a qualquer pessoa com acesso físico ao dispositivo ou a extensões de navegador maliciosas.

Este item da 6819 é o fundamento direto de por que a **RFC 6750** desaconselha explicitamente carregar o *access token* na query string (permitindo apenas como último recurso, com ressalvas), e prefere o cabeçalho `Authorization: Bearer`. O mesmo raciocínio, aplicado ao *authorization code* — que inevitavelmente viaja na query string do `redirect_uri`, já que não há alternativa de protocolo para isso — é o motivo de o code ter vida curtíssima (minutos, uso único) em vez de qualquer TTL mais longo: o vazamento *vai* acontecer estruturalmente (é a natureza do canal), a mitigação é reduzir a janela de validade a quase zero.

5.5 Reuso do authorization code

O cenário. Um authorization code já trocado por token com sucesso é apresentado de novo ao token endpoint. Duas leituras possíveis: ou é um erro de implementação do client (bug, retry duplicado por timeout), ou é um sinal de que o code vazou e um segundo ator (o atacante) está tentando completar a troca depois do client legítimo já ter feito isso.

A mitigação. Rejeitar peremptoriamente qualquer segunda tentativa de troca do mesmo code — não apenas ignorá-la silenciosamente, mas tratá-la como evento de segurança. A RFC 6819 já recomenda que, ao detectar reuso, o authorization server deveria revogar retroativamente todos os tokens já emitidos a partir daquele code, sob a premissa de que se o code foi reapresentado é porque pode ter havido duas partes de posse dele — e não há como saber com certeza qual delas era a legítima. O laboratório deste projeto já implementa o code como recurso de uso único (removido do armazenamento assim que trocado).

5.6 "Confused deputy" / substituição de token entre resource servers

O cenário. Um access token emitido pelo authorization server com a intenção de ser aceito apenas pelo Resource Server A é apresentado — e aceito — pelo Resource Server B. Isso acontece quando o token não carrega (ou o RS não valida) nenhuma informação de "para qual audiência este token foi emitido", e quando múltiplos resource servers confiam cegamente em qualquer token assinado/válido pelo mesmo authorization server, sem checar se aquele token específico foi de fato destinado a eles. O nome "confused deputy" vem do padrão clássico de segurança de sistemas: um agente com autoridade legítima (o RS, que confia no AS) é enganado a exercer essa autoridade em benefício de um pedido que não deveria atender.

Este é um dos itens do catálogo com a trajetória histórica mais longa dentro do próprio roteiro deste livro: a RFC 6819 já o identifica como risco estrutural em 2013; a **RFC 8707 (Resource Indicators)**, anos depois, oferece o mecanismo formal de mitigação (o parâmetro `resource`, que permite ao client declarar — e ao AS registrar no próprio token — para qual resource server aquele token específico é válido); e a **RFC 9700** reforça a recomendação como parte do endurecimento geral de restrição de audience/escopo.

5.7 Clickjacking na tela de consentimento

O cenário. A tela onde o resource owner decide "autorizar" ou "negar" o acesso do client pode ser embutida, via `<iframe>`, dentro de uma página controlada pelo atacante — tornada invisível ou disfarçada de outro elemento de UI (um botão de "jogo", um "play" de vídeo). O usuário pensa que está clicando em algo inofensivo, mas na verdade seu clique atinge o botão "Autorizar" da tela de consentimento posicionado exatamente sob o cursor.

A mitigação, hoje trivial de implementar mas que a RFC 6819 já recomendava explicitamente em 2013: o authorization server deve enviar o cabeçalho `X-Frame-Options: DENY` (ou o equivalente moderno via `Content-Security-Policy: frame-ancestors 'none'`) em toda resposta da tela de autorização/consentimento, impedindo que ela seja embutida em iframe de terceiros.

5.8 Força bruta contra `client_secret`

O cenário. Clients públicos, por definição, não têm `client_secret` — mas mesmo clients confidenciais mal configurados (segredo curto, sem rate limiting no endpoint de token, sem logout após tentativas falhas) ficam expostos a ataques de força bruta ou dicionário contra o segredo. A RFC 6819 usa este item para reforçar uma tensão que já vinha da própria RFC 6749: a distinção *confidential/public* nunca foi uma garantia binária de segurança — é uma classificação de *capacidade* de manter segredo, e mesmo dentro da categoria "confidencial" há uma escala enorme de robustez efetiva.

A mitigação. Segredos de entropia adequada (nunca senhas memorizáveis por humanos), rate limiting agressivo no endpoint de token, e — o caminho que a RFC 9700 viria a preferir estruturalmente — migrar para autenticação assimétrica (assinatura com chave privada, `private_key_jwt` ou mTLS) em vez de depender de um segredo compartilhado estático que precisa trafegar em toda requisição de token.

5.9 Comprometimento do armazenamento do authorization server

O cenário. Um vazamento de banco de dados, backup ou dump de memória do authorization server expõe `client_secret`s, access tokens e refresh tokens armazenados sem proteção adequada — tipicamente em texto plano, quando deveriam estar ao menos com hash (para segredos que só precisam ser *verificados*, nunca reapresentados) ou criptografados em repouso (para tokens que precisam ser recuperados para validação).

Este item merece destaque porque é o único do catálogo cuja mitigação depende inteiramente de práticas de infraestrutura e operação — nada que o protocolo OAuth em si consiga resolver. É também o motivo pelo qual "confiar no AS" é uma decisão de arquitetura tão pesada: comprometer o AS compromete transitivamente todo client e todo resource server que dependem dele.

6. Relação com outras RFCs

A RFC 6819 funciona como a base conceitual de praticamente todo o resto do roteiro de segurança deste livro:

- **RFC 7636 (PKCE)** resolve diretamente a ameaça de interceptação de authorization code (§5.1 acima).
- **RFC 8707 (Resource Indicators)** resolve a ameaça de confused deputy/substituição de token entre resource servers (§5.6 acima).
- **RFC 9700 (BCP)** é, em essência, esta mesma análise revisitada treze anos depois — pega dezenas de recomendações "deveria" catalogadas aqui e as promove a "deve" obrigatório, à luz de mais de uma década de exploração real documentada.
- Complementa diretamente a **RFC 6749/6750** — assume o papel de "e se algo der errado?" que as specs de protocolo, deliberadamente, não cobrem.
- Deu origem, mais tarde, a análises de ameaça mais específicas que ficam fora do escopo deste roteiro mas merecem nota: ataques de "**mix-up**" entre múltiplos authorization servers (quando um client fala com mais de um AS e pode ser enganado sobre qual deles respondeu a uma dada requisição), endereçados formalmente pela RFC 9207 (AS Issuer Identification).

7. O que mudou desde a publicação (2013)

- A RFC nunca foi revisada — permanece, textualmente, um retrato de ameaças de mais de uma década atrás. Isso não a invalida: a arquitetura fundamental que ela analisa não mudou, e a maioria das ameaças catalogadas continua tão relevante hoje quanto em 2013.
- Muitas mitigações que aqui eram apenas "recomendadas" viraram **obrigatórias** na RFC 9700: PKCE universal (não mais restrito a clients públicos), exact-match de `redirect_uri`, rotação de refresh token.
- Ameaças ligadas ao ROPC (Resource Owner Password Credentials), que aqui recebiam apenas mitigação paliativa (recomendações de uso restrito, TLS obrigatório, sem armazenamento de senha), hoje são resolvidas de forma mais radical: a RFC 9700 simplesmente deprecia o grant type inteiro (`MUST NOT` ser usado) em vez de tentar mitigar riscos em torno de um design considerado estruturalmente falho.
- Classes de ameaça inteiramente novas surgiram depois e não podiam constar num documento de 2013: ataques específicos de manipulação de JWT (`alg=none` , confusão entre chave simétrica e assimétrica em validação de assinatura), phishing de device code (só relevante depois da RFC 8628, publicada em 2019), e ataques de *downgrade* de PKCE (forçar um client a negociar sem PKCE quando ele o suportaria) — nenhuma dessas categorias existia no vocabulário de ameaças de 2013.

8. Relação com os laboratórios deste projeto

A RFC 6819 não tem — nem poderia ter — um laboratório dedicado: é um documento analítico, sem protocolo próprio para implementar. Mas suas mitigações aparecem, na prática, espalhadas por vários laboratórios já construídos neste projeto: a validação de `state` e o code de uso único já estão no lab base da RFC 6749; a mitigação de interceptação de code via PKCE está implementada no lab `rfc7636/`; a revogação em cascata que mitiga o dano de um refresh token comprometido está também no `rfc7636/` (e será aprofundada no capítulo da RFC 7009, a seguir). Ler este capítulo antes de mexer em qualquer um desses laboratórios ajuda a enxergar *por que* cada uma dessas peças de código existe — cada uma delas é a resposta direta a uma ameaça específica catalogada aqui.

Exemplo em Java (JDK 21, sem frameworks)

Os exemplos abaixo são didáticos — ilustram, em Java puro (sem Spring, sem bibliotecas de terceiros), os mecanismos de mitigação discutidos acima. Não são código de produção: faltam tratamento de concorrência real de nível industrial, persistência durável, e configuração de TLS, que num sistema real ficariam a cargo da infraestrutura.

Exemplo 1 — geração de `state` com `SecureRandom` e comparação em tempo constante

Mitiga a ameaça de CSRF no endpoint de redirecionamento (§5.2). Dois pontos merecem atenção: o gerador de aleatoriedade precisa ser criptograficamente seguro (nunca `java.util.Random`), e a comparação do `state` recebido contra o `state` esperado precisa ser em tempo constante — comparar

strings com `String.equals` pode, em tese, vaziar informação por *timing* (a comparação de `String.equals` interrompe no primeiro byte diferente), o que é uma prática ruim para qualquer comparação de valor sensível, mesmo que aqui o `state` não seja secreto (é apenas imprevisível) — o hábito de comparação constante-tempo evita repetir o erro em outros lugares onde o valor comparado é sensível (como um token).

```

import java.security.SecureRandom;
import java.security.MessageDigest;
import java.util.Base64;
import java.util.concurrent.ConcurrentHashMap;
import java.util.Map;

/**
 * Gera e valida o parametro `state` (RFC 6819 secao 5.2 – mitigacao de CSRF
 * no endpoint de redirecionamento).
 */
public final class StateProtection {

    // SecureRandom, NUNCA java.util.Random: precisa ser imprevisivel para um
    // atacante, nao apenas "bem distribuido" estatisticamente.
    private static final SecureRandom RNG = new SecureRandom();

    // Em producao isto seria um store com TTL (Redis, por exemplo) atrelado
    // a sessao do navegador. Aqui, um mapa em memoria basta para ilustrar.
    private final Map<String, String> statePorSessao = new ConcurrentHashMap<>();

    /** Gera um `state` opaco de 256 bits e o vincula a sessao do usuario. */
    public String gerarEArmazenar(String sessionId) {
        byte[] bytes = new byte[32]; // 256 bits de entropia
        RNG.nextBytes(bytes);
        String state = Base64.getUrlEncoder().withoutPadding().encodeToString(bytes);
        statePorSessao.put(sessionId, state);
        return state;
    }

    /**
     * Valida o `state` recebido no redirect contra o que foi gravado para
     * aquela sessao. Comparacao em tempo constante via MessageDigest.isEqual
     * – evita que o tempo de resposta vaze quantos bytes iniciais batem.
     */
    public boolean validar(String sessionId, String stateRecebido) {
        String stateEsperado = statePorSessao.remove(sessionId); // uso unico
        if (stateEsperado == null || stateRecebido == null) {
            return false;
        }
        byte[] a = stateEsperado.getBytes(java.nio.charset.StandardCharsets.UTF_8);
        byte[] b = stateRecebido.getBytes(java.nio.charset.StandardCharsets.UTF_8);
        // MessageDigest.isEqual ja e' implementado para nao dar "early exit"
        // no primeiro byte diferente, ao contrario de Arrays.equals/String.equals.
        return MessageDigest.isEqual(a, b);
    }
}

```

Exemplo 2 — comparação exata (não por prefixo) de `redirect_uri`

Mitiga a ameaça de open redirector via validação flexível (§5.3). O erro mais comum em implementações reais é usar `startsWith` ou `contains` "para suportar múltiplos subcaminhos" — o exemplo abaixo mostra a forma correta (comparação de URI normalizada e exata contra uma lista de valores registrados) e comenta explicitamente por que a alternativa "conveniente" é perigosa.


```

import java.net.URI;
import java.util.Set;

public final class RedirectUriValidator {

    private final Set<String> uriRegistradas; // valores EXATOS cadastrados pelo client

    public RedirectUriValidator(Set<String> uriRegistradas) {
        this.uriRegistradas = Set.copyOf(uriRegistradas);
    }

    /**
     * RFC 6819 §5.3 / RFC 9700: comparacao deve ser exata (string a string,
     * apos normalizacao minima), nunca por prefixo ou substring.
     *
     * Exemplo do que NAO fazer (comentado de proposito, para servir de
     * contraste didatico):
     *
     * // ERRADO: abre brecha para "https://client.com/callback.evill.com/"
     * // return redirectUri.startsWith(uriRegistrada);
     *
     * // ERRADO: abre brecha para "https://client.com/callback/../../evil"
     * // return redirectUri.contains(uriRegistrada);
     */
    public boolean ehValido(String redirectUriRecebida) {
        URI normalizada = normalizar(redirectUriRecebida);
        if (normalizada == null) {
            return false;
        }
        // Comparacao EXATA da forma normalizada contra o conjunto registrado.
        return uriRegistradas.contains(normalizada.toString());
    }

    private URI normalizar(String bruta) {
        try {
            // URI.normalize() resolve "." e ".." no path, mas NAO substitui
            // a necessidade de comparacao exata depois – normalizar apenas
            // evita falsos negativos triviais (ex: barra final redundante
            // deve ser tratada como parte do cadastro, nao "normalizada para
            // sumir", ou a validacao fica frouxa de novo).
            return URI.create(bruta).normalize();
        } catch (IllegalArgumentException e) {
            return null; // URI malformada: rejeita, nunca tenta "consertar".
        }
    }
}

```

Exemplo 3 — authorization code de uso único (detecção de reuso)

Mitiga a ameaça descrita em §5.5. A ideia central: remover o code do armazenamento **atomicamente** no momento da primeira troca bem-sucedida, e tratar uma segunda apresentação como evento de segurança (não apenas como "código inválido" — o AS deveria, numa implementação real, revogar retroativamente qualquer token já emitido a partir daquele code).

```
import java.util.concurrent.ConcurrentHashMap;
import java.util.Map;

public final class AuthorizationCodeStore {

    private record CodeInfo(String clientId, String username, long expiraEm) {}

    private final Map<String, CodeInfo> codigos = new ConcurrentHashMap<>();

    public void emitir(String code, String clientId, String username) {
        codigos.put(code, new CodeInfo(clientId, username,
            System.currentTimeMillis() + 60_000)); // 60s de vida, uso unico
    }

    /**
     * Troca o code por informacoes de sessao – remove atomicamente do mapa.
     * `Map.remove` retorna null se a chave ja nao existir, o que cobre tanto
     * "nunca existiu" quanto "ja foi trocado antes" com uma unica chamada
     * atomica (sem race condition entre "checar" e "remover").
     */
    public CodeInfo trocarPorTokenOuNull(String code) {
        CodeInfo info = codigos.remove(code);
        if (info == null) {
            // RFC 6819 §5.5: reuso ou code inexistente. Numa implementacao
            // real, aqui e' o gatilho para revogar em cascata qualquer token
            // ja emitido a partir deste code, pois a segunda apresentacao
            // e' sinal de possivel comprometimento.
            registrarEventoSuspeito(code);
            return null;
        }
        if (info.expiraEm() < System.currentTimeMillis()) {
            return null; // expirado: tambem rejeita, sem distinguir o motivo ao chamador
        }
        return info;
    }

    private void registrarEventoSuspeito(String code) {
        // Em producao: alerta de seguranca, nao apenas um log de debug.
        System.err.println("ALERTA: tentativa de reuso de authorization code detectada");
    }
}
```

Exemplo 4 — cabeçalho anti-clickjacking na tela de consentimento

Mitiga a ameaça descrita em §5.7, usando apenas `com.sun.net.httpserver.HttpServer` (embutido no JDK, sem dependência externa) para ilustrar onde o cabeçalho deveria ser adicionado.

```
import com.sun.net.httpserver.HttpExchange;
import com.sun.net.httpserver.HttpHandler;

public final class TelaConsentimentoHandler implements HttpHandler {

    @Override
    public void handle(HttpExchange exchange) throws java.io.IOException {
        // RFC 6819 §5.7: impede que esta tela seja embutida em <iframe> de
        // um site atacante para induzir "clickjacking" sobre o botao Autorizar.
        exchange.getResponseHeaders().add("X-Frame-Options", "DENY");
        exchange.getResponseHeaders().add(
            "Content-Security-Policy", "frame-ancestors 'none'");

        String html = "<html><body><h1>Autorizar acesso?</h1>"
            + "<form method=\"POST\"><button>Autorizar</button></form>"
            + "</body></html>";
        byte[] corpo = html.getBytes(java.nio.charset.StandardCharsets.UTF_8);
        exchange.getResponseHeaders().add("Content-Type", "text/html; charset=utf-8");
        exchange.sendResponseHeaders(200, corpo.length);
        try (var os = exchange.getResponseBody()) {
            os.write(corpo);
        }
    }
}
```

RFC 9700 — Best Current Practice for OAuth 2.0 Security (BCP 240)

1. Contexto e motivação

RFC 9700, publicada em janeiro de 2025, é uma **BCP** (Best Current Practice) — uma categoria de documento IETF distinta tanto de uma spec de protocolo (como a RFC 6749) quanto de um documento puramente informativo (como a RFC 6819). Uma BCP não define mensagens ou parâmetros novos; ela **consolida consenso operacional** sobre como usar um protocolo já existente com segurança, e o faz com a mesma força normativa de linguagem (`MUST` , `MUST NOT` , `SHOULD`) que uma spec de protocolo usaria. É esse detalhe que separa a RFC 9700 da RFC 6819: onde a 6819 apenas sugeria e catalogava, a 9700 manda.

A origem deste documento é longa e vale contar, porque explica por que ele carrega tanto peso de autoridade coletiva. Antes de virar RFC, o conteúdo circulou por quase uma década como um *Internet-Draft* de nome informal `draft-ietf-oauth-security-topics` , atualizado incrementalmente por praticamente vinte revisões entre 2016 e 2024. Cada revisão incorporava lições de ataques documentados em produção, discussões do grupo de trabalho OAuth da IETF, e a experiência coletiva de operadores de authorization servers em escala — não é o trabalho de um único autor sentado numa mesa, é o resultado de um processo de atrito contínuo entre teoria e exploração real por praticamente todo o tempo de vida do OAuth 2.0 em produção massiva.

O efeito prático de virar RFC 9700 é que dezenas de recomendações que a RFC 6819 fazia como sugestão informal ("SHOULD", ou nem isso — apenas prosa descritiva) aqui viram **obrigação normativa explícita**. Isto não é uma mudança cosmética de redação: é a diferença entre "considere fazer X" e "uma implementação não conforme com esta BCP faz X errado, ponto". Para um arquiteto avaliando ou auditando um authorization server de terceiros, ou decidindo requisitos para um AS próprio, a RFC 9700 é hoje a referência de linha de base — o que ela exige é, para efeitos práticos, o piso mínimo aceitável de segurança OAuth em 2025 em diante.

Vale destacar também o parentesco desta RFC com o esforço de consolidação **OAuth 2.1** — um draft que reescreve a RFC 6749 + RFC 6750 + PKCE num único documento, já incorporando exatamente os mandatos que a 9700 formaliza aqui. Não é coincidência: a 9700 é, na prática, **o capítulo de segurança do OAuth 2.1** — quem entender profundamente este documento já entende a maior parte do que muda entre "OAuth 2.0 como definido em 2012" e "OAuth como se pratica com seriedade em 2025".

Uma nota importante para quem lê este capítulo antes de ter processado tudo que ele sintetiza: a RFC 9700 pressupõe familiaridade com PKCE (RFC 7636), mTLS aplicado a tokens (RFC 8705), DPoP (RFC 9449) e Resource Indicators (RFC 8707) — mas este capítulo foi escrito para ser autocontido. Cada mandato abaixo é explicado com o suficiente de contexto para fazer sentido mesmo que você ainda não tenha lido os capítulos dedicados a essas outras RFCs; quando chegar a eles, este capítulo vai se tornar ainda mais nítido em retrospecto, porque terá o "porquê" já internalizado antes do "como".

2. Terminologia e papéis

Nenhum papel novo. A contribuição terminológica real da RFC 9700 é endurecer — não redefinir — a distinção entre client **confidential** e **public** que já vinha da RFC 6749. Vale relembrar essa distinção com precisão, porque a maioria dos mandatos deste capítulo nasce diretamente da fragilidade estrutural do lado "public" dessa fronteira: um client confidencial roda em um ambiente que ele controla e pode manter um segredo (um servidor backend, por exemplo); um client público — uma SPA rodando no navegador, um app mobile, um app desktop distribuído ao usuário final — não tem como manter segredo algum de forma confiável, porque o binário ou o código-fonte do client está, por definição, nas mãos de quem poderia querer extrair esse segredo.

A RFC 9700 também introduz, sem lhe dar um nome formal de "papel", uma nova categoria de decisão de design: **sender-constraining** — a ideia de que um token deveria estar amarrado criptograficamente a quem o solicitou, de forma que posse do token sozinho (o modelo *bearer* puro da RFC 6750) deixe de ser suficiente para usá-lo. Isso não é um papel, mas é um conceito novo que perpassa vários dos mandatos de tokens abaixo.

3. Conteúdo normativo — os mandatos, agrupados e explicados a fundo

Como BCP, o documento não introduz mensagens novas — ele restringe e endurece o que já existe. O que segue não é apenas a lista de mandatos, mas uma discussão de *por que* cada um deixou de ser opcional.

3.1 Grant types: dois banidos, um universalizado

Implicit Grant (`response_type=token`) — **MUST NOT ser usado.**

O Implicit Grant devolvia o access token diretamente no fragmento (`#`) da URL de redirecionamento, sem passar por uma troca server-to-server via token endpoint. A racional original (2012) era acomodar clients em JavaScript puro rodando no navegador, que na época não tinham como fazer uma segunda requisição autenticada de forma prática sem CORS maduro. O problema estrutural: um token no fragmento da URL fica exposto a vazamento via histórico do navegador, via extensões instaladas, via qualquer script de terceiro que rode na mesma página (o fragmento é acessível via `window.location.hash` a qualquer JavaScript da página, não só ao client legítimo), e não existe nenhuma prova de posse do client na entrega — qualquer parte que intercepte o redirecionamento tem o token completo, pronto para uso imediato. Não há mitigação parcial aqui: o design é substituído inteiramente pelo Authorization Code Grant com PKCE, que hoje funciona perfeitamente bem para SPAs graças a CORS e `fetch` maduros — a justificativa original do Implicit Grant simplesmente deixou de existir.

Resource Owner Password Credentials (ROPC) — **MUST NOT ser usado.**

ROPC pede ao usuário para digitar usuário e senha diretamente no client, que os repassa ao token endpoint. O problema estrutural (já apontado, com mitigação paliativa, na RFC 6819) é que isso treina o usuário a confiar suas credenciais primárias a qualquer client que peça — destruindo a premissa central de segurança do OAuth, que é justamente o usuário *nunca* revelar sua senha a um client de terceiro, apenas ao authorization server. Diferente do Implicit Grant, aqui não há um substituto direto de "mesma conveniência, design melhor" — a mensagem da RFC 9700 é que se um client precisa de ROPC, o design do sistema como um todo está errado, e a solução é repensar a arquitetura (client confiável de primeira parte deveria ter uma via de autenticação direta com o AS, não mascarada de OAuth).

PKCE — obrigatório para TODO client no Authorization Code Grant, independentemente do tipo.

Esta é talvez a mudança de maior alcance prático do documento. A RFC 7636 original escopava PKCE como proteção primariamente voltada a clients públicos (a interceptação de code, RFC 6819 §5.1, é mais fácil de explorar contra eles). A RFC 9700 remove essa distinção: mesmo um client confidencial — com `client_secret` — deve usar PKCE em toda requisição de autorização. O racional: PKCE fecha a interceptação de code de forma independente de qualquer outra camada de proteção, e como o custo de implementá-lo é baixo (gerar um par verifier/challenge, dois parâmetros a mais), não há razão para deixar de exigí-lo universalmente só porque um client *também* tem outra camada de defesa. Segurança em profundidade, não segurança condicional.

3.2 Redirect e CSRF

`redirect_uri` — **exact string match obrigatório.**

Como discutido no capítulo anterior (RFC 6819 §5.3), qualquer flexibilização (prefixo, wildcard, substring) historicamente se revelou explorável para redirecionar o authorization code (ou, em designs antigos, o access token) para um destino sob controle do atacante. A RFC 9700 encerra a discussão: comparação deve ser string-a-string exata contra o valor registrado, sem exceção. Isso vale mesmo quando o client legitimamente precisa de múltiplos callbacks — nesse caso, cada um deve ser cadastrado explicitamente como uma entrada distinta e exata, nunca inferido por padrão.

`state` continua recomendado; `code_verifier` do PKCE pode reforçar (mas não substitui) a proteção CSRF.

Um ponto sutil que a RFC 9700 esclarece e que vale destacar: como o `code_verifier` do PKCE é gerado pelo client antes da requisição de autorização e só é revelado na troca final por token, ele *poderia*, em teoria, servir como uma segunda camada de vínculo entre a requisição original e a resposta — mas **somente se o client também vincular esse verifier à sessão do navegador daquele usuário específico** (por exemplo, guardando-o num cookie de sessão). PKCE sozinho, sem esse vínculo explícito à sessão, não substitui o papel do `state`: ele prova que quem está trocando o code por token é quem gerou o par verifier/challenge original, mas não prova, por si só, que essa troca está acontecendo na mesma sessão de navegador que iniciou o fluxo — que é exatamente o que o ataque de CSRF de login (RFC 6819 §5.2) explora.

Ataques de "mix-up".

Quando um client interage com mais de um authorization server (cenário comum em arquiteturas multi-tenant ou multi-provedor), um atacante pode induzir o client a acreditar que uma resposta veio do AS legítimo A quando na verdade veio de um AS malicioso ou comprometido B, levando o client a enviar o code (ou credenciais de troca) para o destino errado. A RFC 9700 recomenda mitigar isso com identificação explícita de qual AS emitiu a resposta — mecanismo formalizado com mais detalhe pela RFC 9207 (AS Issuer Identification, fora do escopo deste roteiro, mas vale saber que existe e resolve exatamente este problema).

3.3 Tokens

Refresh token rotation — obrigatória, com detecção de reuso recomendada.

O mandato tem duas camadas, e é importante não confundi-las:

1. **Rotação em si**: cada uso de um refresh token para obter um novo access token deve invalidar o refresh token usado e emitir um novo em seu lugar. O laboratório `rfc7636/` deste projeto já implementa esta camada.
2. **Detecção de reuso** (recomendação mais forte, ainda não presente no lab): se um refresh token *já invalidado* (porque já foi rotacionado uma vez) for apresentado de novo, isso é sinal inequívoco de que alguém mais, além do client legítimo, está de posse de uma cópia antiga do token — ou o client atual perdeu uma corrida de concorrência (bug), ou um atacante roubou uma cópia anterior e está tentando usá-la. Como não há como distinguir os dois casos com certeza, a mitigação recomendada é tratar qualquer reuso como comprometimento confirmado e **revogar toda a família de tokens** descendente daquele refresh token original — todo access token e todo refresh token subsequente gerado a partir dele, não apenas o token reapresentado.

Essa segunda camada é o que a RFC 9700 chama, em espírito, de tornar o refresh token um mecanismo de **detecção ativa de roubo**, não apenas de renovação de sessão: o AS não está apenas confiando que o refresh token está seguro, está *verificando continuamente* que só uma cópia dele está em circulação a qualquer momento.

Tokens sender-constrained (DPoP ou mTLS) — recomendados onde viável.

A RFC 6750 já apontava, como fraqueza estrutural, que um access token *bearer* — puramente por posse — dá acesso a qualquer parte que o apresente, sem prova adicional de que é o client legítimo apresentando. A RFC 9700 eleva essa preocupação de "considerar no futuro" para recomendação ativa hoje, especialmente para clients públicos (que não têm `client_secret` para se autenticar de outra forma) e cenários de alto valor (dados financeiros, PII sensível). Os dois mecanismos que atendem essa recomendação — DPoP (RFC 9449, prova de posse via par de chaves gerado pelo client, sem precisar de infraestrutura de certificados) e mTLS (RFC 8705, vínculo do token ao certificado de cliente usado na conexão TLS) — têm capítulos dedicados neste livro; aqui basta reter que "sender-constrained" quer dizer, em qualquer um dos dois casos, que um token roubado em trânsito ou em repouso (sem o material criptográfico associado) é inútil para o atacante.

Restrição de audience/escopo.

Reforça diretamente a preocupação de "confused deputy" já catalogada na RFC 6819 (§5.6 do capítulo anterior): um token emitido com a intenção de ser usado no Resource Server A não deveria ser aceito pelo Resource Server B. A RFC 9700 recomenda que o AS restrinja explicitamente a audiência de cada token emitido — mecanismo que ganha forma concreta na RFC 8707 (Resource Indicators), tratada em capítulo próprio deste livro.

Vida curta e uso único para authorization codes; tempos de expiração curtos para access tokens.

Reforço do que já era prática recomendada desde a RFC 6749/6819 — a RFC 9700 apenas formaliza como mandato o que a experiência de mais de uma década já havia consolidado como consenso: quanto menor a janela de validade de um artefato que trafega por um canal exposto (o authorization code, no redirecionamento), menor o dano possível de uma interceptação bem-sucedida.

3.4 Autenticação do client

A RFC 9700 expressa preferência clara por métodos de autenticação de client **assimétricos** — `private_key_jwt` (RFC 7523, o client assina uma asserção JWT com uma chave privada que nunca trafega pela rede) ou mTLS (RFC 8705) — em vez de `client_secret` estático transmitido a cada requisição de token. O racional é direto: um segredo compartilhado estático que precisa ser enviado repetidamente pela rede é, estruturalmente, mais fácil de vazar (interceptação, log acidental, exposição em configuração) e mais fácil de atacar por força bruta (é um valor fixo que pode ser tentado offline indefinidamente se vazar um hash dele) do que uma assinatura criptográfica gerada sob demanda com uma chave que nunca precisa deixar o ambiente do client.

3.5 Apps nativos

A requisição de autorização de um app nativo (mobile ou desktop) **não deve** ser feita dentro de uma *webview* embutida no próprio app — deve abrir o navegador do sistema operacional (Chrome Custom Tabs no Android, `ASWebAuthenticationSession` / Safari View Controller no iOS, ou o navegador padrão em desktop). O racional: uma *webview* embutida está sob controle total do app que a hospeda — ele pode inspecionar o DOM, injetar JavaScript, capturar o que o usuário digita (incluindo a senha no formulário de login do AS), e não oferece nenhuma das garantias de isolamento de sessão/cookies que o navegador do sistema oferece (por exemplo, reaproveitar uma sessão já autenticada no navegador para permitir *single sign-on* silencioso). Este mandato é formalizado com mais detalhe pela RFC 8252 (OAuth 2.0 for Native Apps), fora do escopo direto deste roteiro, mas a RFC 9700 o reforça como parte do conjunto obrigatório de boas práticas.

4. Parâmetros e endpoints normativos

Nenhum endpoint novo é criado por esta RFC. O efeito prático mais próximo de "novidade protocolar" é que ela promove `code_challenge` e `code_challenge_method` — originalmente parâmetros da RFC 7636, opcionais e restritos a clients públicos — à condição de parâmetros que devem estar presentes em

toda requisição de autorização, independentemente do tipo de client. Um authorization server em conformidade com a RFC 9700 deveria, na prática, rejeitar ou alertar sobre qualquer requisição de autorização que omita PKCE, mesmo vinda de um client confidencial.

5. Segurança — este documento inteiro É a seção de segurança

Diferente dos outros capítulos deste livro, aqui não existe uma seção separada "5. Segurança" com conteúdo adicional além do que já foi discutido na seção 3: como BCP, a RFC 9700 apresenta ameaça e mitigação obrigatória juntas, ponto a ponto, ao longo de todo o documento — não há um "conteúdo de protocolo" e depois uma "reflexão de segurança" à parte. O documento inteiro é, estruturalmente, uma seção de segurança.

6. Relação com outras RFCs

- Constrói diretamente sobre a **RFC 6819** — pega dezenas de recomendações informais de lá e as torna obrigatórias, como discutido ao longo deste capítulo.
- Torna a **RFC 7636 (PKCE)** universal, removendo a restrição original a clients públicos.
- Deprecia formalmente 2 dos 4 grant types originais da **RFC 6749** (Implicit Grant e ROPC).
- Recomenda ativamente **RFC 8705 (mTLS)** e **RFC 9449 (DPoP)** para sender-constraining de tokens.
- Reforça a preocupação de audiance que a **RFC 8707 (Resource Indicators)** resolve com um mecanismo concreto.
- É, na prática, o capítulo de segurança do **OAuth 2.1** (o draft de consolidação que reescreve RFC 6749 + 6750 + PKCE).
- Relacionada, fora do escopo direto deste roteiro: RFC 9207 (AS Issuer Identification, mitigação de ataques de mix-up) e RFC 8252 (OAuth 2.0 for Native Apps, guidance detalhado de webview).

7. O que essa RFC representa no arco do tempo

Diferente dos capítulos que tratam de RFCs mais antigas, aqui a pergunta habitual "o que mudou desde a publicação" se inverte: esta RFC é recente (2025) e é justamente o marco que **consolida** mais de dez anos de mudança incremental (2012–2024) em um único documento normativo. Antes dela, "as boas práticas" viviam fragmentadas — parte num draft de segurança de vida longa que só especialistas acompanhavam revisão a revisão, parte em blog posts de equipes de segurança de grandes provedores, parte em catálogos como a RFC 6819, e boa parte apenas na experiência tácita de quem operava authorization servers em produção e via os ataques acontecerem.

O que já se percebe evoluindo **além** dela, e vale ter no radar: perfis de segurança setoriais como o **FAPI** (Financial-grade API, da OpenID Foundation) já tornam **obrigatório** — não apenas recomendado — o que a RFC 9700 apenas sugere: sender-constraining (DPoP ou mTLS) sempre presente, não condicional a "onde viável"; PAR (Pushed Authorization Requests) e JAR (JWT-Secured Authorization Request) sempre usados, movendo os parâmetros de autorização para fora da URL exposta ao navegador; RAR (Rich Authorization Requests) para autorização de granularidade fina,

além do modelo simples de escopo. Para um contexto FSI (financial services), vale internalizar que a RFC 9700 é o **piso mínimo** de 2025 em diante — não o teto de maturidade possível — e que especificações como FAPI 2.0 partem exatamente daqui para exigir mais.

8. Relação com os laboratórios deste projeto

A RFC 9700 não tem laboratório dedicado — como BCP, ela não introduz protocolo novo para implementar, apenas endurece o uso do que já existe. Suas exigências, porém, já aparecem parcialmente distribuídas pelos laboratórios construídos até aqui, e vale mapear explicitamente o que está feito e o que ainda é lacuna: PKCE (agora universal, não mais só para clients públicos) está implementado no lab `rfc7636/`; a rotação de refresh token também está lá, mas a **detecção de reuso com revogação em cascata** — a camada mais avançada do mandato de tokens — ainda não está implementada em nenhum lab deste projeto, e é o gap mais evidente a se fechar caso um laboratório específico venha a ser construído para esta RFC. DPoP (sender-constraining) tem seu próprio capítulo e, quando o lab correspondente existir, será `rfc9449/`. Enquanto isso não acontece, o exemplo de código abaixo ilustra em Java puro exatamente essa lacuna — rotação com detecção de reuso — como referência conceitual de como ela funcionaria.

Exemplo em Java (JDK 21, sem frameworks)

Exemplo 1 — PKCE: geração do par verifier/challenge e verificação (S256)

Ilustra o mecanismo que a RFC 9700 torna obrigatório para **todo** client, não apenas os públicos. O `code_verifier` é uma string aleatória de alta entropia; o `code_challenge` é o hash SHA-256 dele, codificado em Base64 URL-safe sem padding — exatamente como especificado pela RFC 7636, aqui usando somente `java.security.MessageDigest` do próprio JDK.

```

import java.security.MessageDigest;
import java.security.NoSuchAlgorithmException;
import java.security.SecureRandom;
import java.util.Base64;
import java.nio.charset.StandardCharsets;

public final class Pkce {

    private static final SecureRandom RNG = new SecureRandom();

    /** Gera um code_verifier de 256 bits de entropia (RFC 7636 §4.1). */
    public static String gerarCodeVerifier() {
        byte[] bytes = new byte[32];
        RNG.nextBytes(bytes);
        return Base64.getUrlEncoder().withoutPadding().encodeToString(bytes);
    }

    /** Deriva o code_challenge a partir do verifier, metodo S256 (obrigatorio;
     *  "plain" existe na RFC 7636 mas nao deveria ser oferecido na pratica). */
    public static String derivarCodeChallenge(String codeVerifier) {
        try {
            MessageDigest sha256 = MessageDigest.getInstance("SHA-256");
            byte[] hash = sha256.digest(codeVerifier.getBytes(StandardCharsets.US_ASCII));
            return Base64.getUrlEncoder().withoutPadding().encodeToString(hash);
        } catch (NoSuchAlgorithmException e) {
            // SHA-256 e' garantido presente em qualquer JDK conforme; isto
            // nunca deveria disparar, mas o checked exception exige o catch.
            throw new IllegalStateException("SHA-256 indisponivel no JDK", e);
        }
    }

    /**
     * Lado do Authorization Server: na troca final do code por token, recebe
     * o code_verifier em texto puro e confere se ele deriva o mesmo
     * code_challenge que foi apresentado na requisicao de autorizacao original.
     */
    public static boolean verificar(String codeVerifierRecebido, String codeChallengeOriginal) {
        String recalculado = derivarCodeChallenge(codeVerifierRecebido);
        // Comparacao em tempo constante: o challenge nao e' secreto (trafegou
        // em claro na URL de autorizacao), mas o habito de comparar valores
        // criptograficos assim evita repetir o erro onde importa.
        return MessageDigest.isEqual(
            recalculado.getBytes(StandardCharsets.US_ASCII),
            codeChallengeOriginal.getBytes(StandardCharsets.US_ASCII));
    }
}

```

Exemplo 2 — rotação de refresh token com detecção de reuso e revogação em cascata

Este exemplo implementa em memória exatamente a lacuna apontada na seção 8: não apenas rotacionar (invalidar o antigo, emitir um novo), mas **detectar reuso de um refresh token já invalidado** e, ao detectar, revogar toda a família de tokens descendente. A ideia de "família" é a mesma usada no laboratório `rfc7636/` (lá em Redis; aqui, em memória, para fins didáticos).

```

import java.security.SecureRandom;
import java.util.Base64;
import java.util.Set;
import java.util.concurrent.ConcurrentHashMap;
import java.util.concurrent.CopyOnWriteArraySet;

public final class RefreshTokenRotation {

    private static final SecureRandom RNG = new SecureRandom();

    /** Estado de um refresh token: a familia a que pertence e se ja foi usado. */
    private record Estado(String familyId, boolean usado) {}

    private final ConcurrentHashMap<String, Estado> tokens = new ConcurrentHashMap<>();
    // Todos os refresh tokens (validos ou ja usados) que pertenceram a cada familia,
    // para permitir revogacao em cascata quando reuso e' detectado.
    private final ConcurrentHashMap<String, Set<String>> membrosDaFamilia = new ConcurrentHashMap<>();
    private final Set<String> tokensRevogados = new CopyOnWriteArraySet<>();

    private String gerar() {
        byte[] b = new byte[24];
        RNG.nextBytes(b);
        return Base64.getUrlEncoder().withoutPadding().encodeToString(b);
    }

    /** Emissao inicial (ex: primeiro login) - cria uma nova familia. */
    public String emitirNovo() {
        String familyId = gerar();
        String refreshToken = gerar();
        tokens.put(refreshToken, new Estado(familyId, false));
        membrosDaFamilia.put(familyId, new CopyOnWriteArraySet<>(Set.of(refreshToken)));
        return refreshToken;
    }

    /**
     * Troca um refresh token por um novo (rotacao). Se o token apresentado ja
     * tiver sido usado antes, e' sinal de reuso - RFC 9700 §3 (tokens):
     * revoga toda a familia e sinaliza comprometimento ao chamador.
     *
     * @return o novo refresh token, ou Optional.empty() se reuso foi detectado
     *         (a familia inteira foi revogada como consequencia).
     */
    public java.util.Optional<String> rotacionar(String refreshTokenApresentado) {
        Estado estadoAtual = tokens.get(refreshTokenApresentado);

        if (estadoAtual == null || tokensRevogados.contains(refreshTokenApresentado)) {
            // Token desconhecido ou ja revogado: nada a rotacionar.
            return java.util.Optional.empty();
        }

        if (estadoAtual.usado()) {
            // REUSO DETECTADO: alguem apresentou um refresh token que ja

```

```

        // havia sido trocado por um novo anteriormente. Nao ha como
        // distinguir com certeza "bug de concorrencia no client legitimo"
        // de "atacante usando uma copia roubada" – a postura segura e'
        // tratar como comprometimento e revogar a familia inteira.
        revogarFamilia(estadoAtual.familyId());
        registrarAlertaSeguranca(refreshTokenApresentado, estadoAtual.familyId());
        return java.util.Optional.empty();
    }

    // Uso legitimo (primeira vez): marca o atual como usado e emite um novo
    // na mesma familia.
    tokens.put(refreshTokenApresentado, new Estado(estadoAtual.familyId(), true));
    String novoToken = gerar();
    tokens.put(novoToken, new Estado(estadoAtual.familyId(), false));
    membrosDaFamilia.get(estadoAtual.familyId()).add(novoToken);
    return java.util.Optional.of(novoToken);
}

private void revogarFamilia(String familyId) {
    Set<String> membros = membrosDaFamilia.getDefault(familyId, Set.of());
    tokensRevogados.addAll(membros);
    // Numa implementacao real: revogar tambem, em cascata, todo access
    // token emitido a partir de qualquer refresh token desta familia
    // (mesmo mecanismo do endpoint /revoke da RFC 7009 – ver proximo capitulo).
}

private void registrarAlertaSeguranca(String token, String familyId) {
    System.err.println("ALERTA: reuso de refresh_token detectado, familia "
        + familyId + " revogada por completo");
}

public boolean estaRevogado(String refreshToken) {
    return tokensRevogados.contains(refreshToken);
}
}

```

Exemplo 3 — exact-match de `redirect_uri` reforçado com verificação de esquema para apps nativos

Complementa o Exemplo 2 do capítulo anterior (RFC 6819) acrescentando a checagem específica que a RFC 9700 recomenda para apps nativos: além do exact-match, validar que o esquema da URI não é um esquema customizado genérico compartilhável por múltiplos apps no mesmo dispositivo, a menos que o AS também valide "app-links" verificados (fora do escopo deste exemplo simplificado).

```
import java.net.URI;
import java.util.Set;

public final class RedirectUriValidatorReforcado {

    private final Set<String> uriRegistradas;
    private final boolean clientEhAppNativo;

    public RedirectUriValidatorReforcado(Set<String> uriRegistradas, boolean clientEhAppNativo) {
        this.uriRegistradas = Set.copyOf(uriRegistradas);
        this.clientEhAppNativo = clientEhAppNativo;
    }

    public boolean ehValido(String redirectUriRecebida) {
        URI uri;
        try {
            uri = URI.create(redirectUriRecebida).normalize();
        } catch (IllegalArgumentException e) {
            return false;
        }

        // RFC 9700: exact match, sempre – sem excecao para nenhum tipo de client.
        if (!uriRegistradas.contains(uri.toString())) {
            return false;
        }

        if (clientEhAppNativo && uri.getScheme() != null) {
            String esquema = uri.getScheme().toLowerCase();
            // "https" com dominio verificado (App Links/Universal Links) e'
            // preferivel a um esquema customizado (ex: "meuapp://") que
            // multiplos apps no mesmo dispositivo poderiam reivindicar –
            // RFC 8252 (OAuth for Native Apps) e RFC 6819 §5.1 discutem essa
            // ambiguidade de sistema operacional com mais profundidade.
            if (!esquema.equals("https")) {
                System.err.println("AVISO: redirect_uri de app nativo usa esquema "
                    + "customizado '" + esquema + "' – prefira https com "
                    + "App Links/Universal Links verificados quando possivel");
            }
        }

        return true;
    }
}
```

RFC 7009 — OAuth 2.0 Token Revocation

1. Contexto e motivação

A RFC 6749 define como um client obtém um token. A RFC 6750 define como ele o usa para acessar um resource server. Nenhuma das duas, porém, define uma forma padronizada e interoperável de **desfazer** isso — de dizer explicitamente ao authorization server "este token específico não deve mais ser aceito, a partir de agora". Antes da RFC 7009, publicada também em 2013, cada authorization server que quisesse oferecer revogação precisava inventar seu próprio endpoint proprietário, sem nenhuma garantia de que um client genérico soubesse como chamá-lo — e a maioria simplesmente não oferecia nada, deixando o único mecanismo de "morte" de um token ser a expiração natural pelo TTL.

Essa lacuna importa mais do que parece à primeira vista, e vale detalhar os dois cenários motivadores centrais que justificam a existência desta RFC:

Cenário 1 — logout explícito do usuário. Quando um usuário clica em "sair" num client, o comportamento ingênuo (e, sem revogação, o único disponível) é o client simplesmente apagar o token do seu próprio armazenamento local — cookie, localStorage, keychain do dispositivo. Isso é *esquecimento do lado do client*, não *invalidação do lado do servidor*. O token, tecnicamente, continua completamente válido no authorization server e no resource server até expirar naturalmente. Se esse mesmo token vazar depois (backup do dispositivo, log acidental, dump de memória), ele continua utilizável por um atacante até o TTL expirar — mesmo que o usuário tenha, na sua percepção, "saído da conta" há muito tempo. Isso é particularmente grave para refresh tokens, cujo TTL é tipicamente medido em dias ou semanas, não minutos.

Cenário 2 — resposta a comprometimento suspeito. Se um refresh token vaza (por exemplo, através de um dispositivo comprometido, um log de erro mal configurado, ou um ataque de reuso detectado como discutido no capítulo da RFC 9700), a única resposta eficaz é invalidá-lo imediatamente — não esperar o TTL natural passar, que pode ser dias. Sem um endpoint padronizado de revogação, essa resposta rápida dependia inteiramente de mecanismos proprietários e não interoperáveis de cada AS.

A RFC 7009 fecha essa lacuna com uma proposta deliberadamente simples: um único endpoint novo, o **revocation endpoint**, que recebe uma requisição síncrona pedindo a invalidação de um token específico. A simplicidade do design (uma única requisição, uma resposta quase sempre idêntica) é uma escolha consciente — como veremos na seção de segurança, boa parte do valor desta RFC está no que ela decide **não revelar** na resposta, tanto quanto no que ela invalida.

2. Terminologia e papéis

Nenhum papel novo é introduzido. A contribuição conceitual da RFC 7009 é elevar **revogação** à categoria de ação de primeira classe do protocolo — antes dela, o único jeito de um token "morrer" no OAuth 2.0 puro (RFC 6749/6750) era por expiração natural do TTL; não havia verbo para "matar um

token agora, por decisão explícita". Vale notar o paralelo com sistemas de certificados X.509: assim como um certificado tem uma data de expiração natural mas também pode ser revogado antes disso via CRL/OCSP quando a chave é comprometida, um token OAuth tem TTL natural mas agora também tem um mecanismo de revogação antecipada equivalente.

3. Fluxo — uma única requisição, com nuances importantes na resposta

O fluxo é deliberadamente simples e síncrono: o client se autentica perante o authorization server e informa qual token deve ser revogado.

```
POST /revoke HTTP/1.1
Host: as.exemplo.com
Content-Type: application/x-www-form-urlencoded
Authorization: Basic czZCaGRSa3F0MzpnWDFmQmF0M2JW

token=45ghiukldjahdnhdauz&token_type_hint=refresh_token
```

A resposta em caso de sucesso é **200 OK com corpo vazio** — e aqui está o detalhe normativo mais importante de toda a especificação, que merece ser dito com todas as letras: essa resposta **200** é retornada **independentemente** de o token informado existir, já estar expirado, ou já ter sido revogado anteriormente. Do ponto de vista do client, "eu pedi para revogar um token que nunca existiu" e "eu pedi para revogar um token válido que acabou de morrer" são operações indistinguíveis na resposta — ambas retornam exatamente o mesmo **200 OK** vazio. A única classe de erro que a RFC define é para requisições **malformadas** (parâmetro `token` ausente, por exemplo) ou para **falha de autenticação do client** — nesses casos, e apenas nesses, o AS retorna um erro (tipicamente **400** para requisição malformada, ou o erro de autenticação apropriado do token endpoint).

Essa decisão de design — que será detalhada na seção 5 — não é acidental nem uma simplificação preguiçosa; é uma mitigação deliberada contra um vetor de ataque específico, e entender esse raciocínio é mais valioso do que decorar o formato da requisição.

4. Parâmetros e endpoints normativos

- **Revocation endpoint** — endpoint novo, cuja URI é definida pelo authorization server (convencionalmente `/revoke`, mas sem exigência de nome fixo na RFC). Anos depois, com a publicação da RFC 8414 (Authorization Server Metadata), esse endpoint se tornou descobrível de forma padronizada através do campo `revocation_endpoint` no documento de metadados — eliminando a necessidade de configuração manual do client com a URL exata.
- **token** (obrigatório) — o valor do token a ser revogado. Pode ser um access token ou um refresh token; a RFC não distingue sintaticamente entre os dois no corpo da requisição, o que exige que o AS seja capaz de identificar de qual tipo se trata (daí o parâmetro seguinte, que serve apenas de otimização, não de exigência estrutural).

- `token_type_hint` (opcional) — os valores definidos são `access_token` e `refresh_token`. É importante entender que este parâmetro é **apenas uma dica de otimização de busca**, não uma declaração vinculante: se o AS armazena access tokens e refresh tokens em índices ou tabelas separados, o hint evita ter que consultar ambos; mas se o hint informado não corresponder ao tipo real do token (por exemplo, o client erra e informa `access_token` quando na verdade é um `refresh_token`), o AS **deve** tentar o outro tipo antes de desistir e retornar sucesso vazio de qualquer forma. Um AS que rejeitasse a requisição só porque o hint "não bateu" estaria implementando a especificação incorretamente.
- **Autenticação do client** — segue exatamente as mesmas regras já estabelecidas para o token endpoint na RFC 6749: um client confidencial deve se autenticar (tipicamente via `client_secret` em Basic Auth, como no exemplo acima); um client público deve, no mínimo, apresentar seu `client_id`, e o AS deve validar que o token apresentado para revogação pertence de fato àquele client — um client não pode revogar o token de outro client apenas conhecendo o valor do token.
- **Semântica de cascata (§2.1 da RFC)** — este é o ponto mais sutil e mais relevante na prática, e vale destacar com precisão o que a RFC efetivamente diz: revogar um **access token** não implica, necessariamente, revogar o **refresh token** associado a ele — isso fica a critério de política do AS. Revogar um **refresh token**, porém, **deveria** (recomendação forte, não obrigação absoluta na letra da RFC, mas universalmente adotada na prática) também invalidar todo access token que tenha sido derivado dele. O racional é simples: se a revogação de um refresh token não derrubasse também os access tokens já emitidos a partir dele, a operação teria um "meio-efeito" enganoso — o cliente pensaria que revogou o acesso, mas um access token filho continuaria plenamente funcional até expirar sozinho, possivelmente minutos ou horas depois.

5. Segurança já tratada na própria RFC 7009 — análise aprofundada

5.1 `200 OK` sempre, mesmo para token inexistente ou já revogado — o motivo do "anti-oráculo"

Esta é a decisão de design mais importante da especificação, e vale explicar o ataque que ela evita com um exemplo concreto. Imagine que o AS retornasse, em vez do `200` uniforme, uma resposta diferenciada — digamos, `404 Not Found` para "este token não existe" versus `200 OK` para "revogado com sucesso". Um atacante que tivesse capturado parcialmente um token (por exemplo, via um ataque de canal lateral, um log truncado, ou uma tentativa de adivinhação/força bruta contra um espaço de tokens mal dimensionado) poderia usar o endpoint de revogação como **oráculo**: enviar candidatos a token e observar qual resposta volta, inferindo se aquele valor específico é um token válido existente no sistema, mesmo sem nunca ter conseguido *usar* o token de fato. Isso transformaria um endpoint pensado para logout/limpeza em uma ferramenta de reconhecimento para o atacante — exatamente o tipo de vazamento de informação lateral que a RFC 6750 já evita ao não detalhar `error_description` para clients que sequer tentaram se autenticar corretamente (o mesmo princípio de "não dar ao atacante um oráculo de validação gratuito" aparece repetidas vezes na família de RFCs de segurança OAuth).

A resposta uniforme fecha esse vetor por completo: não importa se o token nunca existiu, já expirou naturalmente, ou já havia sido revogado antes — a resposta é sempre idêntica, e um atacante não ganha nenhuma informação distinguível ao testar valores contra o endpoint.

5.2 Autenticação obrigatória do client

Sem essa exigência, o revocation endpoint seria um vetor óbvio de negação de serviço: qualquer parte, sem relação alguma com o token, poderia revogar o token de qualquer outra pessoa apenas conhecendo (ou adivinhando) o valor — bastaria enviar `token=<valor capturado ou adivinhado>` sem nenhuma credencial. A exigência de autenticação do client, e a validação de que o token pertence àquele client específico, fecha esse vetor: revogar o token de terceiros deixa de ser possível sem também comprometer as credenciais daquele client.

5.3 Cascata refresh → access token recomendada

Já discutido na seção 4 — vale reforçar aqui por que é uma questão de segurança, não apenas de conveniência de UX: sem a cascata, um usuário (ou um sistema automatizado de resposta a incidente) que revoga um refresh token comprometido teria uma falsa sensação de que o acesso foi cortado, quando na verdade um access token filho — de posse do mesmo atacante que comprometeu o refresh token, ou de qualquer parte que o tenha capturado independentemente — continuaria válido até seu próprio TTL natural expirar.

5.4 TLS obrigatório

Como em todo endpoint que manipula token ou credencial de autenticação do client, o revocation endpoint exige TLS — sem isso, tanto o `client_secret` (na autenticação) quanto o próprio valor do token a ser revogado trafegariam em texto claro, e um atacante capaz de interceptar essa requisição obteria exatamente o token que o dono estava tentando invalidar, antes mesmo que a revogação surtisse efeito.

6. Relação com outras RFCs

- Completa o ciclo de vida do token iniciado pela **RFC 6749** (emissão) e continuado pela **RFC 6750** (uso) — a RFC 7009 é, nesse ciclo, o "desligar" explícito.
- Tornou-se descobrível de forma padronizada via **RFC 8414** (Authorization Server Metadata), como o campo `revocation_endpoint` no documento de metadados do AS.
- É complementar — não sobreposta — à **RFC 7662 (Token Introspection)**, fora do escopo direto deste roteiro mas útil mencionar a diferença de papel: introspecção responde à pergunta "este token ainda é válido, e quais são suas claims/atributos?"; revogação executa a ação "invalidar este token agora". As duas coexistem tipicamente como endpoints irmãos no mesmo authorization server, servindo propósitos complementares (consulta versus ação).
- A recomendação de **rotação de refresh token com detecção de reuso**, discutida em profundidade no capítulo anterior (RFC 9700), depende estruturalmente da revogação em cascata desta RFC: detectar que um refresh token foi reusado é só metade da mitigação — a outra

metade é ter, por trás dessa detecção, um mecanismo de revogação rápido e eficaz capaz de matar toda a família de tokens comprometida antes que o dano se espalhe.

7. O que mudou desde a publicação (2013)

- A RFC em si permanece estável e amplamente implementada, sem revisões relevantes desde a publicação — ao contrário de outras áreas do OAuth, revogação não teve nenhuma controvérsia de design que exigisse retrabalho.
- O que mudou ao redor dela, não nela mesma: passou de "endpoint proprietário, oferecido só por quem se importava" para campo padrão e esperado no documento de discovery (RFC 8414) de qualquer authorization server moderno — hoje, a ausência de um `revocation_endpoint` publicado é vista como lacuna de maturidade, não como opção neutra de design.
- Com a chegada da rotação de refresh token com detecção de reuso (RFC 9700, 2025), a revogação deixou de ser vista apenas como um "recurso de conveniência para logout" e passou a ser **infraestrutura de segurança ativa**: a velocidade e a confiabilidade com que a revogação em cascata funciona é, literalmente, o que determina a eficácia da resposta a um sinal de comprometimento detectado.
- Tokens sender-constrained (DPoP ou mTLS, discutidos no capítulo da RFC 9700) reduzem — mas não eliminam — a urgência de revogação como *resposta a roubo puro*: se posse do token sozinho já não basta para usá-lo (porque falta a prova de posse da chave associada), um token roubado em trânsito perde grande parte do seu valor para o atacante mesmo sem revogação imediata. Ainda assim, a revogação continua essencial para dois cenários que independem de roubo: logout legítimo do usuário e retirada explícita de consentimento — nenhum dos dois é resolvido por sender-constraining, porque em ambos o token *era* legitimamente utilizável até aquele momento.

8. Relação com o laboratório deste projeto

Diferente da RFC 6819 e da RFC 9700, a RFC 7009 **tem** implementação prática direta neste projeto: o laboratório `rfc7636/` (que primariamente demonstra PKCE, mas incorpora também Token Revocation como parte do mesmo Authorization Server) expõe um endpoint `/revoke` funcional, com revogação em cascata baseada no conceito de **família de tokens**.

O mecanismo, em `rfc7636/authorization-server/src/index.js`, funciona assim: toda vez que um par access token + refresh token é emitido (seja no login inicial, seja numa rotação subsequente), ambos são associados a um `family_id` comum, gerado uma única vez por concessão (`genFamilyId()`, em `pkce.js`) e mantido como um `set` no Redis (`family:<id>` contendo as chaves `access_token:...` e `refresh_token:...` daquela família). Quando o `/revoke` recebe um `refresh_token`, o handler busca o `family_id` associado, lista todos os membros daquele set, revoga cada um deles, e então apaga o set inteiro — implementando exatamente a "cascata refresh → access" que a seção 4 desta RFC recomenda, mas generalizada para qualquer token que já tenha compartilhado a mesma linhagem de emissão, não apenas o par imediato.

Na interface do lab, o botão "Revogar refresh_token (cascata)" no dashboard do client demo dispara essa chamada; o efeito observável é que, logo em seguida, qualquer tentativa de "Chamar Resource Server" com o access token daquela mesma família retorna `401 invalid_token` — provando visualmente que o access token morreu por tabela, mesmo nunca tendo sido revogado diretamente. O log de eventos do lab (`./containers/logs.sh events`) também deixa visível, na prática, o detalhe da seção 5.1 deste capítulo: a resposta do `/revoke` é sempre `200` , independentemente de o token existir ou de quem seja o dono — o log mostra a requisição e a resposta uniforme lado a lado, sem nunca revelar mais informação do que a especificação permite.

Exemplo em Java (JDK 21, sem frameworks)

Os exemplos a seguir usam apenas `java.net.http.HttpClient` (cliente) e `com.sun.net.httpserver.HttpServer` (servidor), ambos embutidos no JDK — sem nenhuma dependência externa. Onde uma resposta precisar ser montada ou lida em JSON (nos erros de protocolo, por exemplo), os exemplos referenciam o utilitário `SimpleJson` , definido no Capítulo 1 (RFC 6749) deste livro, em vez de redefinir a lógica de serialização aqui.

Exemplo 1 — cliente fazendo a requisição de revogação

```
import java.net.URI;
import java.net.http.HttpClient;
import java.net.http.HttpRequest;
import java.net.http.HttpResponse;
import java.util.Base64;
import java.nio.charset.StandardCharsets;

public final class RevocationClient {

    private final HttpClient http = HttpClient.newHttpClient();
    private final URI revocationEndpoint;
    private final String clientId;
    private final String clientSecret;

    public RevocationClient(URI revocationEndpoint, String clientId, String clientSecret) {
        this.revocationEndpoint = revocationEndpoint;
        this.clientId = clientId;
        this.clientSecret = clientSecret;
    }

    /**
     * Revoga um token (access ou refresh) – RFC 7009 §2.1.
     * token_type_hint e' opcional; ajuda o AS a otimizar a busca, mas o AS
     * deve tentar o outro tipo se o hint nao bater (RFC 7009 §2.1).
     */
    public void revogar(String token, String tokenTypeHint) throws Exception {
        String credenciais = Base64.getEncoder().encodeToString(
            (clientId + ":" + clientSecret).getBytes(StandardCharsets.UTF_8));

        String corpo = "token=" + java.net.URLEncoder.encode(token, StandardCharsets.UTF_8)
            + (tokenTypeHint != null
                ? "&token_type_hint=" + java.net.URLEncoder.encode(tokenTypeHint, StandardCharsets.UTF_8)
                : "");

        HttpRequest requisicao = HttpRequest.newBuilder(revocationEndpoint)
            .header("Content-Type", "application/x-www-form-urlencoded")
            .header("Authorization", "Basic " + credenciais)
            .POST(HttpRequest.BodyPublishers.ofString(corpo))
            .build();

        HttpResponse<String> resposta = http.send(requisicao, HttpResponse.BodyHandlers.ofString());

        // RFC 7009: sucesso e' sempre 200, corpo vazio – independentemente de
        // o token existir, ja estar expirado, ou ja ter sido revogado antes.
        // So' ha erro para requisicao malformada ou falha de autenticao do
        // client (tipicamente 400, com corpo JSON de erro OAuth padrao).
        if (resposta.statusCode() == 200) {
            System.out.println("Revogacao processada (o AS nunca revela se o "
                + "token de fato existia ou pertencia a este client).");
        } else {
```

```

// Corpo de erro, quando existe, segue o formato OAuth padrao
// {"error": "...", "error_description": "..."} – usar o
// utilitario SimpleJson (Capitulo 1, RFC 6749) para interpretar.
var campos = SimpleJson.parseFlat(resposta.body());
System.err.println("Falha na revogacao: " + campos.getOrDefault("error", "desconhecido")
    }
}
}

```

Exemplo 2 — servidor: handler do endpoint `/revoke` com resposta uniforme (anti-oráculo)

Implementa, com `com.sun.net.httpserver.HttpServer`, o comportamento normativo central da RFC: autenticar o client, tentar revogar (respeitando cascata refresh → access), e retornar sempre `200` vazio em caso de requisição bem-formada — nunca vazando se o token existia.

```

import com.sun.net.httpserver.HttpExchange;
import com.sun.net.httpserver.HttpHandler;
import java.io.IOException;
import java.nio.charset.StandardCharsets;
import java.util.Map;
import java.util.Set;
import java.util.concurrent.ConcurrentHashMap;

public final class RevokeHandler implements HttpHandler {

    // token -> family_id (simulando o Redis do Lab rfc7636/, aqui em memoria)
    private final Map<String, String> tokenParaFamilia = new ConcurrentHashMap<>();
    private final Map<String, Set<String>> membrosDaFamilia = new ConcurrentHashMap<>();
    private final ClientAuthenticator autenticador;

    public RevokeHandler(ClientAuthenticator autenticador) {
        this.autenticador = autenticador;
    }

    @Override
    public void handle(HttpExchange exchange) throws IOException {
        String corpo = new String(exchange.getRequestBody().readAllBytes(), StandardCharsets.UTF_8);
        Map<String, String> parametros = parseFormUrlEncoded(corpo);

        // Autenticacao do client e' obrigatoria - RFC 7009 §2.1. Falha aqui
        // e' o UNICO caso (junto com requisicao malformada) que retorna erro
        // real, nao o 200 uniforme.
        String clientIdAutenticado = autenticador.autenticarOuNull(exchange);
        if (clientIdAutenticado == null) {
            enviarErro(exchange, 401, "invalid_client");
            return;
        }

        String token = parametros.get("token");
        if (token == null || token.isBlank()) {
            enviarErro(exchange, 400, "invalid_request");
            return;
        }

        // token_type_hint (parametros.get("token_type_hint")) e' apenas dica
        // de otimizacao - nesta implementacao simplificada, nao muda o
        // comportamento porque o lookup por family_id ja cobre os dois casos.

        revogarComCascata(token);

        // RFC 7009 §2.2: 200 OK, corpo vazio, SEMPRE que a requisicao for
        // bem-formada e o client estiver autenticado - mesmo que `token`
        // nunca tenha existido ou ja estivesse revogado. Isto e' deliberado:
        // evita transformar o endpoint num oraculo de validacao de tokens.
        exchange.sendResponseHeaders(200, -1);
        exchange.close();
    }
}

```



```

private void revogarComCascata(String token) {
    String familyId = tokenParaFamilia.remove(token);
    if (familyId == null) {
        return; // token desconhecido: nao ha nada a fazer, mas a resposta e' 200 do mesmo jeito
    }
    // Revogar um refresh_token derruba toda a familia (access tokens
    // derivados inclusos) – RFC 7009 §2.1, cascata recomendada.
    Set<String> membros = membrosDaFamilia.remove(familyId);
    if (membros != null) {
        membros.forEach(tokenParaFamilia::remove);
    }
}

private void enviarErro(HttpExchange exchange, int status, String erro) throws IOException {
    // Reaproveita o utilitario SimpleJson (definido no Capitulo 1, RFC 6749)
    // para montar o corpo JSON de erro no formato OAuth padrao.
    String json = SimpleJson.object(Map.of("error", erro));
    byte[] bytes = json.getBytes(StandardCharsets.UTF_8);
    exchange.getResponseHeaders().add("Content-Type", "application/json");
    exchange.sendResponseHeaders(status, bytes.length);
    try (var os = exchange.getResponseBody()) {
        os.write(bytes);
    }
}

private Map<String, String> parseFormUrlEncoded(String corpo) {
    Map<String, String> resultado = new java.util.HashMap<>();
    for (String par : corpo.split("&")) {
        if (par.isBlank()) continue;
        String[] kv = par.split("=", 2);
        String chave = java.net.URLDecoder.decode(kv[0], StandardCharsets.UTF_8);
        String valor = kv.length > 1 ? java.net.URLDecoder.decode(kv[1], StandardCharsets.UTF_8) : "";
        resultado.put(chave, valor);
    }
    return resultado;
}

/** Abstrai a autenticao do client (Basic Auth ou client_id de client publico). */
public interface ClientAuthenticator {
    String autenticarOuNull(HttpExchange exchange);
}
}

```

Exemplo 3 — nota sobre `token_type_hint` e o dever de tentar o outro tipo

Um trecho isolado, para deixar explícito o detalhe normativo mais frequentemente implementado de forma incorreta: o hint nunca deve ser tratado como filtro que rejeita a requisição se "não bater".

```

/**
 * Ilustra a regra da RFC 7009 §2.1: token_type_hint e' so' uma otimizacao de
 * busca. Se o hint informado nao encontrar o token no armazenamento
 * correspondente, o AS DEVE tentar o outro tipo antes de desistir – nunca
 * deve retornar erro so' porque o hint "nao bateu" (isso vazaria informacao
 * e violaria a semantica de "sempre 200" da secao 5.1 deste capitulo).
 */
public final class TokenLookupComHint {

    public interface TokenStore {
        boolean existeComoAccessToken(String token);
        boolean existeComoRefreshToken(String token);
        void revogarAccessToken(String token);
        void revogarRefreshToken(String token);
    }

    private final TokenStore store;

    public TokenLookupComHint(TokenStore store) {
        this.store = store;
    }

    public void revogarRespeitandoHint(String token, String hintOrNull) {
        String primeiraTentativa = "refresh_token".equals(hintOrNull) ? "refresh" : "access";

        if (primeiraTentativa.equals("refresh")) {
            if (store.existeComoRefreshToken(token)) {
                store.revogarRefreshToken(token);
                return;
            }
            // Hint dizia refresh_token, mas nao encontrado la – tenta access_token
            // antes de desistir, como a RFC exige.
            if (store.existeComoAccessToken(token)) {
                store.revogarAccessToken(token);
            }
            // Se nao encontrar em nenhum dos dois, nao ha erro a reportar –
            // a resposta ao cliente sera 200 de qualquer forma (secao 5.1).
        } else {
            if (store.existeComoAccessToken(token)) {
                store.revogarAccessToken(token);
                return;
            }
            if (store.existeComoRefreshToken(token)) {
                store.revogarRefreshToken(token);
            }
        }
    }
}

```

Parte 3 — Extensões de Grant Type

RFC 7636 — Proof Key for Code Exchange by OAuth Public Clients (PKCE)

1. Contexto e motivação

Para entender por que a RFC 7636 existe, é preciso primeiro entender por que o Authorization Code Grant da RFC 6749, sozinho, é insuficiente para uma classe inteira de clients. A RFC 6749 já distingue **clients confidenciais** (podem guardar um `client_secret` com segurança — um backend web, por exemplo, rodando num servidor que o usuário não controla) de **clients públicos** (não podem — um app mobile, uma SPA rodando inteiramente no browser, uma CLI instalada na máquina do usuário). No token endpoint, um client confidencial autentica a si mesmo com o secret; um client público não tem nada equivalente para apresentar. Esse é o buraco que a RFC 7636 preenche.

O cenário concreto de 2015 que motivou a RFC é bem específico e vale reconstituir em detalhe, porque a lógica do ataque explica exatamente por que a defesa tem o formato que tem. Apps mobile nativos, antes de app links/universal links serem universais, recebiam o redirect de volta do Authorization Server via um **esquema de URI customizado** — algo como `com.exemplo.meuapp://oauth/callback`. O sistema operacional (Android, em particular, era o caso mais discutido) permite que **múltiplos apps instalados registrem o mesmo esquema customizado**. Não há dono exclusivo de `myapp://callback` da forma como há dono de um domínio HTTPS verificado. Isso significa que um app malicioso, sabendo (ou adivinhando, ou lendo o `client_id` público de outro app) qual esquema um app legítimo usa, pode registrar esse mesmo esquema para si. Quando o sistema operacional recebe o redirect do browser/AS de volta para `myapp://oauth/callback?code=xyz`, ele pode entregar essa Intent/URI tanto para o app legítimo quanto para o malicioso — o comportamento exato depende do SO e de detalhes de instalação, mas o ponto estrutural é: **o `redirect_uri` baseado em esquema customizado não é um canal que só o destinatário pretendido consegue ler**.

Agora o problema de fundo: se o app malicioso intercepta esse `code`, o que ele precisa para trocá-lo por um token no `/token`? Segundo a RFC 6749 pura, só o `code` e o `client_id` — e o `client_id` de um client público **não é segredo, é público por definição** (o próprio nome do tipo de client já entrega isso). Não existe `client_secret` para autenticar a troca, porque client público não tem um. Resultado: o interceptador tem literalmente tudo que precisa para completar a troca sozinho, antes ou em vez do app legítimo. A interceptação do `code` é, sozinha, suficiente para roubo de sessão completo.

É importante notar o que esse ataque **não é**: não é uma falha de TLS (o redirect pode estar todo em HTTPS até o momento em que o SO decide para qual app entregar a URI local), não é uma falha do Authorization Server, e não é uma falha de o resource owner ter feito algo errado. É uma falha estrutural do **canal de entrega do `code` no dispositivo do usuário**, canal que a RFC 6749 nunca modelou como potencialmente compartilhado entre múltiplas partes não confiáveis.

A ideia da RFC 7636 é elegante justamente por não tentar consertar o canal (não dá — o SO é quem arbitra esquemas de URI, e a spec não controla isso). Em vez disso, ela garante que **possuir o `code` sozinho não seja mais suficiente**. A defesa: o client legítimo gera, no início do fluxo — antes de

qualquer coisa tocar o browser ou um redirect — um segredo de alta entropia que **nunca trafega em claro** pelo canal vulnerável. Só uma versão derivada e unidirecional dele (o `code_challenge`) vai para a requisição de autorização. Quando chega a hora de trocar o `code` por um token, o client tem que provar posse do segredo original — e o interceptor, que só viu o `code` (não a requisição de autorização original, que não passou pelo mesmo esquema de URI vulnerável, mas sim por HTTPS direto do app para o AS), simplesmente não tem esse segredo.

Vale insistir num ponto sutil que costuma confundir: o `code_challenge` **também** viaja por um canal em tese observável (query string do `/authorize`, que pode acabar em logs de proxy, histórico de browser, `Referer`). Mas isso não importa para o ataque específico que a PKCE resolve — porque quem intercepta o **redirect de volta** (`redirect_uri?code=...`) não necessariamente observou a **ida** (`/authorize?...&code_challenge=...`), e mesmo que observasse, o `code_challenge` com S256 não permite reconstruir o `code_verifier` (hash unidirecional). O modelo de ameaça da PKCE é especificamente "quem só vê o code consegue completar a troca?" — e a resposta vira não.

2. Terminologia e papéis

Não há papel novo em relação à RFC 6749 — client, Authorization Server, resource owner continuam os mesmos. O vocabulário novo é todo criptográfico e merece precisão:

- **code_verifier**: uma string aleatória de alta entropia, gerada pelo client no início de cada fluxo de autorização (um verifier novo por tentativa de login, nunca reaproveitado entre fluxos). A RFC especifica o charset permitido — `[A-Z] [a-z] [0-9] "- " "_ " ~` (o conjunto "unreserved" da RFC 3986, que dispensa qualquer percent-encoding em URLs) — e o comprimento: **mínimo 43, máximo 128 caracteres**. Esses números não são arbitrários: 43 caracteres em base64url corresponde a 256 bits de entropia bruta (32 bytes aleatórios), o mesmo patamar de segurança de uma chave SHA-256 — abaixo disso, a RFC considera a entropia insuficiente para resistir a força bruta num prazo razoável; 128 é um teto prático para não estourar limites de tamanho de URL em implementações mais restritivas.
- **code_challenge**: o que de fato viaja na requisição de autorização — uma transformação do `code_verifier` que o AS consegue verificar depois, mas que não permite (no caso de S256) reconstituir o verifier original a partir dela.
- **code_challenge_method**: qual transformação foi usada. Dois valores definidos:
- **plain** — `code_challenge = code_verifier`, sem transformação nenhuma. Existe por compatibilidade com dispositivos incapazes de computar SHA-256 (a própria RFC cita como motivação hipotética; na prática, praticamente nenhum dispositivo moderno tem essa limitação).
- **S256** — `code_challenge = BASE64URL-ENCODE(SHA256(ASCII(code_verifier)))`, sem padding (=) e usando o alfabeto URL-safe (`- / _` em vez de `+ / /`). É o método recomendado pela RFC desde o dia 1, e hoje é, na prática, o único aceitável — ver §5.

Um detalhe de nomenclatura que vale reter: "PKCE" é pronunciado popularmente "pixy" na comunidade, e o nome completo — **P**roof **K**ey for **C**ode **E**xchange — já entrega a arquitetura: uma prova (`code_verifier`) de posse de uma chave (o segredo gerado), apresentada no momento da troca do code (`code_exchange`, o passo 6 do fluxo).

3. Fluxo

1. Client gera `code_verifier` (aleatório, alta entropia, 43-128 chars)
2. Client calcula `code_challenge` = `BASE64URL(SHA256(code_verifier))` [método S256]
3. Client → AS: GET `/authorize?...&code_challenge=xxx&code_challenge_method=S256`
4. AS associa `code_challenge` (+ método) ao authorization code que vai emitir
5. AS → Client: redirect com `code=...`
6. Client → AS: POST `/token grant_type=authorization_code&code=...&code_verifier=<verifier original>`
7. AS recalcula `BASE64URL(SHA256(code_verifier))` e compara com o `code_challenge` armazenado
8. Bate → emite tokens. Não bate → `invalid_grant`.

Vale detalhar o que acontece dentro do passo 4, porque é onde o AS efetivamente cria o vínculo criptográfico que sustenta toda a garantia: o `code_challenge` (e o método) precisam ficar **atrelados ao code emitido**, tipicamente como um campo a mais no registro do code (junto com `client_id`, `redirect_uri`, `scope`, expiração) guardado server-side — não é algo que o AS possa "confiar de volta" no próprio `code` sem persistir, porque o `code` em si costuma ser só um identificador opaco. Se o AS emitisse o `code` sem persistir o challenge associado, o passo 7 não teria contra o que comparar, e toda a defesa desmoronaria silenciosamente — um erro de implementação real que vale checar em qualquer AS caseiro.

Também vale detalhar o caminho de falha do passo 8 com mais precisão do que o resumo sugere. Existem três formas de o passo 7 falhar, e a RFC trata as três da mesma forma (rejeição com `invalid_grant`), mas elas têm origens diferentes: - **code_verifier ausente** na troca, quando a requisição original trouxe `code_challenge` — o AS deve rejeitar; aceitar cairia num "downgrade" silencioso (ver §5). - **code_verifier presente mas incorreto** — o hash recalculado não bate com o challenge armazenado. É exatamente o caso do ataque de interceptação: quem só tem o `code` não tem o verifier certo, então qualquer tentativa (inclusive um verifier aleatório improvisado) falha aqui. - **code_challenge ausente na requisição original, mas code_verifier presente na troca** (ou vice-versa em alguns perfis mais estritos) — tipicamente também rejeitado; a spec deixa alguma liberdade de política ao AS aqui, mas a prática recomendada (e exigida por OAuth 2.1) é que o AS **sempre** exija `code_challenge` na requisição de autorização, tornando esse caso irrelevante por construção.

Um ponto que costuma gerar confusão: PKCE **não substitui** o passo do usuário se autenticar e consentir — ele roda em paralelo a isso, é ortogonal ao login. O `code_challenge` vai junto com os mesmos parâmetros de sempre (`response_type=code`, `client_id`, `redirect_uri`, `scope`, `state`); a tela de login/consentimento do AS não muda em nada por causa da PKCE. A diferença toda está nos dois pontos: o que acompanha o `/authorize` (o challenge) e o que passa a ser exigido no `/token` (o verifier).

4. Parâmetros e endpoints normativos

Nenhum endpoint novo — PKCE estende os dois endpoints já existentes na RFC 6749:

Parâmetro	Onde	Obrigatoriedade	Descrição
<code>code_challenge</code>	Authorization Request (/authorize)	Obrigatório quando o client usa PKCE	O challenge derivado do verifier
<code>code_challenge_method</code>	Authorization Request (/authorize)	Opcional (mas deveria ser tratado como obrigatório em prática atual)	<code>plain</code> ou <code>S256</code>
<code>code_verifier</code>	Token Request (/token)	Obrigatório se a authorization request usou <code>code_challenge</code>	O verifier original, em claro

Um detalhe normativo com peso prático real: a RFC original (2015), se `code_challenge_method` vem omitido, instrui o AS a **assumir** `plain` como padrão. Isso é hoje amplamente considerado um detalhe infeliz da spec original — um AS que segue essa regra ao pé da letra, sem política adicional, fica vulnerável a um client (ou atacante forçando o client) que simplesmente omita o parâmetro, caindo silenciosamente para o método mais fraco. A prática recomendada atual — e o comportamento que qualquer AS bem-comportado deveria ter — é **rejeitar** a requisição se `code_challenge_method` vier ausente ou vier como algo diferente de `S256`, em vez de aceitar o padrão implícito da spec. Isso é coerente com a RFC 9700, que trata esse padrão de "aceitar silenciosamente o caminho mais fraco quando um parâmetro está ausente" como uma classe geral de risco a evitar em qualquer extensão do protocolo.

Quanto a erros: não há um código de erro novo — a RFC reaproveita o vocabulário de erro já existente do token endpoint (RFC 6749 §5.2). A falha de verificação do PKCE se manifesta como `invalid_grant`, o mesmo erro usado para qualquer outra rejeição de authorization code (code expirado, code já usado, `redirect_uri` que não bate). Do ponto de vista do client, isso significa que **não dá para diferenciar, pela resposta HTTP, se a falha foi "PKCE errado" ou "code expirado"** — o que é intencional: dar esse detalhe ao chamador ajudaria um atacante a refinar tentativas (ver também a discussão equivalente sobre respostas uniformes na RFC 7009, item de revogação).

5. Segurança já tratada na própria RFC 7636 (§5)

- **`plain` vs `S256` — a diferença central de garantia:** com `plain`, `code_challenge` é `code_verifier` — não há transformação nenhuma. Isso significa que qualquer observador da requisição de autorização original (histórico de browser local, um proxy corporativo fazendo TLS termination, logs de servidor que gravam a query string completa) vê o verifier em claro, e para *aquela* tentativa específica, a proteção nunca existiu — o observador tem exatamente o que precisaria para completar a troca. Com `S256`, ver o `code_challenge` não permite computar o `code_verifier` de volta (propriedade de hash unidirecional do SHA-256) — só quem gerou o verifier original o possui. É por isso que a RFC já recomendava `S256` como preferencial desde 2015, e por que a prática atual (reforçada pela RFC 9700 e por OAuth 2.1) é evitar `plain` categoricamente, a menos que o dispositivo seja genuinamente incapaz de computar SHA-256 (praticamente inexistente hoje).

- **Ataque de downgrade:** se um atacante consegue, de alguma forma, fazer o client (ou uma versão adulterada dele) não enviar `code_challenge`, ou enviar com `plain` em vez de `S256`, a janela de vulnerabilidade original reabre. A defesa correta não depende do client "se lembrar" de fazer certo — depende do **AS aplicar a exigência por política**: um AS bem configurado hoje rejeita qualquer authorization request sem `code_challenge` (ou com método diferente de `S256`), independentemente do que o client mandou historicamente. Isso é coerente com o princípio geral (RFC 9700) de que controles de segurança não deveriam ser opt-in do lado do client quando o AS tem poder de os tornar obrigatórios.
- **PKCE não é (sozinha) proteção CSRF:** um erro conceitual comum é achar que, com PKCE, o parâmetro `state` da RFC 6749 se torna dispensável. Não é o caso. PKCE resolve um problema diferente — impedir que **quem intercepta o code** complete a troca. CSRF no fluxo de autorização é outro ataque: induzir a vítima a completar um fluxo de autorização **iniciado pelo atacante**, atrelando a sessão da vítima a uma conta/recurso do atacante. A RFC 9700 discute que o `code_verifier`, por ser mantido pelo client entre o passo 1 e o passo 6 (tipicamente atrelado à sessão do browser via cookie ou armazenamento local), pode incidentalmente **também** ajudar a mitigar CSRF, mas só se o client de fato vincular esse verifier à sessão do usuário que iniciou o fluxo — algo que a spec da PKCE não impõe estruturalmente. Um client que gerasse o verifier de forma desvinculada da sessão do usuário (o que a RFC não proíbe explicitamente) perderia esse benefício incidental. `state` continua sendo o mecanismo desenhado especificamente para CSRF, e não deveria ser removido "porque agora tem PKCE".
- **Reuso de `code_verifier` entre tentativas:** embora a RFC não seja explícita sobre proibir reuso do mesmo verifier em múltiplos fluxos de login, a prática recomendada é gerar um verifier novo a cada tentativa — reuso reduziria a superfície de garantia sem ganho nenhum de usabilidade (gerar um verifier novo é computacionalmente trivial), e um verifier de vida longa é um alvo mais valioso caso algum canal auxiliar (armazenamento local do client, por exemplo) seja comprometido.
- **PKCE não protege contra um client comprometido:** se o próprio app legítimo está comprometido (malware no dispositivo, por exemplo), PKCE não ajuda — o atacante tem acesso ao processo que gera e guarda o verifier. PKCE resolve especificamente a interceptação do `code` num canal compartilhado *fora* do processo do client legítimo, não comprometimento do client em si.

6. Relação com outras RFCs

- É a mitigação direta, nomeada e específica da ameaça de **interceptação de authorization code** catalogada na RFC 6819 (Threat Model) — fecha explicitamente um ciclo aberto por aquele documento: a RFC 6819 identifica o risco, a RFC 7636 é a resposta formal a ele, três anos depois.
- Escopada originalmente (o próprio título da RFC diz: "by OAuth Public Clients") para clients **públicos** — a **RFC 9700** amplia a recomendação para **todo client, inclusive confidencial**. A lógica: mesmo um client com `client_secret` se beneficia de PKCE, porque o verifier é um segredo **adicional e independente** do secret — se o secret vazar (configuração errada, log acidental, repositório de código com credencial hardcoded), PKCE ainda oferece uma segunda camada de defesa contra a troca do code por outra parte.

- No draft **OAuth 2.1**, PKCE deixa de ser uma extensão opcional documentada à parte e passa a fazer parte da **definição própria** do Authorization Code Grant — ou seja, deixa de ser "RFC 6749 + RFC 7636 opcionalmente" e passa a ser só "o Authorization Code Grant, que já inclui PKCE por definição".
- Interage diretamente com as próximas RFCs deste capítulo: **RFC 9126 (PAR)** e **RFC 9101 (JAR)** mudam **como** o `code_challenge / code_challenge_method` são transmitidos ao AS (via um push autenticado por back-channel, ou dentro de um objeto de requisição assinado, em vez de query string crua) — mas não alteram em nada o mecanismo de verificação em si no `/token`. Isso é um padrão recorrente que vale destacar: PAR e JAR resolvem "como os parâmetros chegam ao AS com integridade/confidencialidade"; PKCE resolve "como provar posse de um segredo na troca do code" — são preocupações ortogonais que se compõem sem atrito.
- Combinada com **RFC 8252 (OAuth 2.0 for Native Apps, BCP informativo)** — que recomenda PKCE como parte do conjunto de práticas para apps nativos, junto com uso de `AppAuth`-style browsers externos em vez de WebViews embutidos.

7. O que mudou desde a publicação (2015)

- De "recomendado para clients públicos" para "obrigatório para todo client" — uma década de consolidação via RFC 9700 (2025).
- `plain` como método, embora ainda tecnicamente definido na spec, é hoje considerado essencialmente obsoleto na prática — implementações bem-comportadas de AS **exigem** S256 e rejeitam qualquer coisa diferente, em vez de honrar o `plain` como opção legítima.
- PKCE deixou de ser só uma mitigação para apps nativos mobile (seu caso de uso original) e passou a ser parte central da arquitetura recomendada para **SPAs** também — coincidindo com a recomendação (também consolidada na RFC 9700 e em guias práticos como o próprio OAuth 2.1) de abandonar o Implicit Grant em favor de Authorization Code + PKCE mesmo em contextos onde não há backend confidencial nenhum.
- A ideia de "code_challenge/code_verifier" como padrão de "compromisso criptográfico de mão única, verificado depois" também influenciou, conceitualmente, o desenho de outras extensões de sender-constraining discutidas em capítulos anteriores deste livro (DPoP, mTLS) — todas compartilham a lógica de "provar posse de algo sem revelar o segredo no canal vulnerável".

Observação sobre o lab: o `rfc6749/authorization-server` e o `client-demo` implementam o Authorization Code Grant conforme o piso da RFC 6749, **sem PKCE** — não há `code_challenge / code_verifier` em nenhum passo do fluxo. Esse gap foi fechado no lab `rfc7636/`: uma réplica autocontida do lab da RFC 6749 (mesmos containers, convenções, formatos de token), com PKCE (S256 obrigatório — o AS rejeita qualquer coisa diferente) na Authorization Code Grant. O `authorization-server` calcula `BASE64URL(SHA256(code_verifier))` (arquivo `pkce.js`) e compara contra o `code_challenge` recebido na authorization request. O lab tem um fluxo dedicado — botão "Interceptar code sem o code_verifier" — que simula exatamente o ataque descrito no item 1 deste capítulo: completa um login real, mas na troca final usa um

`code_verifier` propositalmente errado, e a tela de resultado mostra a rejeição `invalid_grant` do AS na prática. O lab também acrescenta Token Revocation (RFC 7009) com cascata por família de tokens, tema de um capítulo anterior deste livro.

Exemplo em Java (JDK 21, sem frameworks)

Os exemplos a seguir usam somente a biblioteca padrão do JDK 21: `java.net.http.HttpClient` para chamadas HTTP, `java.security.SecureRandom / MessageDigest` para a criptografia da PKCE, e `java.util.Base64` para as codificações base64url exigidas pela RFC. O utilitário `SimpleJson` (métodos `SimpleJson.object(Map)` e `SimpleJson.parseFlat(String)`), definido no Capítulo 1 (RFC 6749) deste livro, é reaproveitado para montar e ler corpos JSON simples.

Exemplo 1 — gerar `code_verifier` e `code_challenge` (S256)

```
import java.security.MessageDigest;
import java.security.NoSuchAlgorithmException;
import java.security.SecureRandom;
import java.util.Base64;
import java.nio.charset.StandardCharsets;

public final class Pkce {

    private static final SecureRandom RNG = new SecureRandom();

    // RFC 7636 §4.1: o code_verifier deve ter entre 43 e 128 caracteres,
    // usando apenas o charset "unreserved" (A-Z a-z 0-9 - . _ ~). Gerar
    // 32 bytes aleatórios e codificar em base64url (sem padding) produz
    // uma string de 43 caracteres desse charset – no piso mínimo exigido,
    // mas já carregando 256 bits de entropia bruta, o mesmo patamar de um
    // hash SHA-256. Não há ganho real em ir além disso.
    public static String generateCodeVerifier() {
        byte[] entropy = new byte[32];
        RNG.nextBytes(entropy);
        return Base64.getUrlEncoder().withoutPadding().encodeToString(entropy);
    }

    // RFC 7636 §4.2, método S256: code_challenge = BASE64URL(SHA256(verifier)).
    // O hash é computado sobre os bytes ASCII do verifier (nunca sobre uma
    // representação já codificada) – divergir disso produz um challenge que
    // o AS jamais vai reconhecer como correspondente ao verifier certo.
    public static String computeCodeChallengeS256(String codeVerifier) {
        try {
            MessageDigest sha256 = MessageDigest.getInstance("SHA-256");
            byte[] hash = sha256.digest(codeVerifier.getBytes(StandardCharsets.US_ASCII));
            return Base64.getUrlEncoder().withoutPadding().encodeToString(hash);
        } catch (NoSuchAlgorithmException e) {
            // SHA-256 é garantido presente em qualquer JVM conforme a spec
            // da plataforma – chegar aqui indicaria um runtime quebrado.
            throw new IllegalStateException("SHA-256 indisponível na JVM", e);
        }
    }

    public static void main(String[] args) {
        String verifier = generateCodeVerifier();
        String challenge = computeCodeChallengeS256(verifier);
        System.out.println("code_verifier = " + verifier + " (" + verifier.length() + " chars)");
        System.out.println("code_challenge = " + challenge);
        // O code_verifier fica só na memória do client até o passo do /token.
        // Só o code_challenge (acima) vai para a authorization request.
    }
}
```

Exemplo 2 — montar a authorization request com PKCE

```
import java.net.URI;
import java.net.URLEncoder;
import java.nio.charset.StandardCharsets;
import java.util.LinkedHashMap;
import java.util.Map;

public final class AuthorizationRequestBuilder {

    // Monta a URL de /authorize incluindo code_challenge/code_challenge_method,
    // além dos parametros já conhecidos da RFC 6749 (response_type, client_id,
    // redirect_uri, scope, state). Cada valor passa por URLEncoder – igual a
    // qualquer outro parâmetro de query string, o code_challenge não recebe
    // tratamento especial de encoding (já é base64url, um charset seguro
    // para URL, mas o encoding de query string genérico não faz mal).
    public static URI build(String authorizationEndpoint, String clientId,
        String redirectUri, String scope, String state,
        String codeChallenge) {
        Map<String, String> params = new LinkedHashMap<>();
        params.put("response_type", "code");
        params.put("client_id", clientId);
        params.put("redirect_uri", redirectUri);
        params.put("scope", scope);
        params.put("state", state);
        params.put("code_challenge", codeChallenge);
        params.put("code_challenge_method", "S256"); // nunca "plain" na prática atual

        StringBuilder query = new StringBuilder();
        for (var entry : params.entrySet()) {
            if (!query.isEmpty()) query.append('&');
            query.append(URLEncoder.encode(entry.getKey(), StandardCharsets.UTF_8))
                .append('=')
                .append(URLEncoder.encode(entry.getValue(), StandardCharsets.UTF_8));
        }
        return URI.create(authorizationEndpoint + "?" + query);
    }

    public static void main(String[] args) {
        String verifier = Pkce.generateCodeVerifier();
        String challenge = Pkce.computeCodeChallengeS256(verifier);

        URI authorizeUrl = build(
            "https://as.exemplo.com/authorize",
            "a1b2c3d4e5f6a7b8", // client_id (público, sem segredo)
            "myapp://oauth/callback",
            "profile",
            "xyzABC123", // state, contra CSRF – independente da PKCE
            challenge);

        System.out.println("Abrir no user-agent: " + authorizeUrl);
        // O code_verifier NÃO participa desta URL – fica guardado localmente
    }
}
```

```
} // (memória do processo, ou storage seguro do app) até o passo 6.  
}
```

Exemplo 3 — trocar o `code` por token, enviando o `code_verifier`

```
import java.net.URI;
import java.net.http.HttpClient;
import java.net.http.HttpRequest;
import java.net.http.HttpResponse;
import java.net.URLEncoder;
import java.nio.charset.StandardCharsets;
import java.util.Map;

public final class TokenExchange {

    private static final HttpClient HTTP = HttpClient.newHttpClient();

    public static Map<String, String> trocarCodePorToken(String tokenEndpoint,
                                                         String clientId,
                                                         String code,
                                                         String redirectUri,
                                                         String codeVerifier) throws Exception {

        String body = "grant_type=authorization_code"
            + "&code=" + URLEncoder.encode(code, StandardCharsets.UTF_8)
            + "&redirect_uri=" + URLEncoder.encode(redirectUri, StandardCharsets.UTF_8)
            + "&client_id=" + URLEncoder.encode(clientId, StandardCharsets.UTF_8)
            // client público: sem client_secret. A "autenticação" desta
            // troca é justamente a posse do code_verifier correto.
            + "&code_verifier=" + URLEncoder.encode(codeVerifier, StandardCharsets.UTF_8);

        HttpRequest request = HttpRequest.newBuilder()
            .uri(URI.create(tokenEndpoint))
            .header("Content-Type", "application/x-www-form-urlencoded")
            .POST(HttpRequest.BodyPublishers.ofString(body))
            .build();

        HttpResponse<String> response = HTTP.send(request, HttpResponse.BodyHandlers.ofString());

        // Resposta de sucesso e de erro do /token têm o mesmo formato JSON
        // "achato" (sem aninhamento) – SimpleJson.parseFlat cobre os dois.
        Map<String, String> json = SimpleJson.parseFlat(response.body());

        if (response.statusCode() != 200) {
            // Erro típico quando o code_verifier não bate com o code_challenge
            // registrado: HTTP 400, {"error":"invalid_grant", ...}
            throw new RuntimeException("Token endpoint recusou a troca: "
                + json.getOrDefault("error", "erro desconhecido")
                + " - " + json.getOrDefault("error_description", ""));
        }
        return json; // access_token, token_type, expires_in, refresh_token...
    }
}
```

Exemplo 4 — reproduzindo o ataque de interceptação (verifier incorreto)

```
public final class InterceptacaoDemo {

    // Simula o que um atacante que só capturou o `code` do redirect
    // conseguiria fazer: ele NÃO tem o code_verifier original (que nunca
    // trafegou por esse canal), então só pode tentar um verifier qualquer
    // – aleatório, ou reaproveitado de outra tentativa. Em qualquer caso,
    // o hash recalculado pelo AS não vai bater com o code_challenge
    // registrado, e a troca falha.
    public static void main(String[] args) throws Exception {
        String codeInterceptado = "AUTH_CODE_CAPTURADO_PELo_ATACANTE";
        String verifierAdivinhado = Pkce.generateCodeVerifier(); // um chute qualquer

        try {
            TokenExchange.trocarCodePorToken(
                "https://as.exemplo.com/token",
                "a1b2c3d4e5f6a7b8",
                codeInterceptado,
                "myapp://oauth/callback",
                verifierAdivinhado);

            System.out.println("Troca bem-sucedida – isso NÃO deveria acontecer!");
        } catch (RuntimeException falhaEsperada) {
            // Este é o resultado correto: invalid_grant. É exatamente o
            // comportamento que o lab rfc7636/ demonstra na prática, no
            // fluxo "Interceptar code sem o code_verifier".
            System.out.println("Troca rejeitada como esperado: " + falhaEsperada.getMessage());
        }
    }
}
```

RFC 8628 — OAuth 2.0 Device Authorization Grant

1. Contexto e motivação

Os quatro grants originais da RFC 6749 partem todos de uma premissa implícita: o dispositivo onde o fluxo começa tem um **user-agent** razoável (um browser capaz de navegar, exibir um formulário de login, seguir redirects) e, na maioria dos casos, um **método de entrada de texto** decente (teclado físico ou virtual utilizável). Essa premissa é confortável para desktop e para a maioria dos smartphones — e completamente falsa para uma classe grande e crescente de dispositivos: smart TVs, dispositivos de streaming (set-top boxes), consoles de videogame, impressoras conectadas, terminais de ponto de venda, dispositivos IoT industriais, e ferramentas de linha de comando (CLIs) rodando num terminal sem nenhum browser à disposição.

Antes da RFC 8628 (publicada em 2019, embora a ideia já circulasse informalmente havia anos sob o nome "OAuth Device Flow"), as alternativas disponíveis para esse tipo de dispositivo eram todas insatisfatórias, e vale enumerar por que cada uma falha:

- **Implementar um teclado na tela** para o usuário digitar usuário e senha diretamente no dispositivo limitado (a experiência clássica de "navegar" letra por letra com um controle remoto numa Smart TV). Isso é ruim por dois motivos: a UX é péssima (qualquer um que já tentou digitar uma senha complexa com um controle de TV sabe disso), e — mais grave do ponto de vista arquitetural — isso **reintroduz exatamente o anti-padrão que o OAuth inteiro existe para eliminar**: o dispositivo (um client) coletando a senha do usuário diretamente, em vez de delegar essa coleta para o Authorization Server. É o mesmo problema estrutural do Resource Owner Password Credentials Grant, só que forçado pela limitação de hardware em vez de por preguiça de implementação.
- **Usar o próprio ROPC** — que tem exatamente essa fraqueza estrutural (client vê a senha em claro) somada às limitações já conhecidas do grant (não funciona com MFA de forma nativa, não separa credencial de autorização, e foi formalmente deprecado por práticas atuais, incluindo a RFC 9700).
- **Não ter fluxo nenhum padronizado** — cada fabricante inventando seu próprio mecanismo proprietário de "ativação", sem interoperabilidade entre AS diferentes.

A ideia central da RFC 8628, e o que a torna elegante, é uma inversão simples: em vez de tentar melhorar a entrada de texto no dispositivo limitado, **tirar essa etapa do dispositivo limitado inteiramente** e transferi-la para um **segundo dispositivo** que o usuário quase certamente já tem à mão — o próprio celular, ou um notebook — que tem browser de verdade e teclado de verdade. O dispositivo limitado (o client, no vocabulário OAuth) só precisa fazer duas coisas: mostrar um código curto e uma URL curta na tela, e ficar consultando o Authorization Server em segundo plano até a autorização acontecer em algum outro lugar.

O exemplo que qualquer leitor já viu na prática, mesmo sem saber que é OAuth: ligar uma Smart TV nova e ver "Para ativar o Netflix, acesse `netflix.com/ativar` num computador ou celular e digite o código ABCD-1234". Essa tela é, literalmente, a implementação desta RFC — o "código" é o `user_code`, a URL

é a `verification_uri`, e a TV, por trás da tela, está fazendo polling no token endpoint enquanto espera.

Um ponto de contexto histórico interessante: a RFC 8628 formaliza um padrão que várias grandes plataformas (Google, entre outras) já vinham implementando de forma consistente mas não normatizada havia anos antes da publicação — o documento chega, em boa parte, para dar interoperabilidade e um vocabulário comum a um padrão que o mercado já tinha convergido informalmente.

2. Terminologia e papéis

A característica estrutural mais importante deste fluxo, que molda todo o vocabulário: **em nenhum momento existe um redirect de user-agent envolvendo o dispositivo original**. Isso é uma ruptura real com o modelo dos outros grants baseados em `/authorize` — não há `redirect_uri`, não há `state` no sentido de proteção de CSRF de um redirect (porque não há redirect), e o dispositivo (client) e o "user-agent" que efetivamente interage com o AS **são fisicamente separados**, rodando em hardware diferente, possivelmente em rede diferente.

Vocabulário novo introduzido pela RFC:

- `device_code` — um identificador opaco e de alta entropia, análogo em espírito a um authorization code de vida um pouco mais longa. Fica **só no dispositivo** (o client) — nunca é mostrado ao usuário, nunca sai do dispositivo exceto nas chamadas de polling ao token endpoint. Funciona como a "credencial de sessão" que o dispositivo usa para perguntar repetidamente "já fui autorizado?".
- `user_code` — um código curto, **desenhado explicitamente para ser digitado por um humano**: charset restrito (a RFC recomenda evitar caracteres visualmente ambíguos como `0` vs `O`, `1` vs `I` vs `l`), formato tipicamente com um hífen no meio para facilitar leitura em voz alta ou digitação em duas partes (`WDJB-MJHT`, por exemplo). É o único artefato deste fluxo que o resource owner efetivamente vê e manipula.
- `verification_uri` — a URL curta e fácil de digitar (ou de lembrar) que o usuário acessa no segundo dispositivo. A RFC recomenda que essa URL seja deliberadamente curta e memorável — não é incomum ver domínios dedicados só para isso (`netflix.com/ativar`, por exemplo), porque o usuário pode estar digitando de memória, olhando para uma TV a alguns metros de distância.
- `verification_uri_complete` (opcional) — a mesma URL, mas já com o `user_code` embutido como parâmetro de query. Existe para um caso de uso específico: exibir um **QR code** na tela do dispositivo, que ao ser escaneado leva o usuário direto para a tela de confirmação já com o código preenchido — eliminando completamente a digitação manual do `user_code`. Muitos dispositivos modernos (consoles, algumas Smart TVs) mostram QR code e código textual lado a lado, cobrindo os dois estilos de usuário.

Um ponto sutil sobre papéis: no vocabulário da RFC, o "dispositivo" continua sendo o **client** (no sentido OAuth) — é ele quem faz as chamadas HTTP ao AS. O "segundo aparelho" (celular ou notebook do usuário) **não é um client OAuth novo** — ele só está rodando um browser comum acessando uma página web comum do Authorization Server (a tela de verificação), participando do fluxo de login/consentimento exatamente como participaria em qualquer outro grant baseado em navegador.

3. Fluxo

1. Device → AS: POST /device_authorization client_id=...&scope=...
2. AS → Device: { device_code, user_code, verification_uri, verification_uri_complete, expires_in, ... }
3. Device exibe: "Acesse <verification_uri> e digite o código <user_code>" (ou um QR code)
4. Usuário, em OUTRO dispositivo: abre verification_uri, digita o user_code (ou escaneia o QR)
5. AS autentica o usuário normalmente – mesma tela de login/consentimento do Authorization Code Grant
6. Enquanto isso, Device consulta em loop:
POST /token grant_type=urn:ietf:params:oauth:grant-type:device_code&device_code=...&client_id=...
 - AS responde authorization_pending (400) enquanto o passo 4-5 não terminou
 - Device espera `interval` segundos entre tentativas (AS pode responder slow_down se o poll for muito rápido)
7. Assim que o usuário conclui a autorização, o próximo poll retorna a resposta normal de token (ac...

Vale examinar com mais cuidado a máquina de estados do polling no passo 6, porque é a parte operacionalmente mais delicada de implementar corretamente, e onde a maioria dos bugs de implementação de device flow acontece na prática:

- **Estado inicial:** o `device_code` foi emitido, o usuário ainda não fez nada no segundo dispositivo (ou está no meio do processo). Cada poll retorna `authorization_pending` — um erro HTTP 400 no vocabulário do token endpoint, mas semanticamente "ainda não, continue tentando", não uma falha real.
- **Polling rápido demais:** se o client não respeita o `interval` devolvido no passo 2 (ou continua usando um `interval` desatualizado depois de já ter recebido um `slow_down`), o AS pode responder `slow_down` — que carrega uma instrução implícita: **aumente o intervalo em pelo menos 5 segundos** a partir de agora, e continue. Isso não é um erro fatal, é uma correção de ritmo. Um client bem implementado mantém o `interval` como uma variável mutável ao longo do loop, nunca fixo desde o início.
- **Usuário nega:** se o usuário, na tela de consentimento do segundo dispositivo, explicitamente recusa a autorização, o próximo poll recebe `access_denied` — este sim um erro terminal, o loop deve parar e o dispositivo deve informar o usuário que a autorização foi negada.
- **Expiração:** o `device_code` (e o `user_code` associado) têm um `expires_in` — tipicamente alguns minutos. Se o usuário demora demais (ou simplesmente nunca acessa a `verification_uri`), o próximo poll depois da expiração recebe `expired_token` — também terminal; o dispositivo precisa reiniciar do zero (novo POST /device_authorization, novo user_code exibido).
- **Sucesso:** assim que o passo 5 é concluído do lado do usuário, o **próximo** poll do dispositivo (não instantaneamente — só quando o loop chegar na próxima tentativa, respeitando `interval`) recebe a resposta normal de token, no mesmo formato de qualquer outro grant (`access_token`, `token_type`, `expires_in`, `refresh_token` se aplicável). Esse é o único ponto onde o dispositivo "descobre" que foi autorizado — não há nenhum tipo de callback, webhook ou push notification definido pela spec; é estritamente poll-based.

Um detalhe de timing que vale destacar para quem for implementar um client deste grant: como não há garantia de quando exatamente o usuário vai concluir o passo 5 em relação ao ritmo de polling do dispositivo, o client deve estar preparado para ficar em `authorization_pending` por um tempo

potencialmente longo (minutos) sem tratar isso como anômalo — é o comportamento normal e esperado da maior parte da vida útil de um fluxo bem-sucedido.

4. Parâmetros e endpoints normativos

- **Device authorization endpoint** — endpoint inteiramente novo introduzido por esta RFC, tipicamente `POST /device_authorization` (o caminho exato é decisão do AS; descoberto de forma padronizada via RFC 8414 como o campo `device_authorization_endpoint` no metadata).
- Request (`application/x-www-form-urlencoded`): `client_id` (obrigatório — mesmo `client_id` que seria usado em qualquer outro grant deste client), `scope` (opcional, mesma semântica de sempre).
- Response (JSON): `device_code`, `user_code`, `verification_uri`, `verification_uri_complete` (opcional), `expires_in` (segundos até o `device_code` / `user_code` expirarem), `interval` (segundos mínimos entre polls; se omitido, o padrão assumido pela spec é 5).
- **Reaproveitamento do `/token`** — este é um dos pontos de design mais elegantes da RFC: em vez de inventar um endpoint de emissão de token paralelo, ela só define um **grant_type novo** para o token endpoint já existente da RFC 6749: `urn:ietf:params:oauth:grant-type:device_code`. Toda a infraestrutura de resposta de sucesso/erro do token endpoint (formato do corpo, cabeçalhos de cache, tratamento de erro) é reaproveitada sem modificação — só a semântica de "o que estou pedindo" muda.
- **Erros específicos do polling (§3.5)**, todos devolvidos como erro padrão do token endpoint (HTTP 400, corpo `{"error": "..."}`), com quatro valores próprios deste grant além dos erros genéricos já existentes na RFC 6749:
 - `authorization_pending` — ainda aguardando o usuário completar o passo 4-5; continue tentando no `interval` atual.
 - `slow_down` — o client está consultando rápido demais; aumente o `interval` em pelo menos 5 segundos e continue.
 - `access_denied` — o usuário negou explicitamente a autorização; pare de tentar.
 - `expired_token` — `device_code` / `user_code` expiraram antes da conclusão; pare de tentar, é preciso recomençar do passo 1.

Uma observação de implementação relevante para quem constrói o lado servidor: o AS precisa manter, associados ao mesmo `device_code`, tanto o estado de autorização (pendente/aprovado/negado) quanto o timestamp de expiração e o `interval` atual esperado daquele client específico — inclusive para detectar corretamente quando aplicar `slow_down` (o AS precisa lembrar quando foi o poll anterior daquele `device_code` para saber se o atual veio cedo demais).

5. Segurança já tratada na própria RFC 8628 (§5)

- **Entropia baixa do `user_code` é uma escolha deliberada, não um descuido** — ele precisa ser digitável por um humano em segundos, o que impõe um teto prático de tamanho e um charset restrito bem abaixo da entropia normalmente esperada de um artefato OAuth (compare com os 256 bits recomendados para um `code_verifier` da PKCE, capítulo anterior). Para compensar essa entropia estruturalmente menor, a RFC exige contrapartidas operacionais: **rate limiting**

agressivo no endpoint de verificação (impedindo que um atacante tente forçar bruta muitos

`user_code` s por segundo) e **expiração curta** (minutos, não horas) para o

`user_code / device_code`, reduzindo a janela em que um `user_code` de baixa entropia seria viável de adivinhar. É um perfil de risco genuinamente diferente do resto do OAuth, e a própria RFC é explícita sobre estar fazendo essa troca conscientemente.

- **Confidencialidade do `device_code`** — apesar de não precisar ser digitável (por isso pode ter entropia alta), o `device_code` ainda precisa ficar confidencial no dispositivo durante toda a janela de espera: quem o possuir pode fazer o polling em nome do dispositivo legítimo. Como este grant costuma ser usado por clients públicos (dispositivos sem capacidade de guardar segredo — TVs, consoles), o `device_code` funciona, durante essa janela, quase como uma credencial ao portador de curta duração.
- **`slow_down / interval` como proteção operacional do AS** — sem esse mecanismo, um dispositivo mal implementado (ou deliberadamente malicioso) poderia fazer polling agressivo, sobrecarregando o token endpoint. O `interval` cresce sob demanda do próprio AS, não é uma configuração estática negociada previamente.
- **Reconhecimento (parcial) de risco de phishing já no texto original:** a RFC já observa que mostrar um "código real" de autorização para induzir alguém a autorizar algo que não pediu é um risco conceitual, e recomenda que a tela de consentimento do AS deixe claro **para qual aplicação/dispositivo** a autorização está sendo concedida. Mas essa recomendação, sozinha, se mostrou insuficiente na prática — ver item 7 abaixo, porque é exatamente aqui que a história pós-publicação fica interessante.

6. Relação com outras RFCs

- Adiciona um **quinto grant type** ao vocabulário da RFC 6749, especificamente desenhado para a classe de dispositivo ("input-constrained") que os quatro grants originais não atendiam bem.
- Reaproveita inteiramente a infraestrutura do token endpoint da RFC 6749 — mesmo padrão estrutural de extensão via `grant_type` novo que a **RFC 8693 (Token Exchange)**, próximo capítulo deste livro, também usa. Vale comparar os dois: ambos evitam criar um endpoint de emissão paralelo, preferindo estender a semântica do `/token` já existente — um padrão de design que se repete no ecossistema de extensões do OAuth.
- Descoberto via **RFC 8414** (Authorization Server Metadata) como `device_authorization_endpoint` — o mesmo padrão de descoberta já visto para `revocation_endpoint` e `introspection_endpoint` em capítulos anteriores deste livro.
- **PKCE (RFC 7636) não se aplica a este grant** — e vale entender por quê: PKCE protege contra interceptação de um `code` entregue via redirect de user-agent; aqui não há `redirect_uri` nem redirect nenhum envolvendo o dispositivo original, então a superfície de ataque que a PKCE endereça simplesmente não existe nesta forma. O risco análogo deste grant (phishing de `user_code`) tem uma forma estrutural diferente — ver item 7.
- Fecha um ciclo aberto no capítulo da **RFC 6819** deste livro: naquele capítulo, "phishing de device code" já foi apontado como uma classe de ameaça que só passaria a existir depois da publicação da RFC 8628 (2019) — este é o capítulo onde essa ameaça, antecipada estruturalmente, finalmente ganha um mecanismo concreto ao qual se aplicar.

- Fora do escopo deste livro, mas conceitualmente próximo: **CIBA** (Client-Initiated Backchannel Authentication), uma extensão do OpenID Connect que também desacopla "onde a requisição começa" de "onde a autenticação do usuário acontece" — só que CIBA não depende de o usuário digitar um código: o AS notifica o dispositivo de autenticação do usuário diretamente (push notification, por exemplo), um modelo "decoupled" de geração diferente, mas resolvendo uma preocupação de UX aparentada.

7. O que mudou desde a publicação (2019)

- O texto normativo da RFC em si permanece estável, sem revisão formal desde a publicação — mas um padrão de abuso real e significativo emergiu **depois**, e não estava adequadamente coberto pelo threat model original: **phishing de device code**, também chamado de "device code phishing" ou explorado sob o guarda-chuva de ataques de "adversary-in-the-middle" contra fluxos de autenticação moderna.
- O mecanismo do ataque: o atacante **inicia o próprio fluxo legítimo** (faz o `POST /device_authorization` ele mesmo, com um `client_id` de uma aplicação real e confiável — por exemplo, o próprio client oficial do Microsoft 365, cujo `client_id` é público) e obtém um `user_code` genuíno, válido, gerado pelo AS real. Em seguida, usando engenharia social — tipicamente um pretexto de "suporte técnico" por telefone ou uma campanha de phishing por e-mail/SMS — convence a vítima a acessar a `verification_uri` legítima e digitar **o código do atacante**. A vítima, ao se autenticar normalmente (tudo acontece no domínio real do AS, com TLS válido — não há nada "falso" visualmente para a vítima detectar), efetivamente autoriza a sessão do atacante, não a sua própria. O atacante, do outro lado do polling, recebe o token.
- O que torna esse ataque particularmente insidioso, e por isso vale destacar num livro voltado a arquitetos: **não há nenhum indicador técnico visivelmente errado para a vítima notar** — o domínio é o real, o certificado TLS é válido, a tela de login é a tela de login genuína do provedor de identidade. A única defesa possível é a vítima perceber que está digitando um código que **não veio de um dispositivo que ela mesma está tentando ativar** — uma defesa inteiramente dependente de atenção humana e contexto, exatamente a categoria de defesa que a engenharia social é desenhada para contornar. Esse ataque foi documentado em campanhas reais e amplamente reportadas contra contas Microsoft 365/Entra ID a partir de 2022, com atribuição a grupos patrocinados por estados.
- As mitigações que emergiram são **práticas operacionais e de produto**, não emendas formais ao texto da RFC 8628:
- **"Number matching"** — em vez de (ou além de) o usuário digitar o `user_code` no segundo dispositivo, o fluxo passa a exigir que o usuário **confirme, no dispositivo onde está se autenticando, um número exibido no dispositivo original que iniciou o pedido** — invertendo a direção da verificação e forçando o usuário a olhar concretamente para o dispositivo que ele mesmo está tentando ativar, dificultando (mas não eliminando) o pretexto de phishing.
- Exibir **contexto adicional** durante a tela de consentimento — nome da aplicação, localização geográfica aproximada de onde partiu o pedido, horário — para dar à vítima mais sinais para desconfiar de um pedido que ela não iniciou.
- Reduzir a **vida útil** do `user_code` / `device_code` ainda mais do que o mínimo já recomendado pela spec original, encurtando a janela de oportunidade do atacante.

- Conscientização de segurança organizacional — treinar usuários para desconfiar de qualquer solicitação de "digite este código" que chegue por telefone ou mensagem não solicitada, independentemente de o domínio parecer legítimo.
- Vale reter esta lição como o ponto mais importante do capítulo do ponto de vista arquitetural: este é um exemplo direto e bem documentado de um grant **tecnicamente correto e implementado fielmente conforme a spec**, cujo risco real e explorado em produção só apareceu com a escala de uso — exatamente o tipo de lição que a RFC 6819 tenta antecipar estruturalmente, mas que a prática mostrou que nem sempre um threat model publicado antecipa tudo. Para um arquiteto avaliando adotar este grant hoje, a decisão correta não é "a RFC cobre a segurança, então está resolvido" — é reconhecer que controles operacionais pós-publicação (number matching, em particular) deveriam ser tratados como parte do baseline de adoção responsável, não como um extra opcional.

Nota sobre laboratório: este projeto não possui, até o momento, um laboratório para a RFC 8628 — os labs existentes cobrem RFC 6749 (grant básico), RFC 7636 (PKCE) e RFC 9126 (PAR), todos baseados em redirect de browser. O Device Authorization Grant tem uma forma visualmente muito diferente (sem redirect nenhum, com um loop de polling e dois "dispositivos" separados) e seria um bom candidato a laboratório futuro — simulando um "dispositivo" (uma aba de browser ou processo dedicado, fazendo polling) enquanto um segundo "aparelho" (outra aba, representando o celular do usuário) completa a autorização de forma independente. Por ora, este capítulo se concentra nos exemplos em Java abaixo, que implementam o fluxo do lado do dispositivo (client) contra um Authorization Server hipotético.

Exemplo em Java (JDK 21, sem frameworks)

Os exemplos usam `java.net.http.HttpClient` para as chamadas HTTP e o utilitário `SimpleJson` (métodos `SimpleJson.object(Map)` e `SimpleJson.parseFlat(String)`), definido no Capítulo 1 (RFC 6749) deste livro, para montar/ler os corpos JSON — todos "achatados" (sem aninhamento), o que é suficiente para as respostas deste grant.

Exemplo 1 — iniciar a Device Authorization Request

```
import java.net.URI;
import java.net.URLEncoder;
import java.net.http.HttpClient;
import java.net.http.HttpRequest;
import java.net.http.HttpResponse;
import java.nio.charset.StandardCharsets;
import java.util.Map;

public final class DeviceAuthorizationClient {

    private final HttpClient http = HttpClient.newHttpClient();
    private final String deviceAuthorizationEndpoint;
    private final String tokenEndpoint;
    private final String clientId;

    public DeviceAuthorizationClient(String deviceAuthorizationEndpoint,
                                    String tokenEndpoint, String clientId) {
        this.deviceAuthorizationEndpoint = deviceAuthorizationEndpoint;
        this.tokenEndpoint = tokenEndpoint;
        this.clientId = clientId;
    }

    // RFC 8628 §3.1 – POST /device_authorization. Sem autenticação forte de
    // client aqui (tipicamente clients públicos – TVs, CLIs – não têm
    // client_secret); o client_id sozinho identifica a aplicação.
    public Map<String, String> iniciarDeviceAuthorization(String scope) throws Exception {
        String body = "client_id=" + URLEncoder.encode(clientId, StandardCharsets.UTF_8)
            + "&scope=" + URLEncoder.encode(scope, StandardCharsets.UTF_8);

        HttpRequest request = HttpRequest.newBuilder()
            .uri(URI.create(deviceAuthorizationEndpoint))
            .header("Content-Type", "application/x-www-form-urlencoded")
            .POST(HttpRequest.BodyPublishers.ofString(body))
            .build();

        HttpResponse<String> response = http.send(request, HttpResponse.BodyHandlers.ofString());
        if (response.statusCode() != 200) {
            throw new RuntimeException("Falha ao iniciar device authorization: HTTP "
                + response.statusCode() + " - " + response.body());
        }
        // device_code, user_code, verification_uri, verification_uri_complete,
        // expires_in, interval
        return SimpleJson.parseFlat(response.body());
    }
}
```


Exemplo 2 — exibir instruções ao usuário (simulando a tela do dispositivo)

```
public final class DeviceScreen {

    // Em produção isto seria a UI real do dispositivo (TV, console...). Aqui,
    // simulamos com saída de console – o ponto didático é que o dispositivo
    // não precisa (nem deveria) coletar usuário/senha: só mostra um código
    // curto e uma URL, e delega toda a autenticação para o segundo aparelho.
    public static void exibirInstrucoes(java.util.Map<String, String> deviceAuthResponse) {
        String userCode = deviceAuthResponse.get("user_code");
        String verificationUri = deviceAuthResponse.get("verification_uri");
        String verificationUriComplete = deviceAuthResponse.get("verification_uri_complete");

        System.out.println("=====");
        System.out.println(" Para ativar este dispositivo:");
        System.out.println(" 1. Acesse: " + verificationUri);
        System.out.println(" 2. Digite o código: " + userCode);
        if (verificationUriComplete != null) {
            System.out.println(" (ou escaneie o QR code apontando para "
                               + verificationUriComplete + ")");
        }
        System.out.println("=====");
    }
}
```


Exemplo 3 — loop de polling no `/token`, tratando cada estado do RFC 8628 §3.5

```
import java.net.URI;
import java.net.URLEncoder;
import java.net.http.HttpClient;
import java.net.http.HttpRequest;
import java.net.http.HttpResponse;
import java.nio.charset.StandardCharsets;
import java.util.Map;

public final class DevicePolling {

    private static final String GRANT_TYPE_DEVICE_CODE =
        "urn:ietf:params:oauth:grant-type:device_code";

    private final HttpClient http = HttpClient.newHttpClient();
    private final String tokenEndpoint;
    private final String clientId;

    public DevicePolling(String tokenEndpoint, String clientId) {
        this.tokenEndpoint = tokenEndpoint;
        this.clientId = clientId;
    }

    // Faz o polling até obter sucesso, negação explícita, ou expiração.
    // O `interval` é tratado como mutável – cresce sempre que o AS responde
    // slow_down, nunca diminui, e o Loop dorme por esse tempo entre cada
    // tentativa, exatamente como o §3.5 da RFC prescreve.
    public Map<String, String> aguardarAutorizacao(String deviceCode, int intervalInicialSegundos)
        throws Exception {
        int interval = intervalInicialSegundos;

        while (true) {
            Thread.sleep(interval * 1000L);

            String body = "grant_type=" + URLEncoder.encode(GRANT_TYPE_DEVICE_CODE, StandardCharsets.UTF_8)
                + "&device_code=" + URLEncoder.encode(deviceCode, StandardCharsets.UTF_8)
                + "&client_id=" + URLEncoder.encode(clientId, StandardCharsets.UTF_8);

            HttpRequest request = HttpRequest.newBuilder()
                .uri(URI.create(tokenEndpoint))
                .header("Content-Type", "application/x-www-form-urlencoded")
                .POST(HttpRequest.BodyPublishers.ofString(body))
                .build();

            HttpResponse<String> response = http.send(request, HttpResponse.BodyHandlers.ofString());
            Map<String, String> json = SimpleJson.parseFlat(response.body());

            if (response.statusCode() == 200) {
                // Sucesso: access_token, token_type, expires_in, refresh_token...
                return json;
            }
        }
    }
}
```

```
String error = json.getDefault("error", "");
switch (error) {
    case "authorization_pending" -> {
        // Ainda esperando o usuário completar o passo no segundo
        // dispositivo – continua o loop no mesmo interval.
        System.out.println("Aguardando autorização do usuário...");
    }
    case "slow_down" -> {
        // RFC 8628 §3.5: aumentar o interval em pelo menos 5s.
        interval += 5;
        System.out.println("AS pediu slow_down – novo interval: " + interval + "s");
    }
    case "access_denied" -> throw new RuntimeException(
        "Usuário negou a autorização explicitamente.");
    case "expired_token" -> throw new RuntimeException(
        "device_code/user_code expiraram antes da conclusão – reinicie o fluxo.");
    default -> throw new RuntimeException(
        "Erro inesperado do token endpoint: " + error);
}
}
}
```

Exemplo 4 — orquestrando o fluxo completo

```
public final class DeviceFlowMain {

    public static void main(String[] args) throws Exception {
        String clientId = "tv-app-9f8e7d6c";
        var client = new DeviceAuthorizationClient(
            "https://as.exemplo.com/device_authorization",
            "https://as.exemplo.com/token",
            clientId);

        // Passo 1-2: inicia o fluxo, recebe device_code/user_code/etc.
        var deviceAuth = client.iniciarDeviceAuthorization("profile offline_access");

        // Passo 3: exibe instruções (na TV real, isto ficaria na tela).
        DeviceScreen.exibirInstrucoes(deviceAuth);

        // Passo 6-7: enquanto o usuário age no segundo dispositivo, este
        // processo fica em polling até sucesso, negação ou expiração.
        int intervalInicial = Integer.parseInt(deviceAuth.getDefault("interval", "5"));
        var polling = new DevicePolling("https://as.exemplo.com/token", clientId);
        var tokens = polling.aguardarAutorizacao(deviceAuth.get("device_code"), intervalInicial);

        System.out.println("Autorizado! access_token = " + tokens.get("access_token"));
    }
}
```

RFC 8693 — OAuth 2.0 Token Exchange

1. Contexto e motivação

A RFC 8693 nasce de um problema que praticamente qualquer arquitetura de microsserviços moderna encontra, e que fica particularmente evidente em ambientes bancários/FSI com múltiplas camadas de API: o resource owner (usuário final) autoriza um client uma vez, recebe um `access_token`, e esse token viaja para o primeiro serviço que o recebe (um gateway, um Backend-for-Frontend). Mas o pedido original quase nunca termina ali — o gateway precisa, para atender aquele mesmo pedido, chamar um segundo serviço downstream, e depois talvez um terceiro. A pergunta que a RFC 8693 responde é: **com que credencial o gateway deveria fazer essas chamadas subsequentes?**

A resposta ingênua — repassar exatamente o mesmo `access_token` recebido do usuário — é precisamente o anti-padrão de **"confused deputy"** já catalogado no capítulo da RFC 6819 deste livro. O problema não é hipotético nem sutil: aquele token foi emitido pelo Authorization Server pensando numa audience específica (o primeiro serviço, API-1), com um escopo pensado para as operações que API-1 expõe. Ao repassar o mesmo token para API-2, várias garantias implícitas quebram silenciosamente:

- **API-2 não tem como saber, só olhando o token, que ele não foi emitido pensando nela** — a menos que o AS pratique audience restriction rigorosa (RFC 8707, capítulo anterior deste livro) e API-2 valide isso ativamente, o token "funciona" em qualquer resource server que aceite a mesma chave de assinatura/introspecção, mesmo que ele nunca devesse ter sido chamado com aquele token.
- **O escopo concedido pelo usuário para API-1 pode ser mais amplo do que o necessário para a chamada específica a API-2** — repassar o token inteiro repassa também privilégios que a chamada atual nem precisa exercer, violando least-privilege na própria cadeia interna de chamadas, não só na borda externa.
- **Não há registro de que um intermediário (o gateway) esteve envolvido** — do ponto de vista de API-2, a chamada parece ter vindo diretamente do usuário, quando na verdade veio de um serviço agindo em nome dele. Para auditoria — crítico em ambiente regulado — essa distinção importa.

A RFC 8693 resolve isso formalizando um mecanismo de **troca**: o gateway apresenta o token que recebeu ao token endpoint do AS (não para o usuário se autenticar de novo — o usuário não está envolvido nesta troca) e recebe de volta um **token novo**, especificamente re-audienciado e/ou re-escopado para o destino real da próxima chamada, carregando de forma explícita e auditável tanto **quem originalmente pediu** (o subject — o usuário) quanto, quando aplicável, **quem está agindo agora em nome dele** (o actor — o próprio gateway).

Vale também situar os cenários correlatos que a RFC cobre, além do caso de gateway/BFF que abre o capítulo, porque a spec foi desenhada deliberadamente ampla:

- **Tradução de formato de token** — trocar uma asserção SAML (comum em ambientes corporativos legados, especialmente bancários com SSO baseado em SAML de longa data) por um JWT OAuth, permitindo que um sistema legado de identidade alimente uma arquitetura OAuth moderna sem reescrever o IdP.
- **Confiança cross-domain** — trocar um token emitido por um IdP A por um token aceito num sistema que confia especificamente em IdP B, quando existe uma relação de confiança configurada explicitamente no Authorization Server (uma federação de confiança, não uma tradução automática qualquer).
- **"Downscoping" deliberado** — trocar um token de escopo amplo por um mais restrito antes de repassá-lo adiante, mesmo sem trocar de audience — uma aplicação direta de least-privilege dentro da própria cadeia de chamadas de um mesmo serviço, útil quando um componente interno só precisa de uma fração do que o token original autoriza.

2. Terminologia e papéis

Esta é uma das RFCs do roteiro que introduz vocabulário genuinamente novo, não apenas parâmetros — e a precisão aqui importa porque a distinção central da RFC (impersonação vs. delegação) depende inteiramente desses termos:

- **Subject** — a entidade sobre quem o token efetivamente fala; tipicamente o resource owner/usuário final, mas pode ser qualquer entidade cuja identidade o token representa (inclusive, em cenários machine-to-machine, um serviço).
- **Actor** — quem está de fato exercendo a ação no momento presente, que pode (e no caso de interesse deste capítulo, é) diferente do subject. No exemplo do gateway: o usuário é o subject, o gateway é o actor.
- **Impersonação** — o token resultante da troca representa **só** o subject. O fato de um intermediário (actor) ter solicitado e efetuado a troca não fica registrado em lugar nenhum do token resultante. Do ponto de vista do resource server que recebe esse token novo, a chamada parece ter vindo diretamente do subject — "isto é o usuário", ponto final, como se ele tivesse chamado diretamente, sem intermediário algum visível.
- **Delegação** — o token resultante registra **os dois**, subject e actor, explicitamente. O resource server downstream enxerga uma cadeia de custódia auditável: "isto é o usuário, mas o gateway X está agindo em nome dele agora". É este o padrão que a claim `act` (mencionada informalmente pela RFC e consolidada por perfis JWT posteriores) representa — um encadeamento de "quem age em nome de quem", podendo em teoria se aninhar (um actor agindo em nome de outro actor que já agia em nome de um subject original, numa cadeia de múltiplos hops).
- **Security token** — a RFC evita deliberadamente o termo "access token" ao descrever o que é trocado, preferindo o termo genérico "security token", porque o mecanismo é **agnóstico de formato**: o que entra e o que sai pode ser um access token OAuth comum, um ID Token, um refresh token, um JWT arbitrário, ou até uma asserção SAML — qualquer coisa identificável por uma das URIs de tipo de token pré-registradas.

A escolha entre impersonação e delegação não é do client sozinho decidir livremente — é uma **política do Authorization Server**, tipicamente configurada por combinação de qual client está pedindo a troca, qual subject_token está sendo trocado, e para qual audience/resource. Um AS bem projetado

para ambiente regulado deveria, na prática, favorecer fortemente delegação sobre impersonação sempre que a cadeia de chamadas tiver relevância de auditoria — que, em contexto bancário, é quase sempre.

3. Fluxo

Uma única requisição ao token endpoint já existente, com um `grant_type` novo — sem nenhum redirect de usuário envolvido, porque o Token Exchange é inteiramente back-channel, tipicamente machine-to-machine entre serviços:

```
POST /token
grant_type=urn:ietf:params:oauth:grant-type:token-exchange
&subject_token=<token a trocar>
&subject_token_type=urn:ietf:params:oauth:token-type:access_token
&resource=https://api-2.exemplo.com
&scope=... (opcional, mais restrito que o original)
&actor_token=... &actor_token_type=... (opcional — presente em delegação, representa quem esta pediu)
```

Vale examinar dois cenários concretos lado a lado, porque a diferença entre eles é exatamente a diferença entre impersonação e delegação na prática:

Cenário A — impersonação pura: o gateway envia só `subject_token` (o token do usuário) + `resource` (o destino, API-2). Não envia `actor_token`. O AS, se configurado para permitir impersonação para este client, emite um token novo cujo subject continua sendo o usuário original, sem qualquer registro do gateway ter mediado a troca.

Cenário B — delegação explícita: o gateway envia `subject_token` (do usuário) e `actor_token` (um token que identifica o próprio gateway, tipicamente obtido via Client Credentials Grant previamente) + `actor_token_type`. O AS, reconhecendo que este client tem permissão de atuar como actor para este tipo de subject, emite um token novo cujas claims incluem tanto o subject (usuário) quanto uma claim de ator encadeado — o resource server final consegue diferenciar exatamente quem pediu originalmente e quem mediou.

A resposta, no formato padrão do token endpoint (`access_token`, `token_type`, `expires_in`, `scope`, `refresh_token` opcional), ganha um campo novo e **obrigatório**: `issued_token_type`. Esse campo existe porque, diferente de qualquer grant anterior no roteiro, o Token Exchange pode devolver algo que **não** é um `access_token` OAuth comum — se o `subject_token` de entrada era uma asserção SAML e o cenário é de tradução de formato, o token devolvido pode ser um JWT com semântica diferente de um `access_token` tradicional, e o client precisa de um jeito programático de saber exatamente o que recebeu de volta, sem precisar inferir pelo formato.

4. Parâmetros e endpoints normativos

Nenhum endpoint novo — reaproveita inteiramente o `/token` da RFC 6749, com `grant_type=urn:ietf:params:oauth:grant-type:token-exchange`. A tabela de parâmetros da requisição:

Parâmetro	Obrigatório	Descrição
<code>grant_type</code>	Sim	Sempre <code>urn:ietf:params:oauth:grant-type:token-exchange</code>
<code>subject_token</code>	Sim	O token de segurança a ser trocado
<code>subject_token_type</code>	Sim	URI identificando o tipo do <code>subject_token</code>
<code>actor_token</code>	Não	Presente apenas em cenários de delegação — representa quem está pedindo a troca
<code>actor_token_type</code>	Condicional	Obrigatório se <code>actor_token</code> estiver presente
<code>resource</code>	Não	URI indicando o destino/recurso pretendido do token novo (RFC 8707)
<code>audience</code>	Não	Alternativa/complemento a <code>resource</code> — identifica a audience pretendida de forma mais abstrata que uma URI de recurso
<code>scope</code>	Não	Permite pedir um escopo mais restrito que o do <code>subject_token</code> original — nunca mais amplo
<code>requested_token_type</code>	Não	Permite ao client pedir explicitamente que tipo de token quer de volta

Os valores pré-registrados de tipo de token (usados em `subject_token_type`, `actor_token_type` e na resposta `issued_token_type`) seguem o padrão de URI `urn:ietf:params:oauth:token-type:`, com sufixos: `access_token`, `refresh_token`, `id_token`, `saml1`, `saml2`, `jwt`. Um AS pode, em teoria, suportar tipos adicionais não pré-registrados, mas os seis acima cobrem a esmagadora maioria dos cenários práticos.

A resposta, além dos campos padrão do token endpoint, inclui obrigatoriamente `issued_token_type` — o client deve tratar esse campo como autoritativo sobre o que de fato recebeu, em vez de assumir que voltou um `access_token` só porque pediu um.

Erros seguem o vocabulário genérico já conhecido do token endpoint (`invalid_request`, `invalid_grant`, `invalid_scope`) — a RFC não introduz um código de erro dedicado só para falhas de troca, preferindo reaproveitar a semântica existente: um `subject_token` inválido, expirado ou de um tipo não reconhecido tipicamente resulta em `invalid_grant`; uma combinação de client/subject/resource não autorizada pela política do AS tipicamente resulta em `invalid_target` (definido pela RFC 8707, para quando o `resource` pedido não é um destino válido para aquele token) ou `invalid_scope`.

5. Segurança já tratada na própria RFC 8693

- **Risco central: escalonamento de privilégio via troca em si.** Este é o ponto que a RFC trata com mais insistência, e o que separa uma implementação correta de uma perigosa: o Token Exchange, mal implementado, vira **ele mesmo** um vetor de escalonamento de privilégio. Se o AS aceitar qualquer troca de qualquer client para qualquer resource/audience sem validação rigorosa, um serviço que legitimamente possui um token de baixo privilégio poderia trocá-lo por um de privilégio mais alto — exatamente o oposto do propósito da RFC. A defesa exigida: o AS precisa validar explicitamente se **aquele client específico** está autorizado a trocar **aquele subject_token específico** pelo **resource/audience pedido especificamente** — uma matriz de política de três dimensões (client × subject × destino), não uma regra genérica de "qualquer troca é permitida".
- **Política sobre quem pode atuar como actor:** de forma análoga, o AS deve controlar explicitamente quais clients têm permissão de aparecer como `actor_token` em cenários de delegação, e quais combinações subject/actor fazem sentido — não é uma capacidade que qualquer client registrado deveria ganhar por padrão.
- **Princípio de não-ampliação:** o token resultante de uma troca deve, como regra geral, **restringir ou manter** — nunca ampliar — o escopo ou a audience em relação ao token original. O Token Exchange existe para re-direcionar e reduzir superfície, não para um serviço "upgradar" seu próprio nível de acesso através de uma troca conveniente. Um AS que permitisse `scope` mais amplo no token de saída do que no de entrada estaria invertendo o propósito inteiro da spec.
- **Auditabilidade de cadeias de delegação:** quando múltiplos hops de delegação se encadeiam (um actor trocando em nome de outro actor que já agia em nome de um subject original), a RFC espera que essa cadeia fique registrada de forma rastreável — o conceito de uma claim `act` aninhada (`act` dentro de `act`, representando cada hop) nasce justamente dessa necessidade, embora a estrutura exata da claim tenha sido deixada para perfis JWT subsequentes formalizarem, não a própria RFC 8693.
- **Confidencialidade do `subject_token` / `actor_token` em trânsito:** como qualquer requisição ao token endpoint, esta troca deve acontecer sobre TLS — os tokens de entrada carregam a mesma sensibilidade de qualquer token de segurança, e a RFC não introduz nenhuma exceção a essa exigência básica.
- Vale destacar, para um leitor com background FSI: em cadeias de chamada internas de sistemas bancários — que frequentemente atravessam múltiplos domínios de confiança (core banking, camada de integração, APIs de parceiros) — a disciplina de Token Exchange como **único mecanismo legítimo** de "trocar de contexto" entre domínios, em vez de replay do mesmo token ou de credenciais compartilhadas hardcoded entre serviços, é diretamente relevante para requisitos de segregação e auditoria típicos de regulação financeira.

6. Relação com outras RFCs

- Reaproveita a infraestrutura do token endpoint da RFC 6749 — o mesmo padrão estrutural de extensão via `grant_type` novo que a **RFC 8628 (Device Authorization Grant)**, capítulo anterior deste livro, também usa: em vez de endpoints de emissão paralelos, ambas preferem estender a semântica do `/token` já existente.

- É a solução formal e nomeada para o cenário de "**confused deputy**" catalogado na RFC 6819 (Threat Model): em vez de replay do mesmo token entre resource servers diferentes — o antipadrão descrito na abertura deste capítulo —, o mecanismo correto passa a ser trocar por um token novo, corretamente audienciado e/ou escopado para o destino real.
- Companheira direta da **RFC 8707 (Resource Indicators)**, tema de capítulo anterior deste livro: ambas usam um parâmetro `resource` para indicar o destino pretendido de um token. São specs do mesmo espaço de problema — ambientes com múltiplos resource servers, onde "para quem este token é válido" precisa ser explícito e verificável. O Token Exchange demonstra que a questão de audiente correta não é só "evitar passivamente um ataque" (o enquadramento típico da RFC 8707 sozinha) — é também "ter um mecanismo **ativo** para emitir tokens corretamente audienciados em cadeias de chamada legítimas", uma dimensão complementar da mesma preocupação.
- Formato-agnóstica por design, conecta diretamente com a **RFC 7523 (JWT Profile for OAuth 2.0 Authorization Grants)** — um JWT é um dos tipos de subject/actor token explicitamente reconhecidos, e a RFC 7523 já estabelece o padrão de usar JWTs como portadores de asserção dentro do vocabulário OAuth.
- Conceito equivalente, em espírito, ao padrão "**On-Behalf-Of**" (**OBO**) usado por provedores de identidade corporativos (o exemplo mais citado sendo Azure AD/Microsoft Entra ID) — um padrão de mercado que antecede formalmente a RFC 8693, mas que hoje se alinha e, em implementações mais recentes, mapeia diretamente para o `grant_type` de Token Exchange desta RFC.

7. O que mudou desde a publicação (2020)

- A spec permanece estável, sem revisão formal — mas ganhou relevância crescente e sustentada com a adoção generalizada de arquiteturas de microsserviços e service mesh: Token Exchange se tornou peça padrão em padrões de API gateway e Backend-for-Frontend, exatamente para resolver o problema descrito na abertura deste capítulo — "como chamar a API downstream com um token corretamente escopado, sem repassar o token do usuário cru".
- Ganhou tração particular em arquiteturas **zero-trust**, onde a expectativa arquitetural passou a ser que **cada hop de serviço** carregue um token distintamente escopado/audienciado para aquele hop específico, em vez de um único token amplo circulando sem modificação por toda a cadeia de chamadas internas — um princípio diretamente relevante para o contexto de cadeias de APIs internas bancárias, onde diferentes domínios (pagamentos, core banking, canais digitais) tipicamente exigem essa segregação por exigência regulatória, não só por boa prática.
- A distinção subject/actor e o padrão de claim `act` encadeada foram adotados de forma informal por várias implementações de mercado como a maneira de-facto de representar cadeias de delegação dentro de JWTs — mesmo sem uma RFC separada fechando formalmente a estrutura exata dessa claim. Isso vive hoje mais em documentos de perfil e convenções específicas de fornecedor (Microsoft, Okta, entre outros) construídas sobre a fundação conceitual que a RFC 8693 estabeleceu, do que numa segunda spec IETF dedicada.

Nota sobre laboratório: este projeto não possui, até o momento, um laboratório para a RFC 8693. O cenário natural para um lab desta RFC exigiria pelo menos um segundo resource server (além do já existente nos labs de RFC 6749/7636/9126) e um serviço "gateway" intermediário, demonstrando a troca de token como alternativa concreta ao replay do mesmo token entre APIs — incluindo, idealmente, os dois cenários (impersonação e delegação) lado a lado, com o resource server final mostrando a diferença observável na claim de identidade recebida. Por ora, este capítulo se concentra nos exemplos em Java abaixo, que constroem as requisições de troca do lado do client contra um Authorization Server hipotético.

Exemplo em Java (JDK 21, sem frameworks)

Os exemplos usam `java.net.http.HttpClient` e o utilitário `SimpleJson` (métodos `SimpleJson.object(Map)` e `SimpleJson.parseFlat(String)`), definido no Capítulo 1 (RFC 6749) deste livro, para montar e ler os corpos JSON das requisições e respostas de troca.

Exemplo 1 — troca simples (impersonação), re-audienciando para outra API

```
import java.net.URI;
import java.net.URLEncoder;
import java.net.http.HttpClient;
import java.net.http.HttpRequest;
import java.net.http.HttpResponse;
import java.nio.charset.StandardCharsets;
import java.util.Map;

public final class TokenExchangeClient {

    private static final String GRANT_TYPE_TOKEN_EXCHANGE =
        "urn:ietf:params:oauth:grant-type:token-exchange";
    private static final String TYPE_ACCESS_TOKEN =
        "urn:ietf:params:oauth:token-type:access_token";
    private static final String TYPE_JWT =
        "urn:ietf:params:oauth:token-type:jwt";

    private final HttpClient http = HttpClient.newHttpClient();
    private final String tokenEndpoint;

    public TokenExchangeClient(String tokenEndpoint) {
        this.tokenEndpoint = tokenEndpoint;
    }

    // Troca um access_token recebido da chamada do usuário (subject_token)
    // por um token novo, audienciado para uma API downstream diferente –
    // sem enviar actor_token: cenário de IMPERSONAÇÃO (o token novo
    // representa só o subject, sem registro do gateway ter mediado).
    public Map<String, String> trocarParaOutraAudience(String subjectToken,
                                                         String resourceDestino,
                                                         String scopeRestrito) throws Exception {

        StringBuilder body = new StringBuilder();
        body.append("grant_type=").append(enc(GRANT_TYPE_TOKEN_EXCHANGE));
        body.append("&subject_token=").append(enc(subjectToken));
        body.append("&subject_token_type=").append(enc(TYPE_ACCESS_TOKEN));
        body.append("&resource=").append(enc(resourceDestino));
        if (scopeRestrito != null) {
            body.append("&scope=").append(enc(scopeRestrito));
        }

        HttpRequest request = HttpRequest.newBuilder()
            .uri(URI.create(tokenEndpoint))
            // Autenticação do CLIENT que está pedindo a troca (o gateway,
            // não o usuário) – aqui via client_secret_basic, um dos
            // métodos já vistos no capítulo da RFC 6749.
            .header("Authorization", "Basic " + java.util.Base64.getEncoder()
                .encodeToString("gateway-svc:s3gr3d0-do-gateway".getBytes(StandardCharsets.UTF_8)))
            .header("Content-Type", "application/x-www-form-urlencoded")
            .POST(HttpRequest.BodyPublishers.ofString(body.toString()))
            .build();
```

```
HttpResponse<String> response = http.send(request, HttpResponse.BodyHandlers.ofString());
Map<String, String> json = SimpleJson.parseFlat(response.body());

if (response.statusCode() != 200) {
    throw new RuntimeException("Token exchange falhou: "
        + json.getOrDefault("error", "erro desconhecido"));
}

// Campo obrigatório e específico desta RFC: informa que TIPO de
// token voltou – pode não ser um access_token comum.
String issuedType = json.get("issued_token_type");
System.out.println("Token recebido é do tipo: " + issuedType);
return json;
}

private static String enc(String v) {
    return URLEncoder.encode(v, StandardCharsets.UTF_8);
}
}
```

Exemplo 2 — delegação explícita, com `actor_token` identificando o gateway

```
import java.net.URI;
import java.net.URLEncoder;
import java.net.http.HttpClient;
import java.net.http.HttpRequest;
import java.net.http.HttpResponse;
import java.nio.charset.StandardCharsets;
import java.util.Map;

public final class DelegatedTokenExchange {

    private static final String GRANT_TYPE_TOKEN_EXCHANGE =
        "urn:ietf:params:oauth:grant-type:token-exchange";
    private static final String TYPE_ACCESS_TOKEN =
        "urn:ietf:params:oauth:token-type:access_token";

    private final HttpClient http = HttpClient.newHttpClient();
    private final String tokenEndpoint;

    public DelegatedTokenExchange(String tokenEndpoint) {
        this.tokenEndpoint = tokenEndpoint;
    }

    // Diferença central em relação ao Exemplo 1: envia actor_token, obtido
    // previamente via Client Credentials Grant (o gateway autenticando a si
    // mesmo, sem usuário envolvido). O AS, reconhecendo essa combinação como
    // delegação autorizada, emite um token cujas claims registram TANTO o
    // subject (usuário) QUANTO o actor (o gateway) – auditável no RS final.
    public Map<String, String> trocarComDelegacao(String subjectToken,
                                                  String actorTokenDoGateway,
                                                  String resourceDestino) throws Exception {

        StringBuilder body = new StringBuilder();
        body.append("grant_type=").append(enc(GRANT_TYPE_TOKEN_EXCHANGE));
        body.append("&subject_token=").append(enc(subjectToken));
        body.append("&subject_token_type=").append(enc(TYPE_ACCESS_TOKEN));
        body.append("&actor_token=").append(enc(actorTokenDoGateway));
        body.append("&actor_token_type=").append(enc(TYPE_ACCESS_TOKEN));
        body.append("&resource=").append(enc(resourceDestino));

        HttpRequest request = HttpRequest.newBuilder()
            .uri(URI.create(tokenEndpoint))
            .header("Authorization", "Basic " + java.util.Base64.getEncoder()
                .encodeToString("gateway-svc:s3gr3d0-do-gateway".getBytes(StandardCharsets.
            .header("Content-Type", "application/x-www-form-urlencoded")
            .POST(HttpRequest.BodyPublishers.ofString(body.toString()))
            .build();

        HttpResponse<String> response = http.send(request, HttpResponse.BodyHandlers.ofString());
        Map<String, String> json = SimpleJson.parseFlat(response.body());

        if (response.statusCode() != 200) {
```

```
        throw new RuntimeException("Token exchange (delegação) falhou: "
            + json.getOrDefault("error", "erro desconhecido"));
    }
    // O resource server que receber este token novo, ao decodificar as
    // claims (fora do escopo deste exemplo – ver capítulo de JWT), veria
    // tanto o subject original quanto uma claim de ator identificando o
    // gateway como quem mediu a chamada.
    return json;
}

private static String enc(String v) {
    return URLEncoder.encode(v, StandardCharsets.UTF_8);
}
}
```

Exemplo 3 — obtendo o `actor_token` do gateway via Client Credentials, antes da delegação

```
import java.net.URI;
import java.net.http.HttpClient;
import java.net.http.HttpRequest;
import java.net.http.HttpResponse;
import java.nio.charset.StandardCharsets;
import java.util.Map;

public final class ObterActorToken {

    // O actor_token não é "inventado" pelo gateway – ele precisa ser um
    // token de segurança de verdade, emitido pelo mesmo AS (ou um AS de
    // confiança), tipicamente via Client Credentials Grant (RFC 6749 §4.4),
    // já que aqui não há usuário nenhum envolvido – é o próprio gateway se
    // autenticando como uma identidade de serviço.
    public static String obterActorTokenDoGateway(String tokenEndpoint,
                                                  String gatewayClientId,
                                                  String gatewayClientSecret) throws Exception {

        HttpClient http = HttpClient.newHttpClient();
        String body = "grant_type=client_credentials&scope=service-identity";

        String credenciais = gatewayClientId + ":" + gatewayClientSecret;
        HttpRequest request = HttpRequest.newBuilder()
            .uri(URI.create(tokenEndpoint))
            .header("Authorization", "Basic " + java.util.Base64.getEncoder()
                .encodeToString(credenciais.getBytes(StandardCharsets.UTF_8)))
            .header("Content-Type", "application/x-www-form-urlencoded")
            .POST(HttpRequest.BodyPublishers.ofString(body))
            .build();

        HttpResponse<String> response = http.send(request, HttpResponse.BodyHandlers.ofString());
        Map<String, String> json = SimpleJson.parseFlat(response.body());
        if (response.statusCode() != 200) {
            throw new RuntimeException("Falha ao obter actor_token: " + json.get("error"));
        }
        return json.get("access_token");
    }
}
```

Exemplo 4 — "downscoping" sem trocar de audience

```
public final class DownscopingExample {

    // Cenário: um serviço interno recebeu um token com escopo amplo
    // ("accounts:read accounts:write transfers:execute"), mas só precisa,
    // para uma chamada específica, do escopo "accounts:read". Em vez de
    // repassar o token amplo adiante (violando least-privilege), troca por
    // um token novo, mesma audience, escopo estritamente mais restrito.
    public static void main(String[] args) throws Exception {
        var client = new TokenExchangeClient("https://as.exemplo.com/token");

        var tokenReduzido = client.trocarParaOutraAudience(
            "TOKEN_AMPLO_DO_SERVICO_A",
            "https://api.exemplo.com/contas", // mesma audience de destino
            "accounts:read" // escopo estritamente mais restrito que o original
        );

        System.out.println("Novo access_token (escopo reduzido): "
            + tokenReduzido.get("access_token"));
        // Este token reduzido é o que deveria ser repassado adiante para
        // qualquer componente que só precise de leitura – nunca o original.
    }
}
```


RFC 9126 — OAuth 2.0 Pushed Authorization Requests (PAR)

1. Contexto e motivação

A requisição de autorização tradicional, exatamente como definida na RFC 6749 §4.1.1, transporta **todos** os seus parâmetros como query string, através do user-agent do usuário (o front-channel): `GET /authorize?response_type=code&client_id=...&redirect_uri=...&scope=...&state=...`. Esse desenho fazia sentido em 2012, quando o conjunto de parâmetros era pequeno e estável. O problema é que o ecossistema OAuth não parou de crescer desde então — cada extensão relevante deste próprio livro adiciona parâmetros à mesma requisição: PKCE acrescenta `code_challenge / code_challenge_method`; Resource Indicators acrescenta `resource`; Rich Authorization Requests (fora do escopo deste roteiro, mas real em produção) acrescenta um `authorization_details` potencialmente extenso. O resultado observável é uma URL que só cresce, e que cresce **exposta exatamente onde este próprio livro, no capítulo da RFC 6750, já ensinou que nada sensível deveria estar**: histórico local do browser, logs de servidor e de proxy corporativo, e o cabeçalho `Referer` — que, dependendo da política de referer do browser e do site de destino, pode vaziar a URL completa (com todos os parâmetros) para terceiros simplesmente por o usuário ter clicado num link a partir daquela página.

Há um segundo problema, estruturalmente mais interessante do que "a URL ficou grande", e que vale entender com precisão porque é a motivação central da RFC, não um efeito colateral: os parâmetros da requisição de autorização, ao chegarem ao Authorization Server via o browser do usuário, **passaram por um canal cuja integridade não está de forma alguma atestada como vinda do client**. O AS recebe uma query string e precisa confiar, sem nenhuma prova criptográfica, que ela reflete exatamente o que o client pretendia enviar. Nada estrutural impede uma adulteração no caminho — uma extensão de browser maliciosa reescrevendo a URL antes do redirect efetivo, uma cadeia de redirecionamento intermediária (comum em cenários de single sign-on corporativo com múltiplos hops) alterando um parâmetro, ou simplesmente um proxy de rede fazendo algum tipo de rewrite não intencional. O AS não tem, na RFC 6749 pura, nenhuma forma de detectar isso — ele processa a query string como recebida, e ponto.

A solução da RFC 9126 (publicada em 2021) inverte a direção do transporte dos parâmetros sensíveis: em vez de deixar que eles viajem pelo browser do usuário até o AS, o client os **empurra** (push) diretamente para um endpoint dedicado do AS, por um canal back-channel **autenticado** — o mesmo tipo de canal, com as mesmas garantias de autenticação de client, que já é usado no token endpoint desde a RFC 6749. Esse push acontece **antes** de qualquer coisa tocar o browser. Em troca do push, o client recebe de volta uma referência curta, opaca e de vida curta — o `request_uri` — que é literalmente **tudo** que vai para a URL que o browser do usuário efetivamente vê e navega.

Vale insistir num ponto que separa PAR de uma simples "otimização de tamanho de URL": o ganho real não é só "a URL ficou menor" — é que a integridade e a autenticidade dos parâmetros passam a depender de um canal HTTPS servidor-a-servidor autenticado por credencial de client, exatamente a

mesma garantia de confiança que o token endpoint já tinha desde sempre, em vez de depender das propriedades — muito mais fracas e variáveis — de um redirect de browser passando por múltiplas camadas de software fora do controle direto do client ou do AS.

2. Terminologia e papéis

Nenhum papel novo em relação à RFC 6749. Vocabulário introduzido:

- **Pushed Authorization Request endpoint** (comumente abreviado "PAR endpoint") — um endpoint inteiramente novo, tipicamente exposto como `/par`, onde o client faz o `POST` autenticado carregando os parâmetros da requisição de autorização.
- `request_uri` — uma referência opaca, de vida curta e **uso único**, devolvida pelo PAR endpoint como resposta ao push. Esse valor substitui completamente a lista de parâmetros na requisição de autorização subsequente feita pelo browser.

Um ponto terminológico que merece atenção antecipada, porque volta a aparecer com mais força no próximo capítulo: o termo `request_uri` **também existe, com o mesmo nome mas semântica diferente**, na RFC 9101 (JAR) — lá, é uma URL hospedada pelo *client*, buscada pelo AS; aqui, é uma referência opaca gerada pelo AS depois de receber o push do *client*. A coincidência de nome não é acidental (os dois conceitos de fato têm um parentesco de propósito — reduzir o que trafega na query string do `/authorize`), mas são mecanismos distintos, com fluxo de geração e uso invertidos. O próximo capítulo deste livro trata dessa distinção com mais detalhe.

3. Fluxo

1. Client → AS: `POST /par` (autenticado — `client_secret`, `mTLS`, `private_key_jwt`...)
 `response_type=code&client_id=...&redirect_uri=...&scope=...&state=...&code_challenge=...&code_challenge_method=S256`
2. AS valida os parametros (mesma validacao que faria num `/authorize` normal) e os guarda server-side
3. AS → Client: { `request_uri`: "urn:ietf:params:oauth:request_uri:6esc_11ACC5bwc014l1tc14eY22c", `exp` }
4. Client → User-Agent: redireciona para
 `GET /authorize?client_id=...&request_uri=urn:ietf:params:oauth:request_uri:6esc_11ACC5bwc014l1tc14eY22c`
5. AS busca os parametros guardados pelo `request_uri`, segue com login/consentimento normalmente
6. ... resto do Authorization Code Grant sem mudanca (redirect com code, troca por token...)

Vale examinar com atenção o que acontece dentro do passo 2, porque é onde o AS efetivamente concentra toda a validação que antes só aconteceria no momento do `/authorize` tradicional — e um ponto de design implícito da RFC é que **essa validação deve acontecer integralmente no momento do push**, não ser adiada para quando o `request_uri` for consumido no passo 5. Isso significa: o AS confere `client_id` válido, `redirect_uri` registrado para aquele client, `response_type` suportado, `scope` dentro do que o client tem permissão de pedir, e (se presente)

`code_challenge_method` aceitável — exatamente as mesmas checagens que uma authorization request tradicional receberia, só que antecipadas. Se alguma dessas checagens falhar, o `POST /par` retorna um erro imediatamente (o mesmo vocabulário de erro do `/authorize`, ex.: `invalid_scope`), e o client nunca chega a redirecionar o usuário — uma melhoria de UX incidental: erros de configuração do client são detectados antes de gastar um redirect do usuário.

O passo 4, por contraste, fica deliberadamente minimalista: a requisição de autorização que efetivamente toca o browser carrega só `client_id` (para o AS saber de qual client é essa referência, e não confiar cegamente na referência sozinha) e `request_uri`. Nenhum outro parâmetro da requisição original aparece ali — nem mesmo redundantemente. O AS, ao processar o passo 5, é quem recupera o conjunto completo de parâmetros a partir do que foi armazenado no passo 2, associado àquele `request_uri` específico.

Um detalhe de ciclo de vida que merece destaque: o `request_uri` é **de uso único**. Isso significa que o AS deve invalidar (apagar ou marcar como consumido) o registro associado assim que o passo 5 o consome pela primeira vez — uma segunda tentativa de `GET /authorize` com o mesmo `request_uri` deve falhar, independentemente de o primeiro uso ter tido sucesso ou não. Esse comportamento espelha exatamente a mesma disciplina de uso único já vista no authorization code da RFC 6749 (RFC 6819 já cataloga replay de code como ameaça relevante) — e não é coincidência: ambos são artefatos de vida curta cujo valor de reuso para um atacante deveria ser sempre zero.

4. Parâmetros e endpoints normativos

- **PAR endpoint** — endpoint novo, descoberto de forma padronizada via RFC 8414 (Authorization Server Metadata) como o campo `pushed_authorization_request_endpoint`.
- **Requisição** (`application/x-www-form-urlencoded`, igual ao `/token`): o mesmo conjunto de parâmetros que uma authorization request tradicional carregaria — `response_type`, `client_id`, `redirect_uri`, `scope`, `state`, e qualquer parâmetro de extensão relevante (`code_challenge` / `code_challenge_method` da PKCE, `resource` da RFC 8707, e assim por diante) — **mais autenticação do client**, usando qualquer um dos métodos já suportados no token endpoint (`client_secret_basic`, `client_secret_post`, `private_key_jwt`, mTLS, conforme o perfil do AS).
- **Resposta** (JSON): dois campos —
- `request_uri` — no formato `urn:ietf:params:oauth:request_uri:<valor opaco>` (o prefixo URN é uma convenção da RFC, não obrigatória em todo detalhe de sintaxe, mas amplamente seguida).
- `expires_in` — tempo de vida em segundos, tipicamente muito curto (a própria RFC cita a ordem de grandeza de ~60 segundos como típica — comparável à vida útil de um authorization code).
- A requisição de autorização subsequente (`/authorize`) fica reduzida, na prática, a `client_id` + `request_uri` — nenhum outro parâmetro deveria estar presente; a RFC deixa como decisão de política do AS como tratar parâmetros extras conflitantes que apareçam junto (a prática recomendada é rejeitar, para não abrir uma ambiguidade sobre qual conjunto de parâmetros é a "fonte da verdade").
- **Erros**: se o `request_uri` apresentado ao `/authorize` for desconhecido, expirado, ou já consumido, o AS deve rejeitar — a terminologia comum em implementações de mercado usa `invalid_request_uri` como valor de erro (não formalmente cunhado com esse nome exato no

corpo principal da RFC 9126, mas amplamente adotado na prática como o análogo natural de "sua referência está velha ou já foi usada").

Um ponto normativo importante para quem avalia adotar PAR como requisito de baseline: a RFC define um metadado adicional, `require_pushed_authorization_requests`, que um AS pode publicar em seu documento de metadado (RFC 8414) para declarar que **exige** PAR para todo client — ou seja, rejeita qualquer tentativa de `/authorize` tradicional com parâmetros soltos na query string, mesmo que tecnicamente válidos. Essa é exatamente a postura que perfis regulatórios como o FAPI 2.0 adotam (ver item 6).

5. Segurança já tratada na própria RFC 9126

- **Integridade por autenticação do canal, não por criptografia do conteúdo:** como o push acontece por um back-channel autenticado (o client se autentica no PAR endpoint com as mesmas credenciais que usaria no token endpoint), o AS tem uma garantia forte — comparável à que já tinha para requisições de token — de que os parâmetros vieram efetivamente do client registrado, não de um terceiro no meio do caminho. Essa é uma garantia estruturalmente diferente e mais forte do que confiar num redirect de browser sujeito a log, cache intermediário, extensão de browser maliciosa, ou simplesmente vazamento incidental antes de a requisição sequer chegar ao AS.
- **Redução de exposição em URL:** PAR generaliza, para **todos** os parâmetros de uma authorization request, o mesmo princípio que a RFC 6750 (Bearer Token Usage) já aplicava especificamente a tokens — "nunca em URL logável". Depois de adotar PAR, nada sensível (nem `scope`, nem `code_challenge`, nem `resource`, nem qualquer parâmetro de extensão futura) trafega numa URL que possa acabar em histórico de browser, log de proxy, ou cabeçalho `Referer`.
- **`request_uri` de uso único e vida curta:** segue exatamente a mesma lógica de mitigação de replay já estabelecida para o authorization code na RFC 6749 — limitar tanto **quantas vezes** (uma) quanto **por quanto tempo** (segundos) um valor capturado poderia ser reaproveitado por um atacante que o interceptasse. Vale uma observação importante sobre o que essa proteção **não** cobre: reutilizar um `request_uri` capturado não concede a um atacante nenhum privilégio novo — na melhor das hipóteses para o atacante, ele só reproduziria o mesmo fluxo já iniciado pelo client legítimo, redirecionando para o mesmo `redirect_uri` já registrado para aquele client (não para um destino arbitrário escolhido pelo atacante). Isso significa que capturar um `request_uri` sozinho não é, por si só, um vetor de sequestro de conta — é mais análogo a "reiniciar" o mesmo login que o usuário legítimo já estava fazendo, o que na prática tem valor de ataque limitado, embora ainda seja corretamente tratado como algo a impedir por disciplina defensiva.
- **PAR reduz, mas não elimina, a motivação para assinar a requisição:** no caso simples (parâmetros não sensíveis, cenário de baixo risco), a integridade que vem da autenticação do canal do PAR já é suficiente, tornando desnecessário o esforço adicional de assinar um Request Object (o mecanismo do próximo capítulo, RFC 9101/JAR). Mas as duas técnicas **não são mutuamente exclusivas** — de fato, foram desenhadas para compor: é possível empurrar, via PAR, um objeto de requisição já assinado (um JWT, no formato da RFC 9101) como o corpo do

push. O resultado combina integridade por assinatura criptográfica e opacidade de URL ao mesmo tempo — o padrão que perfis de alta garantia como FAPI 2.0 adotam como baseline (ver item 6 e o próximo capítulo).

- **PAR não substitui autenticação do resource owner nem consentimento** — é importante não confundir "os parâmetros chegaram com integridade" com "o usuário está protegido". PAR resolve exclusivamente o transporte da requisição de autorização; toda a lógica de login, MFA e tela de consentimento do lado do AS continua inteiramente inalterada pelo passo 5 em diante.

6. Relação com outras RFCs

- Generaliza, para **todos** os parâmetros sensíveis de uma authorization request, o princípio de "nunca em URL logável" já estabelecido na **RFC 6750** especificamente para tokens de portador.
- Companheira direta da **RFC 9101 (JAR)**, próximo capítulo deste livro — as duas endereçam a mesma preocupação de fundo (integridade e tamanho do front-channel da requisição de autorização) por mecanismos diferentes e **combináveis**, não concorrentes: PAR ataca o transporte (esconder do browser, via canal autenticado); JAR ataca o conteúdo (provar que os parâmetros não foram adulterados, via assinatura). A combinação das duas — empurrar um Request Object assinado via PAR — é discutida em detalhe no próximo capítulo.
- Descoberta via **RFC 8414** como `pushed_authorization_request_endpoint`, seguindo o mesmo padrão de descoberta já visto para `revocation_endpoint` (RFC 7009), `introspection_endpoint` (RFC 7662) e `device_authorization_endpoint` (RFC 8628), todos capítulos anteriores deste livro.
- Especialmente relevante em conjunto com a **RFC 7636 (PKCE)**, capítulo anterior deste livro: `code_challenge` / `code_challenge_method` são exatamente o tipo de parâmetro que mais se beneficia de sair da query string visível do browser — combinar PAR com PKCE significa que nem mesmo o challenge (que sozinho já não vaza o verifier, mas ainda é informação de protocolo) fica exposto em log ou histórico.
- Recomendada de forma geral pela **RFC 9700**, mas — e este é um ponto que vale destacar especificamente para um leitor com contexto FSI — **exigida, não apenas recomendada**, por perfis setoriais de alta garantia: o **FAPI 2.0** (especificação da OpenID Foundation, amplamente adotada em open banking e mercados financeiros regulados ao redor do mundo) torna PAR **obrigatório** para qualquer AS conforme o perfil, no mesmo padrão de exigência incondicional já visto no capítulo da RFC 9700 para sender-constraining de tokens (DPoP/mTLS).

7. O que mudou desde a publicação (2021)

- Spec relativamente recente e estável, sem revisão formal desde a publicação — mas com adoção crescendo de forma consistente, puxada em boa parte pela exigência incondicional do FAPI 2.0 em mercados regulados (open banking europeu, brasileiro, e outros que seguem perfis derivados do FAPI).
- PAR está seguindo uma trajetória de consolidação muito parecida com a que a própria PKCE seguiu no capítulo anterior deste livro: de "medida de hardening opcional, adotada seletivamente" para "expectativa de baseline" em Authorization Servers voltados ao público,

especialmente — mas não exclusivamente — em contextos de alta sensibilidade como o financeiro.

- A combinação PAR + JAR ("empurrar um objeto de requisição assinado") deixou de ser uma curiosidade de spec e passou a ser um padrão comum, quase um dado adquirido, em implementações conformes a perfis FAPI — o próximo capítulo detalha exatamente como essa combinação funciona na prática.

Nota sobre o lab: este projeto implementa a RFC 9126 no lab `rfc9126/` — uma réplica autocontida da arquitetura do `rfc6749/` (mesmos containers, convenções e formatos de token do restante do projeto), com o Authorization Code Grant exigindo PAR incondicionalmente: o Authorization Server expõe um endpoint `/par` autenticado, que recebe e valida os parâmetros da authorization request e devolve um `request_uri` (`urn:ietf:params:oauth:request_uri:...`) de uso único; o `/authorize` deste lab **só aceita** requisições que carreguem `client_id` + `request_uri` — não há caminho para os parâmetros tradicionais soltos na query string, refletindo exatamente a postura `require_pushed_authorization_requests` discutida no item 4. O README do lab descreve o exercício mais didático dele: comparar lado a lado, no log de eventos (`./containers/logs.sh events`), o tamanho e o conteúdo da URL de `/authorize` deste lab (mínima, opaca) contra a URL equivalente do lab `rfc6749/` (que carrega `scope`, `redirect_uri` e `state` em claro na própria URL) — tornando a redução de exposição literalmente visível na prática, não só teórica. O lab também demonstra o uso único do `request_uri`: uma segunda tentativa com a mesma referência falha explicitamente com a mensagem "request_uri inválido, expirado ou já utilizado".

Exemplo em Java (JDK 21, sem frameworks)

Os exemplos usam `java.net.http.HttpClient` para as chamadas HTTP e o utilitário `SimpleJson` (métodos `SimpleJson.object(Map)` e `SimpleJson.parseFlat(String)`), definido no Capítulo 1 (RFC 6749) deste livro, para ler a resposta JSON do PAR endpoint.

Exemplo 1 — POST /par autenticado, recebendo request_uri

```
import java.net.URI;
import java.net.URLEncoder;
import java.net.http.HttpClient;
import java.net.http.HttpRequest;
import java.net.http.HttpResponse;
import java.nio.charset.StandardCharsets;
import java.util.Base64;
import java.util.Map;

public final class PushedAuthorizationRequestClient {

    private final HttpClient http = HttpClient.newHttpClient();
    private final String parEndpoint;
    private final String clientId;
    private final String clientSecret;

    public PushedAuthorizationRequestClient(String parEndpoint, String clientId, String clientSecret) {
        this.parEndpoint = parEndpoint;
        this.clientId = clientId;
        this.clientSecret = clientSecret;
    }

    // Empurra todos os parametros de uma authorization request tradicional
    // (RFC 6749 §4.1.1 + PKCE da RFC 7636) por um back-channel AUTENTICADO,
    // em vez de deixa-los trafegar soltos na query string vista pelo browser.
    public Map<String, String> empurrarAuthorizationRequest(String redirectUri,
                                                            String scope,
                                                            String state,
                                                            String codeChallenge) throws Exception {

        StringBuilder body = new StringBuilder();
        body.append("response_type=code");
        body.append("&client_id=").append(enc(clientId));
        body.append("&redirect_uri=").append(enc(redirectUri));
        body.append("&scope=").append(enc(scope));
        body.append("&state=").append(enc(state));
        body.append("&code_challenge=").append(enc(codeChallenge));
        body.append("&code_challenge_method=S256");

        String credenciais = clientId + ":" + clientSecret;
        HttpRequest request = HttpRequest.newBuilder()
            .uri(URI.create(parEndpoint))
            // Autenticação do client no PAR endpoint – mesmos métodos já
            // suportados no token endpoint. Isto é o que dá ao AS a
            // garantia de integridade que substitui a confiança (fraca)
            // no canal de um redirect de browser comum.
            .header("Authorization", "Basic " + Base64.getEncoder()
                .encodeToString(credenciais.getBytes(StandardCharsets.UTF_8)))
            .header("Content-Type", "application/x-www-form-urlencoded")
            .POST(HttpRequest.BodyPublishers.ofString(body.toString()))
            .build();
```



```
HttpResponse<String> response = http.send(request, HttpResponse.BodyHandlers.ofString());
Map<String, String> json = SimpleJson.parseFlat(response.body());

// RFC 9126 §2.2: sucesso é HTTP 201 (Created) – não 200 – porque o
// AS de fato CRIOU um recurso novo (o registro da requisição pushed).
if (response.statusCode() != 201) {
    throw new RuntimeException("PAR endpoint recusou o push: "
        + json.getOrDefault("error", "erro desconhecido"));
}
// { request_uri: "urn:ietf:params:oauth:request_uri:...", expires_in: 60 }
return json;
}

private static String enc(String v) {
    return URLEncoder.encode(v, StandardCharsets.UTF_8);
}
}
```


Exemplo 2 — montar a URL minimalista de `/authorize` a partir do `request_uri`

```
import java.net.URI;
import java.net.URLEncoder;
import java.nio.charset.StandardCharsets;

public final class AuthorizeComRequestUri {

    // Compare esta URL com a do capítulo da RFC 7636: aqui, NADA além de
    // client_id e request_uri aparece na query string que o browser navega.
    // scope, redirect_uri, state, code_challenge – tudo já foi entregue via
    // POST /par autenticado; o browser nunca os vê.
    public static URI montarUrlAutorizacao(String authorizationEndpoint,
                                           String clientId,
                                           String requestUri) {
        String query = "client_id=" + URLEncoder.encode(clientId, StandardCharsets.UTF_8)
            + "&request_uri=" + URLEncoder.encode(requestUri, StandardCharsets.UTF_8);
        return URI.create(authorizationEndpoint + "?" + query);
    }

    public static void main(String[] args) throws Exception {
        var parClient = new PushedAuthorizationRequestClient(
            "https://as.exemplo.com/par", "banco-app-123", "segredo-do-client");

        String codeVerifier = Pkce.generateCodeVerifier(); // reaproveitando o capítulo da RFC 7636
        String codeChallenge = Pkce.computeCodeChallengeS256(codeVerifier);

        var parResponse = parClient.empurrarAuthorizationRequest(
            "https://app.exemplo.com/callback",
            "accounts:read",
            "estado-anti-csrf-abc123",
            codeChallenge);

        URI authorizeUrl = montarUrlAutorizacao(
            "https://as.exemplo.com/authorize",
            "banco-app-123",
            parResponse.get("request_uri"));

        System.out.println("URL minimalista para abrir no user-agent: " + authorizeUrl);
        System.out.println("expires_in do request_uri: " + parResponse.get("expires_in") + "s");
        // A partir daqui, o resto do Authorization Code Grant + PKCE segue
        // exatamente como no capítulo anterior – redirect com code, troca
        // por token enviando o code_verifier original.
    }
}
```

Exemplo 3 — comparando o tamanho da URL antes/depois de PAR

```
public final class ComparacaoTamanhoUrl {

    // Ilustra numericamente o ganho de PAR – útil para justificar a adoção
    // em uma revisão de arquitetura: menos bytes sensíveis trafegando por
    // um canal (browser) que a organização não controla totalmente.
    public static void main(String[] args) {
        String urlTradicional = "https://as.exemplo.com/authorize"
            + "?response_type=code&client_id=banco-app-123"
            + "&redirect_uri=https%3A%2F%2Fapp.exemplo.com%2Fcallback"
            + "&scope=accounts%3Aread%20accounts%3Awrite"
            + "&state=estado-anti-csrf-abc123"
            + "&code_challenge=E9Melhor-code-challenge-fake-para-exemplo"
            + "&code_challenge_method=S256";

        String urlComPar = "https://as.exemplo.com/authorize"
            + "?client_id=banco-app-123"
            + "&request_uri=urn%3Aietf%3Aparams%3Aoauth%3Arequest_uri%3A6esc_11ACC5bwc014l1tc14e";

        System.out.println("Sem PAR : " + urlTradicional.length() + " caracteres");
        System.out.println("Com PAR : " + urlComPar.length() + " caracteres");
        System.out.println("Redução : " + (urlTradicional.length() - urlComPar.length())
            + " caracteres a menos expostos em log/histórico/Referer");
    }
}
```

RFC 9101 — The OAuth 2.0 Authorization Framework: JWT-Secured Authorization Request (JAR)

1. Contexto e motivação

O capítulo anterior deste livro (RFC 9126, PAR) abriu com o mesmo diagnóstico que serve de ponto de partida aqui: a requisição de autorização tradicional (RFC 6749 §4.1.1) manda cada parâmetro como um par chave-valor solto na query string, sem nenhuma verificação de integridade embutida. Vale reforçar exatamente o que isso significa na prática, porque a RFC 9101 ataca essa mesma lacuna por um ângulo diferente do PAR: nada garante ao Authorization Server que os parâmetros que ele recebeu chegaram **exatamente** como o client os montou originalmente. Um intermediário de rede em algum ponto do caminho, uma extensão de browser maliciosa instalada na máquina do usuário, ou uma cadeia de redirecionamento manipulada (comum em cenários de SSO corporativo com múltiplos hops de identidade) poderiam, em tese, alterar um único parâmetro individual — trocar `scope=profile` por `scope=profile+admin`, por exemplo, ou trocar um `redirect_uri` por outro sutilmente diferente — sem que essa adulteração deixe rastro algum detectável pelo AS.

A RFC 9101 (JAR — JWT-Secured Authorization Request, publicada em 2021) resolve esse problema de um jeito estruturalmente diferente do PAR, e vale marcar bem essa diferença porque é fácil confundir as duas abordagens à primeira vista: em vez de **esconder** os parâmetros do browser (a estratégia do PAR — movê-los para um back-channel autenticado), a JAR **empacota todos eles num único JWT assinado pelo próprio client** — o chamado **Request Object** — e envia esse JWT como um parâmetro único na requisição. Qualquer adulteração em qualquer campo, por menor que seja, quebra a assinatura criptográfica, e o AS detecta e rejeita isso deterministicamente. A garantia de integridade aqui vem de **criptografia de chave pública**, não de nenhuma propriedade do canal de transporte por onde a requisição passou — o JWT pode, em teoria, transitar por qualquer canal, confiável ou não, que a assinatura continua provando que o conteúdo não foi alterado desde que o client o assinou.

Essa diferença de fundamento — "confiar porque o canal é autenticado" (PAR) versus "provar criptograficamente, independente do canal" (JAR) — não é só uma curiosidade acadêmica: ela é exatamente o que torna as duas técnicas complementares em vez de concorrentes, um ponto que este capítulo retoma em detalhe no item 6.

2. Terminologia e papéis

Nenhum papel novo em relação à RFC 6749. Vocabulário introduzido pela RFC:

- **Request Object** — um JWT cujas claims **são**, literalmente, os parâmetros de uma authorization request comum (`response_type`, `client_id`, `redirect_uri`, `scope`, `state`, `code_challenge`, `code_challenge_method`, e qualquer outro parâmetro de extensão relevante), somadas às claims

padrão de qualquer JWT: `iss` (o `client_id`, identificando quem assinou), `aud` (identificador do Authorization Server, tipicamente seu `issuer`), e `exp` (expiração — crítico para limitar a janela de replay, ver item 5).

- **request** — o parâmetro de query string que carrega o Request Object **por valor**: o JWT compacto inteiro, codificado, diretamente embutido na URL do `/authorize`.
- **request_uri** — o parâmetro que carrega o Request Object **por referência**: uma URL, hospedada pelo próprio client, que o AS busca (um `GET` feito pelo próprio AS) para obter o JWT.

Atenção dobrada ao nome, retomando o aviso do capítulo anterior: o `request_uri` desta RFC (9101) é **conceitualmente diferente** do `request_uri` da RFC 9126 (PAR), apesar do nome idêntico. Vale a comparação lado a lado, porque a confusão entre os dois é um erro real e recorrente em discussões técnicas:

	RFC 9101 (JAR) <code>request_uri</code>	RFC 9126 (PAR) <code>request_uri</code>
Quem hospeda a URL	O client	O Authorization Server
Quem busca o conteúdo	O AS faz <code>GET</code> na URL do client	O client já recebeu o conteúdo de volta no push; o <code>/authorize</code> só referencia
Como o conteúdo chega lá	O client publica o Request Object numa URL própria, antecipadamente	O AS gera a referência como resposta a um <code>POST /par</code> do client
O que garante integridade	A assinatura do JWT (JAR)	A autenticação do client no momento do push (PAR)

Os dois mecanismos usam o mesmo nome de parâmetro porque, superficialmente, resolvem o mesmo tipo de necessidade ("não colocar tudo na query string do `/authorize`") — mas são artefatos com direção de fluxo invertida e origem de confiança diferente. Isso não impede que sejam combinados (ver item 6) — na verdade, a combinação é o padrão recomendado em perfis de alta garantia.

3. Fluxo

Por valor (`request`):

1. Client monta um JWT com `claims` = todos os parametros da authorization request + `iss`, `aud`, `exp`
2. Client assina o JWT com uma chave registrada no AS
3. Client → User-Agent: `GET /authorize?client_id=...&request=<JWT compacto>`
4. AS valida a assinatura (chave do client, via `jwtks/jwks_uri` registrados), confere `exp/aud/iss`, extrai os parametros das `claims` do JWT (nao da query string solta)
5. Segue com login/consentimento normalmente, usando os parametros ja verificados

Por referência (`request_uri`):

```
1-2. mesma construcao do JWT
3. Client hospeda o JWT numa URL propria (ex.: https://client.exemplo/reqs/abc123)
4. Client → User-Agent: GET /authorize?client_id=...&request_uri=https://client.exemplo/reqs/abc123
5. AS busca o JWT nessa URL, entao valida assinatura/claims como acima
```

Vale examinar com mais cuidado o passo 4 em cada modalidade, porque é onde as duas variantes divergem de forma relevante para uma decisão de arquitetura:

No modo **por valor**, o JWT compacto inteiro (header.payload.assinatura, tudo codificado em Base64URL) vai embutido na própria URL do `/authorize`. Isso resolve o problema de integridade (a assinatura protege o conteúdo), mas **não** resolve o problema de tamanho de URL que motivou o capítulo anterior — na verdade, um Request Object assinado tende a ser **maior** do que os parâmetros soltos equivalentes, porque carrega overhead de JWT (header, claims padrão, a assinatura em si, que para RSA facilmente passa de 340 bytes codificados). Ou seja: por valor resolve integridade, mas piora exposição de URL.

No modo **por referência**, a URL do `/authorize` fica pequena (só `client_id` + a URL de referência do client) — só que agora o AS precisa fazer, ele mesmo, uma requisição HTTP de saída para buscar o conteúdo, o que introduz uma superfície de risco nova e específica: SSRF (ver item 5). Além disso, essa variante introduz uma dependência operacional que o modo por valor não tem — o client precisa manter um endpoint disponível e responsivo hospedando o Request Object, para o AS conseguir buscá-lo no momento exato do passo 5; se esse endpoint estiver fora do ar, o fluxo de autorização inteiro falha ali.

O passo de validação (4, em ambas as variantes) é onde o AS precisa ser rigoroso: validar a assinatura contra a chave pública correta do client (obtida via `jwks` embutido no registro do client, ou buscada dinamicamente via `jwks_uri`), conferir que `exp` ainda não passou, e idealmente confirmar que `aud` de fato identifica este AS especificamente (evitando que um Request Object assinado para um AS seja reaproveitado contra outro, num cenário de múltiplos ASs confiando na mesma chave do client — incomum, mas não impossível de configurar incorretamente). Só depois dessa validação completa o AS deveria tratar as claims do JWT como a fonte autoritativa dos parâmetros da requisição — nunca misturar parâmetros da query string solta (fora do JWT) com os das claims verificadas, para não abrir uma brecha onde um atacante injeta um parâmetro adicional não coberto pela assinatura.

4. Parâmetros e endpoints normativos

Nenhum endpoint novo — ao contrário da RFC 9126, que introduz o `/par` como endpoint dedicado, a JAR reaproveita o `/authorize` já existente inteiramente, só adicionando dois parâmetros possíveis de query string: `request` e `request_uri` (mutuamente exclusivos — a RFC não define comportamento de usar os dois juntos, e a prática recomendada é o AS rejeitar se ambos vierem presentes).

- **Claims do Request Object:** `iss` (= `client_id`), `aud` (= identificador do AS, tipicamente o `issuer`), `exp` (padrão JWT, obrigatório para limitar replay) — mais **todos** os parâmetros normais de uma authorization request, como claims adicionais no mesmo nível (não aninhados):

`response_type`, `client_id` (novamente, agora como `claim`), `redirect_uri`, `scope`, `state`, `code_challenge`, `code_challenge_method`, `resource`, e qualquer outro parâmetro de extensão relevante para o AS em questão.

- **Descoberta da chave de assinatura do client:** via `jwks` (o conjunto de chaves públicas embutido diretamente no registro do client, útil para clients com chave estática) ou `jwks_uri` (uma URL que o AS busca dinamicamente para obter o conjunto de chaves, útil para rotação de chave sem precisar re-registrar o client) — ambos campos fazem parte do metadata de registro do client, tema da RFC 7591 (Dynamic Client Registration), categoria de RFCs de descoberta/registro deste roteiro.
- **Validação obrigatória do lado do AS:** verificar a assinatura contra a chave correta, verificar `exp`, e — prática fortemente recomendada, embora a redação da RFC deixe alguma margem — conferir `aud`. Para o modo por referência especificamente, a RFC recomenda que o AS **restringa de onde busca** o `request_uri` (ver item 5).
- **Metadados de descoberta relacionados (RFC 8414):** `request_parameter_supported` (o AS aceita `request`?), `request_uri_parameter_supported` (o AS aceita `request_uri`?), `require_signed_request_object` (o AS exige que toda `authorization request` venha como `Request Object` assinado, recusando parâmetros soltos — a postura equivalente, para JAR, ao `require_pushed_authorization_requests` do capítulo anterior).
- **Erros:** a RFC reaproveita, na medida do possível, o vocabulário de erro já existente do `/authorize` (`invalid_request`, `invalid_request_object` sendo um valor comumente adotado na prática para especificamente sinalizar falha de validação do JWT — assinatura inválida, `exp` vencido, `claims` malformadas).

5. Segurança já tratada na própria RFC 9101

- **Integridade por assinatura, garantia mais forte e mais geral que a do PAR:** mesmo que o canal de transporte da requisição não seja confiável em nenhum ponto do caminho, qualquer adulteração no conteúdo quebra a assinatura de forma detectável — não importa onde ou como a adulteração ocorreu. Vale comparar explicitamente com o mecanismo do capítulo anterior: a PAR confia na integridade **porque o canal do push é autenticado** (uma garantia de "confiança no transporte"); a JAR **prova** a integridade criptograficamente, de forma independente das propriedades de qualquer canal por onde o JWT passe depois de assinado (uma garantia de "prova no conteúdo"). Isso torna a JAR aplicável mesmo em cenários onde o `Request Object` precisaria trafegar por canais que o client não controla totalmente (por exemplo, embutido numa URL que passa por múltiplos redirecionamentos de terceiros antes de chegar ao AS).
- **Mitigação de replay via `exp` (e, opcionalmente, `jti`):** a `claim exp` limita por quanto tempo um `Request Object` capturado poderia ser reaproveitado — um AS bem-comportado rejeita qualquer `Request Object` cujo `exp` já tenha passado, mesmo que a assinatura seja perfeitamente válida. Para AS que queiram impor também uso único (não só limite de tempo), a RFC permite o uso de uma `claim jti` (JWT ID) opcional, que o AS pode rastrear e recusar em caso de reapresentação — análogo em espírito ao uso único do `request_uri` da PAR, mas aqui é uma escolha de política adicional do AS, não uma exigência normativa central da spec.

- **Risco de SSRF no modo por referência — o ponto de segurança mais discutido desta RFC na prática:** como é o **próprio Authorization Server** quem faz a requisição HTTP de saída para buscar o conteúdo apontado por `request_uri`, um client malicioso (ou um valor de `request_uri` não devidamente vinculado e validado contra o client registrado) poderia, em tese, induzir o AS a fazer requisições a URLs arbitrárias — potencialmente endereços internos da rede do próprio AS, serviços de metadata de cloud (o clássico alvo de SSRF em ambientes AWS/GCP/Azure), ou qualquer outro destino que o atacante queira que o AS "visite" com a autoridade de rede do próprio AS. A mitigação recomendada, e que qualquer AS de produção deveria implementar como não-negociável: **restringir de quais hosts/padrões o AS aceita buscar `request_uri`**, tipicamente exigindo que a URL bata com um padrão pré-registrado especificamente para aquele client (não aceitar qualquer URL arbitrária que a requisição de autorização apresente). Este é, de longe, o risco mais citado em análises de segurança desta RFC, e a razão pela qual muitas implementações de AS optam por só suportar o modo por valor, ou por só suportar `request_uri` combinado com PAR (ver abaixo), evitando o SSRF por completo ao nunca deixar o client apontar para uma URL arbitrária de sua escolha.
- **JAR sozinha não dá confidencialidade — só integridade:** um ponto que merece ênfase porque é fácil supor o contrário. O Request Object, na forma básica descrita pela RFC, é um JWS (JSON Web Signature) — **assinado, não criptografado**. Qualquer parte que consiga observar o `request` ou o conteúdo apontado por `request_uri` (um proxy, um log, o histórico de browser no modo por valor) ainda lê `scope`, `redirect_uri`, e qualquer outro parâmetro em **texto claro** — a assinatura impede adulteração não detectada, não impede leitura. Quem precisa também de confidencialidade (esconder o conteúdo, não só protegê-lo de adulteração) precisa adicionalmente **criptografar** o Request Object como um JWE (JSON Web Encryption) — uma combinação (assinar e depois criptografar, "sign-then-encrypt", ou o inverso conforme o perfil) comum em implementações FAPI de alta garantia, mas que a RFC 9101 básica não exige por si só.
- **Confiança na chave do client precisa ser gerenciada com cuidado:** se o `jwks_uri` do client for buscado dinamicamente pelo AS a cada validação (em vez de cacheado com uma política de expiração razoável), isso introduz uma dependência de disponibilidade nova (o AS depende do endpoint do client estar no ar) e uma superfície adicional (o mesmo tipo de risco de host não confiável do `request_uri`, embora aqui o conteúdo buscado — um conjunto de chaves públicas — seja normalmente menos sensível de adulterar do que redirecionar o próprio AS para um destino arbitrário).

6. Relação com outras RFCs

- Companheira direta da **RFC 9126 (PAR)**, capítulo anterior deste livro — e aqui está a combinação mais importante deste par de capítulos, que vale detalhar passo a passo porque é exatamente o padrão que perfis de alta garantia adotam como baseline: o client (1) monta o Request Object — o JWT com todos os parâmetros de autorização como claims (o mecanismo desta RFC, JAR); (2) assina esse JWT com sua chave privada; (3) empurra o JWT assinado inteiro como o corpo de um `POST /par` (o mecanismo do capítulo anterior, PAR), autenticando-se no PAR endpoint da forma usual; (4) recebe de volta o `request_uri` **opaco gerado pelo AS** (o `request_uri` da RFC 9126, não desta RFC); (5) usa esse `request_uri` opaco no `/authorize`. O resultado combina as duas garantias ao mesmo tempo: os parâmetros ficam **escondidos do**

browser (confidencialidade de transporte, cortesia da PAR) e são verificavelmente autênticos e íntegros (cortesia da assinatura da JAR) — e, como efeito colateral bem-vindo, essa combinação especificamente **evita o risco de SSRF** discutido no item 5, porque o AS nunca precisa buscar um `request_uri` hospedado pelo client — o Request Object já chegou inteiro, assinado, no próprio corpo do push.

- A chave de assinatura do client (via `jwks / jwks_uri`) vem do metadata de registro estabelecido pela **RFC 7591 (Dynamic Client Registration)**, categoria de RFCs de descoberta/registro deste roteiro.
- Há um paralelo conceitual direto com **RFC 7523** (uso de `private_key_jwt` para autenticação de client no token endpoint) — a mesma ideia estrutural (um JWT assinado pela chave privada do client, provando que só o client poderia tê-lo produzido), aplicada a um propósito diferente: a RFC 7523 autentica o client no momento da troca de token; a RFC 9101 protege a integridade da requisição de autorização em si. Vale reconhecer o padrão comum às duas: "JWT assinado pelo client" é um primitivo reutilizável do ecossistema OAuth, aplicado em pontos distintos do fluxo conforme a necessidade.
- Descoberta via **RFC 8414**: os campos `request_parameter_supported`, `request_uri_parameter_supported` e `require_signed_request_object` permitem a um client (ou a uma ferramenta de auditoria de conformidade) verificar programaticamente se e como um AS específico suporta JAR, sem precisar de documentação fora de banda.
- Fortemente usada — e, na prática, frequentemente **exigida** — em perfis **FAPI** (FAPI 1.0 Advanced e, de forma ainda mais consolidada, FAPI 2.0), quase sempre já combinada com PAR exatamente no padrão descrito acima. Um arquiteto avaliando conformidade FAPI para um AS ou client deveria tratar "PAR + JAR combinados" como a unidade de análise relevante, não as duas specs isoladamente.

7. O que mudou desde a publicação (2021)

- Spec relativamente recente e estável, sem revisão formal — mas seguiu uma trajetória de adoção **diferente** da do PKCE (capítulo anterior deste livro), e vale marcar bem essa diferença para não generalizar incorretamente o padrão de "toda extensão de segurança vira baseline universal com o tempo": a JAR **não** se tornou obrigatória de forma ampla e incondicional via RFC 9700 — ao contrário do PKCE, que passou de "recomendado para públicos" para "obrigatório para todos". A JAR continua sendo, mesmo hoje, uma ferramenta de **"alta garantia"**, adotada seletivamente onde a organização (ou o regulador) exige integridade estrutural mais forte do que o baseline geral — setor financeiro sendo o exemplo mais citado — em vez de um requisito de linha de base esperado de qualquer AS público, do jeito que PKCE se tornou.
- A combinação PAR + JAR se consolidou, na prática de mercado, como o padrão de fato em implementações conformes a FAPI 2.0 — reforçando a leitura de que essas duas RFCs se entendem melhor, e são adotadas melhor, como um par desenhado para compor, do que isoladamente uma da outra.
- Vale reter, como conclusão de todo este capítulo de Extensões de Grant Type: das cinco RFCs cobertas nesta parte do livro, PKCE é hoje universal (todo client, todo cenário); Device Authorization Grant e Token Exchange são adotados conforme a necessidade arquitetural específica (dispositivo sem browser; cadeias de chamada multi-serviço); e PAR/JAR representam

o par de extensões de "alta garantia", cuja adoção — ainda que crescente — permanece concentrada onde o perfil de risco ou a exigência regulatória especificamente demanda esse nível adicional de rigor.

Nota sobre laboratório: este projeto não possui, até o momento, um laboratório dedicado para a RFC 9101. O cenário natural, e um bom candidato futuro, seria estender o lab já existente `rfc9126/` (que implementa PAR) para também assinar o Request Object antes de empurrá-lo — exercitando exatamente a combinação PAR + JAR descrita no item 6 deste capítulo, e mostrando as duas proteções (opacidade de transporte e integridade por assinatura) lado a lado, na prática, dentro da mesma dança de eventos HTTP que os demais labs do projeto já expõem. Por ora, este capítulo se concentra nos exemplos em Java abaixo, que constroem, assinam e transmitem um Request Object do zero, usando somente a biblioteca padrão do JDK.

Exemplo em Java (JDK 21, sem frameworks)

Os exemplos usam `java.security.KeyPairGenerator` para gerar um par de chaves RSA, `java.security.Signature` (algoritmo `SHA256withRSA`) para assinar, `java.util.Base64` para as codificações Base64URL exigidas pelo formato JWT compacto, e o utilitário `SimpleJson` (método `SimpleJson.object(Map)`), definido no Capítulo 1 (RFC 6749) deste livro, para serializar as claims como JSON antes de codificá-las. Nenhuma biblioteca de JOSE/JWT é usada — a montagem do JWT é feita manualmente, passo a passo, para deixar explícito o que uma biblioteca como Nimbus JOSE normalmente esconde.

Exemplo 1 — gerar o par de chaves RSA do client

```
import java.security.KeyPair;
import java.security.KeyPairGenerator;
import java.security.NoSuchAlgorithmException;

public final class ClientKeyPair {

    // Em produção, este par seria gerado uma vez e persistido com segurança
    // (HSM, keystore protegido) – nunca gerado a cada requisição. Aqui,
    // geramos em memória só para tornar o exemplo autocontido.
    public static KeyPair gerarParRsa() {
        try {
            KeyPairGenerator generator = KeyPairGenerator.getInstance("RSA");
            generator.initialize(2048); // 2048 bits: piso aceitável para RSA hoje
            return generator.generateKeyPair();
        } catch (NoSuchAlgorithmException e) {
            throw new IllegalStateException("Algoritmo RSA indisponível na JVM", e);
        }
    }
}
```

Exemplo 2 — montar e assinar o Request Object (JWT compacto, manualmente)

```
import java.nio.charset.StandardCharsets;
import java.security.KeyPair;
import java.security.Signature;
import java.time.Instant;
import java.util.Base64;
import java.util.LinkedHashMap;
import java.util.Map;

public final class RequestObjectBuilder {

    private static final Base64.Encoder B64URL = Base64.getUrlEncoder().withoutPadding();

    // Monta o Request Object: um JWT cujas claims SÃO os parâmetros da
    // authorization request (RFC 6749 §4.1.1 + PKCE), somados às claims
    // padrão iss/aud/exp exigidas pela RFC 9101 §4.
    public static String montarEAssinar(String clientId, String audienceAs,
                                         String redirectUri, String scope, String state,
                                         String codeChallenge, KeyPair chaveDoClient) throws Exception {

        // --- Header: {"alg":"RS256","typ":"JWT"} ---
        Map<String, Object> header = new LinkedHashMap<>();
        header.put("alg", "RS256");
        header.put("typ", "JWT");
        String headerJson = SimpleJson.object(header);
        String headerB64 = B64URL.encodeToString(headerJson.getBytes(StandardCharsets.UTF_8));

        // --- Payload: claims padrao de JWT + parametros da authorization request ---
        long agora = Instant.now().getEpochSecond();
        Map<String, Object> claims = new LinkedHashMap<>();
        claims.put("iss", clientId); // quem assinou o Request Object
        claims.put("aud", audienceAs); // para qual AS esta requisicao se destina
        claims.put("exp", agora + 60); // vida curta: 60s, limita a janela de replay
        claims.put("response_type", "code");
        claims.put("client_id", clientId);
        claims.put("redirect_uri", redirectUri);
        claims.put("scope", scope);
        claims.put("state", state);
        claims.put("code_challenge", codeChallenge);
        claims.put("code_challenge_method", "S256");

        String payloadJson = SimpleJson.object(claims);
        String payloadB64 = B64URL.encodeToString(payloadJson.getBytes(StandardCharsets.UTF_8));

        // --- Assinatura sobre "header.payload" (a "signing input" do JWT) ---
        String signingInput = headerB64 + "." + payloadB64;
        Signature assinador = Signature.getInstance("SHA256withRSA");
        assinador.initSign(chaveDoClient.getPrivate());
        assinador.update(signingInput.getBytes(StandardCharsets.US_ASCII));
        byte[] assinatura = assinador.sign();
        String assinaturaB64 = B64URL.encodeToString(assinatura);
    }
}
```

```

        // JWT compacto: header.payload.assinatura
        return signingInput + "." + assinaturaB64;
    }
}

```

Exemplo 3 — enviar o Request Object por valor (request=)

```

import java.net.URI;
import java.net.URLEncoder;
import java.nio.charset.StandardCharsets;
import java.security.KeyPair;

public final class EnviarPorValor {

    public static void main(String[] args) throws Exception {
        KeyPair chaveDoClient = ClientKeyPair.gerarParRsa();
        // Em produção, a chave PÚBLICA correspondente seria registrada
        // previamente no AS (via jwks ou jwks_uri no cadastro do client,
        // RFC 7591) – é contra ela que o AS valida a assinatura no passo 4.

        String verifier = Pkce.generateCodeVerifier(); // reaproveitando o capítulo da RFC 7636
        String challenge = Pkce.computeCodeChallengeS256(verifier);

        String requestObject = RequestObjectBuilder.montarESassinar(
            "banco-app-123",
            "https://as.exemplo.com", // aud: identifica o AS
            "https://app.exemplo.com/callback",
            "accounts:read",
            "estado-anti-csrf-abc123",
            challenge,
            chaveDoClient);

        // O JWT inteiro (grande, mas assinado) vai como o valor do
        // parametro `request` – note que isto NAO reduz o tamanho da URL,
        // so garante que qualquer adulteracao no caminho quebra a assinatura.
        String query = "client_id=" + URLEncoder.encode("banco-app-123", StandardCharsets.UTF_8)
            + "&request=" + URLEncoder.encode(requestObject, StandardCharsets.UTF_8);

        URI authorizeUrl = URI.create("https://as.exemplo.com/authorize?" + query);
        System.out.println("Abrir no user-agent (modo por valor): " + authorizeUrl);
        System.out.println("Tamanho do Request Object (JWT compacto): "
            + requestObject.length() + " caracteres");
    }
}

```

Exemplo 4 — enviar por referência (request_uri=) e combinar com PAR (FAPI 2.0)

```
import java.net.URI;
import java.net.URLEncoder;
import java.nio.charset.StandardCharsets;

public final class EnviarPorReferenciaOuComPar {

    // Modo por referência "puro" (RFC 9101 sozinha): o client hospeda o JWT
    // numa URL própria, e o AS é quem faz o GET para buscá-lo. Note o
    // request_uri aqui É uma URL do CLIENT – o oposto do request_uri da
    // RFC 9126 (PAR), que é uma referência OPACA gerada pelo AS. Um AS de
    // producao so deveria aceitar isto se a URL bater com um padrao
    // pre-registrado para este client (mitigacao de SSRF, ver item 5).
    public static URI montarUrlPorReferencia(String clientId, String requestObjectUrlDoClient) {
        String query = "client_id=" + URLEncoder.encode(clientId, StandardCharsets.UTF_8)
            + "&request_uri=" + URLEncoder.encode(requestObjectUrlDoClient, StandardCharsets.UTF_8);
        return URI.create("https://as.exemplo.com/authorize?" + query);
    }

    // Combinação recomendada por perfis FAPI 2.0: em vez de hospedar o JWT
    // numa URL propria (arriscando SSRF do lado do AS), o client empurra o
    // Request Object assinado via PAR (capítulo anterior) e usa o
    // request_uri OPACO devolvido pelo AS – nunca uma URL escolhida pelo
    // client. Isto some as duas protecoes (integridade da JAR + opacidade
    // da PAR) e elimina o risco de SSRF por completo.
    public static void main(String[] args) throws Exception {
        var chaveDoClient = ClientKeyPair.gerarParRsa();
        String verifier = Pkce.generateCodeVerifier();
        String challenge = Pkce.computeCodeChallengeS256(verifier);

        String requestObject = RequestObjectBuilder.montarESassinar(
            "banco-app-123", "https://as.exemplo.com",
            "https://app.exemplo.com/callback", "accounts:read",
            "estado-anti-csrf-abc123", challenge, chaveDoClient);

        // Passo PAR (capítulo anterior): empurra o Request Object assinado
        // inteiro como corpo do POST /par autenticado. Aqui o corpo do push
        // NAO é form-urlencoded parametro-a-parametro – é o proprio JWT,
        // enviado atraves do parametro "request" dentro do corpo do /par.
        var parClient = new PushedAuthorizationRequestClient(
            "https://as.exemplo.com/par", "banco-app-123", "segredo-do-client");

        // (Ilustrativo: em uma implementacao real, o metodo do PAR client
        // seria adaptado para aceitar o JWT pronto em vez de parametros
        // soltos – omitido aqui por brevidade, o ponto central é a ordem:
        // JAR monta e assina; PAR transporta; o resultado é um request_uri
        // OPACO, gerado pelo AS, usado no /authorize final.)
        System.out.println("Request Object assinado, pronto para push via PAR: "
            + requestObject.substring(0, 40) + "...(truncado)");
        System.out.println("Padrao FAPI 2.0: JAR (integridade) + PAR (opacidade) combinados.");
    }
}
```

```
}  
}
```

Parte 4 — Metadata e Registro de Clientes

RFC 8414 — OAuth 2.0 Authorization Server Metadata

1. Contexto e motivação

Todo protocolo de integração cliente-servidor enfrenta, mais cedo ou mais tarde, o mesmo problema de fundo: como o cliente sabe *onde* falar com o servidor? Em OAuth 2.0 "puro" (RFC 6749), a resposta original era simplesmente "fora de banda" — o desenvolvedor do client lê a documentação do provedor, copia manualmente as URLs de `/authorize` e `/token` para um arquivo de configuração, e torce para que elas não mudem. Essa abordagem sobrevive razoavelmente bem quando existe **um** provedor fixo e conhecido (o client foi escrito especificamente para ele), mas começa a ranger nas costuras assim que o ecossistema cresce em qualquer uma destas direções:

- **Bibliotecas OAuth genéricas.** Uma biblioteca cliente que pretenda suportar "qualquer Authorization Server compatível" não pode hardcodar URL nenhuma — ela precisa de um mecanismo em tempo de execução para aprender a topologia do AS específico que está sendo integrado.
- **Crescimento orgânico de endpoints.** RFC 6749 definiu só `/authorize` e `/token`. Depois vieram `/revoke` (RFC 7009), `/introspect` (RFC 7662), `/device_authorization` (RFC 8628), `/par` (RFC 9126), e a lista não para de crescer. Sem um documento central, cada nova capacidade exigiria seu próprio mecanismo de descoberta ad hoc, ou pior, mais um parâmetro de configuração manual que alguém vai esquecer de preencher.
- **Multi-tenancy e ambientes federados.** Em cenários com múltiplos "issuers" lógicos atrás da mesma infraestrutura (ex.: um AS multi-tenant, um por cliente/organização), cada tenant pode ter endpoints ligeiramente diferentes — path prefixado por tenant, conjunto de scopes distinto, algoritmos de assinatura habilitados diferentes. Configuração estática por tenant é operacionalmente inviável em escala.
- **Rotação e migração de infraestrutura.** Qualquer mudança de URL no AS — uma migração de domínio, uma reorganização de path, a introdução de um load balancer com path diferente — quebra, silenciosamente, toda configuração estática espalhada pelos clients. Não há como avisar "atualizem suas configurações" de forma centralizada quando a configuração é, por definição, distribuída.

A RFC 8414 (publicada em 2018, mas formalizando uma prática já comum informalmente) resolve isso com um **documento de descoberta**: um único JSON, publicado numa URL *well-known* padronizada, que descreve de uma vez todos os endpoints e capacidades relevantes do AS. Em vez de N configurações manuais (uma por endpoint, replicada em cada client), existe **um** ponto de verdade que qualquer client pode consultar em tempo de execução.

Vale explicitar a genealogia: esse padrão não nasceu com a 8414. O OpenID Connect Discovery — que expõe `/.well-known/openid-configuration` — é anterior e já resolvia exatamente esse problema para o mundo OIDC. A RFC 8414 é, essencialmente, a extração desse mesmo padrão para o OAuth 2.0 "puro", sem exigir a pilha OIDC inteira (sem exigir `id_token`, sem exigir um `userinfo_endpoint`, sem as claims

de identidade). Isso significa que muita gente que já lida com OIDC reconhece a 8414 imediatamente — os nomes de campo são, em grande parte, os mesmos, e não por acaso: a RFC 8414 foi desenhada deliberadamente para que os dois documentos pudessem, na prática, ser o **mesmo objeto JSON** quando um AS suporta ambos os protocolos (um AS que também é OpenID Provider tipicamente publica o mesmo conteúdo, ou um superconjunto dele, em ambos os paths well-known).

2. Terminologia e papéis

A RFC 8414 não introduz nenhum papel novo em relação ao vocabulário OAuth já estabelecido (Resource Owner, Client, Authorization Server, Resource Server) — ela introduz apenas conceitos ligados ao próprio mecanismo de descoberta:

- **Authorization Server Metadata** — o documento JSON em si: um conjunto de pares chave-valor descrevendo endpoints, capacidades e políticas suportadas pelo AS.
- **Well-Known URI** — o mecanismo de localização do documento, definido pela RFC 8615 (*Well-Known Uniform Resource Identifiers*), da qual a 8414 é consumidora. O caminho padronizado é `/.well-known/oauth-authorization-server`.
- **issuer** — de longe o campo mais importante do documento, e o único genuinamente obrigatório. É uma URL (sem *fragment*, tipicamente sem *query*) que identifica o AS de forma única e global. Esse valor não é decorativo: ele precisa bater, byte a byte, com qualquer lugar onde o AS se identifica como emissor — o claim `iss` de um JWT (RFC 9068, tokens JWT introspectáveis), o `aud` esperado de um Request Object assinado (RFC 9101), ou o valor usado para localizar o próprio documento de metadata (ver §5 abaixo — essa auto-referência é o cerne da mitigação de mix-up attack).

2.1 A regra de composição do path well-known

Um detalhe que confunde bastante gente na primeira leitura, porque é contraintuitivo comparado a outras convenções de URL: quando o `issuer` **tem um path** (por exemplo, um AS multi-tenant publicado em `https://as.exemplo.com/tenant1`), o segmento `.well-known/oauth-authorization-server` é **inserido antes** do path do issuer, não anexado depois dele.

<code>issuer:</code>	<code>https://as.exemplo.com/tenant1</code>
<code>well-known CORRETO:</code>	<code>https://as.exemplo.com/.well-known/oauth-authorization-server/tenant1</code>
<code>well-known ERRADO:</code>	<code>https://as.exemplo.com/tenant1/.well-known/oauth-authorization-server</code>

Essa regra existe porque `.well-known` é, por definição da RFC 8615, um path reservado no **nível da origem** (scheme + host + porta) — não algo que se relativiza a um subpath arbitrário. A justificativa prática é operacional: um servidor HTTP típico já tem uma rota fixa para tudo que começa com `/.well-known/`, independentemente de quantos tenants ou subpaths existam depois. Se a convenção fosse "anexar depois", cada tenant precisaria de sua própria rota `.well-known` configurada separadamente — voltando ao mesmo problema de escala que a RFC tenta resolver. Um AS de tenant único, sem path no `issuer` (o caso mais comum, e o caso do lab deste livro), não precisa se preocupar com essa regra: o well-known fica simplesmente em `https://as.exemplo.com/.well-known/oauth-authorization-server`.

2.2 Metadata "genuína" vs. metadata "descritiva"

Vale uma distinção conceitual que a RFC deixa implícita: alguns campos do documento são **descritivos** (informam algo que já é verdade independentemente do documento — ex.: `token_endpoint`, que é só "onde bater"), enquanto outros são **normativos sobre política** (`require_pushed_authorization_requests: true`, por exemplo, muda o comportamento exigido do client). Um client bem-comportado deve tratar campos de política com o mesmo peso que teria uma exigência hardcoded — ignorá-los porque "são só metadata" é um erro de interpretação comum em implementações apressadas.

3. Fluxo

Não é, estritamente falando, um "fluxo" OAuth no sentido de trocar código de autorização por token — é uma consulta HTTP simples, pública, sem estado e sem autenticação:

```
GET /.well-known/oauth-authorization-server HTTP/1.1
Host: as.exemplo.com

HTTP/1.1 200 OK
Content-Type: application/json

{
  "issuer": "https://as.exemplo.com",
  "authorization_endpoint": "https://as.exemplo.com/authorize",
  "token_endpoint": "https://as.exemplo.com/token",
  "registration_endpoint": "https://as.exemplo.com/register",
  "revocation_endpoint": "https://as.exemplo.com/revoke",
  "introspection_endpoint": "https://as.exemplo.com/introspect",
  "jwks_uri": "https://as.exemplo.com/jwks.json",
  "scopes_supported": ["profile", "email", "offline_access"],
  "response_types_supported": ["code"],
  "grant_types_supported": ["authorization_code", "refresh_token", "client_credentials"],
  "token_endpoint_auth_methods_supported": ["client_secret_basic", "private_key_jwt"],
  "code_challenge_methods_supported": ["S256"],
  "pushed_authorization_request_endpoint": "https://as.exemplo.com/par",
  "require_pushed_authorization_requests": false,
  "device_authorization_endpoint": "https://as.exemplo.com/device_authorization"
}
```

O consumidor típico deste documento não é um humano lendo no browser (embora isso também funcione, já que é um GET público) — é código de biblioteca cliente que, na inicialização, busca esse JSON uma vez, faz cache dele por um período razoável, e a partir daí resolve `token_endpoint` / `authorization_endpoint` /etc. dinamicamente em vez de tê-los hardcoded. É esse padrão — "configure só o issuer, a biblioteca descobre o resto" — que se tornou o modo dominante de configurar integrações OAuth/OIDC em produção bem depois de 2018 (ver §7).

3.1 O que acontece quando o documento não existe

A RFC não prescreve um comportamento único para "AS não publica metadata" — mas na prática, um client que recebe `404 Not Found` na URL well-known tem duas saídas legítimas: (a) cair de volta para configuração manual dos endpoints (o modo "pré-8414", ainda plenamente válido), ou (b) tratar isso como erro fatal de integração, se o client foi desenhado para depender inteiramente de descoberta. Ambas são escolhas de implementação, não normas da RFC.

4. Parâmetros e endpoints normativos

- **A well-known URI é o próprio "endpoint"** — não há verbo além de `GET`, não há autenticação (o documento é público por design — ver §5), e a resposta é sempre `application/json`.
- **issuer** — único campo genuinamente **obrigatório** em todo o documento. Todos os demais são opcionais na letra da RFC, ainda que praticamente universais na prática (nenhum AS sério publica um documento de metadata sem `token_endpoint`, por exemplo, mesmo que tecnicamente a RFC permitisse).
- Campos que aparecem ao longo de praticamente todo o roteiro deste livro, cada um sendo a forma "descoberta" de algo que, em RFCs anteriores, era configuração manual:
- `authorization_endpoint`, `token_endpoint` — os dois endpoints fundacionais (RFC 6749).
- `jwks_uri` — onde buscar as chaves públicas do AS para validar assinaturas (usado por RFC 9068, RFC 9101, JWT-based client auth).
- `scopes_supported` — lista informativa (não normativa em termos de enforcement) dos scopes que o AS reconhece.
- `response_types_supported`, `grant_types_supported` — quais fluxos o AS de fato implementa; um client pode usar isso para decidir, em runtime, se um determinado grant type está disponível antes de tentar usá-lo.
- `token_endpoint_auth_methods_supported` — quais métodos de autenticação de client o `/token` aceita (`client_secret_basic`, `client_secret_post`, `private_key_jwt`, `none`, `tls_client_auth` — RFC 8705).
- `code_challenge_methods_supported` — como o AS anuncia suporte a PKCE (RFC 7636) e com quais métodos de transformação (tipicamente só `S256`; `plain` é desencorajado e alguns ASs nem o anunciam mesmo aceitando-o por retrocompatibilidade).
- `revocation_endpoint` (RFC 7009), `introspection_endpoint` (RFC 7662), `device_authorization_endpoint` (RFC 8628), `pushed_authorization_request_endpoint` + `require_pushed_authorization_requests` (RFC 9126), `request_parameter_supported` / `request_uri_parameter_supported` / `require_signed_request_object` (RFC 9101), `registration_endpoint` (RFC 7591 — capítulo seguinte deste livro).
- Campos adicionais comuns na prática, ainda que fora do escopo estrito dos labs deste projeto: `service_documentation`, `ui_locales_supported`, `op_policy_uri`, `op_tos_uri`, `revocation_endpoint_auth_methods_supported`, `introspection_endpoint_auth_methods_supported` (cada endpoint secundário pode ter seu **próprio** conjunto de métodos de autenticação suportados, potencialmente diferente do `/token`).

Esse documento é, na prática, o índice vivo que amarra quase todo endpoint introduzido pelas RFCs deste roteiro — é o "mapa do tesouro" que substitui a necessidade de ler documentação estática de integração.

4.1 Extensibilidade

A RFC define explicitamente que novos campos podem ser registrados no **IANA "OAuth Authorization Server Metadata" registry** sem exigir uma revisão da RFC 8414 em si — é assim que campos de RFCs posteriores (PAR, JAR, DPoP) puderam ser adicionados ao vocabulário do documento sem qualquer mudança na 8414 original. Um client robusto deve, portanto, **ignorar silenciosamente** campos desconhecidos em vez de tratá-los como erro — o documento é projetado para crescer.

5. Segurança já tratada na própria RFC 8414

- **Validação do issuer é a mitigação central de mix-up attack.** Ao buscar metadata em `https://{host}/.well-known/oauth-authorization-server`, o campo `issuer` devolvido no corpo **deve** bater exatamente — mesmo scheme, mesmo host, mesma porta (se explícita), sem barra final divergente — com o valor efetivamente consultado. Sem essa checagem, um client poderia ser induzido (por DNS spoofing, por um proxy malicioso, por um AS comprometido reaproveitado em outro contexto) a aceitar um documento de metadata que na verdade descreve um AS diferente do pretendido, redirecionando silenciosamente todo o fluxo de autorização para infraestrutura sob controle de um atacante. Essa é exatamente a mesma família de preocupação tratada com mais profundidade no capítulo da RFC 9700 (identificação inequívoca do emissor como defesa contra mix-up attacks) — a validação do `issuer` no documento de metadata é, historicamente, uma das primeiras instâncias concretas dessa mitigação no ecossistema OAuth.
- **TLS é obrigatório** para buscar o documento em qualquer implantação de produção. Um documento de metadata adulterado em trânsito (ataque man-in-the-middle sobre HTTP puro) poderia redirecionar o client inteiro para endpoints controlados por um atacante — trocar `token_endpoint` por uma URL maliciosa é, em essência, um vetor de personificação completa do AS, capaz de capturar `client_secret`, códigos de autorização, ou tokens inteiros. (Os labs deste projeto rodam em HTTP puro deliberadamente, como simplificação didática — ver a Parte de mTLS/RFC 8705 para quando TLS entra na conversa de forma explícita.)
- **Cache não deve ser cego nem eterno.** A RFC não prescreve uma política de cache específica (não há `Cache-Control` normativo definido pela 8414 em si), mas alerta para os dois riscos opostos: cache **desatualizado demais** pode quebrar integração silenciosamente quando o AS migra endpoints; cache de um documento **comprometido** (obtido uma vez sob condições adversas, depois reutilizado indefinidamente) prolonga a janela de um redirecionamento malicioso muito além do momento do ataque original. Um TTL razoável (minutos a poucas horas, revalidado periodicamente) é a prática comum, ainda que não normatizada em número exato.
- **Ausência de autenticação para buscar o documento é proposital, não uma omissão.** O documento de metadata funciona como uma âncora de confiança pública — similar a uma chave pública, ou ao próprio certificado TLS do servidor: ele existe justamente para ser lido por qualquer um, sem barreira, para *bootstrapar* confiança antes de qualquer outra interação

acontecer. O efeito colateral é que qualquer parte — incluindo um atacante fazendo reconhecimento — pode ver exatamente quais capacidades o AS suporta (quais grant types, quais métodos de auth, se PAR é obrigatório). Isso é considerado risco de reconhecimento de baixo impacto, não uma vulnerabilidade em si: a informação exposta é, por natureza, a mesma que qualquer integrador legítimo precisaria descobrir de qualquer forma.

- **Confusão de porta/scheme como variante sutil do mix-up.** Um detalhe menos óbvio: `https://as.exemplo.com` e `https://as.exemplo.com:443` podem ser semanticamente equivalentes para um navegador, mas a comparação de string do `issuer` deve ser feita de forma canônica e consistente — implementações ingênuas que comparam strings brutas sem normalizar porta padrão podem, em teoria, criar uma janela de bypass da validação. A RFC não detalha algoritmo de canonicalização; delega ao bom senso de comparação de URLs já estabelecido por outras especificações (RFC 3986).

6. Relação com outras RFCs

- **Modelo direto do OpenID Connect Discovery** (`/.well-known/openid-configuration`), que é anterior e continua sendo o "primo" mais rico em campos (inclui `userinfo_endpoint` , `id_token_signing_alg_values_supported` , claims de identidade). A 8414 é, deliberadamente, a versão "OAuth puro" do mesmo padrão — compatível o suficiente para que um AS que suporte ambos os protocolos frequentemente publique conteúdo idêntico ou sobreposto nos dois paths `well-known`.
- **É o mecanismo de descoberta para praticamente todo endpoint já coberto neste roteiro:** RFC 7009 (`revocation_endpoint`), RFC 7662 (`introspection_endpoint`), RFC 8628 (`device_authorization_endpoint`), RFC 9126 (`pushed_authorization_request_endpoint` + a flag de exigência), RFC 9101 (flags de suporte a Request Object).
- **Complementa diretamente RFC 7591/7592 (próximos dois capítulos)** — um client que descobre as capacidades do AS via 8414 pode se registrar dinamicamente de forma informada: por exemplo, só pedir `token_endpoint_auth_method: "private_key_jwt"` se `token_endpoint_auth_methods_supported` de fato listar esse valor, evitando um registro fadado a falhar depois na hora de autenticar no `/token`.
- **Conecta diretamente com a preocupação de mix-up attack da RFC 9700** através da validação do `issuer` — este capítulo cobre o mecanismo (o campo, a comparação de string); o capítulo da 9700 cobre o *raciocínio de ameaça* mais amplo por trás dessa exigência.
- **RFC 9449 (DPoP)**, tratada mais adiante no roteiro, estende o vocabulário de metadata com `dpop_signing_alg_values_supported` — mais um exemplo do documento crescendo organicamente sem exigir revisão da 8414 original.

7. O que mudou desde a publicação (2018)

- A especificação em si é estável e hoje **essencialmente universal** — praticamente todo Authorization Server de produção séria publica esse documento; deixou de ser um "extra opcional" e virou expectativa de baseline mínima de qualquer integração OAuth nova.

- O conjunto de campos que o documento pode descrever cresceu organicamente conforme cada RFC companheira foi publicada (PAR em 2021, e mais adiante DPoP) — o mecanismo de extensibilidade via registro IANA (§4.1) permitiu essa evolução sem precisar de um novo padrão de descoberta a cada nova capacidade adicionada ao ecossistema.
- **O ecossistema de bibliotecas cliente amadureceu para "descoberta primeiro" como modo padrão de configuração.** Bibliotecas OAuth/OIDC modernas tipicamente pedem só a URL do `issuer` como entrada de configuração, e resolvem todo o resto (endpoints, algoritmos suportados, chaves) automaticamente a partir do documento de metadata — um padrão de configuração que só se tornou dominante bem depois de 2018, à medida que a adoção da RFC (e da prática equivalente em OIDC) se espalhou.
- Em ambientes de nuvem gerenciados (identity-as-a-service), o documento de metadata frequentemente é o **único artefato de configuração** exposto ao integrador — o provedor nem sempre documenta endpoints individuais em texto livre, esperando que o integrador consuma o JSON diretamente.

Nota de laboratório: implementado em `rfc7592/`, `rfc7523/` e `rfc9449/` — nos três, o Authorization Server publica `/well-known/oauth-authorization-server`, com as URLs montadas dinamicamente a partir do cabeçalho `Host` da própria requisição recebida (o `issuer` sempre bate exatamente com quem está perguntando, seja de dentro da rede interna do projeto — via `oauth7592-as:3301` — ou de fora, via `localhost:3301`). Nos dois últimos labs, o `client-demo` efetivamente **usa** a metadata descoberta para localizar `registration_endpoint` / `token_endpoint` em tempo de execução, em vez de existir só como documento passivo consultável por `curl`; o `rfc9449/` também anuncia `dpop_signing_alg_values_supported`. Os demais labs do projeto (`rfc6749/`, `rfc7636/`, `rfc9126/`) continuam sem esse endpoint, usando configuração fora de banda — o contraste entre os dois modos, lado a lado no mesmo projeto, é deliberado.

Exemplo em Java (JDK 21, sem frameworks)

Os exemplos a seguir usam exclusivamente `java.net.http.HttpClient` (embutido no JDK desde a versão 11) e o utilitário `SimpleJson`, definido no Capítulo 1 (RFC 6749) deste livro, para parse do documento de metadata. Nenhuma biblioteca externa é usada — nem para HTTP, nem para JSON.

7.1 Buscar e fazer parse do documento de metadata

```
import java.net.URI;
import java.net.http.HttpClient;
import java.net.http.HttpRequest;
import java.net.http.HttpResponse;
import java.time.Duration;
import java.util.Map;

/**
 * Busca o documento de Authorization Server Metadata (RFC 8414) a partir
 * do issuer informado, aplicando a regra de composicao de path da RFC 8615
 * (well-known e inserido ANTES do path do issuer, nao depois).
 */
public class MetadataFetcher {

    private final HttpClient http = HttpClient.newBuilder()
        .connectTimeout(Duration.ofSeconds(5))
        .build();

    /**
     * @param issuer ex.: "https://as.exemplo.com" ou "https://as.exemplo.com/tenant1"
     */
    public Map<String, String> fetchMetadata(String issuer) throws Exception {
        URI issuerUri = URI.create(issuer);
        String origin = issuerUri.getScheme() + "://" + issuerUri.getAuthority();
        String issuerPath = issuerUri.getPath(); // "" ou "/tenant1"

        // Regra RFC 8615 + 8414: .well-known entra ANTES do path do issuer.
        String wellKnownUrl = origin + "/.well-known/oauth-authorization-server" + issuerPath;

        HttpRequest request = HttpRequest.newBuilder()
            .uri(URI.create(wellKnownUrl))
            .header("Accept", "application/json")
            .GET()
            .build();

        HttpResponse<String> response = http.send(request, HttpResponse.BodyHandlers.ofString());

        if (response.statusCode() != 200) {
            // Sem metadata publicada: o integrador decide se cai para
            // configuracao manual ou trata como erro fatal (a RFC nao normatiza).
            throw new IllegalStateException(
                "Nao foi possivel obter metadata: HTTP " + response.statusCode());
        }

        // SimpleJson.parseFlat assume um objeto JSON "raso" (sem aninhamento) -
        // suficiente para os campos escalares do documento de metadata que
        // este exemplo consome. Arrays como scopes_supported exigiriam um
        // parser mais completo, fora do escopo didatico deste exemplo.
        return SimpleJson.parseFlat(response.body());
    }
}
```

```
}  
}
```

7.2 Validar que o issuer bate com o host consultado (mitigação de mix-up)

```
import java.net.URI;
import java.util.Map;

/**
 * Implementa a validacao central de seguranca da RFC 8414 (secao 5 deste
 * capitulo): o campo "issuer" devolvido no corpo DEVE bater exatamente com
 * a URL que foi consultada. Pular esta checagem abre a porta para um
 * ataque de mix-up – o client passaria a confiar em endpoints (token_endpoint,
 * authorization_endpoint) de um Authorization Server diferente do pretendido.
 */
public class IssuerValidator {

    public static void validar(String issuerEsperado, Map<String, String> metadata) {
        String issuerRecebido = metadata.get("issuer");

        if (issuerRecebido == null) {
            // "issuer" e o UNICO campo obrigatorio do documento (RFC 8414 §3.2).
            // A ausencia dele torna o documento inteiro nao confiavel.
            throw new SecurityException("Documento de metadata sem campo 'issuer' obrigatorio");
        }

        // Comparacao exata de string, nao apenas "mesmo host". Um issuer com
        // path (multi-tenant) precisa bater ATE o path, nao so scheme+host.
        // Barra final divergente ja e o suficiente para reprovar (RFC nao
        // define normalizacao implicita de barra final).
        URI esperado = URI.create(issuerEsperado);
        URI recebido = URI.create(issuerRecebido);

        boolean bateSchemeHostPorta =
            esperado.getScheme().equalsIgnoreCase(recebido.getScheme())
            && esperado.getAuthority().equalsIgnoreCase(recebido.getAuthority());
        boolean batePath = normalizarPath(esperado.getPath()).equals(normalizarPath(recebido.getPath()));

        if (!bateSchemeHostPorta || !batePath) {
            throw new SecurityException(
                "Possivel mix-up attack: issuer esperado='" + issuerEsperado
                + "' mas documento declara issuer='" + issuerRecebido + "'");
        }
    }

    private static String normalizarPath(String path) {
        if (path == null || path.isEmpty()) return "";
        return path.endsWith("/") ? path.substring(0, path.length() - 1) : path;
    }
}
```


7.3 Uso combinado — descoberta seguida de resolução de endpoint

```
import java.util.Map;

/**
 * Exemplo de uso ponta a ponta: descobre a metadata, valida o issuer, e so
 * entao resolve o token_endpoint a partir do documento – em vez de ter essa
 * URL hardcoded na configuracao do client. E este o padrao "descoberta
 * primeiro" que o ecossistema de bibliotecas OAuth adotou como default
 * (secao 7 deste capitulo).
 */
public class DescobertaExemplo {

    public static void main(String[] args) throws Exception {
        String issuer = "http://localhost:3301"; // igual ao Lab rfc7592/

        MetadataFetcher fetcher = new MetadataFetcher();
        Map<String, String> metadata = fetcher.fetchMetadata(issuer);

        IssuerValidator.validar(issuer, metadata);

        String tokenEndpoint = metadata.get("token_endpoint");
        String registrationEndpoint = metadata.get("registration_endpoint");

        if (tokenEndpoint == null) {
            throw new IllegalStateException("AS nao anuncia token_endpoint – integracao impossivel")
        }

        System.out.println("issuer confirmado: " + metadata.get("issuer"));
        System.out.println("token_endpoint descoberto: " + tokenEndpoint);
        System.out.println("registration_endpoint descoberto: " + registrationEndpoint);
        // A partir daqui, o restante do client (Capitulo 1 / RFC 6749) usa
        // tokenEndpoint em vez de uma constante hardcoded.
    }
}
```

RFC 7591 — OAuth 2.0 Dynamic Client Registration Protocol

1. Contexto e motivação

Em todo capítulo e em todo laboratório deste livro até este ponto, registro de client foi tratado como um passo **manual, fora de banda**: alguém — um administrador, um script de seed rodando na inicialização do ambiente — configura `client_id`, `client_secret` e `redirect_uris` diretamente no armazenamento do Authorization Server, **antes** de qualquer fluxo OAuth acontecer. Nos labs deste projeto, isso é literalmente um script escrevendo um registro pronto direto no Redis, executado no boot do ambiente.

Esse modelo funciona bem — e continua sendo a escolha certa — quando existe um conjunto pequeno e relativamente estático de clients conhecidos de antemão (o cenário típico de "integração B2B combinada por contrato"). Mas ele deixa de escalar em pelo menos três cenários realistas:

1. **Software distribuído que aponta para um AS de escolha do usuário final.** Pense num client de e-mail, de chat, ou um agregador de "traga sua própria conta" (bring-your-own-server): o mesmo binário de client precisa, em tempo de execução, se registrar contra qualquer Authorization Server que o usuário decida usar — o desenvolvedor do client não tem como pré-negociar credenciais com cada AS existente no mundo.
2. **Onboarding self-service de integrações de terceiros.** Uma plataforma que expõe uma API OAuth-protégida a desenvolvedores externos quer, tipicamente, um portal onde qualquer integrador possa criar um novo client sem depender de um humano do lado do provedor aprovando e digitando credenciais manualmente a cada pedido.
3. **Provisionamento programático multi-tenant.** Uma plataforma SaaS que cria um tenant novo por cliente frequentemente precisa, como parte desse provisionamento automatizado, criar também um (ou vários) client OAuth associado — sem intervenção humana no caminho crítico.

A RFC 7591 (publicada em 2015, junto com a RFC 7592) resolve isso substituindo o processo manual por um **endpoint de registro**: o client (ou o software que o provisiona em nome de um usuário/tenant) faz um `POST` descrevendo a metadata desejada, e o Authorization Server responde criando o registro e devolvendo as credenciais recém-geradas. O que antes era um ticket de suporte ou um script administrativo vira uma chamada de API — com todas as implicações de segurança que isso traz (tratadas em detalhe na §5).

1.1 Por que isso não é "menos seguro por definição"

Vale desfazer uma intuição comum: registro dinâmico **não é**, em si, menos seguro que registro manual — é apenas um mecanismo diferente de *como* a credencial chega a existir. Um registro manual malfeito (segredo fraco, `redirect_uri` mal validado no cadastro) é tão vulnerável quanto um registro dinâmico malfeito. O que muda é a **superfície de ataque**: registro manual tem, implicitamente, um portão humano no caminho (alguém decidiu criar aquele client); registro dinâmico só tem esse portão se a

implantação explicitamente o construir (via initial access token, aprovação, ou outro controle — ver §5). A RFC é honesta sobre isso: ela padroniza o mecanismo, mas deixa a decisão de *quem pode chamá-lo* inteiramente a critério de cada implantação.

2. Terminologia e papéis

A RFC 7591 não introduz papéis novos além dos já conhecidos (Client, Authorization Server) — introduz um vocabulário específico do próprio processo de registro:

- **Client Registration Endpoint** — o endpoint novo, aceitando `POST`, cuja localização é tipicamente descoberta via o campo `registration_endpoint` do documento de metadata da RFC 8414 (capítulo anterior deste livro).
- **Client Metadata** — o conjunto de campos descritivos que caracterizam um client: `redirect_uris`, `client_name`, `grant_types`, `response_types`, `token_endpoint_auth_method`, `jwt / jwks_uri`, `scope`, `contacts`, `tos_uri`, `policy_uri`, `logo_uri`, `client_uri`, `software_id`, `software_version`. Praticamente todos esses nomes já apareceram ao longo do roteiro deste livro como coisas "configuradas previamente" em algum lugar — esta RFC é justamente onde esse ato de configuração vira uma chamada de API, em vez de um passo administrativo silencioso.
- **Initial Access Token** — uma credencial **opcional**, cuja existência e formato ficam inteiramente a critério de cada implantação (a RFC não define seu formato). Serve como portão de entrada: um AS pode exigir que quem chama `POST /register` já possua algum token previamente emitido (por um processo de aprovação separado, por exemplo), impedindo registro totalmente aberto. Não é exigido pela letra da RFC, mas é explicitamente antecipado como padrão comum e recomendado de implantação — a RFC até dedica uma seção não-normativa a discutir esse tipo de controle (ver §5).
- **Software Statement** — um JWT **assinado por um terceiro de confiança** (tipicamente o próprio publicador/fabricante do software, ou uma autoridade de federação), contendo afirmações (claims) sobre o software que está sendo registrado. A ideia central: em vez de o AS confiar cegamente em campos auto-declarados no corpo do `POST` (qualquer um pode escrever `"client_name": "Banco Legítimo S.A."` sem que isso seja verdade), o software statement permite que essas afirmações venham *assinadas* por alguém que o AS já confia previamente — deslocando a âncora de confiança de "o que o requisitante diz" para "o que um terceiro atestou".

2.1 O "escopo do registro" vs. "escopo de uma requisição"

Um ponto sutil que vale destacar cedo: o campo `scope` enviado no registro **não é** uma concessão — é o **teto** do que aquele client poderá, no futuro, solicitar em requisições de autorização individuais. Um client registrado com `"scope": "profile email"` não ganha automaticamente esses scopes em todo token emitido; ele só passa a ser **elegível** para pedi-los. A concessão efetiva continua acontecendo fluxo a fluxo, exatamente como descrito no capítulo da RFC 6749. Confundir essas duas coisas — achar que registrar com um scope amplo já concede acesso amplo — é um erro conceitual comum em quem lê a RFC pela primeira vez.

3. Fluxo

1. Client (ou quem o provisiona em seu nome) -> AS: POST /register
Content-Type: application/json
Authorization: Bearer <initial_access_token> (opcional, depende da implantacao)

```
{
  "redirect_uris": ["https://client.exemplo.org/callback"],
  "client_name": "Meu App",
  "grant_types": ["authorization_code", "refresh_token"],
  "response_types": ["code"],
  "token_endpoint_auth_method": "client_secret_basic",
  "scope": "profile email"
}
```

2. AS valida a metadata solicitada:
 - redirect_uris sao URIs absolutas, sem fragmento, coerentes com o token_endpoint_auth_method (client publico tipicamente nao deveria usar client_secret_basic, por exemplo)
 - grant_types e response_types sao mutuamente consistentes (ex.: response_types=["code"] implica grant_types deve incluir "authorization_code")
 - se initial_access_token for exigido pela implantacao, valida-o
 - se software_statement estiver presente, valida a assinatura contra a chave do emissor confiavel
3. AS cria o registro internamente, gera client_id (e client_secret, se o client for confidencial) e responde:

AS -> Client: 201 Created
Content-Type: application/json

```
{
  "client_id": "s6BhdRkqt3",
  "client_secret": "cf136dc3c1fc93f31185e5885805d",
  "client_id_issued_at": 2893256800,
  "client_secret_expires_at": 0,
  "registration_access_token": "reg-23410913-abewfq.123483",
  "registration_client_uri": "https://as.exemplo.com/register/s6BhdRkqt3",
  "redirect_uris": ["https://client.exemplo.org/callback"],
  "client_name": "Meu App",
  "grant_types": ["authorization_code", "refresh_token"],
  "response_types": ["code"],
  "token_endpoint_auth_method": "client_secret_basic",
  "scope": "profile email"
}
```

Note que a resposta **ecoa** a metadata que foi efetivamente aceita — que pode ser idêntica à solicitada, ou ajustada pelo AS (por exemplo, um AS pode aceitar o registro mas remover um `grant_type` que não suporta, em vez de rejeitar o pedido inteiro; a RFC permite ambos os comportamentos, deixando a

critério da implantação qual delas seguir).

O par `registration_access_token` + `registration_client_uri`, devolvido aqui na criação, é exatamente a credencial e a URL que a RFC companheira — **RFC 7592**, próximo capítulo — usa depois para ler, atualizar ou apagar esse registro específico. A 7591 cobre exclusivamente a **criação**; tudo que acontece depois do `201 Created` é território da 7592.

3.1 Erros de registro

Os erros seguem o mesmo padrão OAuth de `error` + `error_description` (RFC 6749 §5.2), com códigos específicos deste protocolo:

- `invalid_redirect_uri` — a URI fornecida é inválida ou não atende a alguma exigência da implantação (esquema não permitido, fragmento presente, etc.).
- `invalid_client_metadata` — algum campo de metadata é inválido ou inconsistente (ex.: `token_endpoint_auth_method` que o AS não suporta, conforme anunciado em `token_endpoint_auth_methods_supported` na metadata RFC 8414).
- `invalid_software_statement` — o software statement fornecido não pôde ser validado (assinatura inválida, claims malformadas).
- `unapproved_software_statement` — o software statement é criptograficamente válido, mas o AS optou por não aprovar esse software específico (ex.: uma allowlist de emissores de software statement).

4. Parâmetros e endpoints normativos

- **Client Registration Endpoint** — novo, tipicamente descoberto via RFC 8414 como o campo `registration_endpoint` do documento de metadata.
- **Detalhe que chama atenção imediatamente a quem já percorreu os outros capítulos deste livro:** diferente de praticamente todo outro endpoint OAuth coberto até aqui (que usa `application/x-www-form-urlencoded` — `/token`, `/revoke`, `/introspect`), o registro usa `application/json`. Essa é uma exceção deliberada da RFC, não um descuido: a metadata do client tem campos naturalmente estruturados — arrays (`redirect_uris`, `grant_types`, `response_types`, `contacts`) e potencialmente objetos aninhados (`jwtks`, que é um JSON Web Key Set completo) — que não se prestam bem à codificação achatada de `form-urlencoded`. É por isso que este capítulo (e o próximo) são os únicos deste livro onde o utilitário `SimpleJson` — definido no Capítulo 1 (RFC 6749) — é genuinamente indispensável para o corpo da requisição, não só da resposta.
- **Campos de request mais relevantes:**
 - `redirect_uris` (array de strings) — obrigatório se o client pretende usar qualquer grant baseado em redirecionamento (`authorization_code`). Cada URI deve ser absoluta e sem fragmento.
 - `token_endpoint_auth_method` — como o client se autenticará no `/token`: `client_secret_basic`, `client_secret_post`, `private_key_jwt`, `client_secret_jwt`, `none` (client público, sem segredo), `tls_client_auth` / `self_signed_tls_client_auth` (RFC 8705, mais adiante no roteiro).
 - `grant_types` — lista de grant types que o client pretende usar (default `["authorization_code"]` se omitido).

- `response_types` — tipicamente `["code"]`; deve ser consistente com `grant_types` (um client pedindo `response_types: ["code"]` precisa ter `"authorization_code"` em `grant_types`).
- `client_name`, `client_uri`, `logo_uri`, `contacts`, `tos_uri`, `policy_uri` — campos majoritariamente informativos/de exibição, usados por exemplo numa tela de consentimento para identificar o client ao Resource Owner.
- `scope` — string separada por espaços, o teto do que o client poderá solicitar depois (ver §2.1).
- `jwks_uri` / `jwks` — chave pública (ou conjunto de chaves) do client, usada para verificar assinaturas quando `token_endpoint_auth_method` é `private_key_jwt`, ou para verificar Request Objects assinados (RFC 9101). São mutuamente exclusivos — um client não deve fornecer os dois.
- `software_statement` — o JWT assinado por terceiro, cobrindo (potencialmente) qualquer subconjunto dos campos acima; quando presente, os valores do software statement tipicamente prevalecem sobre valores conflitantes enviados soltos no corpo.
- **Campos de response:** `client_id` (único campo de resposta genuinamente obrigatório), `client_secret` (presente apenas se o client for confidencial), `client_id_issued_at` (timestamp Unix), `client_secret_expires_at` (`0` significa "nunca expira" — um valor positivo é um timestamp Unix de expiração), `registration_access_token`, `registration_client_uri`, além de toda a metadata efetivamente aceita, ecoada de volta.

5. Segurança já tratada na própria RFC 7591

- **Registro aberto sem nenhum portão é um risco concreto, não hipotético.** Se qualquer parte não autenticada pode chamar `POST /register` livremente, torna-se trivial para um atacante criar `client_id`s arbitrários em volume. Isso é diretamente explorável para phishing — registrar um client com `client_name` e `logo_uri` deliberadamente enganosos, que serão exibidos numa tela de consentimento legítima do AS, fazendo um usuário confiar num client malicioso porque a *interface* de consentimento parece confiável — ou simplesmente para poluir/sobrecarregar o armazenamento de clients do AS (um vetor de negação de serviço de baixo esforço). A RFC discute isso na sua §3 (de forma não-normativa, deixando a decisão a cada implantação): implementações **deveriam** considerar algum tipo de barreira antes de permitir o registro — um initial access token previamente distribuído por um canal confiável, um passo de aprovação manual pós-registro antes do client se tornar utilizável, ou simplesmente restringir o endpoint a um conjunto fechado de partes já autenticadas por outro mecanismo (mTLS de rede, allowlist de IP, etc.).
- **Software Statement mitiga o problema fundamental de "quem é você de verdade".** Como todos os campos de metadata são, por padrão, inteiramente auto-declarados por quem chama o endpoint, um AS que precise de garantia mais forte sobre a autenticidade de certas afirmações (por exemplo, "este software é realmente o app oficial publicado pelo Fornecedor X", relevante em contextos regulados como Open Banking/FAPI) pode exigir que essas afirmações venham dentro de um software statement assinado por uma autoridade de emissão em quem o AS já confia previamente — deixando de depender da palavra não verificada de quem preenche o formulário de registro.
- `registration_access_token` **precisa de proteção equivalente a qualquer credencial bearer** (as mesmas considerações da RFC 6750 se aplicam integralmente): quem o possui pode gerenciar — e, com a RFC 7592, efetivamente apagar — aquele registro de client específico.

Vazamento desse token tem, na prática, um raio de impacto comparável a vaziar o próprio `client_secret`.

- **Validação de `redirect_uri` não é relaxada pelo fato do registro ser dinâmico.** É um erro conceitual pensar que "dinâmico" significa "menos rigoroso" — a exigência de correspondência exata entre o `redirect_uri` usado numa requisição de autorização e algum valor previamente registrado (RFC 6749 §3.1.2.3, reforçada pela RFC 9700) se aplica exatamente da mesma forma a um client registrado dinamicamente. O que muda é só *como* aquele valor chegou a existir no armazenamento do AS — a checagem no momento do `/authorize` é idêntica.
- **Confidencialidade do `client_secret` na resposta é uma janela de exposição única.** O `client_secret` só é devolvido **uma vez**, na resposta de criação — nunca novamente em uma consulta subsequente (ver RFC 7592, `GET` explicitamente omite esse campo). Isso significa que o cliente que perde essa resposta original (ex.: falha de rede após o `201`, log truncado) perde o segredo para sempre, exigindo rotação (via `PUT`, RFC 7592) para recuperar acesso — um comportamento operacionalmente relevante, não só teórico.

6. Relação com outras RFCs

- **Companheira direta e indissociável da RFC 7592** (próximo capítulo deste livro), publicada na mesma data — a 7592 estende exatamente este protocolo com leitura, atualização e exclusão do registro, usando o `registration_access_token` + `registration_client_uri` que a 7591 devolve na criação. As duas RFCs formam, na prática, um único ciclo de vida partido em dois documentos por conveniência de escopo.
- **Descoberta via RFC 8414** — o `registration_endpoint` é um dos campos do documento de metadata (capítulo anterior); um client que queira descobrir automaticamente onde registrar-se consulta a metadata primeiro.
- **É exatamente o que a maioria dos labs deste projeto ainda não faz:** `rfc6749/`, `rfc7636/` e `rfc9126/` registram o client de demonstração fora de banda, via script de seed escrevendo diretamente no Redis na subida do ambiente — o padrão manual que esta RFC existe precisamente para substituir por uma chamada de API real.
- **`jwtks_uri` / `jwtks` registrados aqui alimentam diretamente a RFC 9101 (JAR) e a RFC 7523** (autenticação de client via JWT assertion, seção adiante do roteiro) — é neste ponto do ciclo de vida que essas chaves passam a existir associadas formalmente a um `client_id` específico no AS.
- **`token_endpoint_auth_method` registrado aqui determina, de forma vinculante, qual método de autenticação de client se aplica** a partir daí:
`client_secret_basic` / `client_secret_post` (segredo compartilhado), `private_key_jwt` (RFC 7523), `none` (client público, tipicamente combinado com PKCE — RFC 7636), `tls_client_auth` (mTLS, RFC 8705, ainda por vir no roteiro).
- **RFC 9126 (PAR)** — um client registrado dinamicamente pode, em seguida, ser instruído (via metadata) a sempre empurrar seus parâmetros de autorização antes de redirecionar o Resource Owner; não há dependência direta entre os protocolos, mas ambos convivem naturalmente no mesmo AS.

7. O que mudou desde a publicação (2015)

- Spec estável e fundacional, mas a adoção de registro **totalmente aberto** (sem qualquer portão de entrada) permanece relativamente rara em produção — precisamente pelos riscos que a própria RFC já sinaliza na sua seção de considerações de segurança. A esmagadora maioria das implantações que suportam RFC 7591 combinam-na com initial access token, aprovação manual pós-registro, ou restringem o endpoint a contextos fechados de federação/enterprise.
- É mais visível, na prática, em ecossistemas onde registro dinâmico resolve um problema real e não apenas teórico: contextos de **OpenID Connect Federation** (onde a confiança vem de uma cadeia de assinaturas, não de um portal humano), alguns ecossistemas de **IoT e apps nativos** com provisionamento em massa, e crescentemente em contextos adjacentes a **FAPI** (Financial-grade API), onde software statements fornecem exatamente a âncora de confiança forte que integrações financeiras reguladas exigem antes de conceder qualquer credencial.
- O formato geral do protocolo (`POST JSON` → `client_id / client_secret` + credenciais de gerenciamento) permaneceu estável desde 2015 e efetivamente virou o **modelo de referência** para outras APIs de "auto-registro" em especificações adjacentes publicadas depois — a forma do request/response praticamente não mudou, mesmo com o ecossistema ao redor evoluindo bastante.

Nota de laboratório: implementado em três labs deste projeto, cada um exercitando um `token_endpoint_auth_method` diferente — uma escolha deliberada para cobrir o espectro de uso real do protocolo. Em `rfc7592/` — `client_secret_basic` : não há script de seed nem `.demo-client.env` ; o Authorization Server expõe `POST /register` , e o client-demo se registra **sob demanda**, como ação manual do usuário (botão "Registrar client agora" — ver detalhes no capítulo da RFC 7592 a seguir). Em `rfc7523/` — `private_key_jwt` : o corpo do registro carrega `jwks` (a chave pública do client) em vez de o client receber um `client_secret` de volta, coerente com autenticação baseada em assinatura JWT em vez de segredo compartilhado. Em `rfc9449/` — `none` (client público): nem `client_secret` nem `jwks` são trocados; o sender-constraining do token fica inteiramente a cargo do DPoP (RFC 9449), não da autenticação do client no `/token` .

Exemplo em Java (JDK 21, sem frameworks)

Os exemplos usam `java.net.http.HttpClient` e o utilitário `SimpleJson` (`SimpleJson.object(Map)` para serializar o corpo, `SimpleJson.parseFlat(String)` para ler a resposta), ambos definidos no Capítulo 1 (RFC 6749) deste livro. Note que este é o **único** endpoint, entre todos os labs do projeto, cujo corpo de requisição é JSON — todo o resto usa `application/x-www-form-urlencoded` .

7.1 Registrar um client novo com POST /register

```
import java.net.URI;
import java.net.http.HttpClient;
import java.net.http.HttpRequest;
import java.net.http.HttpResponse;
import java.net.http.HttpRequest.BodyPublishers;
import java.util.LinkedHashMap;
import java.util.List;
import java.util.Map;

/**
 * Registra dinamicamente um novo client no Authorization Server (RFC 7591).
 * Diferente de /token, /revoke, /introspect (form-urlencoded), este endpoint
 * usa application/json – a metadata tem campos naturalmente estruturados
 * (arrays como redirect_uris) que nao se prestam a form-encoding achatado.
 */
public class ClientRegistrationClient {

    private final HttpClient http = HttpClient.newHttpClient();
    private final String registrationEndpoint;

    public ClientRegistrationClient(String registrationEndpoint) {
        this.registrationEndpoint = registrationEndpoint;
    }

    public Map<String, String> registrar(String redirectUri, String clientName) throws Exception {
        // Monta a metadata desejada. redirect_uris e um array – e exatamente
        // por causa de campos assim que o corpo precisa ser JSON, nao
        // form-urlencoded (que so representa pares chave-valor escalares).
        Map<String, Object> metadata = new LinkedHashMap<>();
        metadata.put("redirect_uris", List.of(redirectUri));
        metadata.put("client_name", clientName);
        metadata.put("grant_types", List.of("authorization_code", "refresh_token"));
        metadata.put("response_types", List.of("code"));
        metadata.put("token_endpoint_auth_method", "client_secret_basic");
        metadata.put("scope", "profile email");

        String corpoJson = SimpleJson.object(metadata);

        HttpRequest request = HttpRequest.newBuilder()
            .uri(URI.create(registrationEndpoint))
            .header("Content-Type", "application/json")
            .header("Accept", "application/json")
            // Se a implantacao exigir initial_access_token, seria aqui:
            // .header("Authorization", "Bearer " + initialAccessToken)
            .POST(BodyPublishers.ofString(corpoJson))
            .build();

        HttpResponse<String> response = http.send(request, HttpResponse.BodyHandlers.ofString());

        if (response.statusCode() != 201) {
```

```
// Erros seguem o padrao OAuth: error + error_description no corpo,  
// ex.: invalid_redirect_uri, invalid_client_metadata,  
// invalid_software_statement, unapproved_software_statement.  
throw new IllegalStateException(  
    "Registro falhou: HTTP " + response.statusCode() + " - " + response.body();  
}  
  
return SimpleJson.parseFlat(response.body());  
}  
}
```

7.2 Ler de volta as credenciais e a URI de gerenciamento

```
import java.util.Map;

/**
 * Extrai e organiza os campos criticos da resposta de registro. O
 * client_secret so aparece AQUI, uma unica vez – nao e devolvido de novo
 * em nenhuma consulta subsequente (ver RFC 7592, capitulo seguinte: o GET
 * de gerenciamento omite deliberadamente esse campo).
 */
public class RegistroRecemCriado {

    public final String clientId;
    public final String clientSecret;           // pode ser null (client publico)
    public final String registrationAccessToken;
    public final String registrationClientUri;
    public final long clientSecretExpiresAt;    // 0 = nunca expira

    public RegistroRecemCriado(Map<String, String> resposta) {
        this.clientId = resposta.get("client_id");
        this.clientSecret = resposta.get("client_secret"); // ausente => client publico
        this.registrationAccessToken = resposta.get("registration_access_token");
        this.registrationClientUri = resposta.get("registration_client_uri");

        String expiresAt = resposta.get("client_secret_expires_at");
        this.clientSecretExpiresAt = expiresAt != null ? Long.parseLong(expiresAt) : 0L;

        if (clientId == null) {
            throw new IllegalStateException("Resposta de registro sem client_id – resposta invalida");
        }
        if (registrationAccessToken == null || registrationClientUri == null) {
            // Sem esse par, e impossivel gerenciar o registro depois (RFC 7592)
            // – o client precisaria de um novo registro do zero se os perder.
            System.err.println("Aviso: AS nao devolveu credenciais de gerenciamento (RFC 7592)");
        }
    }

    public void persistirComoSegredo() {
        // PONTO DE ATENCAO DIDATICO: client_secret e registration_access_token
        // sao credenciais de alto valor (secao 5 deste capitulo) e devem ser
        // guardadas com o mesmo cuidado que qualquer segredo de longa duracao
        // – nunca logadas em texto claro, nunca versionadas em codigo-fonte.
        System.out.println("client_id: " + clientId);
        System.out.println("client_secret: [omitido do log deliberadamente]");
        System.out.println("registration_client_uri: " + registrationClientUri);
    }
}
```

7.3 Exemplo completo de uso

```
public class RegistroExemplo {
    public static void main(String[] args) throws Exception {
        // registration_endpoint tipicamente descoberto via RFC 8414
        // (metadata.get("registration_endpoint")), aqui hardcoded por brevidade.
        String registrationEndpoint = "http://localhost:3301/register";

        ClientRegistrationClient registrador = new ClientRegistrationClient(registrationEndpoint);
        var resposta = registrador.registrar(
            "http://localhost:3300/callback",
            "Client de Demonstracao - Capitulo RFC 7591");

        RegistroRecemCriado registro = new RegistroRecemCriado(resposta);
        registro.persistirComoSegredo();

        // A partir daqui, registro.clientId + registro.clientSecret alimentam
        // o fluxo Authorization Code (Capitulo 1, RFC 6749) normalmente; e
        // registro.registrationAccessToken + registro.registrationClientUri
        // alimentam o gerenciamento do proprio registro (proximo capitulo,
        // RFC 7592).
    }
}
```

RFC 7592 — OAuth 2.0 Dynamic Client Registration Management Protocol

1. Contexto e motivação

A RFC 7591, tratada no capítulo anterior, resolve exatamente um problema: como criar um registro de client dinamicamente, via API, em vez de fora de banda. Mas criação é só o primeiro momento do ciclo de vida de qualquer entidade que persiste — depois de criado, um registro de client inevitavelmente precisa ser **lido** (para exibir num painel de administração, por exemplo), **atualizado** (o `redirect_uri` mudou porque o app migrou de domínio; é hora de rotacionar o `client_secret` por política; um novo `grant_type` precisa ser habilitado) e, eventualmente, **apagado** (o client foi descomissionado e não deve mais existir).

A RFC 7591, sozinha, não define nenhuma dessas três operações — ela devolve `registration_access_token` e `registration_client_uri` na resposta de criação, mas não especifica formalmente o que fazer com eles depois. É exatamente essa lacuna que a **RFC 7592** fecha, publicada **na mesma data** que a 7591 (ambas de julho de 2015) — não por coincidência: as duas foram desenhadas em conjunto como duas metades de um único ciclo de vida, apenas divididas em documentos separados por conveniência de escopo e para permitir que uma implantação suporte só criação (7591) sem necessariamente suportar gerenciamento self-service ao vivo (7592), se assim preferir.

Cenários concretos que motivam esta RFC, além dos já citados:

- **Migração de infraestrutura do lado do client.** Um app migra de domínio ou reestrutura suas rotas de callback; sem um mecanismo de atualização padronizado, a única saída seria abrir um novo registro do zero e desativar o antigo manualmente — perdendo a continuidade de `client_id` que outros sistemas (logs, auditoria, integrações downstream) podem depender.
- **Portais self-service de gerenciamento.** Uma organização que já oferece registro dinâmico (7591) naturalmente quer, no mesmo portal, permitir que o próprio integrador veja e edite a configuração do client que criou — sem depender de suporte técnico para cada ajuste pequeno.
- **Baixa formal de um client descomissionado.** Deixar tokens simplesmente expirarem sozinhos não é o mesmo que dar baixa formal no registro: um client apagado explicitamente não pode mais, por exemplo, ser usado para *criar novos* tokens via `client_credentials` ou `authorization_code`, mesmo que algum token antigo ainda não tenha expirado (ver §5 sobre a relação, não automática, entre apagar o registro e revogar tokens já emitidos).

2. Terminologia e papéis

A RFC 7592 não introduz nenhum papel novo, nem nenhum conceito genuinamente novo em termos de vocabulário — ela reaproveita integralmente o que a RFC 7591 já definiu: `registration_access_token` (a credencial) e `registration_client_uri` (a localização), ambos devolvidos exclusivamente na resposta

de criação da 7591, são exatamente a credencial e a URL que esta RFC usa para todas as operações subsequentes sobre aquele registro específico.

Vale destacar uma distinção estrutural importante entre os dois documentos de metadata envolvidos neste ecossistema:

- `registration_endpoint` (RFC 7591) é uma capacidade **geral** do Authorization Server — um único endpoint, o mesmo para qualquer client que queira se registrar, descoberto genericamente via RFC 8414.
- `registration_client_uri` (RFC 7592) é **específico por instância de client** — uma URL diferente para cada registro individual, que só existe (e só é conhecida) depois que aquele client específico foi criado. Não há, nem poderia haver, um campo de metadata RFC 8414 genérico apontando para "o" `registration_client_uri`, porque não existe um único — existe um por client.

Essa distinção é o motivo pelo qual a RFC 7592 não aparece na lista de campos descobertos via `/.well-known/oauth-authorization-server` (capítulo da RFC 8414) — não é uma omissão, é uma consequência necessária da própria natureza da informação.

3. Fluxo — três operações sobre `registration_client_uri`

Todas as três operações são autenticadas exclusivamente por `Authorization: Bearer <registration_access_token>` — não há outro mecanismo de autenticação previsto pela RFC para este conjunto de endpoints (nem `client_secret_basic`, nem mTLS específico deste protocolo; o token bearer é, deliberadamente, a única credencial reconhecida aqui).

3.1 Leitura (GET)

```
GET /register/s6BhdRkqt3 HTTP/1.1
Authorization: Bearer reg-23410913-abewfq.123483

HTTP/1.1 200 OK
Content-Type: application/json

{
  "client_id": "s6BhdRkqt3",
  "client_id_issued_at": 2893256800,
  "redirect_uris": ["https://client.exemplo.org/callback"],
  "client_name": "Meu App",
  "grant_types": ["authorization_code", "refresh_token"],
  "response_types": ["code"],
  "token_endpoint_auth_method": "client_secret_basic",
  "scope": "profile email",
  "registration_client_uri": "https://as.exemplo.com/register/s6BhdRkqt3",
  "registration_access_token": "reg-23410913-abewfq.123483"
}
```

Um detalhe de segurança relevante que a especificação deixa implícito, mas que toda implementação séria observa na prática: a resposta do `GET` **omite o `client_secret`**. O segredo foi devolvido uma única vez, no momento da criação (RFC 7591); a partir daí, nenhuma consulta subsequente o revela de novo — mesmo para o dono legítimo do `registration_access_token`. Se o `client_secret` for perdido, a única saída é rotacioná-lo via `PUT` (§3.2), não recuperá-lo via `GET`.

3.2 Atualização (`PUT` — substituição completa, não patch parcial)

```
PUT /register/s6BhdRkqt3 HTTP/1.1
Authorization: Bearer reg-23410913-abewfq.123483
Content-Type: application/json

{
  "client_id": "s6BhdRkqt3",
  "redirect_uris": ["https://client.exemplo.org/novo-callback"],
  "client_name": "Meu App",
  "grant_types": ["authorization_code", "refresh_token"],
  "response_types": ["code"],
  "token_endpoint_auth_method": "client_secret_basic",
  "scope": "profile email"
}

HTTP/1.1 200 OK
Content-Type: application/json

{
  "client_id": "s6BhdRkqt3",
  "redirect_uris": ["https://client.exemplo.org/novo-callback"],
  "client_name": "Meu App",
  "grant_types": ["authorization_code", "refresh_token"],
  "response_types": ["code"],
  "token_endpoint_auth_method": "client_secret_basic",
  "scope": "profile email",
  "registration_access_token": "reg-NOVO-token-98765.abcdef",
  "registration_client_uri": "https://as.exemplo.com/register/s6BhdRkqt3"
}
```

O ponto normativo mais importante — e o que mais gera confusão prática — é que a semântica do `PUT` aqui é **substituição completa do recurso**, no sentido HTTP mais estrito do verbo, e não um `PATCH` parcial. Isso significa que o client precisa reenviar **toda** a metadata que deseja manter, não apenas o campo que mudou. Um corpo de `PUT` que omite `token_endpoint_auth_method` não significa "não altere esse campo" — na prática (dependendo de como a implantação trata omissão), pode significar "reverta para o default", ou ser rejeitado como metadata incompleta. A responsabilidade de recuperar o estado atual completo (via `GET` prévio) e só então editar o campo desejado antes de reenviar tudo é do client — a RFC não protege contra um `PUT` malfeito que apaga configuração por omissão acidental (ver §5).

O segundo ponto normativo relevante: a resposta a um `PUT` bem-sucedido **pode** vir com um `registration_access_token` **novo**, diferente do que foi usado na própria requisição. Isso não é um bug nem uma coincidência — é uma opção explícita de design da RFC (§2.2): o AS pode rotacionar essa

credencial a cada atualização, como defesa em profundidade. Se isso acontecer, o client **precisa** passar a usar o token novo a partir daquele momento — o token antigo deixa de ser válido para chamadas futuras.

3.3 Exclusão (DELETE)

```
DELETE /register/s6BhdRkqt3 HTTP/1.1
Authorization: Bearer reg-23410913-abewfq.123483

HTTP/1.1 204 No Content
```

Sem corpo de resposta. O código `204 No Content` confirma que o registro foi removido; chamadas subsequentes de `GET` / `PUT` / `DELETE` para o mesmo `registration_client_uri` devem, a partir daí, responder com algo equivalente a "não encontrado" ou "não autorizado" (a RFC não prescreve qual dos dois exatamente — implementações variam entre `401` / `403` e `404`, cada escolha com uma implicação de information disclosure ligeiramente diferente).

4. Parâmetros e endpoints normativos

- **Não há um endpoint genérico novo anunciado no documento de metadata da RFC 8414** — como discutido em §2, `registration_client_uri` é específico por client, não uma capacidade geral e uniforme do AS.
- **Métodos HTTP:** `GET` (ler o estado atual), `PUT` (atualizar — substituição completa), `DELETE` (apagar permanentemente). Não há `POST` neste conjunto de endpoints (criação já aconteceu via RFC 7591) nem `PATCH` (a RFC não define atualização parcial como alternativa ao `PUT`).
- **Autenticação:** exclusivamente `Bearer <registration_access_token>` em toda chamada, sem exceção — inclusive no `GET`, que muita gente presume (erroneamente) ser "só leitura, menos sensível". Não é: ler a configuração completa de um client (`redirect_uris`, `scopes`, métodos de auth) já é, em si, informação sensível o bastante para exigir a mesma credencial das operações destrutivas.
- **Códigos de status relevantes:** `200 OK` (`GET` e `PUT` bem-sucedidos, ambos com corpo), `204 No Content` (`DELETE` bem-sucedido, sem corpo), `401 Unauthorized` (token ausente ou inválido), `403 Forbidden` (token válido mas sem permissão sobre este registro específico — situação que, na prática, um AS bem implementado tende a tratar de forma indistinguível de "não encontrado", para não vazsar a existência de um `client_id` a quem não deveria sabê-la).

5. Segurança já tratada na própria RFC 7592

- O `registration_access_token` é uma credencial de altíssimo valor — **arguivelmente mais sensível que o próprio `client_secret`**. Quem o possui pode ler a configuração completa do client (incluindo, potencialmente, ver que um `client_secret` existe e quando expira — ainda que não seu valor), **modificar** `redirect_uris` (o que efetivamente redireciona, do ponto de vista do AS, para onde ele está disposto a entregar authorization codes e tokens em nome desse client) e **apagar** o registro inteiro. A RFC é explícita: essa credencial merece proteção pelo menos

equivalente à de um `client_secret`; alguns argumentam que merece proteção *maior*, porque o dano de um vazamento não se limita a personificar o client numa troca de token — permite reconfigurar silenciosamente o comportamento do client do ponto de vista do AS, ou eliminá-lo de vez.

- **A semântica de substituição completa do `PUT` é, na prática, um risco operacional tão relevante quanto qualquer risco puramente criptográfico.** Como não existe um `PATCH` parcial, um client que reenvia um objeto de metadata incompleto — por exemplo, esquecendo de incluir um `redirect_uri` secundário que ainda está em uso ativo em outro ambiente — corre o risco real de **apagar silenciosamente** configuração de segurança já existente: resetar `token_endpoint_auth_method` para um valor default menos restritivo, ou perder uma entrada válida de `redirect_uris` sem qualquer aviso ou confirmação. A mitigação prática (não normativa, mas amplamente recomendada) é sempre fazer um `GET` imediatamente antes de qualquer `PUT`, editar o objeto completo recebido, e só então reenviá-lo — nunca construir o corpo do `PUT` "do zero" a partir de memória do que se pretendia originalmente.
- **Apagar o registro do client não garante, por si só, que tokens já emitidos parem de funcionar.** Este é um ponto de nuance frequentemente mal-entendido: `DELETE /register/{id}` remove o **registro** (a entidade "client" no armazenamento do AS) — mas access tokens e refresh tokens já emitidos anteriormente para aquele client podem, dependendo estritamente de como a implementação do AS foi construída, continuar tecnicamente válidos até expirarem naturalmente, a menos que o AS explicitamente encadeie a exclusão do registro com uma revogação em massa dos tokens associados (RFC 7009). A RFC 7592 não exige esse encadeamento — é uma prática recomendada de implementação madura, não uma obrigação normativa do protocolo. Um AS que queira uma baixa verdadeiramente completa precisa implementar essa ligação deliberadamente.
- **A rotação do `registration_access_token` a cada `PUT` é uma defesa em profundidade opcional**, seguindo o mesmo princípio já visto para refresh tokens no capítulo da RFC 6749/RFC 9700: mesmo que o token antigo tenha vazado em algum momento entre duas atualizações, rotacioná-lo a cada `PUT` limita a janela de validade útil de uma credencial comprometida sem exigir nenhuma ação extra do lado do client (além de, claro, passar a usar o valor novo devolvido).
- **Enumeração de `client_id` via tentativa-e-erro é uma preocupação lateral, mas real.** Como `registration_client_uri` inclui o `client_id` na própria URL (`/register/{clientId}`), um AS que responda de forma diferenciável a "client_id existe mas token errado" (`403`) versus "client_id não existe" (`404`) está, ainda que sutilmente, vazando quais `client_id`s são válidos a quem tenta adivinhar. Implementações cuidadosas tendem a normalizar essa resposta.

6. Relação com outras RFCs

- **Companheira direta e indissociável da RFC 7591** (capítulo anterior deste livro) — a 7591 cria, a 7592 gerencia, usando exatamente as credenciais que a 7591 já devolve na resposta de criação. Foram publicadas juntas precisamente por serem as duas metades inseparáveis de um único ciclo de vida de registro.

- **Conecta diretamente com a RFC 7009 (Token Revocation)**, tratada em capítulo anterior deste roteiro: apagar um registro de client deveria, por política de implementação madura (não por exigência normativa formal entre os dois protocolos), levar à revogação em cadeia de qualquer token pendente daquele client. São problemas adjacentes mas distintos — revogar um **token** individual versus dar baixa formal num **client** inteiro — e uma implantação bem projetada tipicamente trata a segunda ação como gatilho para a primeira, mesmo que os textos normativos de ambas as RFCs não exijam essa ligação explicitamente.
- O `registration_access_token` **em si é, funcionalmente, um bearer token nos termos exatos da RFC 6750** — as mesmas considerações de transporte seguro, de não vazamento em logs, de proteção equivalente a qualquer outra credencial de portador se aplicam integralmente, ainda que o propósito específico desse token (gerenciar um registro, não acessar um recurso protegido) seja exclusivo deste par de RFCs.
- **Diferente de `registration_endpoint`, `registration_client_uri` não é descoberto genericamente via RFC 8414** — como já discutido em §2, é inerentemente específico por instância de client, incompatível com a natureza de "capacidade geral do AS" que o documento de metadata descreve.

7. O que mudou desde a publicação (2015)

- Publicada junto com a RFC 7591, manteve-se igualmente estável ao longo dos anos; sua taxa de adoção acompanha de perto a da 7591 — onde há criação dinâmica de clients, o gerenciamento via 7592 é o complemento natural e esperado, embora nem toda implantação escolha suportar as duas simetricamente.
- É comum, na prática, encontrar implantações que suportam **só** a criação (7591) e tratam atualização/exclusão como um processo administrativo fora de banda (um humano intervindo diretamente no armazenamento, ou um processo de suporte) — nesses casos, o gerenciamento self-service ao vivo previsto pela 7592 é tratado como *hardening opcional*, não como requisito obrigatório de toda integração dinâmica. Isso reflete, em parte, a superfície de risco adicional que expor gerenciamento ao vivo introduz (§5) — algumas organizações preferem deliberadamente manter esse controle mais próximo, mesmo tendo adotado registro dinâmico para a criação.
- Sem mudança significativa de uso desde a publicação — continua sendo uma peça relativamente nicho do ecossistema OAuth (comparada à onipresença de RFC 6749 ou mesmo de PKCE), mas estável e bem definida, mais visível em contextos de federação e portais self-service corporativos, sempre lado a lado com a 7591.

Nota de laboratório: implementado em `rfc7592/` — o Authorization Server expõe

`GET / PUT / DELETE` em `/register/:clientId`, todos autenticados pelo `registration_access_token` recebido na criação (RFC 7591). O `GET` omite deliberadamente o `client_secret` da resposta, exatamente como discutido em §3.1; o `PUT` rotaciona o `registration_access_token` a cada chamada bem-sucedida, exatamente como discutido em §3.2 e §5. No client-demo, o próprio ato de registro é uma **ação manual do usuário** ("Registrar client agora" na home) — nenhuma outra funcionalidade (login, chamadas machine-to-machine, ou a própria seção de gerenciamento) fica disponível até essa ação ser concluída. Uma vez registrado, a home exibe a seção "Gerenciar registro" com três ações espelhando exatamente os três verbos desta RFC: "Consultar (GET)"

mostra a metadata atual (sem o segredo); "Atualizar (PUT)" reenvia a metadata completa com um `client_name` alterado e exibe o `registration_access_token` recém-rotacionado na resposta; "Apagar (DELETE)" remove o registro e devolve a aplicação inteira ao estado "não registrado" — exigindo um novo registro manual do zero. Um teste didático sugerido pelo próprio README do lab: executar "Apagar (DELETE)" e, em seguida, tentar "Executar chamada machine-to-machine" — o `/token` responde com `invalid_client`, demonstrando concretamente que apagar o registro efetivamente invalida o client para **novas** emissões de token (ainda que, como discutido em §5, isso não seja garantia automática de que tokens antigos já emitidos parem de funcionar por si só).

Exemplo em Java (JDK 21, sem frameworks)

Os exemplos usam `java.net.http.HttpClient` e o utilitário `SimpleJson` (`SimpleJson.object(Map)` / `SimpleJson.parseFlat(String)`), definido no Capítulo 1 (RFC 6749) deste livro. Todas as chamadas carregam `Authorization: Bearer <registration_access_token>` — não há outro método de autenticação previsto por esta RFC.

7.1 Consultar o registro atual (GET)

```
import java.net.URI;
import java.net.http.HttpClient;
import java.net.http.HttpRequest;
import java.net.http.HttpResponse;
import java.util.Map;

/**
 * Cliente de gerenciamento de registro (RFC 7592). Todas as operacoes usam
 * o registration_client_uri especifico deste client (devolvido apenas na
 * criacao via RFC 7591) e sao autenticadas com o registration_access_token
 * correspondente – nunca com client_secret_basic ou qualquer outro metodo
 * de autenticacao de client usado no /token.
 */
public class RegistrationManagementClient {

    private final HttpClient http = HttpClient.newHttpClient();

    /**
     * GET: Le a configuracao atual do client. Repare que o client_secret
     * NUNCA volta aqui – foi devolvido uma unica vez, na criacao (RFC 7591).
     * Se foi perdido, a unica saida e rotacionar via PUT, nao recuperar via GET.
     */
    public Map<String, String> consultar(String registrationClientUri, String registrationAccessToken)
        throws Exception {
        HttpRequest request = HttpRequest.newBuilder()
            .uri(URI.create(registrationClientUri))
            .header("Authorization", "Bearer " + registrationAccessToken)
            .header("Accept", "application/json")
            .GET()
            .build();

        HttpResponse<String> response = http.send(request, HttpResponse.BodyHandlers.ofString());

        if (response.statusCode() != 200) {
            throw new IllegalStateException(
                "Consulta ao registro falhou: HTTP " + response.statusCode());
        }
        return SimpleJson.parseFlat(response.body());
    }
}
```

7.2 Atualizar o registro (PUT — substituição completa, nunca patch)

```
import java.net.URI;
import java.net.http.HttpClient;
import java.net.http.HttpRequest;
import java.net.http.HttpResponse;
import java.net.http.HttpRequest.BodyPublishers;
import java.util.LinkedHashMap;
import java.util.List;
import java.util.Map;

public class RegistrationUpdater {

    private final HttpClient http = HttpClient.newHttpClient();

    /**
     * PUT: ATENCAO – este e um replace completo do recurso, nao um PATCH
     * parcial (secao 3.2 deste capitulo). O corpo precisa conter TODA a
     * metadata que se deseja manter, nao so o campo alterado. Por isso este
     * metodo exige que o chamador passe a configuracao atual completa (tipicamente
     * obtida via um GET imediatamente anterior) e so entao aplica a mudanca
     * desejada em cima dela – nunca construa o corpo do zero.
     */
    public Map<String, String> atualizar(
        String registrationClientUri,
        String registrationAccessToken,
        Map<String, String> configuracaoAtualCompleta,
        String novoRedirectUri) throws Exception {

        // Parte da config JA EXISTENTE (obtida via GET previo) e so altera
        // o campo desejado – nunca monta um objeto novo do zero, sob risco
        // de apagar silenciosamente campos que nao foram tocados aqui.
        Map<String, Object> corpo = new LinkedHashMap<>();
        corpo.put("client_id", configuracaoAtualCompleta.get("client_id"));
        corpo.put("redirect_uris", List.of(novoRedirectUri));
        corpo.put("client_name", configuracaoAtualCompleta.get("client_name"));
        corpo.put("grant_types", List.of("authorization_code", "refresh_token"));
        corpo.put("response_types", List.of("code"));
        corpo.put("token_endpoint_auth_method", configuracaoAtualCompleta.get("token_endpoint_auth"));
        corpo.put("scope", configuracaoAtualCompleta.get("scope"));

        String corpoJson = SimpleJson.object(corpo);

        HttpRequest request = HttpRequest.newBuilder()
            .uri(URI.create(registrationClientUri))
            .header("Authorization", "Bearer " + registrationAccessToken)
            .header("Content-Type", "application/json")
            .PUT(BodyPublishers.ofString(corpoJson))
            .build();

        HttpResponse<String> response = http.send(request, HttpResponse.BodyHandlers.ofString());
    }
}
```

```
if (response.statusCode() != 200) {
    throw new IllegalStateException(
        "Atualizacao do registro falhou: HTTP " + response.statusCode());
}

Map<String, String> resposta = SimpleJson.parseFlat(response.body());

// PONTO CRITICO: o AS pode rotacionar o registration_access_token a
// cada PUT bem-sucedido (secao 3.2 / 5 deste capitulo – comportamento
// efetivamente usado pelo lab rfc7592/). O token usado NESTA chamada
// pode ja nao servir para a PROXIMA – o chamador precisa persistir
// o valor novo antes de fazer qualquer outra operacao.
String tokenRotacionado = resposta.get("registration_access_token");
if (tokenRotacionado != null && !tokenRotacionado.equals(registrationAccessToken)) {
    System.out.println("registration_access_token foi rotacionado – atualize o valor armazena
}

return resposta;
}
}
```

7.3 Apagar o registro (DELETE)

```
import java.net.URI;
import java.net.http.HttpClient;
import java.net.http.HttpRequest;
import java.net.http.HttpResponse;

public class RegistrationDeleter {

    private final HttpClient http = HttpClient.newHttpClient();

    /**
     * DELETE: remove permanentemente o registro. Resposta esperada e 204 sem
     * corpo. IMPORTANTE (secao 5 deste capitulo): isto NAO garante, por si
     * so, que tokens ja emitidos para este client parem de funcionar – a
     * menos que o Authorization Server encadeie explicitamente uma revogacao
     * em massa (RFC 7009) a este DELETE. E uma pratica recomendada de
     * implementacao, nao uma obrigacao normativa da RFC 7592.
     */
    public void apagar(String registrationClientUri, String registrationAccessToken) throws Exception {
        HttpRequest request = HttpRequest.newBuilder()
            .uri(URI.create(registrationClientUri))
            .header("Authorization", "Bearer " + registrationAccessToken)
            .DELETE()
            .build();

        HttpResponse<String> response = http.send(request, HttpResponse.BodyHandlers.ofString());

        if (response.statusCode() != 204) {
            throw new IllegalStateException(
                "Exclusao do registro falhou: HTTP " + response.statusCode());
        }
        // Sem corpo a processar. A partir daqui, este client_id deixa de ser
        // aceito no /token para NOVAS emissoes (invalid_client), replicando
        // o comportamento observavel no Lab rfc7592/.
    }
}
```

7.4 Uso combinado — ciclo de vida completo

```
import java.util.Map;

/**
 * Demonstra o ciclo de vida completo de gerenciamento sobre um registro ja
 * existente (criado previamente via RFC 7591, capitulo anterior): consulta,
 * atualizacao com rotacao de credencial, e exclusao final.
 */
public class CicloDeVidaExemplo {
    public static void main(String[] args) throws Exception {
        // Valores tipicamente obtidos da resposta de criacao (RFC 7591) e
        // persistidos pelo client – aqui hardcoded so para fins didaticos.
        String registrationClientUri = "http://localhost:3301/register/abcdef0123456789";
        String registrationAccessToken = "reg-token-inicial-devolvido-na-criacao";

        var gerenciador = new RegistrationManagementClient();
        Map<String, String> atual = gerenciador.consultar(registrationClientUri, registrationAccessToken);
        System.out.println("Configuracao atual: " + atual);
        // Note: atual.get("client_secret") sera sempre null aqui – o GET
        // nunca devolve o segredo de volta (secao 3.1 deste capitulo).

        var atualizador = new RegistrationUpdater();
        Map<String, String> resposta = atualizador.atualizar(
            registrationClientUri,
            registrationAccessToken,
            atual,
            "http://localhost:3300/novo-callback");

        // A partir daqui, o token de gerenciamento pode ter mudado –
        // sempre use o valor mais recente devolvido pela ultima chamada.
        String tokenAtual = resposta.getDefault("registration_access_token", registrationAccessToken);

        var removedor = new RegistrationDeleter();
        removedor.apagar(registrationClientUri, tokenAtual);
        System.out.println("Registro apagado – client nao pode mais obter novos tokens");
    }
}
```


Parte 5 — Formato e Introspecção de Tokens

RFC 7523 — JSON Web Token (JWT) Profile for OAuth 2.0 Client Authentication and Authorization Grants

1. Contexto e motivação

Esta RFC define, na verdade, **duas extensões independentes** que só compartilham o mesmo mecanismo de base — um JWT assinado apresentado ao token endpoint. É fácil confundir as duas ao ler o título, e vale separar isso com cuidado desde a primeira linha, porque o resto do capítulo depende dessa distinção ficar firme:

- **§2.1 — JWT como Authorization Grant.** Um jeito de trocar uma asserção assinada por um emissor terceiro confiável (um JWT, ou uma asserção SAML — a RFC 7521, o documento "guarda-chuva" que a 7523 concretiza para JWT, também tem uma irmã para SAML na RFC 7522) diretamente por um `access_token`, sem passar pelo fluxo interativo de `/authorize`. É pensado como uma ponte entre sistemas de identidade baseados em asserção — tipicamente federações corporativas, SAML Identity Providers legados — e o modelo de token do OAuth 2.0.
- **§2.2 — JWT como Autenticação de Client.** Um jeito do **client** provar sua identidade no token endpoint assinando um JWT com sua própria chave privada, em vez de apresentar um `client_secret` estático. É o método amplamente conhecido pelo nome que o OpenID Connect deu a ele — `private_key_jwt` — embora a RFC 7523 em si não use esse rótulo; ela apenas define o mecanismo genérico de "cliente se autentica com um JWT assinado".

Essas duas metades resolvem problemas completamente diferentes, e é comum uma implantação usar só uma delas (como o próprio lab deste projeto faz — só §2.2). Não há nada que force as duas a andarem juntas.

1.1 Por que o grant JWT-bearer (§2.1) existe

Antes desta RFC, uma organização que já tinha investido pesado em infraestrutura de identidade federada (SAML 2.0, WS-Federation, um IdP corporativo que já emite asserções assinadas para dezenas de sistemas internos) e queria adotar OAuth 2.0 para suas APIs enfrentava um dilema: ou reimplementava toda aquela lógica de confiança dentro do fluxo `/authorize` (redirecionamentos, telas de login duplicadas, um segundo sistema de sessão), ou aceitava que os dois mundos ficassem desconectados. O grant JWT-bearer oferece uma terceira via: pegue a asserção que o IdP já emite, apresente-a diretamente ao token endpoint do OAuth AS, e receba um `access_token` de volta — sem o resource owner jamais ver um navegador. É, na essência, uma forma de **federação de confiança encurtada em uma única chamada HTTP**, adequada sobretudo para comunicação servidor-a-servidor onde o "usuário" já foi autenticado por outro sistema e essa autenticação precisa ser traduzida para o vocabulário OAuth.

1.2 Por que a autenticação de client via JWT (§2.2) existe

A motivação da segunda metade conecta direto com um problema que atravessa todo o núcleo do OAuth: um `client_secret` estático, se vazar, é uma credencial de impersonação completa e permanente até ser rotacionado manualmente. Pior — o vazamento não precisa acontecer em trânsito; um comprometimento do **armazenamento do lado do AS** (um dump de banco de dados, um backup mal protegido) também expõe esse segredo, porque o AS precisa guardá-lo em algum formato recuperável (ou ao menos um hash comparável) para poder validar `client_secret_basic` / `client_secret_post`.

Com `private_key_jwt`, a inversão é completa: o AS só precisa conhecer a **chave pública** do client. Um vazamento do armazenamento do AS expõe apenas chaves públicas — que, por definição, não servem para impersonar ninguém. A prova de posse da chave privada acontece a cada chamada, via assinatura, e nunca trafega o segredo em si (diferente de `client_secret_post`, que literalmente coloca o segredo no corpo da requisição a cada vez). Esse é o mesmo argumento de fundo, aplicado ao client, que levou o ecossistema a preferir assinaturas assimétricas sobre segredos compartilhados em praticamente todo lugar onde a escolha existe (mTLS na RFC 8705 é a outra grande alternativa assimétrica, indo por um caminho de camada de transporte em vez de um artefato JWT).

1.3 Onde isso se encaixa historicamente

A RFC 7523 é de 2015, contemporânea de um período em que o ecossistema OAuth começava a amadurecer para cenários de alto valor (financeiro, saúde, governo) onde `client_secret` estático já soava frágil demais. Não por acaso, é também a época em que perfis de segurança mais rígidos (que décadas depois viriam a influenciar a RFC 9700) começaram a exigir ou fortemente recomendar métodos assimétricos de autenticação de client.

2. Terminologia e papéis

Nenhum papel novo em relação à RFC 6749 — client, AS, resource owner continuam os mesmos. A RFC introduz vocabulário específico para os artefatos que ela define:

- **Assertion (asserção)** — o termo genérico, herdado da RFC 7521, para "um pacote de claims assinado que uma parte apresenta como evidência de algo". Um JWT é uma forma concreta de assertion; SAML é outra.
- **JWT Bearer Token / JWT Bearer Grant** — quando a assertion é usada como grant (§2.1):
`grant_type=urn:ietf:params:oauth:grant-type:jwt-bearer`, parâmetro `assertion` carregando o JWT completo e compacto.
- **Client Assertion** — quando a assertion é usada para autenticar o client (§2.2):
`client_assertion_type=urn:ietf:params:oauth:client-assertion-type:jwt-bearer` + `client_assertion` (o JWT assinado pelo client). Isso é usado **junto** com qualquer `grant_type` normal — não substitui o grant, substitui *como o client se autentica* dentro dele.
- **Issuer da assertion vs. Issuer do AS** — vale notar uma inversão sutil de papéis entre as duas metades: no grant §2.1, o `iss` do JWT identifica quem **emitiu a asserção externa** (o IdP SAML/federação, uma parte terceira do ponto de vista do AS OAuth). No client assertion §2.2, o

`iss` (e também o `sub`) do JWT é o **próprio client_id** — o client está assinando algo sobre si mesmo, não repassando uma asserção de terceiro. É fácil, ao ler rapidamente as duas seções da RFC, achar que `iss` sempre significa "uma autoridade externa" — mas em §2.2 ele significa "o próprio ator que está se autenticando".

- **Bearer** — no sentido usado aqui (como em "Bearer Grant"), significa que a posse da assertion assinada corretamente é suficiente para uso — não há um desafio adicional de prova de posse em tempo real além da assinatura já embutida. É o mesmo sentido de "bearer" que qualifica o `access_token` Bearer da RFC 6750, generalizado para este artefato.

3. Fluxo

3.1 §2.1 — JWT como Authorization Grant

```
[Emissor de confianca, ex: IdP SAML/federacao corporativa]
| emite/assina um JWT com claims sobre uma identidade
v
Client
|
| POST /token
| grant_type=urn:ietf:params:oauth:grant-type:jwt-bearer
| &assertion=<JWT compacto: header.payload.signature>
v
Authorization Server
| 1. decodifica o JWT, identifica o "iss" (emissor)
| 2. verifica se ha relacao de confianca PRE-ESTABELECIDADA com esse iss
|    (chave publica pre-registrada, ou descoberta via federacao)
| 3. valida assinatura, exp, aud (= este AS), demais claims
v
Client <- access_token (e, opcionalmente, refresh_token — a RFC deixa a
                        criterio do AS; muitas implementacoes NAO emitem
                        refresh_token aqui, ja que a assertion original
                        pode ser reapresentada quando precisar de novo token)
```

Note o que **não** acontece: nenhum redirecionamento de navegador, nenhuma tela de login, nenhuma interação síncrona do resource owner. Tudo acontece em uma única chamada servidor-a-servidor. Isso torna esse grant adequado para integração de sistemas (B2B, pipelines batch, pontes de federação), mas inadequado como substituto do Authorization Code Grant em cenários onde o consentimento explícito e em tempo real do usuário é necessário — a própria confiança no emissor da assertion é que faz o papel do consentimento aqui, implicitamente.

3.2 §2.2 — JWT como Autenticação de Client (combinável com qualquer grant)

```
Client (já possui um par de chaves RSA/EC proprio, gerado/gerenciado por ele mesmo)
|
| monta um JWT novo a cada chamada ao /token:
|   header: {"alg":"RS256","typ":"JWT"}
|   payload: {
|     "iss": "<client_id>",
|     "sub": "<client_id>",          (identico ao iss, por definicao da RFC)
|     "aud": "<identificador do token endpoint/AS>",
|     "exp": <curto, tipicamente 60-120s no futuro>,
|     "iat": <agora>,
|     "jti": "<identificador aleatorio unico>"
|   }
|   assina com a CHAVE PRIVADA do client
v
POST /token
grant_type=authorization_code (ou refresh_token, client_credentials, ...)
&code=... (ou refresh_token=..., conforme o grant)
&client_assertion_type=urn:ietf:params:oauth:client-assertion-type:jwt-bearer
&client_assertion=<o JWT montado acima, compacto>
v
Authorization Server
| 1. extrai client_id do "iss"/"sub" do client_assertion (ainda sem
|    confiar nele – e so um indice para buscar a chave publica)
| 2. busca a chave PUBLICA registrada para esse client_id
|    (via jwks estatico ou jwks_uri, registrados pela RFC 7591)
| 3. verifica assinatura com essa chave publica
| 4. valida claims: iss==sub==client_id esperado, aud==este AS,
|    exp no futuro (mas dentro de uma janela maxima aceitavel),
|    jti nao reutilizado
| 5. SO ENTAO processa o grant_type normalmente, como se o client
|    tivesse sido autenticado por client_secret_basic
v
Client <- access_token (resposta identica a de qualquer outro metodo de
                        autenticacao de client – RFC 7523 nao muda o
                        formato da resposta do /token, so como o client
                        provou sua identidade antes de chegar la)
```

A observação central: **cada chamada ao /token carrega uma client_assertion nova**, montada e assinada na hora. Não é um token de longa duração reaproveitado — é mais parecido, conceitualmente, com uma prova de posse renovada a cada uso (o mesmo espírito de "prove de novo, a cada vez" que aparece, em formato diferente, na DPOP).

4. Parâmetros e endpoints normativos

Nenhum endpoint novo — as duas metades reaproveitam o /token da RFC 6749. Toda a superfície nova é de **parâmetros** e de **claims dentro do JWT**.

4.1 Parâmetros de requisição

Parâmetro	Usado em	Obrigatório	Descrição
<code>grant_type=urn:ietf:params:oauth:grant-type:jwt-bearer</code>	§2.1	sim	Identifica o grant como JWT-bearer
<code>assertion</code>	§2.1	sim	O JWT (ou asserção SAML, na RFC irmã 7522) compacto
<code>client_assertion_type=urn:ietf:params:oauth:client-assertion-type:jwt-bearer</code>	§2.2	sim, quando usando este método	Identifica o formato da <code>client_assertion</code>
<code>client_assertion</code>	§2.2	sim, quando usando este método	O JWT assinado pelo client
<code>scope</code>	ambos	opcional	Mesmo parâmetro de sempre — escopo requisitado

Um detalhe fácil de escorregar: em §2.2, `client_assertion_type` e `client_assertion` **acompanham** o `grant_type` real da operação (`authorization_code`, `refresh_token`, `client_credentials` ...) — eles não competem com ele. Uma requisição de Authorization Code Grant autenticada via `private_key_jwt` carrega **cinco** parâmetros relevantes ao mesmo tempo: `grant_type=authorization_code`, `code`, `redirect_uri`, `client_assertion_type`, `client_assertion`.

4.2 Claims dentro do JWT (ambos os usos)

Claim	§2.1 (grant)	§2.2 (client assertion)
<code>iss</code>	Emissor da asserção externa (o IdP, a federação)	<code>client_id</code> do próprio client
<code>sub</code>	O sujeito de quem a asserção fala (pode ou não ser igual ao <code>iss</code>)	<code>client_id</code> — idêntico ao <code>iss</code> aqui, por definição
<code>aud</code>	Identificador do AS (tipicamente a URL do token endpoint)	Idem — identificador do AS/token endpoint
<code>exp</code>	Obrigatória	Obrigatória, e tipicamente com janela bem mais curta (minutos, não horas)
<code>nbf</code>	Opcional ("not before")	Opcional
<code>iat</code>	Opcional, mas recomendada	Recomendada — usada por implementações para impor uma janela máxima de "quão no passado" a assertion pode ter sido emitida

Claim	§2.1 (grant)	§2.2 (client assertion)
jti	Opcional, mas fortemente recomendada contra replay	Fortemente recomendada — é o principal mecanismo antirreplay de §2.2

Vale insistir num ponto que a RFC deixa implícito mas que gera confusão prática: em §2.2, **aud** **deve ser o identificador do próprio Authorization Server/token endpoint** — não do resource server, não de um recurso qualquer. É comum, num sistema com múltiplos ASs (ambientes, regiões, tenants), um erro de configuração colocar o **aud** errado e o AS legitimamente rejeitar a assertion. Isso não é um bug do protocolo — é exatamente o que **aud** foi desenhado para impedir: uma assertion cunhada para o AS de homologação não deveria, por acidente ou má-fé, ser aceita pelo AS de produção.

4.3 Respostas de erro

Como a assertion (em ambos os casos) é parte do processo de **autenticação/autorização da requisição**, e não do fluxo interativo, os erros seguem o vocabulário já estabelecido do `/token` (RFC 6749 §5.2):

- **invalid_client** — a assertion de §2.2 falhou (assinatura inválida, **aud** / **iss** / **sub** incorretos, expirada, **jti** já usado). É o código correto porque o problema é a **identidade do client**, não o grant em si.
- **invalid_grant** — a assertion de §2.1 falhou (emissor não confiável, assinatura inválida, expirada). Aqui o problema é o **grant/assertão apresentada**, não a identidade de quem chama.

Essa distinção entre **invalid_client** (§2.2) e **invalid_grant** (§2.1) é sutil mas semanticamente importante: no primeiro caso, é o *chamador* que falhou em se provar; no segundo, é o *material apresentado como prova de autorização* que falhou em ser aceito.

5. Segurança já tratada na própria RFC 7523

5.1 Nenhum segredo compartilhado necessário para autenticação de client

Como já discutido na motivação: o AS só precisa da chave **pública**. Um vazamento do armazenamento do AS não dá capacidade de impersonação — muito diferente de um **client_secret** vazado, que é diretamente reutilizável por quem o obtiver.

5.2 Proteção contra replay via **exp** + **jti**

Uma **client_assertion** capturada (por um proxy comprometido, um log mal protegido, um ataque de rede) só vale por uma janela curta — e o **jti**, se rastreado pelo AS, permite rejeitar reapresentação da mesma asserção mesmo **dentro** dessa janela de validade. É o mesmo espírito do código de uso único (**authorization_code**) aplicado aqui ao próprio artefato de autenticação: uma coisa é a assertion estar dentro do prazo de validade; outra, independente, é ela já ter sido gasta.

Dois detalhes operacionais que a RFC não especifica, mas que toda implementação real precisa decidir:

- **Por quanto tempo guardar o `jti` ?** Guardar para sempre é impraticável; guardar só até o `exp` da própria assertion é o mínimo necessário e suficiente — depois que ela expira, reapresentá-la já falharia na checagem de `exp` de qualquer forma, então rastrear o `jti` além desse ponto não agrega proteção. O lab deste projeto usa exatamente essa estratégia: TTL do registro de `jti` no Redis = TTL máximo aceito para o `exp` da assertion.
- **`jti` é obrigatório?** A RFC 7523 o lista como opcional na letra da norma, mas sem `jti` — ou algum mecanismo equivalente — a proteção antirreplay dentro da janela de `exp` simplesmente não existe. Implementações sérias (e o próprio lab deste projeto) tratam `jti` como obrigatório na prática, mesmo a RFC não exigindo.

5.3 `aud` amarra a asserção a um AS específico

Uma `client_assertion` construída para um AS não deveria ser aceita por outro, mesmo que apresentada lá — é mais uma instância do mesmo padrão recorrente de restrição de audiance que aparece na RFC 8707 (Resource Indicators) e no `aud` dos Request Objects da RFC 9101 (JAR). O caso concreto de risco: uma organização com AS de sandbox e AS de produção compartilhando o mesmo par de chaves de client (reuso indevido de credenciais entre ambientes) — sem `aud`, uma assertion vazada do ambiente de sandbox seria diretamente utilizável em produção.

5.4 Confiança pré-estabelecida no emissor (§2.1)

Para o grant do §2.1, o AS precisa de uma relação de confiança **pré-estabelecida** com o emissor (`iss`) da asserção — JWTs autoassinados de partes desconhecidas obviamente não deveriam ser aceitos. Isso normalmente significa chave/certificado do emissor pré-registrado (fora de banda, num processo administrativo) ou descoberto via federação confiável (um metadado de federação SAML, por exemplo). Um AS que aceitasse **qualquer** `iss` sem essa checagem estaria, na prática, permitindo que qualquer parte cunhasse tokens para qualquer sujeito — uma falha catastrófica de design, não uma nuance.

5.5 Nuances de segurança que a RFC pressupõe mas não detalha exaustivamente

Vale a pena o leitor ter em mente, porque são exatamente os pontos onde implementações reais tropeçam:

- **Confusão de algoritmo ("alg confusion").** Um JWT declara seu próprio algoritmo de assinatura no header (`alg`). Um validador ingênuo que lê esse campo e o usa cegamente para decidir *como* verificar abre uma classe conhecida de vulnerabilidade: se o validador aceita `alg: none` (sem assinatura) ou permite que uma chave pública RSA seja reinterpretada como segredo HMAC (confundindo RS256 com HS256 — o ataque clássico "RS256-to-HS256"), um atacante pode forjar assertions sem jamais possuir a chave privada. A defesa correta é o **validador** — não o token — decidir de antemão qual algoritmo é esperado para aquele `client_id`, e rejeitar qualquer coisa diferente, nunca delegar essa decisão ao próprio `alg` do header recebido.
- **Ataques de temporização (timing attacks) na verificação de assinatura.** Implementações de baixo nível de comparação criptográfica que não são de tempo constante podem, em tese, vaziar informação através de quanto tempo a verificação leva. Bibliotecas JOSE

maduras já tratam disso; uma implementação artesanal (como o exemplo didático deste capítulo) tipicamente não se preocupa com isso — outro motivo para nunca usar código manual em produção.

- **Deriva de relógio (clock skew).** Como `exp / iat / nbf` dependem de sincronismo de relógio entre client e AS, implementações costumam tolerar uma pequena margem (segundos a poucos minutos) — mas essa margem, se grande demais, amplia a janela de replay possível mesmo com `jti` funcionando corretamente.
- **Rotação de chave do client sem downtime.** Como o AS precisa conhecer a chave pública vigente do client, revogar/rotacionar uma chave comprometida exige um mecanismo de atualização (reregistro via RFC 7591, ou um `jwks_uri` que o AS resolve dinamicamente) — se o client só tem uma chave estática registrada uma vez, uma chave privada comprometida vira um segredo estático de fato, na prática, até alguém rotacionar manualmente.

6. Relação com outras RFCs

- A metade de autenticação de client é exatamente o `private_key_jwt` — é aqui que esse método, mencionado no capítulo da RFC 9700 ("prefere métodos assimétricos a `client_secret` estático"), é definido concretamente.
- `jwks / jwks_uri` usados para validar assinaturas vêm do metadata de registro da **RFC 7591** — conexão direta com aquele capítulo: é lá que o client informa sua chave pública ao AS pela primeira vez.
- Descoberto via **RFC 8414**: `token_endpoint_auth_methods_supported` listaria `private_key_jwt` se suportado, e `token_endpoint_auth_signing_alg_values_supported` indica quais algoritmos de assinatura o AS aceita (o lab deste projeto anuncia só `["RS256"]`).
- O grant do §2.1 é conceitualmente antecessora/irmã da **RFC 8693 (Token Exchange)** — ambas fazem ponte entre "tenho algum tipo de token/asserção externa" e "quero um token OAuth", mas a 8693 é mais geral (qualquer tipo de token, distinção subject/actor explícita, direcionamento por audience/resource via RFC 8707). A 8693 não substituiu a 7523 — coexistem para níveis de complexidade diferentes; a 7523 permanece mais simples para o caso específico "tenho um JWT/SAML assertion, quero trocar por `access_token`", sem a generalidade (e complexidade) adicional do Token Exchange.
- **RFC 8705 (mTLS)** é a outra grande alternativa assimétrica de autenticação de client — em vez de assinar um JWT a cada chamada, o client apresenta um certificado de cliente na própria camada TLS. As duas soluções resolvem o mesmo problema de fundo (eliminar o segredo estático) por caminhos completamente diferentes: uma no nível de aplicação (JWT no corpo da requisição), outra no nível de transporte (handshake TLS). Uma implantação pode até suportar ambas e deixar o client escolher.
- Vale distinguir das **outras** duas assinaturas de JWT que aparecem no roteiro, para não confundir: o Request Object da **RFC 9101 (JAR)** protege a integridade da *authorization request*; o `client_assertion` desta RFC autentica o *client* no token endpoint; e o JWT-bearer grant desta mesma RFC troca uma *asserção externa* por token. Três propósitos diferentes, todos usando "um JWT assinado" como mecanismo — o leitor que só lembra "JWT assinado no OAuth" sem saber qual dos três está diante de si é um convite a erro de design.

7. O que mudou desde a publicação (2015)

- A metade de **autenticação de client** (`private_key_jwt`) cresceu bastante em relevância — explicitamente recomendada sobre segredo estático pela RFC 9700 (2025), uma década depois, e exigida por perfis FAPI (Financial-grade API) para clients confidenciais em muitos casos de alto valor. Segue a mesma trajetória "recomendado → cada vez mais esperado" já vista com PKCE, embora `private_key_jwt` não tenha se tornado obrigatório universal como o PKCE se tornou.
- A metade do **grant JWT-bearer** (§2.1) cresceu comparativamente menos — seu caso de uso de "ponte" foi em boa parte absorvido por designs novos usando a RFC 8693 (Token Exchange), mais flexível e mais expressiva para cenários modernos de delegação/impersonação. A 7523 continua válida e mais simples, ainda usada especialmente em pontes legadas SAML-para-OAuth que já existiam antes da 8693 existir.
- A discussão de "qual método de autenticação de client escolher" hoje frequentemente se resume a uma comparação de três vias assimétricas praticamente equivalentes em garantia de segurança, mas bem diferentes em modelo operacional: `private_key_jwt` (RFC 7523, nível de aplicação), `tls_client_auth` / `self_signed_tls_client_auth` (RFC 8705, nível de transporte), e — cada vez mais discutido, embora fora do escopo direto deste projeto — atestação de dispositivo/chave em hardware. A escolha entre as duas primeiras costuma depender de qual infraestrutura já existe: se a organização já tem PKI de mTLS madura, RFC 8705 é natural; se o client já opera em ambiente onde gerar e assinar JWTs é trivial (a maioria dos runtimes modernos), `private_key_jwt` é mais simples de operacionalizar sem tocar na camada de transporte.

Nota de laboratório: implementado em `rfc7523/` — réplica do lab básico da RFC 6749, mas **sem `client_secret` nenhum**: o client-demo gera seu próprio par de chaves RSA-2048, registra a pública via `jwtks` (RFC 7591), e autentica todo `grant_type` (Authorization Code, refresh, Client Credentials) via `client_assertion` (§2.2). Só a metade de autenticação de client é implementada — o grant JWT-bearer do §2.1 fica de fora, por escolha de escopo. O Authorization Server (`rfc7523/authorization-server/src/index.js`) valida a `client_assertion` exigindo `alg=RS256`, checando `iss` / `sub` (ambos = `client_id`), `aud` (= URL base do próprio AS, montada a partir do header `Host` da requisição) e `exp` (com uma janela máxima aceita de 120 segundos), e rastreia o `jti` já usado no Redis via `SET NX` com TTL igual a essa mesma janela — o mesmo prazo usado tanto para validar a assertion quanto para lembrar que ela já foi gasta. Há inclusive um botão na UI que reenvia deliberadamente a mesma `client_assertion` para provar a rejeição por replay. Os demais labs (`rfc6749/`, `rfc7636/`, `rfc9126/`, `rfc7592/`) continuam usando `client_secret_basic` / `client_secret_post`.

Exemplo em Java (JDK 21, sem frameworks)

Os exemplos abaixo constroem, assinam e enviam uma `client_assertion` (RFC 7523 §2.2) usando **apenas** a biblioteca padrão do JDK — `java.security` para geração de chaves e assinatura, `java.util.Base64` para a codificação Base64URL sem padding que o formato JWT compacto exige, e `java.net.http.HttpClient` para a chamada ao token endpoint. O utilitário `SimpleJson` (métodos

`SimpleJson.object(Map)` e `SimpleJson.parseFlat(String)`), definido no Capítulo 1 (RFC 6749) deste livro, é reaproveitado para montar/interpretar os objetos JSON planos envolvidos — não é redefinido aqui.

Aviso didático, para não deixar dúvida: montar e verificar JWTs "na mão" com `java.security.Signature`, como fazemos abaixo, serve para **expor a mecânica** do formato — três partes Base64URL separadas por ponto, cada uma com um papel específico — e não deve ser copiado para um sistema real. Uma biblioteca JOSE madura (a referência do ecossistema Java é o **Nimbus JOSE+JWT**) trata dezenas de detalhes sutis que uma implementação artesanal tende a errar: rejeição categórica de `alg: none`, prevenção de confusão entre algoritmos assimétricos e simétricos (o ataque "RS256-para-HS256" mencionado na seção 5.5), comparação de assinatura em tempo constante, parsing robusto contra entradas malformadas ou adversariais, e suporte correto a todas as variantes de curva/tamanho de chave. Cada uma dessas é uma vulnerabilidade em potencial se implementada incorretamente — e "incorretamente" aqui não significa "com bugs óbvios", significa "sutilmente errado de um jeito que só aparece sob ataque".

4.1 Gerando o par de chaves do client (RSA-2048)

```
import java.security.KeyPair;
import java.security.KeyPairGenerator;
import java.security.NoSuchAlgorithmException;
import java.security.SecureRandom;

/**
 * O client (nao o AS!) gera e guarda este par de chaves. A publica e' o que
 * vai para o AS via registro dinamico (RFC 7591, campo "jwks"); a privada
 * NUNCA sai do processo do client - e' ela que assina cada client_assertion.
 *
 * RSA-2048 e RS256 (RSASSA-PKCS1-v1_5 com SHA-256) e' o par minimo que a
 * maioria dos ASs aceita hoje; perfis mais rigorosos (FAPI) frequentemente
 * exigem 2048 bits como piso e recomendam considerar PS256 (RSASSA-PSS)
 * ou ES256 (curva eliptica) como alternativas mais modernas.
 */
public final class ClientKeyMaterial {

    public static KeyPair gerarParDeChaves() throws NoSuchAlgorithmException {
        KeyPairGenerator gerador = KeyPairGenerator.getInstance("RSA");
        gerador.initialize(2048, SecureRandom.getInstanceStrong());
        return gerador.generateKeyPair();
        // Em producao: a chave privada resultante deveria ser protegida em
        // repouso (HSM, KMS, ou ao menos um keystore criptografado) - jamais
        // gravada em texto plano em disco ou em variavel de ambiente.
    }
}
```

4.2 Montando e assinando o `client_assertion` (RFC 7523 §2.2)

```
import java.nio.charset.StandardCharsets;
import java.security.PrivateKey;
import java.security.Signature;
import java.time.Instant;
import java.util.Base64;
import java.util.LinkedHashMap;
import java.util.Map;
import java.util.UUID;

/**
 * Constroi o JWT compacto (header.payload.assinatura, cada parte em
 * Base64URL sem padding – e' isso que RFC 7515 §2 chama de "JWS Compact
 * Serialization") que serve como client_assertion.
 *
 * Claims conforme RFC 7523 §3:
 *   iss/sub = client_id (o client se autenticando, nao um terceiro)
 *   aud      = identificador do token endpoint/AS – nunca do resource server
 *   exp      = janela CURTA (aqui, 90s) – quanto menor, menor a superficie
 *              de replay caso a assertion vaze antes de ser usada
 *   jti      = identificador aleatorio unico – e' o que permite ao AS
 *              detectar reapresentacao da MESMA assertion (replay)
 */
public final class ClientAssertionBuilder {

    private static final Base64.Encoder B64URL =
        Base64.getUrlEncoder().withoutPadding();

    public static String montarESinar(String clientId, String audienceTokenEndpoint,
        PrivateKey chavePrivada) throws Exception {

        // Header: alg deve refletir o que a chave privada realmente suporta –
        // nunca aceite "alg" vindo de configuracao externa nao confiavel na
        // hora de assinar (isso e' preocupacao de quem VALIDA, mas quem
        // assina tambem deveria fixar o algoritmo, nao inferi-lo de outrem).
        Map<String, Object> header = new LinkedHashMap<>();
        header.put("alg", "RS256");
        header.put("typ", "JWT");

        long agora = Instant.now().getEpochSecond();
        Map<String, Object> payload = new LinkedHashMap<>();
        payload.put("iss", clientId);
        payload.put("sub", clientId); // identico ao iss, por definicao (§2.2)
        payload.put("aud", audienceTokenEndpoint); // amarra a assertion a ESTE AS
        payload.put("iat", agora);
        payload.put("exp", agora + 90); // janela curta de validade
        payload.put("jti", UUID.randomUUID().toString()); // defesa antirreplay

        String headerB64 = B64URL.encodeToString(
            SimpleJson.object(header).getBytes(StandardCharsets.UTF_8));
        String payloadB64 = B64URL.encodeToString(
```

```
SimpleJson.object(payload).getBytes(StandardCharsets.UTF_8));

String assinavel = headerB64 + "." + payloadB64;

Signature assinador = Signature.getInstance("SHA256withRSA");
assinador.initSign(chavePrivada);
assinador.update(assinavel.getBytes(StandardCharsets.US_ASCII));
byte[] assinatura = assinador.sign();

String assinaturaB64 = B64URL.encodeToString(assinatura);

return assinavel + "." + assinaturaB64; // JWT compacto completo
}
```

4.3 Enviando o `client_assertion` ao token endpoint

```
import java.net.URI;
import java.net.http.HttpClient;
import java.net.http.HttpRequest;
import java.net.http.HttpResponse;
import java.net.http.HttpRequest.BodyPublishers;
import java.nio.charset.StandardCharsets;
import java.security.PrivateKey;
import java.util.Map;

/**
 * Troca um authorization_code por access_token, autenticando o client via
 * private_key_jwt em vez de client_secret_basic/post – note que
 * client_assertion_type e client_assertion viajam ao LADO do grant_type
 * normal, nunca no lugar dele (RFC 7523 §2.2).
 */
public final class TokenRequestComPrivateKeyJwt {

    public static Map<String, String> trocarCodigoPorToken(
        String tokenEndpoint, String clientId, String code, String redirectUri,
        PrivateKey chavePrivada) throws Exception {

        String clientAssertion = ClientAssertionBuilder.montarESinar(
            clientId, tokenEndpoint, chavePrivada);

        String corpo = "grant_type=authorization_code"
            + "&code=" + code
            + "&redirect_uri=" + redirectUri
            + "&client_assertion_type=" +
                "urn:ietf:params:oauth:client-assertion-type:jwt-bearer"
            + "&client_assertion=" + clientAssertion;
        // Em producao: os valores acima (code, redirect_uri, JWT) devem
        // passar por URL-encoding – omitido aqui por brevidade didatica.

        HttpClient http = HttpClient.newHttpClient();
        HttpRequest req = HttpRequest.newBuilder()
            .uri(URI.create(tokenEndpoint))
            .header("Content-Type", "application/x-www-form-urlencoded")
            .POST(BodyPublishers.ofString(corpo, StandardCharsets.UTF_8))
            .build();

        HttpResponse<String> resp = http.send(req, HttpResponse.BodyHandlers.ofString());

        if (resp.statusCode() != 200) {
            // invalid_client aqui tipicamente significa: assinatura invalida,
            // aud/iss/sub incorretos, exp expirado, ou jti ja usado (replay).
            throw new IllegalStateException(
                "Token endpoint recusou a client_assertion: " + resp.body());
        }

        return SimpleJson.parseFlat(resp.body()); // access_token, token_type, expires_in...
```

```
}  
}
```

Note o contraste com o Capítulo 1 (RFC 6749): lá, a autenticação do client é uma única linha (`Authorization: Basic base64(client_id:client_secret)` ou dois parâmetros no corpo). Aqui, a "credencial" é reconstruída — assinada do zero — a cada chamada, o que é exatamente o ponto: nada estático e reutilizável trafega pela rede.

RFC 7662 — OAuth 2.0 Token Introspection

1. Contexto e motivação

Quando o `access_token` é **opaco** — uma string aleatória sem estrutura interpretável, como em todos os labs deste projeto até este capítulo — o resource server não tem como, sozinho, saber se ele é válido, qual escopo carrega, ou a quem foi emitido. Ele precisa perguntar a alguém que saiba. Antes desta RFC (2015), essa pergunta era resolvida de forma **proprietária** por cada Authorization Server — o mesmo tipo de lacuna de interoperabilidade que motivou a RFC 7009 para revogação e a RFC 8414 para descoberta: sem um formato comum, cada fornecedor de AS expunha seu próprio endpoint ad-hoc de validação, com seu próprio esquema de request/response, e um resource server escrito para um AS não funcionava com outro.

A RFC 7662 formaliza um **endpoint de introspecção**: o resource server apresenta o token, o AS responde com um JSON estruturado dizendo se está ativo e, se estiver, seus metadados (escopo, client, usuário, expiração...). É a terceira peça que completa o ciclo de vida do token, ao lado da RFC 6749 (emissão) e da RFC 7009 (revogação): **emitir, validar, revogar** — três operações distintas, três RFCs, um só objeto de vida (o token) atravessando todas elas.

1.1 O problema de arquitetura que a introspecção resolve

Vale nomear explicitamente o desenho arquitetural que está por trás disso. Num sistema OAuth real, o Authorization Server e o Resource Server são frequentemente **processos separados, mantidos por times separados, às vezes empresas separadas** (pense num AS central de uma organização emitindo tokens que múltiplos microsserviços, escritos por times diferentes, precisam aceitar). O AS é o único que sabe, com autoridade, se um token ainda é válido — ele tem acesso ao armazenamento onde tokens são emitidos e revogados. O resource server, por definição, não tem esse acesso direto (e não deveria ter — compartilhar o armazenamento interno do AS com N resource servers diferentes seria um acoplamento arquitetural ruim, além de um risco de segurança, pois qualquer RS comprometido teria acesso de leitura a todos os tokens de todos os clients).

A introspecção resolve isso com o padrão mais simples possível: uma chamada HTTP síncrona, autenticada, do RS para o AS, perguntando "este token específico ainda vale?". É o equivalente, em desenho de sistemas, de um serviço consultando a fonte de verdade em vez de manter uma cópia local potencialmente desatualizada.

1.2 Por que não simplesmente compartilhar o token opaco "batendo" contra algum banco comum

Alguém poderia perguntar: por que não deixar o RS ler diretamente o armazenamento (banco/cache) onde o AS guarda os tokens, evitando a chamada de rede? É exatamente isso que **todos os labs deste projeto fazem hoje** (ver nota de laboratório, seção 6) — e funciona *só porque é o mesmo laboratório*, com AS e RS compartilhando deliberadamente o mesmo Redis para fins didáticos. Num sistema de produção real, com AS e RS mantidos por partes diferentes, isso é inviável: exigiria dar a cada RS

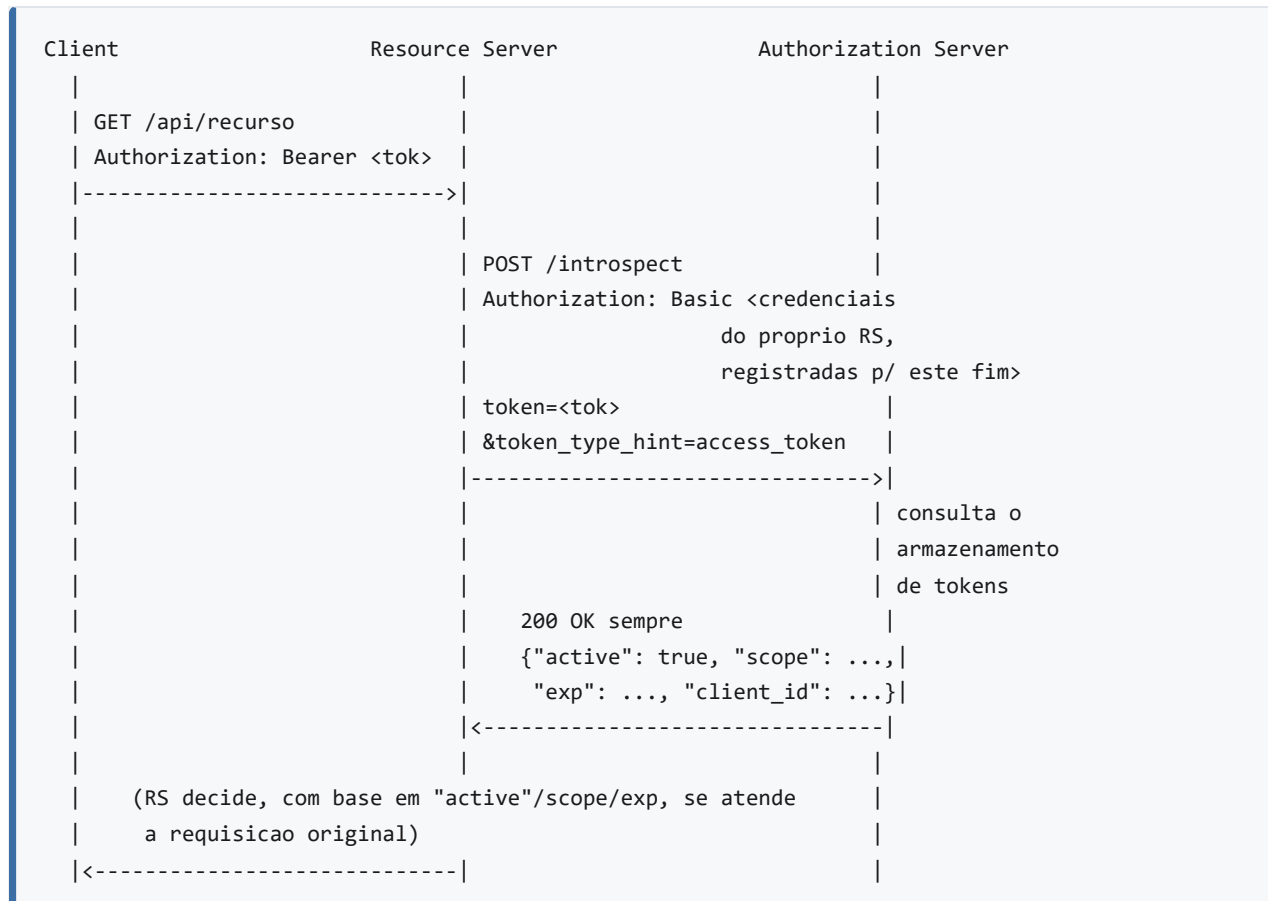
acesso de leitura direto à infraestrutura de dados do AS, quebrando o isolamento entre serviços, criando uma superfície de ataque enorme (qualquer RS comprometido veria os tokens de todo mundo) e acoplando o esquema de armazenamento do AS à API que os RSs consomem (uma mudança interna no AS quebraria RSs). A introspecção existe precisamente para dar ao RS uma **interface estável e desacoplada** para a mesma pergunta, sem esses custos.

2. Terminologia e papéis

Nenhum papel novo — mas a RFC usa consistentemente o termo "**protected resource**" para quem chama a introspecção, reforçando que o público-alvo pretendido é o resource server, não qualquer parte arbitrária. Isso tem uma consequência de design que vale destacar: a RFC **não** define introspecção como um serviço de "autoatendimento" onde o portador de um token pergunta sobre o próprio token. O padrão de uso pretendido é sempre "uma parte que **recebeu** o token de outra (o client, numa requisição de API) quer validá-lo perante quem o emitiu" — nunca "eu, que possuo este token, quero saber sobre ele".

Essa distinção importa na prática: implementações que expõem introspecção sem autenticação adequada, ou que a tratam como um endpoint de conveniência para qualquer chamador, estão se afastando do modelo de confiança pretendido pela RFC (ver seção 5.1).

3. Fluxo



Dois pontos centrais no fluxo:

1. **Quem chama `/introspect` é o Resource Server, autenticado como tal** — não o client, não o resource owner. As credenciais usadas (`Authorization: Basic` no exemplo, mas a RFC não fixa o método) são do próprio RS, registradas junto ao AS especificamente para esse propósito.
2. **A resposta é sempre HTTP 200**, mesmo quando o token é inválido, expirado ou desconhecido — o corpo é que carrega a informação de validade, nunca o código de status HTTP (ver seção 5.2 para a razão de design).

3.1 Exemplo de resposta — token ativo

```
{
  "active": true,
  "scope": "profile email",
  "client_id": "s6BhdRkqt3",
  "username": "alice",
  "token_type": "Bearer",
  "exp": 1787510000,
  "iat": 1787506400,
  "sub": "alice",
  "aud": "https://rs.exemplo.com",
  "iss": "https://as.exemplo.com"
}
```

3.2 Exemplo de resposta — token inválido, expirado ou desconhecido

```
{
  "active": false
}
```

Se o token não existe, expirou ou foi revogado: `{"active": false}` — e **nenhum outro campo** deveria acompanhar isso. A RFC é explícita que, quando `active` é `false`, o servidor **não deveria** incluir outros campos — é uma medida deliberada de minimização de informação (ver seção 5.2).

4. Parâmetros e endpoints normativos

4.1 Endpoint

- **Introspection endpoint** — novo, descoberto via **RFC 8414** como `introspection_endpoint` no metadata do AS.

4.2 Parâmetros de requisição

Parâmetro	Obrigatório	Descrição
token	sim	O valor do token a verificar (access ou refresh)
token_type_hint	não	access_token ou refresh_token — mesma lógica de "dica" já vista na RFC 7009: acelera a busca no armazenamento do AS, mas o AS deve tentar os demais tipos se a dica não bater, nunca tratá-la como garantia

A RFC é deliberadamente **genérica quanto ao tipo de token** que pode ser introspectado — tanto `access_token` quanto `refresh_token` são candidatos válidos, e o `token_type_hint` existe justamente para não obrigar o AS a varrer todos os armazenamentos possíveis quando o chamador já sabe qual tipo está enviando.

4.3 Autenticação do chamador

Obrigatória — o endpoint precisa estar protegido; não é aberto a qualquer parte. A RFC não fixa um único método (poderia ser `client_secret_basic`, um certificado mTLS, ou qualquer outro método de autenticação de client já suportado pelo AS) — o ponto normativo é que **deve haver** autenticação, delegando o método específico para o mesmo cardápio já usado no restante do AS.

4.4 Campos da resposta

Campo	Presente quando active=false ?	Descrição
active	sempre (obrigatório)	Booleano — o único campo garantido em toda resposta
scope	não deveria	String espaço-delimitada, mesmo formato de sempre
client_id	não deveria	A quem o token foi emitido
username	não deveria	Identificador legível do resource owner, se aplicável
token_type	não deveria	Tipicamente <code>Bearer</code>
exp, iat, nbf	não deveria	Timestamps Unix — expiração, emissão, "não antes de"
sub	não deveria	De quem o token fala
aud	não deveria	Para quem o token foi destinado
iss	não deveria	Quem emitiu
jti	não deveria	Identificador único do token

Não é acaso que boa parte desses nomes espelha exatamente as claims padrão de JWT definidas na RFC 7519 (`iss`, `sub`, `aud`, `exp`, `iat`, `nbf`, `jti`) — a RFC 7662 reaproveita deliberadamente esse vocabulário semântico, de modo que o **mesmo conjunto de conceitos** ("quem emitiu, para quem,

até quando, com que escopo") sirva tanto para um token opaco introspectado quanto para um JWT autocontido (RFC 9068, capítulo seguinte). Um resource server que já sabe interpretar essas claims de um JWT não precisa reaprender um vocabulário novo para interpretar a resposta de introspecção de um token opaco.

5. Segurança já tratada na própria RFC 7662

5.1 O endpoint precisa estar protegido

Introspecção revela informação potencialmente sensível sobre um token — escopo concedido, identidade do usuário, client ao qual pertence — e não deveria estar aberta a chamadores arbitrários ou não autenticados. Isso conecta direto com a distinção de papéis da seção 2: introspecção **não é** pensada para "o próprio portador do token perguntar sobre si mesmo" — é para o **resource server**, que recebeu o token de outra parte, validá-lo perante quem o emitiu. Um endpoint de introspecção aberto seria, na prática, um oráculo de metadados de token para qualquer parte que conseguisse capturar (ou adivinhar) um valor de token — mesmo sem conseguir *usar* esse token para acessar recursos, aprenderia coisas sobre ele (a quem pertence, que escopo tem, quando expira).

5.2 `active: false` revela o mínimo possível

A RFC recomenda que, para um token inválido, a resposta traga só o campo `active: false`, sem mais nada — mesmo princípio anti-oráculo já visto na RFC 7009 (ali operacionalizado via "sempre retorna 200, mesmo para token já inexistente"; aqui via um campo `active` minimalista quando falso). O raciocínio de fundo é idêntico ao da revogação: se a resposta para "token inexistente" fosse diferente (em forma, em código de status, ou em quais outros campos aparecem) da resposta para "token existe mas está expirado" ou "token existe mas foi revogado", um chamador malicioso poderia usar essa diferença para **sondar** quais tokens existem no sistema, mesmo sem autoridade para vê-los de outra forma — um vetor de enumeração. Manter a resposta uniforme (`{"active": false}`), sempre, para qualquer razão de invalidade) fecha esse canal lateral de informação.

5.3 TLS obrigatório

Como em qualquer endpoint que manipula material de token — o valor do `token` viaja no corpo da requisição, e a resposta carrega metadados sensíveis —, TLS é mandatório. Sem ele, tanto o token quanto os metadados de introspecção (identidade do usuário, escopo) trafegariam em claro.

5.4 Custo operacional e o trade-off de atualidade vs. desempenho

Como a introspecção tipicamente acontece a cada requisição recebida pelo resource server (a menos que haja cache local), ela cria uma **dependência de rede síncrona** no caminho crítico de toda chamada de API, e um possível vetor de sobrecarga no AS se muitos RSs introspectarem com alta frequência. Isso não é um detalhe de implementação — é uma característica estrutural do modelo: cada requisição ao RS agora depende da disponibilidade e latência do AS, mesmo que a operação em si nada tenha a ver com autenticação.

Implantações costumam mitigar isso com **cache local de curta duração** no lado do RS (segundos a poucos minutos), trocando uma fração de atualidade da informação de revogação por uma redução considerável de round-trips. O trade-off é explícito e precisa ser uma decisão consciente: um cache de 60 segundos significa que um token revogado no AS pode continuar sendo aceito pelo RS por até 60 segundos depois — uma janela de exposição real, ainda que pequena, que a equipe de segurança precisa aprovar conscientemente, não descobrir depois de um incidente.

5.5 Nuances adicionais que a prática revela

- **Introspecção de refresh_token.** Como a RFC é genérica quanto ao tipo de token, um AS pode (e alguns fazem) permitir introspectar um `refresh_token` também — mas isso levanta uma questão de escopo de autorização adicional: nem todo RS deveria ter permissão de introspectar refresh tokens (que costumam ter vida muito mais longa e maior poder), então implementações reais tipicamente restringem quem pode introspectar o quê, com base em qual credencial de RS está autenticada.
- **token_type_hint como otimização, não como garantia.** Assim como na RFC 7009, um chamador pode enviar a dica errada (ou nenhuma) — o AS deve continuar funcional, só potencialmente mais lento, nunca deve simplesmente falhar ou assumir que a dica está correta sem verificar.
- **Quem pode introspectar o token de quem.** A RFC não impõe uma regra única aqui, mas implementações sérias costumam restringir a introspecção ao(s) resource server(s) para os quais aquele token foi efetivamente emitido (via `aud`, quando presente) — evitando que um RS "curioso" introspece tokens destinados a outros RSs só porque possui credenciais válidas de chamador.
- **Introspecção não substitui verificação de escopo do lado do RS.** Saber que `active: true` e que o `scope` inclui `read:orders` não dispensa o RS de aplicar sua própria lógica de autorização (ex.: o usuário específico tem permissão sobre *aquele* recurso específico?) — introspecção resolve "este token é válido e o que ele autoriza em termos amplos", não "esta operação específica é permitida".

6. Relação com outras RFCs

- Fecha o trio de gerenciamento de ciclo de vida com **RFC 6749** (emissão) e **RFC 7009** (revogação) — emitir, revogar, e agora validar. As três RFCs, lidas em conjunto, cobrem completamente o que significa "gerenciar um token do nascimento à morte".
- É exatamente o que os resource servers de **todos os labs deste projeto** ainda não fazem: cada `rfc*/resource-server` valida o `access_token` lendo **diretamente** do mesmo Redis que o AS escreve — só funciona porque é o mesmo laboratório, com AS e RS compartilhando deliberadamente a mesma infraestrutura de dados para fins didáticos. Um resource server de verdade, desacoplado do AS (mantido por outro time, outra empresa, sem acesso a essa infraestrutura), chamaria `/introspect` no AS em vez disso.
- Descoberta via **RFC 8414** como `introspection_endpoint`.

- Contraste direto com o próximo capítulo, **RFC 9068 (JWT Profile for Access Tokens)**: se o AS emite tokens em formato JWT em vez de opaco, o resource server pode validar **localmente**, por assinatura, sem round-trip de rede nenhum — introspecção e token-como-JWT são as duas estratégias concorrentes para o mesmo problema de fundo ("como o RS sabe que o token é válido, e o que ele autoriza").
- `token_type_hint` reaproveita exatamente o mesmo padrão de dica opcional já visto na RFC 7009 — mesmo desenho, RFCs irmãs, resolvendo problemas espelhados (revogar vs. validar).
- O vocabulário de campos da resposta (`iss` , `sub` , `aud` , `exp` , `iat` , `nbf` , `jti`) é deliberadamente compartilhado com a **RFC 7519 (JWT)** e, por extensão, com a **RFC 9068** — a mesma semântica de claims serve nos dois formatos.

7. O que mudou desde a publicação (2015)

- Spec estável e essencialmente universal onde quer que tokens opacos sejam usados — companheira natural de qualquer implantação que não use JWT autocontido como formato de access token.
- A ascensão de access tokens em formato JWT (formalizada depois pela RFC 9068 em 2021, e adotada por convenção de mercado até antes disso) reduziu a **necessidade** de introspecção em cenários de alto throughput, já que valida localmente evita o round-trip por requisição — mas introspecção continua essencial sempre que o status de revogação **em tempo real** importa mais do que evitar aquele round-trip, ou quando o token é mantido opaco de propósito (tamanho menor no cabeçalho HTTP, nenhuma informação vazando do próprio conteúdo do token para quem o interceptar).
- Um padrão híbrido comum hoje, que não era bem o desenho original de 2015 mas surgiu como prática de mercado: JWT para o caso geral (validação local, rápida, sem depender do AS estar no ar) **mais** uma checagem de revogação — via introspecção ou uma lista de revogação consultada esporadicamente — só quando a operação em questão exige garantia extra (uma transação financeira de alto valor, uma operação administrativa irreversível). Esse meio-termo reconhece que "JWT vs. opaco" não precisa ser uma escolha binária e permanente para toda e qualquer chamada.

Nota de laboratório: nenhum lab deste projeto implementa `/introspect` ainda — todos os resource servers (`rfc6749/resource-server` , `rfc7636/resource-server` , `rfc7523/resource-server` , etc.) validam o `access_token` lendo diretamente do Redis compartilhado com o respectivo Authorization Server, o atalho já descrito na seção 6. Um gap natural para um laboratório futuro seria desacoplar essa leitura direta, substituindo-a por uma chamada real a `POST /introspect` — nesse ponto, o resource server deixaria de conhecer o esquema interno de armazenamento do AS e passaria a depender só do contrato JSON desta RFC.

Exemplo em Java (JDK 21, sem frameworks)

Os exemplos a seguir implementam, do lado do resource server, uma chamada de introspecção completa usando apenas `java.net.http.HttpClient` para a chamada HTTP e o utilitário `SimpleJson` (`SimpleJson.object`, `SimpleJson.parseFlat`), definido no Capítulo 1 (RFC 6749) deste livro, para montar o corpo e interpretar a resposta.

4.1 Chamando POST /introspect autenticado

```
import java.net.URI;
import java.net.http.HttpClient;
import java.net.http.HttpRequest;
import java.net.http.HttpResponse;
import java.net.http.HttpRequest.BodyPublishers;
import java.nio.charset.StandardCharsets;
import java.util.Base64;
import java.util.Map;

/**
 * O RESOURCE SERVER (nao o client!) chama este endpoint, autenticado com
 * suas PROPRIAS credenciais (registradas junto ao AS especificamente para
 * este fim) – nunca com o token do usuario que esta sendo verificado.
 *
 * A resposta HTTP e' SEMPRE 200 – mesmo para token invalido/expirado/
 * revogado. O corpo (via campo "active") e' que carrega essa informacao;
 * tratar um status != 200 aqui como "token invalido" seria um erro de
 * implementacao – um != 200 significa problema na PROPRIA chamada de
 * introspeccao (ex.: credenciais do RS invalidas), nao no token investigado.
 */
public final class TokenIntrospectionClient {

    private final HttpClient http = HttpClient.newHttpClient();
    private final String introspectionEndpoint;
    private final String rsCredencialBasic; // Base64("resource_server_id:senha")

    public TokenIntrospectionClient(String introspectionEndpoint,
                                    String resourceId, String resourceServerSenha) {
        this.introspectionEndpoint = introspectionEndpoint;
        this.rsCredencialBasic = Base64.getEncoder().encodeToString(
            (resourceId + ":" + resourceServerSenha)
                .getBytes(StandardCharsets.UTF_8));
    }

    public Map<String, String> introspectar(String token, String tokenTypeHint)
        throws Exception {

        String corpo = "token=" + token
            + (tokenTypeHint != null ? "&token_type_hint=" + tokenTypeHint : "");

        HttpRequest req = HttpRequest.newBuilder()
            .uri(URI.create(introspectionEndpoint))
            .header("Content-Type", "application/x-www-form-urlencoded")
            .header("Authorization", "Basic " + rsCredencialBasic) // credencial do RS, nao do
            .POST(BodyPublishers.ofString(corpo, StandardCharsets.UTF_8))
            .build();

        HttpResponse<String> resp = http.send(req, HttpResponse.BodyHandlers.ofString());

        if (resp.statusCode() != 200) {
```



```
// Isto indica falha na PRÓPRIA chamada (ex.: credenciais do RS
// rejeitadas) – não confundir com "token inválido", que vem
// sempre como 200 + {"active": false}.
throw new IllegalStateException(
    "Falha ao chamar introspection endpoint: HTTP " + resp.statusCode());
}

return SimpleJson.parseFlat(resp.body());
}
}
```

4.2 Interpretando a resposta e aplicando a decisão de autorização

```
import java.time.Instant;
import java.util.Map;
import java.util.Set;

/**
 * A resposta de /introspect nunca deveria ser tratada como "true = deixa
 * passar" sem mais checagens – active=true so' significa que o token EXISTE
 * e nao expirou/foi revogado; ainda cabe ao resource server checar se o
 * escopo cobre a operacao pedida, exatamente como faria com um JWT local.
 */
public final class DecisaoDeAutorizacao {

    public static boolean tokenAutorizaOperacao(Map<String, String> introspeccao,
                                                String escopoNecessario) {

        boolean ativo = "true".equals(introspeccao.get("active"));
        if (!ativo) {
            // Cobre TODOS os motivos de invalidade – expirado, revogado,
            // nunca existiu – a resposta e' deliberadamente uniforme
            // (RFC 7662 §2.2) para nao revelar qual dos tres aconteceu.
            return false;
        }

        String expStr = introspeccao.get("exp");
        if (expStr != null) {
            long exp = Long.parseLong(expStr);
            if (Instant.now().getEpochSecond() >= exp) {
                // Defesa em profundidade: mesmo que o AS diga active=true,
                // um RS cauteloso pode revalidar exp localmente (util se
                // o resultado veio de um cache local do proprio RS, e nao
                // de uma chamada fresca – ver secao 4.3 a seguir).
                return false;
            }
        }

        String scopeString = introspeccao.getOrDefault("scope", "");
        Set<String> escoposConcedidos = Set.of(scopeString.split(" "));
        return escoposConcedidos.contains(escopoNecessario);
    }
}
```

4.3 Cache local de curta duração (mitigando o custo do round-trip)

```
import java.time.Instant;
import java.util.Map;
import java.util.concurrent.ConcurrentHashMap;

/**
 * Implementa a mitigacao discutida na secao 5.4: cache de curtissima duracao
 * para nao pagar o round-trip de rede a CADA requisicao recebida pelo RS.
 * O trade-off e' explicito – um token revogado no AS pode continuar sendo
 * aceito aqui por ate CACHE_TTL_SEGUNDOS depois da revogacao. Escolher esse
 * numero e' uma decisao de seguranca, nao so' de performance.
 */
public final class IntrospeccaoComCache {

    private static final long CACHE_TTL_SEGUNDOS = 30;

    private record Entrada(Map<String, String> resposta, long expiraEm) {}

    private final ConcurrentHashMap<String, Entrada> cache = new ConcurrentHashMap<>();
    private final TokenIntrospectionClient clienteIntrospeccao;

    public IntrospeccaoComCache(TokenIntrospectionClient clienteIntrospeccao) {
        this.clienteIntrospeccao = clienteIntrospeccao;
    }

    public Map<String, String> introspectarComCache(String token) throws Exception {
        long agora = Instant.now().getEpochSecond();

        Entrada emCache = cache.get(token);
        if (emCache != null && emCache.expiraEm() > agora) {
            return emCache.resposta(); // evita o round-trip desta vez
        }

        Map<String, String> fresca = clienteIntrospeccao.introspectar(token, "access_token");
        cache.put(token, new Entrada(fresca, agora + CACHE_TTL_SEGUNDOS));
        return fresca;
        // Nota: um cache ingenuo como este NAO invalida entradas quando o
        // token e' revogado no meio da janela de TTL – e' exatamente a
        // janela de exposicao que a secao 5.4 pede para ser uma decisao
        // consciente da equipe de seguranca, nao um acidente de implementacao.
    }
}
```

RFC 9068 — JSON Web Token (JWT) Profile for OAuth

2.0 Access Tokens

1. Contexto e motivação

A RFC 6749 deixa o formato do `access_token` deliberadamente indefinido — é uma string opaca, ponto final, e o resource server "simplesmente sabe" como validá-la por algum meio fora do escopo da especificação (tipicamente, um dos dois caminhos que este livro já cobriu: leitura direta de um armazenamento compartilhado, como nos labs deste projeto, ou introspecção via RFC 7662). Essa flexibilidade foi proposital — o OAuth 2.0 nunca quis prescrever formato de token, só o protocolo de obtenção — mas criou um problema real de interoperabilidade ao longo da década seguinte: **vários fornecedores passaram a usar JWT como access token de forma independente, cada um com seu próprio conjunto de claims, sem um vocabulário comum**. Um resource server construído para entender o JWT de um fornecedor não entendia o de outro, mesmo os dois sendo nominalmente "access token em formato JWT" — a mesma tecnologia de base (JWS/JWT, RFC 7515/7519), mas nomes de claim, obrigаторiedades e convenções divergentes o suficiente para inviabilizar reuso de bibliotecas de validação entre ecossistemas.

A RFC 9068 (2021) formaliza a **estrutura** de um JWT quando usado como access token — claims obrigatórias/recomendadas e, principalmente, um valor de `typ` no header que **autodeclara** "isto é especificamente um access token JWT", evitando confusão com outros tipos de JWT circulando pelo mesmo sistema (notavelmente, o ID Token do OpenID Connect, que compartilha boa parte do vocabulário de claims mas tem um propósito semântico completamente diferente).

1.1 A diferença entre "JWT como access token" e "JWT como formato universal"

É importante não interpretar esta RFC como "todo access token deveria ser JWT". A própria RFC é explícita que continua sendo uma escolha legítima do AS emitir tokens opacos — o que ela resolve é **quando** o AS decide usar JWT, garantindo que essa escolha produza um artefato interoperável, em vez de mais um formato proprietário reinventado. Em outras palavras: RFC 9068 não é uma RFC sobre "usar JWT", é uma RFC sobre "se você vai usar JWT como access token, faça assim, e não de outro jeito qualquer".

1.2 O problema concreto de "confusão de JWT" que motiva o `typ`

Antes de existir um marcador de tipo explícito, um sistema com múltiplos emissores de JWT (o próprio AS emitindo ID Tokens do OIDC e access tokens, por exemplo) corre o risco de um componente de validação aceitar por engano um JWT emitido para um propósito diferente do que ele espera. Um ID Token, por design, tem claims como `sub` (o usuário), `aud` (o `client_id`, não o resource server!), `iss` — muito parecido em forma com um access token JWT, mas com **semântica de audience completamente diferente**: o público-alvo pretendido de um ID Token é o *client* que fez o login, não um resource server de API. Um resource server que, por erro de implementação, aceitasse um ID Token

como se fosse um access token válido estaria confiando numa prova de identidade pensada para outro consumidor e outro propósito — uma classe real e documentada de vulnerabilidade ("confusão de tipo de token"/JWT confusion), não uma preocupação teórica.

2. Terminologia e papéis

Nenhum papel novo. Introduz:

- **JWT Access Token** — um JWT que segue este perfil específico: conjunto de claims definido, header com `typ` marcado.
- **typ: `at+jwt`** no header — marcador de tipo explícito (o valor completo, quando usado como cabeçalho `Content-Type` de uma requisição HTTP que carregue o token, seria `application/at+jwt`; dentro do header JOSE do próprio JWT, o valor abreviado `at+jwt` é o convencional, seguindo a mesma convenção de tipos estruturados definida pela RFC 8725/RFC 7515).
- **Media Type Structured Syntax Suffix** — o sufixo `+jwt` em `at+jwt` segue a convenção de nomenclatura de tipos de mídia estruturados (RFC 6839), o mesmo padrão que dá nomes como `application/at+jwt` ao invés de simplesmente reaproveitar `application/jwt` genérico para tudo. A ideia é permitir que ferramentas genéricas de processamento de JWT reconheçam "isto é um JWT" (pelo sufixo) e ferramentas específicas de OAuth reconheçam "isto é especificamente um access token" (pelo prefixo `at`).

3. "Fluxo" — não há fluxo novo, é um formato aplicado ao que já existe

Esta RFC não define endpoint nem grant novo — ela governa o **formato** do valor que já volta no campo `access_token` da resposta padrão do token endpoint (RFC 6749 §5.1), quando o AS escolhe emitir tokens em JWT em vez de opacos. A resposta do `/token` continua tendo exatamente a mesma forma:

```
{
  "access_token": "<agora um JWT compacto, em vez de uma string aleatoria>",
  "token_type": "Bearer",
  "expires_in": 300,
  "scope": "profile email"
}
```

A única mudança está **dentro** do valor de `access_token` — antes 48 caracteres hexadecimais aleatórios sem estrutura, agora três segmentos Base64URL separados por ponto (header.payload.assinatura), estruturados conforme as regras abaixo.

3.1 Estrutura do header

```
{
  "typ": "at+jwt",
  "alg": "RS256",
  "kid": "chave-do-as-2026-01"
}
```

O `kid` (Key ID) não é exigido pela RFC 9068 diretamente, mas é praticamente indispensável na prática: como o resource server precisa saber **qual** chave pública usar para verificar a assinatura (o AS tipicamente tem mais de uma chave ao longo do tempo, por rotação), o `kid` funciona como um índice dentro do conjunto de chaves publicado via `jwks_uri` (RFC 8414 + o formato JWK Set da RFC 7517).

3.2 Claims exigidas/recomendadas pelo perfil

Claim	Status	Significado
<code>iss</code>	obrigatória	Identificador do AS — mesmo valor do <code>issuer</code> publicado via metadata da RFC 8414
<code>exp</code>	obrigatória	Expiração
<code>iat</code>	obrigatória	Momento de emissão
<code>aud</code>	obrigatória	Identifica o(s) resource server(s) destinatário(s) — pode ser um array, se o token é destinado a mais de um RS
<code>sub</code>	obrigatória	De quem o token fala — o resource owner, em grants delegados (Authorization Code); o próprio client, em Client Credentials Grant (onde não há resource owner)
<code>client_id</code>	obrigatória	Deliberadamente distinta de <code>sub</code> — num grant delegado, <code>sub</code> é o usuário e <code>client_id</code> é o app agindo em nome dele; confundir os dois é um erro sutil e recorrente de implementações que tentam "economizar" uma claim
<code>jti</code>	obrigatória	Identificador único do token — mesmo conceito de unicidade já visto no <code>client_assertion</code> da RFC 7523, aplicado agora ao próprio access token
<code>scope</code>	recomendada	Mesma string espaço-delimitada de sempre
<code>auth_time</code>	opcional	Quando o resource owner efetivamente se autenticou (não quando o token foi emitido — podem ser momentos bem diferentes, por exemplo num refresh)
<code>acr</code>	opcional	Authentication Context Class Reference — "com que nível de garantia" a autenticação ocorreu
<code>amr</code>	opcional	Authentication Methods References — "por quais métodos" (senha, biometria, segundo fator)

Os três últimos (`auth_time`, `acr`, `amr`) são emprestados conceitualmente do ID Token do OpenID Connect — e essa vizinhança semântica é exatamente o motivo pelo qual o marcador `typ` é tão importante: um access token JWT e um ID Token OIDC podem, na prática, compartilhar boa parte do

vocabulário de claims, tornando ainda mais fácil confundir um pelo outro sem um marcador de tipo explícito no header.

3.3 Por que `sub` e `client_id` precisam ser claims separadas

Vale insistir neste ponto porque é um erro real e recorrente. Considere um Authorization Code Grant onde Alice autoriza o aplicativo "MeuApp" a acessar sua conta:

- `sub = "alice"` — de quem o token fala, o resource owner que concedeu a autorização.
- `client_id = "meuapp-web"` — quem está de fato segurando o token e fazendo a chamada de API.

Um resource server que precise decidir "este client tem permissão para fazer isto, independente do usuário" (uma checagem de allowlist de aplicações, por exemplo) usa `client_id`. Um resource server que precise decidir "este usuário tem permissão para ver este recurso" usa `sub`. Confundir as duas claims — por exemplo, um RS mal implementado que aplica uma regra de autorização usando `sub` quando deveria usar `client_id` — pode abrir brechas onde a identidade errada é usada para a decisão de autorização errada. Já no Client Credentials Grant (onde não existe resource owner), `sub` costuma ser igual ao próprio `client_id`, porque não há "outro" de quem falar.

4. Parâmetros e endpoints normativos

Nenhum endpoint novo. O único parâmetro normativo relevante é de **header do próprio JWT**, não de protocolo HTTP: `typ` deve ser `at+jwt` (ou, em contextos de `Content-Type` HTTP, `application/at+jwt`), e `alg` precisa ser um algoritmo de assinatura de verdade — **nunca** `none`. A RFC reforça explicitamente essa proibição porque `alg: none` é uma das armadilhas mais conhecidas e mais fáceis de introduzir acidentalmente numa implementação de validação de JWT malfeita (uma biblioteca que, ao ver `alg: none`, simplesmente pula a etapa de verificação de assinatura).

4.1 O que a RFC não normatiza (e por que isso importa)

A RFC 9068 não define: - **Qual algoritmo de assinatura usar** (RS256, PS256, ES256 — todos são compatíveis com o perfil, desde que não seja `none` e desde que o RS saiba qual esperar para aquele `iss / kid`). - **Como o resource server descobre a chave pública** — isso é delegado à RFC 8414 (`jwtks_uri`) e RFC 7517 (formato JWK). - **Se o token deve ou não ser criptografado (JWE)** além de assinado (JWS) — deixado como uma opção, não uma exigência (ver seção 5.3).

Essa omissão deliberada é consistente com o espírito geral do ecossistema de RFCs deste livro: cada especificação resolve uma fatia específica do problema, e composição entre elas é o que produz um sistema completo.

5. Segurança já tratada na própria RFC 9068

5.1 `typ: at+jwt` previne confusão de tipo de token

Sem essa autodeclaração explícita, um resource server (ou uma biblioteca mal implementada, ou um desenvolvedor que copiou código de validação de ID Token achando que servia igualmente para access token) poderia aceitar por engano um JWT emitido para **outro propósito** como se fosse um access token válido — uma classe real de vulnerabilidade histórica ("confusão de JWT"), já discutida na seção 1.2. O `typ` explícito permite ao código de validação do RS checar, **antes de confiar em qualquer outra coisa** (antes mesmo de verificar a assinatura, idealmente, como uma checagem rápida de "isto sequer é o tipo certo de artefato"), se aquele JWT realmente foi emitido para ser um access token.

A ordem de validação recomendada na prática costuma ser: (1) parsear a estrutura, (2) checar `typ` no header, (3) verificar assinatura com a chave correta para o `alg / kid` declarados, (4) só então validar as claims de conteúdo (`exp`, `aud`, `iss`, `sub` ...). Fazer a checagem de `typ` antes da assinatura é uma otimização de "fail fast" — rejeitar rápido tokens do tipo errado sem gastar o custo de uma verificação criptográfica que de qualquer forma levaria à rejeição por outro motivo.

5.2 `aud` restringe quais resource servers podem aceitar o token

Operacionaliza diretamente a preocupação de restrição de audience já discutida na RFC 6819 (o cenário do "confused deputy" — um RS que aceita um token pensado para outro RS, agindo como intermediário involuntário de um ataque) e que a RFC 8707 (Resource Indicators) formaliza do lado da **requisição** (o client pede um token para um recurso específico). A RFC 9068 é o "ponto de pouso" dessa preocupação: é aqui que a decisão de audience fica efetivamente registrada **dentro do próprio token**, como uma claim que qualquer RS pode e deve checar antes de aceitar o token como seu.

Um resource server correto verifica que **o próprio identificador dele** está presente em `aud` (que pode ser uma string única ou um array de valores, quando o token é válido para múltiplos RSs) — e rejeita qualquer token onde isso não seja verdade, mesmo que a assinatura seja perfeitamente válida e o token não tenha expirado. Um token assinado corretamente pelo AS, mas destinado a outro RS, **não é válido para este RS** — a assinatura prova quem emitiu, não para quem foi destinado.

5.3 Sem criptografia por padrão — a troca entre legibilidade e confidencialidade

Um JWT access token normalmente é assinado (JWS), não criptografado — suas claims são legíveis em texto (Base64URL, tecnicamente, mas Base64 não é criptografia, é só codificação) por qualquer um que tenha o token, diferente de um token opaco (cujo "significado" fica trancado dentro do AS, inacessível a quem não tem acesso ao armazenamento interno). É uma troca real:

- **Legibilidade/validação local ganha** — o RS não precisa perguntar a ninguém, decodifica e valida sozinho.
- **Confidencialidade do conteúdo perde** — qualquer intermediário que veja o token (um proxy, um log de acesso mal configurado que grava cabeçalhos `Authorization`, o próprio navegador do usuário em certos fluxos) consegue ler `sub`, `scope`, e qualquer outra claim, mesmo

sem conseguir *usar* o token (a assinatura impede forjar um novo, mas não impede ler o existente).

A RFC permite (não exige) usar JWE — JSON Web Encryption, o par criptográfico do JWS — por cima da assinatura quando as claims forem sensíveis o suficiente para justificar esse custo adicional. É o mesmo par JWS/JWE já visto no contexto do Request Object da RFC 9101 (JAR), aplicado aqui ao access token em vez de à requisição de autorização.

5.4 Revogação fica mais difícil de aplicar em tempo real

Validação local por assinatura significa que o RS confia no token até o `exp`, sem checar de novo com o AS — a mesma troca já apontada no capítulo da RFC 7662: JWT evita o round-trip de rede, mas perde a capacidade do AS dizer "na verdade, acabei de revogar esse token" no meio da vida dele. Se um token JWT com `exp` de 1 hora é comprometido (roubado) 5 minutos depois de emitido, ele continua válido — do ponto de vista de qualquer RS que o valide localmente — pelos 55 minutos restantes, mesmo que o AS já tenha sido informado do comprometimento e queira revogá-lo.

Mitigações práticas, já discutidas de forma mais geral no capítulo da RFC 7662: - **Tempos de vida curtos** — reduzir a janela de exposição ao custo de refresh mais frequente. - **Padrão híbrido** — JWT para o caso comum (validação rápida, local) + checagem extra de revogação (introspecção, ou consulta a uma lista de revogação) só em operações de alto valor (transações financeiras, mudanças administrativas irreversíveis). - **cnf (confirmation) e sender-constraining** — amarrar o token a uma chave/certificado específico (RFC 8705 mTLS, RFC 9449 DPoP — ver seção 7) reduz o *impacto* de um token roubado mesmo sem revogação em tempo real, porque o token roubado sozinho não basta para uso — ainda seria necessário possuir a chave/certificado associado.

5.5 Nuances adicionais de validação que a prática exige

- **Verificação de `alg` no header nunca deve ser delegada ao próprio token.** Assim como discutido no capítulo da RFC 7523, um resource server deve fixar de antemão quais algoritmos aceita para aquele `iss / kid`, nunca "confiar" cegamente no `alg` declarado no header recebido — a mesma classe de ataque de confusão de algoritmo (RS256-para-HS256, ou aceitação acidental de `none`) se aplica aqui integralmente.
- **Tamanho do token cresce em relação ao opaco.** Um access token JWT tipicamente ocupa várias centenas de bytes a mais de um caso opaco de 32-48 caracteres — relevante em cenários com limites estritos de tamanho de cabeçalho HTTP, ou alto volume de tráfego onde esse overhead se acumula.
- **`jti` aqui serve outro propósito além do antirreplay já visto na RFC 7523** — para um access token, `jti` é sobretudo um identificador para fins de auditoria/correlação de logs e, eventualmente, para permitir uma lista de revogação granular por token individual (em vez de por client ou por usuário inteiro).

6. Relação com outras RFCs

- Estratégia concorrente/complementar direta à **RFC 7662 (Introspecção)** — o "ponto de pouso" da troca já discutida no capítulo anterior: validar localmente por assinatura (esta RFC) vs. perguntar ao AS a cada requisição (RFC 7662). Uma implementação pode inclusive combinar as duas, como discutido na seção 5.4.
- `iss` amarra diretamente ao `issuer` do metadata da **RFC 8414** — mesmo valor, mesma âncora de identidade, usada pelo RS para descobrir onde buscar as chaves de verificação (`jwks_uri`).
- `aud` é o mecanismo concreto que operacionaliza a preocupação de audience da **RFC 6819**, e é o alvo natural do que a **RFC 8707 (Resource Indicators)** deixa o *client* pedir do lado da requisição — vale reter a divisão de responsabilidades: 8707 é sobre o pedido ("quero um token para ESTE recurso"), 9068 é sobre o token emitido ("é aqui que essa decisão fica registrada, de forma que o próprio RS possa verificar").
- É um dos formatos que o `issued_token_type` da **RFC 8693 (Token Exchange)** pode indicar — a troca de token pode devolver exatamente um JWT seguindo este perfil, sinalizado por um valor de `issued_token_type` como `urn:ietf:params:oauth:token-type:access_token` combinado com o próprio conteúdo do token seguindo esta RFC.
- `jti` espelha o mesmo conceito de identificador único já visto no `client_assertion` da RFC 7523 — a ideia de "toda vez que assinamos algo, damos um identificador único a ele" se repete em múltiplas RFCs deste livro, sempre com o mesmo propósito de fundo (permitir rastreamento/rejeição individual).
- Não precisa de relação especial com a **RFC 7519 (JWT)** além de ser a spec de formato-base sobre a qual este perfil se apoia — a 7519 define o que é um JWT genericamente (estrutura, claims registradas, processamento); a 9068 é um **perfil** que restringe e especializa esse formato genérico para um propósito específico (ser um access token OAuth). É exatamente por isso que a 7519 não tem capítulo próprio neste livro, mas aparece referenciada em vários pontos como a fundação sobre a qual RFC 7523 (client assertions) e RFC 9068 (access tokens) constroem.

7. O que mudou desde a publicação (2021)

- Relativamente recente entre as RFCs deste livro, mas com espírito retroativo: formaliza algo que **muitos fornecedores já faziam de forma ad-hoc havia anos**. JWT como access token já existia na prática bem antes de 2021 — grandes provedores de identidade já emitiam JWTs como access tokens desde meados da década anterior, só sem estrutura comum entre eles. Por isso a adoção da RFC 9068 foi relativamente rápida: ela formalizou uma prática de mercado já madura, em vez de introduzir algo totalmente novo que o mercado precisasse assimilar do zero.
- Cada vez mais tratada como expectativa padrão para novas implementações de AS que optam por tokens em formato JWT, em vez de cada fornecedor reinventar nomes de claim — o mesmo ganho de interoperabilidade que a RFC 8414 trouxe para descoberta de metadata, agora aplicado ao formato do próprio token.
- Conecta com uma tendência maior no ecossistema, que aparece em capítulos futuros deste livro (RFC 8705 mTLS, RFC 9449 DPoP): amarrar o access token JWT a uma chave/certificado específico via uma claim `cnf` (confirmation) — exatamente o ponto de extensão que este perfil habilita para **sender-constraining** (tornar o token inútil, mesmo se roubado, para quem não

possui a chave/certificado correspondente). Essa é, provavelmente, a evolução mais significativa em curso sobre o formato definido por esta RFC: de "prova de que o AS autorizou algo" para "prova de que o AS autorizou algo e de que quem apresenta o token é quem deveria possuí-lo".

Nota de laboratório: nenhum lab construído ainda para esta RFC neste projeto — todos os labs emitem tokens **opacos** (48 caracteres hexadecimais), nunca JWT. Seria o gap natural para um laboratório futuro: trocar o formato do `access_token` por um JWT seguindo este perfil, e então o resource server passaria a validar por assinatura local em vez de ler o Redis diretamente (fechando também o gap equivalente já apontado no capítulo da RFC 7662, mas por um caminho diferente — validação local em vez de introspecção remota).

Exemplo em Java (JDK 21, sem frameworks)

Os exemplos abaixo cobrem os dois lados do ciclo de vida: o Authorization Server **emitindo** um access token JWT seguindo o perfil desta RFC, e o Resource Server **validando-o localmente** — sem nenhuma chamada de rede, em contraste direto com o custo de round-trip do `/introspect` da RFC 7662 discutido no capítulo anterior. Usamos apenas `java.security` (`KeyPairGenerator`, `Signature`) para a assinatura RS256, `java.util.Base64` para a codificação compacta, e o utilitário `SimpleJson` (`SimpleJson.object`, `SimpleJson.parseFlat`), definido no Capítulo 1 (RFC 6749) deste livro, para montar/interpretar os objetos JSON envolvidos.

Vale repetir aqui a mesma ressalva do capítulo da RFC 7523: assinar e verificar JWTs manualmente com `Signature`, como fazemos abaixo, é apenas para expor a mecânica — em produção, use uma biblioteca JOSE validada (ex.: Nimbus JOSE+JWT), que trata corretamente casos como confusão de algoritmo, `alg: none`, e as demais nuances já discutidas na seção 5.5 deste capítulo e na seção 5.5 do capítulo da RFC 7523.

4.1 Emitindo um access token JWT (lado do Authorization Server)

```
import java.nio.charset.StandardCharsets;
import java.security.PrivateKey;
import java.security.Signature;
import java.time.Instant;
import java.util.Base64;
import java.util.LinkedHashMap;
import java.util.Map;
import java.util.UUID;

/**
 * Monta um access token JWT conforme RFC 9068: header com "typ":"at+jwt"
 * (o marcador que distingue este JWT de um ID Token OIDC ou de qualquer
 * outro JWT que por acaso circule pelo mesmo sistema – secao 5.1) e as
 * claims obrigatorias do perfil (iss, exp, iat, aud, sub, client_id, jti).
 */
public final class JwtAccessTokenIssuer {

    private static final Base64.Encoder B64URL =
        Base64.getUrlEncoder().withoutPadding();

    public static String emitir(String issuer, String audienceResourceServer,
                               String subjectResourceOwner, String clientId,
                               String scope, int ttlSegundos,
                               PrivateKey chavePrivadaDoAS, String kid) throws Exception {

        Map<String, Object> header = new LinkedHashMap<>();
        header.put("typ", "at+jwt"); // <- e' este marcador que a secao 5.1 explica
        header.put("alg", "RS256");
        header.put("kid", kid); // permite ao RS escolher a chave publica certa

        long agora = Instant.now().getEpochSecond();
        Map<String, Object> payload = new LinkedHashMap<>();
        payload.put("iss", issuer); // = "issuer" do metadata RFC 8414
        payload.put("aud", audienceResourceServer); // RS(s) destinatario(s) – secao 5.2
        payload.put("sub", subjectResourceOwner); // o USUARIO (nao o client!)
        payload.put("client_id", clientId); // o APP agindo – claim SEPARADA de sub
        payload.put("iat", agora);
        payload.put("exp", agora + ttlSegundos);
        payload.put("jti", UUID.randomUUID().toString()); // identificador unico do token
        payload.put("scope", scope);

        String headerB64 = B64URL.encodeToString(
            SimpleJson.object(header).getBytes(StandardCharsets.UTF_8));
        String payloadB64 = B64URL.encodeToString(
            SimpleJson.object(payload).getBytes(StandardCharsets.UTF_8));
        String assinavel = headerB64 + "." + payloadB64;

        Signature assinador = Signature.getInstance("SHA256withRSA");
        assinador.initSign(chavePrivadaDoAS);
        assinador.update(assinavel.getBytes(StandardCharsets.US_ASCII));
```

```
String assinaturaB64 = B64URL.encodeToString(assinador.sign());  
  
return assinavel + "." + assinaturaB64;  
}  
}
```

4.2 Validando o token localmente (lado do Resource Server) — sem round-trip de rede

```
import java.nio.charset.StandardCharsets;
import java.security.PublicKey;
import java.security.Signature;
import java.time.Instant;
import java.util.Base64;
import java.util.Map;
import java.util.Set;

/**
 * Contraste direto com o capítulo da RFC 7662: ali, o resource server fazia
 * um POST /introspect e ESPERAVA uma resposta de rede do AS a cada chamada
 * (TokenIntrospectionClient.introspectar, no capítulo anterior). Aqui, com
 * o access token em formato JWT (RFC 9068), a validação inteira acontece
 * DENTRO DO PROCESSO do resource server – nenhuma chamada HTTP, nenhuma
 * dependência de o AS estar no ar no momento da requisição. O custo que se
 * paga em troca está descrito na seção 5.4: revogar este token específico
 * antes do "exp" não tem efeito imediato aqui, porque ninguém consulta o AS
 * de novo depois da emissão.
 */
public final class JwtAccessTokenValidator {

    public record TokenValidado(Map<String, Object> claims) {}

    public static TokenValidado validar(String jwtCompacto, PublicKey chavePublicaDoAS,
                                       String issuerEsperado, String audienceEsperada)
        throws Exception {

        String[] partes = jwtCompacto.split("\\.");
        if (partes.length != 3) {
            throw new IllegalArgumentException("JWT mal formado – esperado 3 segmentos");
        }

        Base64.Decoder b64url = Base64.getUrlDecoder();
        String headerJson = new String(b64url.decode(partes[0]), StandardCharsets.UTF_8);
        Map<String, String> header = SimpleJson.parseFlat(headerJson);

        // 1) Checagem de TIPO primeiro (fail fast, seção 5.1) – antes mesmo
        // de gastar o custo de uma verificação de assinatura, rejeita
        // qualquer JWT que não se autodeclare um access token deste perfil.
        if (!"at+jwt".equals(header.get("typ"))) {
            throw new SecurityException(
                "typ inesperado (" + header.get("typ") + ") – isto pode ser um "
                + "JWT de outro propósito, ex.: ID Token OIDC (RFC 9068 seção 5.1)");
        }

        // 2) O algoritmo aceito é FIXADO pelo validador, nunca lido cegamente
        // do header e usado sem checagem – previne confusão de algoritmo
        // (RS256-para-HS256, ou aceitação acidental de "alg":"none").
        if (!"RS256".equals(header.get("alg"))) {
            throw new SecurityException(
```

```

        "Algoritmo nao suportado por esta politica de validacao: " + header.get("alg"));
    }

    // 3) SO ENTAO verifica a assinatura criptografica.
    String assinavel = partes[0] + "." + partes[1];
    byte[] assinatura = b64url.decode(partes[2]);
    Signature verificador = Signature.getInstance("SHA256withRSA");
    verificador.initVerify(chavePublicaDoAS);
    verificador.update(assinavel.getBytes(StandardCharsets.US_ASCII));
    if (!verificador.verify(assinatura)) {
        throw new SecurityException("Assinatura invalida – token forjado ou corrompido");
    }

    // 4) So depois da assinatura confirmada e' que as CLAIMS podem ser
    // consideradas confiaveis o suficiente para orientar decisoes.
    String payloadJson = new String(b64url.decode(partes[1]), StandardCharsets.UTF_8);
    Map<String, String> claims = SimpleJson.parseFlat(payloadJson);

    if (!issuerEsperado.equals(claims.get("iss"))) {
        throw new SecurityException("iss inesperado – token nao veio do AS esperado");
    }

    String audienceDoToken = claims.get("aud");
    if (audienceDoToken == null || !audienceDoToken.contains(audienceEsperada)) {
        // secao 5.2 – um token valido para OUTRO resource server nao e'
        // valido para este, mesmo com assinatura perfeita.
        throw new SecurityException(
            "aud nao inclui este resource server – token nao foi emitido para aqui");
    }

    long exp = Long.parseLong(claims.get("exp"));
    if (Instant.now().getEpochSecond() >= exp) {
        throw new SecurityException("Token expirado");
    }

    return new TokenValidado(Map.copyOf(claims));
}

public static boolean autorizaEscopo(TokenValidado validado, String escopoNecessario) {
    Object scopeObj = validado.claims().get("scope");
    String scopeString = scopeObj == null ? "" : scopeObj.toString();
    Set<String> escopos = Set.of(scopeString.split(" "));
    return escopos.contains(escopoNecessario);
}
}

```

O contraste com o capítulo anterior é o ponto central deste exemplo: `JwtAccessTokenValidator.validar` acima não faz **nenhuma** chamada de rede — todo o trabalho é decodificação Base64URL e verificação criptográfica local, executado inteiramente dentro do processo do resource server, contra `TokenIntrospectionClient.introspectar` (RFC 7662, capítulo anterior), que dependia de uma chamada

HTTP síncrona ao Authorization Server a cada requisição recebida. É exatamente a troca discutida na seção 5.4 e na seção 6 deste capítulo: velocidade e desacoplamento de disponibilidade do AS, ao custo de não conseguir mais que o AS diga "revoguei esse token" no meio da vida dele.

Parte 6 — Segurança Avançada

RFC 8705 — OAuth 2.0 Mutual-TLS Client Authentication and Certificate-Bound Access Tokens

1. Contexto e motivação

Esta RFC define **duas coisas distintas**, e vale insistir na separação logo de início porque é comum vê-las confundidas em discussões informais:

1. **mTLS como método de autenticação de client** no token endpoint — o client se autentica apresentando um certificado X.509 durante o próprio handshake TLS, em vez de (ou além de) qualquer credencial transportada dentro da requisição HTTP. É uma alternativa assimétrica ao `client_secret`, ocupando o mesmo espaço de decisão que `private_key_jwt` (RFC 7523 §2.2): "como o AS sabe que é realmente este client falando?" — só que a prova é feita na camada de transporte (TLS), não dentro do corpo da requisição.
2. **Certificate-bound access tokens** — independentemente de como o client se autenticou, o AS pode amarrar (*bind*) o access token emitido ao certificado X.509 usado na conexão em que o token foi obtido. Um token vinculado assim só é aceito pelo resource server se a **mesma** identidade de certificado for reapresentada na conexão que efetivamente usa o token.

A primeira metade resolve "quem é o client". A segunda resolve um problema completamente diferente e mais fundamental, que atravessa este roteiro inteiro desde o capítulo da RFC 6750: **bearer token roubado = acesso total até a expiração**, porque um bearer token é, por definição, uma credencial "quem tiver, pode usar" (*possession-based*). A RFC 6750 já registrava esse risco residual como algo que fica fora do escopo dela — ficou como uma dívida técnica explícita, que a RFC 8705 (e a RFC 9449, no capítulo seguinte) vêm pagar.

É perfeitamente possível usar as duas metades de forma independente:

- **mTLS só para autenticação de client, sem token vinculado** — o client prova quem é usando certificado, mas o access_token resultante continua sendo um bearer comum. Reduz o risco de personificação do client (ninguém sem a chave privada consegue se passar por ele no token endpoint), mas não protege o token depois de emitido.
- **Token vinculado a certificado, com client autenticado por outro método** — teoricamente possível, mas raro na prática: a vinculação normalmente nasce naturalmente da própria conexão mTLS que já está sendo usada para autenticar o client, então a maioria das implantações reais faz as duas coisas **juntas**, porque é onde mora o valor real: autenticar via mTLS e vincular o token resultante ao mesmo certificado, numa única conexão.

Por que mTLS e não só "confiar no client_secret"?

Vale reforçar o contraste com o modelo mais simples (`client_secret_basic` / `client_secret_post`, RFC 6749): um segredo compartilhado, transmitido a cada requisição, que precisa ser guardado por ambos os lados (client e AS) e que, se vazarem de qualquer um dos dois lados (log, repositório, banco de dados do

AS comprometido), permite personificação completa do client até ser rotacionado. Um certificado X.509 com chave privada nunca sai do client (fica em HSM, keystore protegido, ou pelo menos um arquivo de chave local) — o AS nunca vê nem guarda a chave privada, só a chave pública embutida no certificado. É a mesma vantagem estrutural que `private_key_jwt` já trazia; mTLS só move essa prova para a camada de transporte.

2. Terminologia e papéis

Nenhum papel novo — client, AS e RS continuam os mesmos. Conceitos novos:

- `tls_client_auth` — método de autenticação de client em que o certificado apresentado no handshake é validado por uma cadeia de confiança **PKI tradicional**: o AS verifica que o certificado foi emitido por uma CA (ou cadeia de CAs) em que ele confia, normalmente também checando um campo esperado do subject/SAN contra o `client_id` registrado. É o modelo natural para ambientes corporativos que já operam uma PKI interna (comum em bancos, exatamente o seu contexto FSI).
- `self_signed_tls_client_auth` — o client apresenta um certificado **autoassinado** (sem cadeia de CA nenhuma). O AS não valida cadeia; em vez disso, compara a chave pública do certificado apresentado contra uma chave pública **pré-registrada** para aquele `client_id`, fora de banda, no momento do registro do client. É estruturalmente idêntico, em espírito de confiança, ao modelo de JWKS pré-registrado do `private_key_jwt` — a diferença é só o empacotamento (certificado X.509 vs. JWK cru).
- **Certificate-bound access token** — um `access_token` (opaco ou JWT) associado a um certificado X.509 específico. A associação pode viver:
- **Server-side**, para tokens opacos: o AS guarda, junto ao registro interno do token, o thumbprint do certificado que o gerou; a introspecção (RFC 7662) expõe isso.
- **Embutida no próprio token**, para JWT access tokens (RFC 9068): claim `cnf` (confirmation), com membro `x5t#S256` contendo o thumbprint.
- **Thumbprint do certificado (x5t#S256)** — SHA-256 sobre a codificação **DER** do certificado X.509 completo, codificado em Base64URL sem padding. É determinístico: o mesmo certificado sempre produz o mesmo thumbprint, permitindo comparação simples byte-a-byte do hash em vez de comparar certificados inteiros.

Onde o certificado "mora" durante a requisição

Um ponto frequentemente mal entendido: o certificado **não** é enviado como um header HTTP nem como parâmetro do corpo. Ele é parte do handshake TLS em si — o servidor solicita (`CertificateRequest`) e o client responde com sua cadeia de certificados como parte do protocolo TLS, antes mesmo da primeira linha HTTP ser trocada. Isso implica algo operacionalmente importante: **qualquer coisa que termine o TLS antes do AS/RS** (um load balancer, um API gateway, um proxy reverso) precisa, de alguma forma, repassar a informação do certificado do client para a aplicação — porque a aplicação, rodando atrás desse ponto de terminação, nunca vê o handshake TLS original.

3. Fluxo

Autenticação do client via mTLS, no token endpoint:

```
Handshake TLS: client apresenta certificado X.509
                (validado via cadeia de CA – tls_client_auth –
                 ou via chave pré-registrada – self_signed_tls_client_auth)

POST /token      (sobre essa mesma conexão mTLS)
grant_type=...&client_id=...
    – sem client_secret, sem client_assertion:
      o certificado apresentado no handshake É a credencial
```

Emissão do token vinculado (automática, quando o AS suporta o recurso e o client foi autenticado ou pelo menos identificado via mTLS):

```
Ao emitir o access_token, o AS calcula x5t#S256 = Base64URL(SHA-256(DER(certificado)))
e vincula:
- token opaco: thumbprint guardado no registro interno do token, no AS
- token JWT:   embutido como claim cnf: { "x5t#S256": "<thumbprint>" }
```

Validação no resource server, no uso do token:

```
Client → RS: mesma disciplina mTLS (apresenta o certificado de novo, nesta nova conexão)
              + Authorization: Bearer <token vinculado>

RS:
1) valida o token normalmente (introspecção RFC 7662, ou verificação local se for JWT)
2) calcula o thumbprint do certificado apresentado NESTA conexão específica
3) compara com o thumbprint vinculado ao token (cnf.x5t#S256, ou resposta de introspecção)
4) thumbprint diferente (ou ausente) → rejeita, mesmo que o token em si seja "válido"
```

O passo 4 é a propriedade de segurança central deste mecanismo inteiro: **possuir a string do bearer token deixa de ser suficiente**. É preciso também controlar a chave privada correspondente ao certificado, para conseguir reestabelecer uma conexão mTLS válida com aquele mesmo certificado — algo que um atacante que só roubou o token (de um log, de um proxy mal configurado, de um cache) não tem como fazer.

Caso de borda: renovação de token (refresh_token)

Quando um client usa um `refresh_token` para obter um novo `access_token`, a requisição ao token endpoint também passa pela mesma conexão mTLS. A RFC recomenda que o AS confira que o certificado apresentado nesta renovação é **o mesmo** que originou o `refresh_token` (ou pelo menos um certificado ainda válido e associado ao mesmo client) — do contrário, um `refresh_token` roubado

(mesmo sem token vinculado ele mesmo, já que `refresh_token`s tipicamente não trafegam para o resource server) ainda poderia ser trocado por um `access_token` novo por quem não tem a chave privada original. Vincular também a renovação fecha essa lacuna.

Caso de borda: múltiplos certificados válidos para o mesmo client

Nada na RFC impede um AS de aceitar mais de um certificado registrado para o mesmo `client_id` (útil durante rotação de certificado, ver seção 5). Cabe à implantação decidir a política — a RFC não normatiza um mecanismo de "múltiplos certificados simultaneamente válidos", deixando isso como decisão operacional de cada AS.

4. Parâmetros e endpoints normativos

- **Nenhum endpoint novo.** Estende a autenticação do token endpoint com dois métodos novos, registráveis via `token_endpoint_auth_method` no registro dinâmico de client (RFC 7591): `tls_client_auth` e `self_signed_tls_client_auth`.
- **Metadados de registro do client** — quando `tls_client_auth` é usado, o registro do client pode carregar campos adicionais indicando qual atributo do certificado deve ser conferido contra qual valor esperado (ex.: DN do subject, ou uma SAN específica) — a RFC define um pequeno vocabulário de campos para isso (`tls_client_auth_subject_dn` e variantes), permitindo ao AS saber exatamente o que comparar no certificado apresentado.
- **Resposta de introspecção (RFC 7662)** ganha um membro `cnf` com `x5t#S256` quando o token introspectado está vinculado a certificado — é assim que um resource server que usa introspecção (em vez de validar um JWT localmente) descobre o thumbprint esperado.
- **JWT access tokens (RFC 9068)** carregam `cnf.x5t#S256` diretamente no payload, quando vinculados — resource servers que validam o JWT localmente (sem round-trip de introspecção) leem o thumbprint direto do próprio token.
- **Descoberta via RFC 8414:** `tls_client_certificate_bound_access_tokens` (booleano, indicando suporte a tokens vinculados) e os dois métodos novos aparecendo em `token_endpoint_auth_methods_supported`. Também pode existir um endpoint alternativo dedicado a mTLS (`mtls_endpoint_aliases`), para AS que hospedam o token endpoint "normal" (sem exigir certificado de client) e uma variante mTLS-only em uma URL distinta — útil quando nem todo cliente do AS usa mTLS.

5. Segurança já tratada na própria RFC 8705

Fechamento da lacuna de bearer token

Fecha exatamente a fraqueza estrutural que a RFC 6750 já registrava como fora de escopo desde o início deste roteiro: bearer token roubado deixa de bastar sem a chave privada correspondente ao certificado. Esse é, de longe, o ganho de segurança mais importante desta RFC.

Terminação de TLS quebra tudo

Este é o ponto operacional mais citado (e mais frequentemente esquecido) da RFC: **se um proxy, load balancer ou API gateway termina a conexão TLS antes da requisição chegar ao AS/RS**, a aplicação nunca vê o certificado real do client — o handshake mTLS aconteceu entre o client e o gateway, não entre o client e a aplicação. Isso quebra silenciosamente qualquer verificação de `cnf.x5t#S256`, a menos que o certificado seja repassado por um canal explicitamente confiável — tipicamente um header HTTP (`X-Client-Cert` ou similar) contendo o certificado ou seu thumbprint, que a aplicação só aceita porque existe uma segunda camada de confiança entre gateway e aplicação (por exemplo, mTLS *também* entre gateway e backend, ou uma rede estritamente isolada). Sem esse cuidado, um atacante que conseguisse falar diretamente com a aplicação (contornando o gateway) poderia forjar esse header e personificar qualquer certificado.

Esse é um ponto arquitetural real, não teórico: provedores de nuvem e gateways de API adicionaram, ao longo do tempo, suporte específico para "passthrough de mTLS" (encaminhar o handshake original até o backend) ou para "encaminhamento de certificado via header confiável" justamente por causa dessa fricção repetida em implantações reais.

Rotação de certificado

Como o token está amarrado a um certificado específico (via thumbprint), quando o client renova ou troca seu certificado (expiração natural, comprometimento suspeito, troca de CA), **qualquer token já emitido e vinculado ao certificado antigo se torna inutilizável** — o thumbprint apresentado na próxima conexão não vai mais bater. Na prática, isso raramente é um problema porque access tokens tendem a ter vida curta (minutos), então a rotação de certificado naturalmente "varre" tokens antigos sem precisar de um mecanismo explícito de "migração de vínculo". Para `refresh_token`s de vida mais longa, uma implantação cuidadosa precisa antecipar esse efeito: trocar o certificado de um client efetivamente revoga (silenciosamente) todos os `refresh_token`s vinculados ao certificado anterior.

Modelo de confiança do `self_signed_tls_client_auth`

Sem cadeia de CA, o AS depende inteiramente de ter pré-registrado a chave pública exata esperada para aquele `client_id`, por um canal de registro que ele confia (fora de banda, geralmente no momento do onboarding do client). Se esse pré-registro for feito de forma frouxa (aceitar qualquer certificado que o client mandar por e-mail, por exemplo, sem verificação adicional de identidade), a "autenticação" vira teatro — a mesma disciplina de confiança prévia que já vimos com JWKS estático na RFC 7523.

Certificados expirados ou revogados

A RFC não define, ela mesma, como o AS deve tratar validação de revogação (CRL, OCSP) — isso é delegado à infraestrutura PKI subjacente, do mesmo jeito que qualquer verificação TLS "normal" trataria. Uma implantação que usa `tls_client_auth` precisa ter uma política operacional clara sobre isso (checar OCSP a cada handshake? confiar apenas em CRLs atualizadas periodicamente?) — é uma responsabilidade que sai do escopo direto da RFC 8705 e cai inteiramente sobre a camada TLS/PKI da implantação.

6. Relação com outras RFCs

- Resolve diretamente a fraqueza que a **RFC 6750** já sinalizava como fora de escopo desde o capítulo 2 deste roteiro — um bom fechamento de ciclo para quem seguiu o roteiro em ordem.
- Estende a **RFC 9068** (JWT access tokens) via `cnf.x5t#S256`, e a **RFC 7662** (introspecção) via o membro `cnf` na resposta — ambos os pontos de extensão já haviam sido antecipados nos capítulos correspondentes deste roteiro.
- Método de autenticação de client alternativo ao `private_key_jwt` (RFC 7523 §2.2): os dois são baseados em chave assimétrica (sem segredo estático compartilhado), mas mTLS reaproveita infraestrutura de PKI/TLS já existente, enquanto `private_key_jwt` usa JWS/JWKS — sem exigir handshake TLS especial. A RFC 9700 recomenda qualquer um dos dois sobre `client_secret` estático, sem preferir um especificamente; a escolha é de implantação, conforme a infraestrutura já disponível (uma organização com PKI corporativa madura tende a mTLS; uma sem PKI tende a `private_key_jwt`).
- Irmã direta da **RFC 9449 (DPoP)**, próximo capítulo — as duas são mecanismos de *sender-constraining*, resolvendo o mesmo problema (roubo de bearer token) por caminhos diferentes: mTLS amarra o token à camada de **transporte** (exige PKI completa e infraestrutura mTLS, mais pesado, mas reaproveita controles de PKI já maduros como revogação de certificado); DPoP amarra o token à camada de **aplicação** (mais leve, sem PKI nenhuma, funciona sobre HTTPS comum, sem sofrer com a complicação de terminação de mTLS em gateways).
- Descoberta via **RFC 8414** através dos novos campos de metadata.

7. O que mudou desde a publicação (2020)

- Spec estável, cada vez mais **exigida** (não só recomendada) em setores de alta garantia. No seu contexto FSI, vale reter: **FAPI 2.0 trata sender-constraining (mTLS OU DPoP) como praticamente obrigatório** no perfil baseline, não como hardening opcional — não é mais "se vocês quiserem", é requisito de conformidade em muitos perfis regulatórios de open banking.
- A armadilha operacional de "terminação de TLS no gateway quebra mTLS" virou um problema bem documentado conforme mais organizações tentaram implantar isso na prática. Provedores de nuvem (AWS ALB/NLB, vários API gateways populares) adicionaram suporte específico a "passthrough de mTLS" ou "encaminhamento de certificado via header confiável", justamente para essa fricção repetida.
- A **RFC 9449 (DPoP)**, próximo capítulo, nasceu em boa parte porque a exigência de infraestrutura do mTLS (PKI completa, configuração cuidadosa de gateway, gestão de ciclo de vida de certificado de cliente) é pesada para muitas implantações — especialmente SPAs e apps mobile, onde gerenciar um certificado de cliente é operacionalmente incômodo (rotação, distribuição segura, revogação). DPoP costuma ser descrito como "a alternativa mais leve ao mTLS para sender-constraining".

Nota de laboratório: nenhum lab construído ainda para esta RFC neste projeto. Diferente de todos os capítulos anteriores do roteiro, este é o primeiro que exigiria genuinamente TLS real (certificados de cliente, possivelmente uma CA pequena para emití-los) para fazer sentido —

rompendo deliberadamente a simplificação de HTTP puro mantida em todos os labs até aqui (ver decisão de manter TLS adiado até este capítulo). Os exemplos de código abaixo, portanto, são standalone — não integrados a nenhum lab existente deste projeto.

Exemplo em Java (JDK 21, sem frameworks)

Os exemplos a seguir usam somente `java.security.*`, `javax.net.ssl.*` e `java.net.http.HttpClient` — nada de bibliotecas externas. Para os campos JSON de saída (quando necessário), reaproveitamos o utilitário `SimpleJson`, definido no Capítulo 1 (RFC 6749) deste livro.

Nota didática importante: os exemplos abaixo usam `KeyStore` / `SSLContext` do próprio JDK para o handshake TLS em si (isso é produção-grade — é exatamente como uma aplicação Java real faria mTLS). Já o cálculo manual do thumbprint (`x5t#S256`) e a comparação com uma claim `cnf` de um JWT são feitos "na mão" apenas para fins didáticos, para expor o mecanismo. Em produção, a extração e validação de claims de um JWT (incluindo verificação de assinatura, `exp`, `aud`, etc.) deve usar uma biblioteca JOSE validada (ex.: Nimbus JOSE+JWT) — nunca parsing manual de JSON de um JWT em produção.

1. Cliente HTTP com certificado mTLS (`tls_client_auth`)

Configura um `HttpClient` para apresentar um certificado de cliente (com sua chave privada) durante o handshake TLS, ao chamar o token endpoint.


```

import javax.net.ssl.KeyManagerFactory;
import javax.net.ssl.SSLContext;
import javax.net.ssl.TrustManagerFactory;
import java.io.FileInputStream;
import java.net.URI;
import java.net.http.HttpClient;
import java.net.http.HttpRequest;
import java.net.http.HttpResponse;
import java.security.KeyStore;
import java.time.Duration;

public class ClienteMtls {

    /**
     * Monta um HttpClient que apresenta um certificado de cliente (X.509 +
     * chave privada) durante o handshake TLS – a prova de identidade do
     * client, no lugar de client_secret, acontece aqui, na camada de
     * transporte, ANTES de qualquer byte HTTP ser trocado.
     */
    static HttpClient criarClienteComCertificado(
        String keystorePath, char[] keystoreSenha,
        String truststorePath, char[] truststoreSenha) throws Exception {

        // KeyStore do CLIENT: contém o certificado X.509 do client + sua
        // chave privada correspondente. E a chave privada nunca sai deste
        // processo – só a assinatura implícita do handshake TLS viaja.
        KeyStore keyStore = KeyStore.getInstance("PKCS12");
        try (var in = new FileInputStream(keystorePath)) {
            keyStore.load(in, keystoreSenha);
        }
        KeyManagerFactory kmf = KeyManagerFactory.getInstance(
            KeyManagerFactory.getDefaultAlgorithm());
        kmf.init(keyStore, keystoreSenha);

        // TrustStore: quais CAs este client confia PARA VALIDAR O SERVIDOR
        // (o "outro lado" do mTLS – o client também precisa confiar no
        // certificado do AS, como em qualquer TLS normal).
        KeyStore trustStore = KeyStore.getInstance("PKCS12");
        try (var in = new FileInputStream(truststorePath)) {
            trustStore.load(in, truststoreSenha);
        }
        TrustManagerFactory tmf = TrustManagerFactory.getInstance(
            TrustManagerFactory.getDefaultAlgorithm());
        tmf.init(trustStore);

        SSLContext sslContext = SSLContext.getInstance("TLSv1.3");
        sslContext.init(kmf.getKeyManagers(), tmf.getTrustManagers(), null);

        return HttpClient.newBuilder()
            .sslContext(sslContext)
            .connectTimeout(Duration.ofSeconds(10))
            .build();
    }
}

```

```

    }

    /**
     * POST /token autenticado por mTLS: repare que NÃO há client_secret nem
     * client_assertion no corpo – o certificado apresentado no handshake
     * (configurado acima) É a credencial do client.
     */
    static HttpResponse<String> requisitarToken(HttpClient client, String tokenEndpoint,
                                                String clientId, String code,
                                                String redirectUri) throws Exception {
        String corpo = "grant_type=authorization_code"
            + "&client_id=" + clientId
            + "&code=" + code
            + "&redirect_uri=" + redirectUri;

        HttpRequest request = HttpRequest.newBuilder()
            .uri(URI.create(tokenEndpoint))
            .header("Content-Type", "application/x-www-form-urlencoded")
            .POST(HttpRequest.BodyPublishers.ofString(corpo))
            .build();

        return client.send(request, HttpResponse.BodyHandlers.ofString());
    }
}

```

2. Calculando o thumbprint x5t#S256 de um certificado

Do lado do resource server (ou de um validador standalone), calcula o SHA-256 sobre a codificação DER do certificado — exatamente o que o AS calculou ao emitir o token, e o que precisa bater com a claim `cnf.x5t#S256` recebida.

```

import java.security.MessageDigest;
import java.security.cert.Certificate;
import java.security.cert.CertificateFactory;
import java.util.Base64;
import java.io.FileInputStream;

public class ThumbprintCertificado {

    /**
     * x5t#S256 (RFC 8705 §3.1): SHA-256 sobre a codificação DER do
     * certificado inteiro, em Base64URL sem padding. Determinístico – o
     * mesmo certificado sempre produz o mesmo thumbprint, permitindo
     * comparação simples de string em vez de comparar certificados inteiros.
     */
    static String calcularX5tS256(Certificate certificado) throws Exception {
        byte[] der = certificado.getEncoded(); // getEncoded() já retorna DER p/ X.509
        MessageDigest sha256 = MessageDigest.getInstance("SHA-256");
        byte[] hash = sha256.digest(der);
        return Base64.getUrlEncoder().withoutPadding().encodeToString(hash);
    }

    static Certificate carregarCertificadoDoKeystore(String path, char[] senha,
                                                    String alias) throws Exception {
        var keyStore = java.security.KeyStore.getInstance("PKCS12");
        try (var in = new FileInputStream(path)) {
            keyStore.load(in, senha);
        }
        return keyStore.getCertificate(alias);
    }

    // Alternativa: a partir de bytes PEM/DER recebidos por outro canal
    // (ex.: header de um gateway que repassa o certificado do client –
    // ver a discussão de "terminação de TLS quebra mTLS" na seção 5).
    static Certificate carregarCertificadoDePem(byte[] pemBytes) throws Exception {
        CertificateFactory cf = CertificateFactory.getInstance("X.509");
        return cf.generateCertificate(new java.io.ByteArrayInputStream(pemBytes));
    }
}

```

3. Validando um access token JWT com `cnf.x5t#S256` (resource server)

Extraí a claim `cnf.x5t#S256` de um JWT (parsing manual, só didático — ver nota acima) e compara com o thumbprint do certificado apresentado nesta conexão TLS.

```

import java.util.Base64;
import java.util.Map;

public class ValidadorTokenVinculado {

    /**
     * Extrai o payload (segunda parte) de um JWT compacto e localiza
     * cnf.x5t#S256. Parsing manual, só didático – em produção, valide
     * assinatura/exp/aud/etc. com uma biblioteca JOSE antes de sequer olhar
     * para claims de negócio.
     */
    static String extrairThumbprintEsperado(String jwtCompacto) {
        String[] partes = jwtCompacto.split("\\.");
        if (partes.length != 3) {
            throw new IllegalArgumentException("JWT malformado");
        }
        String payloadJson = new String(
            Base64.getUrlDecoder().decode(partes[1]),
            java.nio.charset.StandardCharsets.UTF_8);

        // parseFlat aqui é ilustrativo – um payload de JWT real tem
        // estrutura aninhada (cnf é um objeto dentro do payload); um parser
        // de produção precisaria de suporte a objetos aninhados, que o
        // SimpleJson (utilitário do Capítulo 1) não cobre por ser
        // deliberadamente simples. Adaptar conforme a estrutura real.
        Map<String, String> payload = SimpleJson.parseFlat(payloadJson);
        return payload.get("cnf.x5t#S256"); // ilustrativo – depende do parser usado
    }

    /**
     * O passo de segurança central da RFC 8705: token só é aceito se o
     * thumbprint do certificado NESTA conexão bater com o vinculado ao token.
     */
    static boolean tokenValidoParaEstaConexao(String thumbprintEsperado,
                                                String thumbprintDaConexaoAtual) {

        if (thumbprintEsperado == null) {
            // Token não é vinculado a certificado – decisão de política:
            // uma implantação que EXIGE sender-constraining deve rejeitar
            // aqui, não tratar como "válido sem vínculo".
            return false;
        }
        // Comparação de thumbprints – strings já normalizadas (Base64URL),
        // então comparação direta é suficiente (evite MessageDigest.isEqual
        // aqui não ser necessário: não estamos comparando segredos, e sim
        // valores públicos e determinísticos).
        return thumbprintEsperado.equals(thumbprintDaConexaoAtual);
    }
}

```

RFC 9449 — OAuth 2.0 Demonstrating Proof of Possession (DPoP)

1. Contexto e motivação

Mesmo problema de fundo do capítulo anterior (RFC 8705, mTLS): bearer token roubado é acesso total até a expiração, fraqueza que a RFC 6750 já registrava como fora de seu próprio escopo desde o capítulo 2 deste roteiro. Mas mTLS tem um custo de infraestrutura real — PKI completa, gestão de ciclo de vida de certificado de cliente, e o problema recorrente de terminação de TLS quebrando o vínculo quando há um gateway no meio do caminho, como vimos em detalhe no capítulo anterior.

O DPoP propõe uma alternativa **deliberadamente mais leve**: sender-constraining implementado inteiramente na **camada de aplicação**, usando um par de chaves assimétrico comum — sem PKI, sem certificado, sem CA, sem cadeia de confiança nenhuma — que funciona sobre HTTPS normal e sobrevive a qualquer proxy no meio do caminho, porque **nada depende da conexão TLS em si**, só de um objeto assinado (um JWT) enviado como header HTTP comum, ao lado de qualquer requisição.

Isso torna o DPoP particularmente atraente para exatamente as categorias de client onde o mTLS mais dói:

- **SPAs no browser** — já têm acesso à Web Crypto API, que consegue gerar pares de chave e assinar sem nunca expor a chave privada ao JavaScript da página (chaves não-extraíveis, `extractable: false`).
- **Apps mobile** — já têm acesso a keystores seguros (Android Keystore, iOS Secure Enclave) para gerar e usar pares de chave sem exportar a chave privada.
- **CLIs e ferramentas de desenvolvedor** — já geram pares de chave locais para outros fins (SSH, por exemplo) e não têm ciclo de vida de certificado X.509 para gerenciar.

Nenhuma dessas categorias tradicionalmente gerencia certificados de cliente — exigir mTLS delas seria adicionar uma peça de infraestrutura nova e cara. DPoP reaproveita algo que muitas delas já sabem fazer (gerar e usar um par de chaves) e adiciona só a disciplina de assinar uma prova nova a cada chamada.

Por que uma prova por requisição, e não uma vez por token?

Esse é o ponto mais frequentemente mal entendido do DPoP na primeira leitura: a prova DPoP **não é assinada uma vez, quando o token é emitido** — ela é assinada **de novo, a cada requisição HTTP individual**, tanto ao obter o token quanto ao usá-lo depois no resource server. Isso parece redundante à primeira vista (por que assinar de novo se a chave é a mesma?), mas é exatamente o que permite ao servidor aplicar uma janela de frescor bem apertada e detectar replay de forma muito mais rígida do que seria possível com uma única prova reaproveitada — ver seção 5.

2. Terminologia e papéis

Nenhum papel novo. Introduz:

- **DPoP proof** — um JWT assinado a cada requisição individual, carregando `htm` (método HTTP em maiúsculas), `htu` (URI de destino, **sem** query string nem fragmento), `iat` (timestamp de criação), `jti` (identificador único por prova, nunca reaproveitado) e, no **header** do próprio JWT (não no payload), a chave pública do client embutida como JWK (`jwk`).
- **Header HTTP DPoP** — carrega essa prova, tanto na requisição ao token endpoint (provando posse da chave que vai ser vinculada ao token que está sendo emitido) quanto em cada requisição subsequente ao resource server (provando, de novo, posse da mesma chave, a cada chamada).
- **Esquema de autorização DPoP** — tokens vinculados a DPoP usam `Authorization: DPoP <token>`, não `Authorization: Bearer <token>`. Isso é deliberado: o esquema no header já denuncia, para qualquer observador da mensagem (incluindo logs, proxies, ferramentas de depuração), que aquele token exige uma prova de posse para ser aceito — não é "só mais um bearer".
- `ath` (access token hash) — quando a prova acompanha uma chamada ao resource server (não ao token endpoint), carrega o hash SHA-256 (Base64URL) do próprio access token apresentado junto, amarrando explicitamente a prova àquele token específico.
- `jkt` (JWK Thumbprint, RFC 7638) — thumbprint canônico da chave pública, calculado a partir dos membros obrigatórios do JWK (varia por tipo de chave — para EC: `crv`, `ktu`, `x`, `y`; para RSA: `e`, `ktu`, `n`), ordenados alfabeticamente, serializados sem espaços, SHA-256, Base64URL. Aparece na claim `cnf.jkt` do access token emitido — mesma estrutura de extensão `cnf` que a RFC 8705 já usava com `x5t#S256`, só que aqui o que é vinculado é uma chave pública crua, não um certificado.
- `typ: dpop+jwt` — o header do JWT de prova se autodeclara como prova DPoP, não um JWT genérico. Mesmo princípio de autodeclaração de tipo já visto em `typ: at+jwt` (RFC 9068): evita que uma prova DPoP seja aceita em um contexto onde qualquer outro JWT seria válido, ou vice-versa.

Por que a chave pública viaja no header do JWT, e não é pré-registrada?

Diferente de `private_key_jwt` (RFC 7523) ou de JWKS pré-registrado, o DPoP **não exige registro prévio de chave nenhuma**. A chave pública viaja embutida em cada prova (`jwk` no header). Isso é o que torna o DPoP utilizável por clients totalmente públicos, sem nenhum passo de provisionamento de chave: o client simplesmente gera um par de chaves na primeira vez que precisa, e a chave pública "se apresenta" na primeira prova que ele envia. O vínculo de confiança não vem de "o AS já conhecia essa chave" — vem de "o AS emitiu o token vendo, naquele exato momento, uma prova de posse daquela chave, e vinculou o thumbprint dela ao token resultante". A partir daí, qualquer uso subsequente do token precisa reapresentar prova da mesma chave.

3. Fluxo

1. Client gera um par de chaves (uma vez, guardado localmente — Web Crypto API não-extraível no browser, keystore seguro no mobile, ou um arquivo de chave local numa CLI).

2. Requisição de token, com prova DPoP:

```
POST /token
DPoP: <JWT: header={typ:"dpop+jwt", alg, jwk:<chave publica>},
      payload={htm:"POST", htu:"https://as.exemplo.com/token", iat, jti}>
grant_type=...&code=...
```

AS valida a prova:

- assinatura bate com o jwk embutido no proprio header da prova
- typ = dpop+jwt
- htm/htu conferem com esta requisicao exata
- iat dentro de uma janela de frescor aceitavel
- jti nao foi visto antes (nesta janela)

AS emite o access_token com token_type=DPoP, vinculado:
cnf.jkt = thumbprint (RFC 7638) dessa chave publica

3. Requisição de recurso, com uma prova NOVA (fresca por requisição) + o token vinculado:

```
GET /api/recurso
Authorization: DPoP <access_token>      - esquema "DPoP", nao "Bearer"
DPoP: <JWT: header={jwk: mesma chave publica},
      payload={htm:"GET", htu:"https://rs.exemplo.com/api/recurso",
               iat, jti, ath:<hash do access_token>}>
```

RS valida, nesta ordem:

- 1) o access_token em si (introspeccao ou verificacao local do JWT) -> confirma cnf.jkt = X
- 2) assinatura da prova bate com o jwk do proprio header da prova
- 3) thumbprint desse jwk == cnf.jkt do token (precisa bater)
- 4) htm/htu conferem com esta requisicao exata
- 5) iat dentro da janela de frescor
- 6) jti nao foi reaproveitado (dentro de uma janela de rastreo)
- 7) ath bate com o hash do access_token apresentado nesta mesma chamada

A diferença mecânica central em relação ao mTLS (capítulo anterior): em vez de amarrar o token a uma **conexão** (identidade de certificado TLS), o DPoP amarra o token a uma **chave usada para assinar uma prova nova a cada requisição individual** — sem exigência nenhuma de camada de transporte. Isso significa que o vínculo sobrevive a qualquer proxy, load balancer ou gateway no meio do caminho, porque a prova é só mais um header HTTP comum, visível e intacto de ponta a ponta — nada equivalente ao problema de "terminação de TLS quebra mTLS" existe aqui.

Caso de borda: renovação de token (refresh_token)

Quando um client troca um refresh_token por um novo access_token , a requisição ao token endpoint também precisa vir acompanhada de uma prova DPoP — e o AS deve confirmar que essa prova usa **a mesma chave** já vinculada ao refresh_token originalmente (via cnf.jkt associado ao registro do refresh_token , não só do access_token). Sem essa checagem na renovação, um refresh_token

roubado (que tipicamente não é ele mesmo enviado ao resource server, então não se beneficiaria diretamente do `ath`) ainda poderia ser trocado por um `access_token` novo por quem não tem a chave privada original — exatamente o mesmo raciocínio do caso de borda equivalente na RFC 8705.

Caso de borda: troca de chave no meio de uma sessão

Nada na RFC proíbe um client de gerar uma chave nova (por exemplo, ao reinstalar, ou por rotação periódica de segurança). Mas isso invalida qualquer token já vinculado à chave antiga — o client precisa passar por um novo fluxo de obtenção de token (ou renovação, se ainda tiver um `refresh_token` — mas aí esbarra na checagem do parágrafo anterior). Na prática, a maioria dos clients gera a chave uma vez e a mantém pela vida útil da instalação/sessão.

4. Parâmetros e endpoints normativos

- **Header `DPoP`** — novo, carrega a prova compacta (JWT serializado), tanto na requisição de token quanto na de recurso.
- **Esquema `Authorization: DPoP <token>`** — substitui `Bearer` para tokens vinculados assim; visível diretamente na mensagem HTTP, sem precisar inspecionar o token em si para saber que exige prova.
- `token_type` na resposta do token endpoint vira `"DPoP"`, não `"Bearer"` — outro sinal explícito, desta vez na resposta de emissão.
- Descoberto via RFC 8414: `dpop_signing_alg_values_supported`, listando quais algoritmos de assinatura o AS aceita para provas DPoP (tipicamente `ES256` é o mais recomendado por gerar assinaturas menores que RSA, mas a RFC não exige um algoritmo específico).
- Erro característico no resource server: token DPoP apresentado sem prova válida/correspondente → `401` com header `WWW-Authenticate: DPoP ...` e um código de erro como `error="invalid_dpop_proof"`, seguindo o mesmo formato de erro que a RFC 6750 já definia para `WWW-Authenticate: Bearer`, adaptado ao novo esquema.
- O header da prova deve trazer `typ: dpop+jwt` — se ausente ou com outro valor, a prova deve ser rejeitada mesmo que a assinatura seja válida (proteção contra confusão de tipo de token, mesmo princípio de `typ: at+jwt` na RFC 9068).
- Um servidor de autorização também pode emitir um **nonce** de servidor (header `DPoP-Nonce`) que o client deve ecoar na claim `nonce` da próxima prova — mecanismo opcional adicional contra certas formas de replay antecipado ("pré-computar" provas offline); não é obrigatório em toda implantação, mas está previsto na RFC para AS que queiram essa camada extra.

5. Segurança já tratada na própria RFC 9449

Proteção contra replay via `jti` + janela curta de `iat`

Como cada requisição exige uma prova nova, o servidor pode aplicar uma janela de frescor bem apertada (rejeitar provas com `iat` de mais de alguns segundos/poucos minutos atrás) e rastrear os `jti`s vistos recentemente (tipicamente num cache com TTL curto, como Redis) — proteção de replay

muito mais rígida do que a de um único bearer token de vida longa, já que uma prova capturada por um atacante (de um log, de um proxy comprometido) fica inútil quase imediatamente, tanto pela janela de tempo quanto pelo `jti` já consumido.

`htm` / `htu` amarram a prova à requisição exata

Uma prova feita para uma chamada específica (combinação de método HTTP + URL, sem query nem fragmento) não serve para nenhuma outra — mesmo que capturada a tempo (dentro da janela de frescor), não pode ser reaproveitada para chamar um endpoint diferente ou usar um método HTTP diferente no mesmo endpoint.

`ath` amarra a prova ao token específico

Impede que um atacante que tenha capturado, separadamente, um `access_token` válido e alguma prova DPoP não relacionada (de outra chamada, de outro token, potencialmente até de outro client com a mesma chave — cenário artificial, mas ilustra o ponto) consiga combinar os dois. A prova precisa referenciar explicitamente o hash do token específico com que está sendo usada — sem `ath` correto, a prova não vale para aquele token, mesmo sendo uma prova DPoP legítima e recente.

A chave privada nunca sai do client

Mesma garantia central do mTLS/ `private_key_jwt` : só a prova (uma assinatura sobre um payload pequeno) trafega pela rede, nunca a chave privada em si. Clients em browser podem usar chaves não-extraíveis via Web Crypto API — mesmo um script malicioso injetado na mesma página (via alguma vulnerabilidade de XSS, por exemplo) não consegue exfiltrar a chave privada propriamente dita, porque o próprio browser recusa a operação de exportação. Isso não é proteção total, porém: um script malicioso rodando no mesmo contexto **ainda pode pedir ao browser para assinar provas arbitrárias** enquanto tiver acesso à página — DPoP protege contra roubo/replay de token capturado na rede ou em um log, não contra um runtime de client totalmente comprometido, que consegue abusar da própria API de assinatura em tempo real.

Comparação com mTLS em termos de revogação

DPoP não tem equivalente aos controles de revogação em nível de PKI que o mTLS herda naturalmente (listas de revogação de certificado, controles de CA, verificação OCSP). Se uma chave DPoP for comprometida, não existe um mecanismo padronizado de "lista de chaves revogadas" — a mitigação prática é a vida curta do `access_token` e a recusa em renovar `refresh_token`s vinculados a uma chave suspeita (decisão operacional do AS, não normatizada pela RFC). Para a ameaça específica que motivou a RFC — "token roubado da rede ou de um log" — DPoP é igualmente eficaz ao mTLS, a um custo de infraestrutura muito menor; mas não é um substituto direto para todos os controles que uma PKI completa oferece.

6. Relação com outras RFCs

- Irmã direta/alternativa da **RFC 8705 (mTLS)**, capítulo anterior — mesmo objetivo (sender-constraining contra a fraqueza de bearer da RFC 6750), mecanismo diferente (prova assinada na camada de aplicação vs. certificado na camada de transporte TLS). A RFC 9700 recomenda qualquer um dos dois, sem preferência estrutural.
- `cnf.jkt` em access tokens JWT é a instância específica do DPoP do mesmo ponto de extensão `cnf` que a RFC 9068 definiu e que a RFC 8705 já usava (com `x5t#S256`) — mesma estrutura de claim, mecanismos de vínculo diferentes.
- `typ: dpop+jwt` espelha exatamente o mesmo princípio de autodeclaração de tipo do `typ: at+jwt` da RFC 9068.
- Descoberto via `dpop_signing_alg_values_supported` da RFC 8414.
- Particularmente adequado a clients públicos (SPAs, mobile, CLI) que já geram uma chave efêmera localmente — resolve a lacuna de autenticação forte para esse tipo de client (que não pode guardar um `client_secret` com segurança) já discutida nos capítulos da RFC 6749 e da RFC 9700, sem exigir `client_secret` nem PKI.
- No seu contexto FSI: **FAPI 2.0 trata DPoP como um dos dois mecanismos aceitáveis de sender-constraining** (ao lado de mTLS) no perfil baseline — implantações que não têm PKI de client madura tendem a preferir DPoP por esse motivo.

7. O que mudou desde a publicação (2023)

- Uma das RFCs mais recentes deste roteiro — adoção crescendo ativamente, sobretudo em arquiteturas com muito SPA/mobile, onde o custo de PKI do mTLS sempre foi mal ajustado à realidade operacional desses clients.
- Cada vez mais vista como recomendação padrão para sender-constraining em implantações novas que ainda não têm infraestrutura de mTLS, dado o custo operacional bem menor de "gerar um par de chaves" comparado a "operar uma PKI de client".
- Suporte de ferramentas amadureceu rápido desde a publicação — integração com Web Crypto API em bibliotecas OAuth de front-end, e SDKs mobile incorporando geração/uso de chave DPoP nativamente.

Nota de laboratório: implementado em [rfc9449/](#) — confirmando a observação acima, roda inteiramente em HTTP puro, sem quebrar a simplificação de TLS deliberadamente adiada até a RFC 8705. O client é público (sem `client_secret`, sem JWKS pré-registrada — a chave viaja embutida em cada prova, como descrito na seção 2). Curiosamente, o lab usa chaves **RSA com alg: RS256** para as provas (não EC/ ES256, que é a escolha mais comum na prática por gerar assinaturas menores) — a RFC não exige um algoritmo específico, só que ele apareça em `dpop_signing_alg_values_supported`. Todo `access_token` / `refresh_token` carrega `cnf.jkt`; o AS valida a prova na emissão (inclusive conferindo, na renovação, que a prova vem da mesma chave já vinculada ao `refresh_token` — exatamente o caso de borda discutido na seção 3), e o RS valida na chamada ao recurso.

Os logs de montagem (`logs.sh client`) e de validação passo a passo (`logs.sh as / logs.sh rs`) foram um requisito explícito deste lab — cada checagem (assinatura, `typ`, `htm / htu`, janela de `iat`, `jti`, `ath`, thumbprint vs. `cnf.jkt`) é logada individualmente pelo módulo `dpop.js` (compartilhado entre AS e RS para a validação, e usado pelo client para a montagem), com o motivo exato de rejeição quando falha — por exemplo, `"htu nao confere com a requisicao"` ou `"prova ja utilizada antes - replay detectado via jti"`. Dois botões de ataque na UI do lab exercitam exatamente as proteções da seção 5: usar o token como `Bearer` simples (sem prova — corretamente rejeitado, já que o `token_type` é `DPoP`) e reenviar a mesma prova capturada (replay — corretamente rejeitado pela checagem de `jti`).

Exemplo em Java (JDK 21, sem frameworks)

Usamos `java.security.*` (para o par de chaves EC e assinatura), `java.net.http.HttpClient` e `java.util.Base64`. Para serialização/parsing JSON simples, reaproveitamos o utilitário `SimpleJson`, definido no Capítulo 1 (RFC 6749) deste livro.

Nota didática importante: assim como no capítulo anterior, montar e assinar um JWT "na mão" abaixo é só para fins didáticos, para expor exatamente os passos que uma prova DPoP exige. Em produção, use uma biblioteca JOSE validada (ex.: Nimbus JOSE+JWT) tanto para assinar quanto para validar — parsing manual de JSON dentro de um JWT em produção é uma fonte comum de vulnerabilidades sutis (confusão de algoritmo, canonicalização incorreta de JSON, etc.).

1. Gerando o par de chaves EC (P-256)

```
import java.security.KeyPair;
import java.security.KeyPairGenerator;
import java.security.interfaces.ECPublicKey;
import java.security.spec.ECGenParameterSpec;

public class ChaveDpop {

    /**
     * P-256 (secp256r1) é a curva mais comumente usada para provas DPoP,
     * combinada com o algoritmo de assinatura ES256 – assinaturas bem
     * menores que RSA equivalente, o que importa porque a prova viaja
     * inteira em um header HTTP a CADA requisição.
     */
    static KeyPair gerarParDeChaves() throws Exception {
        KeyPairGenerator gerador = KeyPairGenerator.getInstance("EC");
        gerador.initialize(new ECGenParameterSpec("secp256r1"));
        return gerador.generateKeyPair();
    }

    /**
     * Extrai as coordenadas x/y da chave pública EC, em Base64URL sem
     * padding – os campos que compõem um JWK de chave EC (RFC 7518 §6.2).
     */
    static String[] coordenadasXY(ECPublicKey chavePublica) {
        var ponto = chavePublica.getW();
        int tamanhoCampo = (chavePublica.getParams().getCurve().getField()
            .getFieldSize() + 7) / 8; // 32 bytes para P-256

        return new String[] {
            java.util.Base64.getUrlEncoder().withoutPadding()
                .encodeToString(paraBytesTamanhoFixo(ponto.getAffineX(), tamanhoCampo)),
            java.util.Base64.getUrlEncoder().withoutPadding()
                .encodeToString(paraBytesTamanhoFixo(ponto.getAffineY(), tamanhoCampo))
        };
    }

    private static byte[] paraBytesTamanhoFixo(java.math.BigInteger valor, int tamanho) {
        byte[] bruto = valor.toByteArray();
        byte[] resultado = new byte[tamanho];
        int copiarDe = Math.max(0, bruto.length - tamanho);
        int copiarPara = Math.max(0, tamanho - bruto.length);
        System.arraycopy(bruto, copiarDe, resultado, copiarPara, bruto.length - copiarDe);
        return resultado;
    }
}
```

2. Montando e assinando a prova DPOP, calculando jkt

```
import java.security.Signature;
import java.security.interfaces.ECPrivateKey;
import java.security.interfaces.ECPublicKey;
import java.security.MessageDigest;
import java.util.Base64;
import java.util.LinkedHashMap;
import java.util.Map;
import java.util.UUID;

public class ProvaDpop {

    /**
     * Monta o JWK (público) da chave EC, como um mapa ORDENADO – a ordem
     * importa para o calculo do thumbprint (RFC 7638), que exige os
     * membros obrigatorios em ordem alfabetica antes de fazer o hash.
     */
    static Map<String, String> montarJwkPublico(ECPublicKey chavePublica) {
        String[] xy = ChaveDpop.coordenadasXY(chavePublica);
        // Ordem alfabetica dos membros obrigatorios: crv, kty, x, y
        Map<String, String> jwk = new LinkedHashMap<>();
        jwk.put("crv", "P-256");
        jwk.put("kty", "EC");
        jwk.put("x", xy[0]);
        jwk.put("y", xy[1]);
        return jwk;
    }

    /**
     * jkt (RFC 7638): SHA-256 sobre a serializacao JSON canonica (membros
     * ordenados, sem espacos) dos campos obrigatorios do JWK, em Base64URL.
     * E o valor que o AS embute em cnf.jkt no token, e que qualquer
     * validador recalcula a partir do jwk apresentado numa prova para
     * conferir contra o token.
     */
    static String calcularJkt(Map<String, String> jwkOrdenado) throws Exception {
        String json = SimpleJson.object(new LinkedHashMap<>(jwkOrdenado));
        MessageDigest sha256 = MessageDigest.getInstance("SHA-256");
        byte[] hash = sha256.digest(json.getBytes(java.nio.charset.StandardCharsets.UTF_8));
        return Base64.getUrlEncoder().withoutPadding().encodeToString(hash);
    }

    /**
     * Monta e assina uma prova DPOP NOVA para uma requisicao especifica.
     * accessToken, se informado, ativa o claim ath (uso ao chamar o
     * resource server; ausente na requisicao ao token endpoint).
     */
    static String montarEAssinarProva(KeyPair par, String htm, String htu,
                                     String accessToken) throws Exception {
        ECPublicKey chavePublica = (ECPublicKey) par.getPublic();
        ECPrivateKey chavePrivada = (ECPrivateKey) par.getPrivate();
    }
```

```

Map<String, String> jwk = montarJwkPublico(chavePublica);

// Header do JWT: typ autodeclarado + alg + a chave publica embutida
Map<String, Object> header = new LinkedHashMap<>();
header.put("typ", "dpop+jwt");
header.put("alg", "ES256");
header.put("jwk", jwk);
String headerJson = montarJsonComObjetoAninhado(header);
String headerB64 = Base64.getUrlEncoder().withoutPadding()
    .encodeToString(headerJson.getBytes(java.nio.charset.StandardCharsets.UTF_8));

long iat = System.currentTimeMillis() / 1000;
String jti = UUID.randomUUID().toString();

Map<String, String> payload = new LinkedHashMap<>();
payload.put("htm", htm);
payload.put("htu", htu);
payload.put("iat", String.valueOf(iat));
payload.put("jti", jti);
if (accessToken != null) {
    // ath: hash do access_token apresentado nesta mesma chamada -
    // amarra a prova aquele token especifico (secao 5).
    MessageDigest sha256 = MessageDigest.getInstance("SHA-256");
    byte[] hashToken = sha256.digest(
        accessToken.getBytes(java.nio.charset.StandardCharsets.UTF_8));
    payload.put("ath", Base64.getUrlEncoder().withoutPadding().encodeToString(hashToken));
}
String payloadJson = SimpleJson.object(new LinkedHashMap<>(payload));
String payloadB64 = Base64.getUrlEncoder().withoutPadding()
    .encodeToString(payloadJson.getBytes(java.nio.charset.StandardCharsets.UTF_8));

String assinandoSobreEsteConteudo = headerB64 + "." + payloadB64;

// ES256 = ECDSA sobre SHA-256 - a chave privada NUNCA sai deste
// metodo: so a assinatura resultante e retornada.
Signature assinatura = Signature.getInstance("SHA256withECDSA");
assinatura.initSign(chavePrivada);
assinatura.update(assinandoSobreEsteConteudo.getBytes(java.nio.charset.StandardCharsets.UTF_8));
byte[] assinaturaDer = assinatura.sign();
// Nota: Signature.sign() com ECDSA produz DER; JWS exige o formato
// R||S de tamanho fixo (JOSE) - conversao omitida aqui por brevidade,
// mas obrigatoria numa implementacao real (mais um motivo para usar
// uma biblioteca JOSE em producao, que ja faz essa conversao).
String assinaturaB64 = Base64.getUrlEncoder().withoutPadding().encodeToString(assinaturaDer);

return assinandoSobreEsteConteudo + "." + assinaturaB64;
}

// Serializacao minima s[ó para permitir um objeto aninhado (jwk) dentro
// do header - SimpleJson.object (Cap. 1) assume valores string/flat;
// esta variante e um complemento local, so para este exemplo.
private static String montarJsonComObjetoAninhado(Map<String, Object> mapa) {
    StringBuilder sb = new StringBuilder("{");

```

```
boolean primeiro = true;
for (var entrada : mapa.entrySet()) {
    if (!primeiro) sb.append(",");
    primeiro = false;
    sb.append("\"").append(entrada.getKey()).append("\":");
    if (entrada.getValue() instanceof Map<?, ?> aninhado) {
        @SuppressWarnings("unchecked")
        Map<String, String> m = (Map<String, String>) aninhado;
        sb.append(SimpleJson.object(new LinkedHashMap<>(m)));
    } else {
        sb.append("\"").append(entrada.getValue()).append("\"");
    }
}
return sb.append("}").toString();
}
```

3. Validando `jkt` contra `cnf.jkt` do token (resource server)

```
import java.util.Map;

public class ValidadorDpop {

    /**
     * Passo central de validacao no resource server: o thumbprint da chave
     * usada NESTA prova precisa bater com o cnf.jkt vinculado ao token.
     * Sem essa igualdade, o token nao serve para esta chamada, mesmo com
     * assinatura da prova valida e token em si nao expirado.
     */
    static boolean chaveDaProvaConfereComToken(Map<String, String> jwkDaProva,
                                                String jktEsperadoDoToken) throws Exception {
        String jktCalculado = ProvaDpop.calcularJkt(
            new java.util.LinkedHashMap<>(jwkDaProva));
        return jktCalculado.equals(jktEsperadoDoToken);
    }

    /**
     * ath: confere que a prova foi feita especificamente para o
     * access_token apresentado nesta chamada (nao para outro token).
     */
    static boolean athConfereComToken(String athDaProva, String accessTokenApresentado)
        throws Exception {
        var sha256 = java.security.MessageDigest.getInstance("SHA-256");
        byte[] hash = sha256.digest(
            accessTokenApresentado.getBytes(java.nio.charset.StandardCharsets.UTF_8));
        String athEsperado = java.util.Base64.getUrlEncoder().withoutPadding()
            .encodeToString(hash);
        return athEsperado.equals(athDaProva);
    }

    /**
     * htm/htu: a prova so vale para o metodo+URI exatos desta requisicao.
     * htu NUNCA inclui query string ou fragmento (RFC 9449 §4.2).
     */
    static boolean htmHtuConferem(String htmDaProva, String htuDaProva,
                                    String metodoRequisicaoAtual, String uriRequisicaoAtual) {
        return htmDaProva.equals(metodoRequisicaoAtual)
            && htuDaProva.equals(uriRequisicaoAtual);
    }
}
```


RFC 8707 — Resource Indicators for OAuth 2.0

1. Contexto e motivação

Este é o capítulo onde fechamos, com profundidade, a pergunta que ficou em aberto desde o capítulo da RFC 6819: **a segmentação clara de nomes de scope por recurso já não resolveria o problema de "confused deputy" sozinha, sem precisar de um mecanismo dedicado de audience?**

A análise completa: scope segregado resolve o problema?

Considere o cenário canônico de confused deputy em OAuth: um token emitido com `scope=accounts:read` (pensado, no momento da emissão, para ser usado contra a API de contas) é apresentado — por engano do client, por um proxy mal configurado, ou por má-fé de um atacante que o interceptou — à API de pagamentos. A pergunta que a RFC 6819 deixou em aberto informalmente é: se os nomes de scope forem bem segregados (`accounts:read` nunca significando nada em nenhum outro serviço), isso já não bastaria?

No caso concreto isolado, sim, funciona: a API de pagamentos, ao checar se o token apresentado tem `payments:read` ou `payments:write`, não vai encontrar nenhum desses scopes — porque o token só tem `accounts:read` — e rejeita a chamada. Nomenclatura limpa de scope, sozinha, evita essa ação indevida específica.

O problema é que essa proteção depende inteiramente de **disciplina de implementação em cada resource server individual**, não de uma garantia estrutural do protocolo. Vamos decompor por que isso é uma diferença importante, não só uma nuance acadêmica:

(a) Nada no protocolo obriga um resource server a checar o nome de scope específico dele. Um serviço mal implementado — por bug de regex numa checagem de scope, por uma checagem frouxa do tipo "aceita qualquer scope que comece com o prefixo `read`", ou, pior ainda, por só conferir `active: true` numa resposta de introspecção sem sequer olhar o campo `scope` retornado — aceita o token mesmo com nomenclatura perfeitamente segregada. A proteção nunca foi do protocolo: foi do código específico de quem escreveu aquele resource server, e código específico tem bugs, tem pressa de deadline, e não é auditado com a mesma disciplina em toda uma organização.

(b) A unicidade global de nomes de scope é, na prática, um problema de governança organizacional — não um problema técnico resolvido pelo protocolo. Numa organização grande, com dezenas ou centenas de times construindo APIs ao longo de vários anos, garantir que ninguém nunca reaproveite um nome de scope (seja por coincidência, seja copiando um serviço existente como ponto de partida e esquecendo de renomear) é uma disciplina que **degrada com o tempo**. Não existe um registro central de scopes que o protocolo exija ou valide — é convenção pura, sujeita a toda a entropia normal de organizações grandes: rotatividade de equipe, documentação desatualizada, pressão de prazo.

(c) Confused deputy não é só "o escopo errado foi concedido" — é "a parte errada aceitou essa credencial". Este é o ponto mais sutil e o mais importante de reter: mesmo um token com scope **corretamente nomeado** e **corretamente checado** pelo resource server pode, ainda assim, ser aceito por um resource server que **não deveria**, estruturalmente, receber tokens daquele tipo.

Exemplo concreto: um ambiente de staging, menos confiável que produção (talvez rodando código de um branch de feature ainda não revisado), que por coincidência de arquitetura compartilhada também expõe um endpoint que checa `accounts:read` — e aceita, porque o scope bate, mesmo que esse ambiente jamais devesse estar processando tokens de produção. Nomenclatura de scope distingue **"o que pode ser feito"** (a ação/permissão), mas não distingue **"com quem o token pode falar"** (o destinatário pretendido) — são dois eixos ortogonais, e scope só cobre um deles.

Conclusão da análise: segmentação de nomes de scope é uma mitigação real, vale a pena manter como defesa em profundidade, e reduz consideravelmente o risco prático. Mas não é um substituto estrutural para uma restrição de audiance, exatamente porque depende de disciplina de nomenclatura e de implementação correta em **absolutamente todo** resource server do ecossistema, em vez de uma garantia reforçada pelo próprio protocolo — uma garantia que não depende de como os scopes foram nomeados, nem de quão disciplinado foi o time que implementou aquele resource server específico. É exatamente essa lacuna estrutural que a RFC 8707 fecha, introduzindo um sinal explícito, no próprio pedido, de qual recurso o token deve servir — sinal que depois "pousa" na claim `aud` do token, verificável de forma padronizada por qualquer resource server, independentemente de como seus scopes foram nomeados.

2. Terminologia e papéis

Nenhum papel novo. Introduz:

- **resource** — parâmetro novo, de valor **URI absoluta** (sem fragmento), identificando o resource server que o client pretende usar com o token resultante daquela requisição específica.
- **Resource Indicator** — o nome que a RFC dá a essa declaração explícita, feita pelo client no momento do pedido: "este token é para conversar com **este** destinatário, e não com qualquer outro que aceite o mesmo scope".

Por que URI, e não um identificador arbitrário?

A escolha de usar uma URI absoluta (tipicamente a própria base URL do resource server, ex.

`https://api.exemplo.com/pagamentos`) em vez de um identificador opaco arbitrário tem duas vantagens práticas: (1) reaproveita um formato que o ecossistema HTTP já entende e já valida (sintaxe de URI é padronizada, RFC 3986); (2) frequentemente já corresponde a algo que o resource server já sabe reconhecer sobre si mesmo (sua própria base URL), simplificando a checagem de audiance do lado do RS — comparar a claim `aud` recebida contra "minha própria URI conhecida" é uma operação direta.

3. Fluxo

```
GET /authorize?response_type=code&client_id=...
    &resource=https://api.exemplo.com/pedidos&scope=...
```

ou, no token endpoint (qualquer grant, incluindo client_credentials):

```
POST /token
grant_type=...&resource=https://api.exemplo.com/pedidos&...
```

AS valida se o client tem permissao de obter token para ESSE resource, com ESSE scope
AS emite o token restrito aquele resource (aud = o resource pedido, ou uma lista, se multiplo)

O parâmetro `resource` pode aparecer **mais de uma vez** na mesma requisição, se o client quiser (ou precisar) de um token válido para vários recursos simultaneamente. A RFC deixa como decisão de implantação o que o AS faz nesse caso:

- **Emitir um único token multi-audience** — `aud` vira uma lista com todos os recursos pedidos, e cada resource server precisa checar se **seu próprio** identificador está presente na lista (não checar igualdade estrita, e sim pertencimento).
- **Recusar por ambiguidade** — alguns AS, especialmente em perfis de alta garantia (FAPI, por exemplo), preferem restringir a **um único** `resource` por token, simplificando a checagem de audience do lado do RS para uma comparação de igualdade direta, e reduzindo a superfície de um token "poderoso demais" que fala com múltiplos serviços simultaneamente.

Caso de borda: `resource` ausente

Quando o client não informa `resource` nenhum, o comportamento cai para o que o AS já fazia antes desta RFC existir — normalmente emitir um token sem restrição explícita de audience (ou com uma audience padrão pré-configurada por client/scope, dependendo da implantação). A RFC não obriga que `resource` seja sempre exigido; ela introduz o mecanismo, e cabe a cada AS (ou a um perfil de segurança específico, como FAPI) decidir se torna esse parâmetro obrigatório.

Caso de borda: `resource` pedido mas não autorizado para aquele client

O AS deve validar se o client em questão está de fato autorizado a obter tokens para aquele `resource` específico — um client registrado para falar apenas com a API de contas não deveria conseguir simplesmente pedir `resource=https://api.exemplo.com/pagamentos` e receber um token válido para pagamentos só porque pediu. Essa validação é responsabilidade do AS (ver seção 5) — sem ela, o parâmetro `resource` seria apenas decorativo, sem nenhum reforço real de segurança.

Caso de borda: `resource` e `scope` interagindo

Um mesmo client pode ter diferentes scopes válidos para diferentes recursos (ex.: `accounts:read` faz sentido para `resource=https://api.exemplo.com/contas`, mas não para `resource=https://api.exemplo.com/pagamentos`). O AS precisa avaliar a combinação (`client`,

`resource`, `scope`) como um todo — não apenas `(client, scope)` isoladamente, como fazia antes desta RFC. Isso pode exigir uma tabela de política mais rica no AS (matriz `client × resource × scope` permitido), mas é exatamente o ganho de granularidade que a RFC entrega.

4. Parâmetros e endpoints normativos

Nenhum endpoint novo — estende tanto a requisição de autorização (RFC 6749 §4.1.1) quanto a requisição de token, com o parâmetro `resource`, repetível.

- **Também utilizável na RFC 8693 (Token Exchange)** — que já usava exatamente esse mesmo parâmetro `resource` antes desta RFC existir formalmente. A RFC 8707 generaliza o parâmetro (que nasceu ligado ao Token Exchange) para os grants padrão do núcleo OAuth (Authorization Code, Client Credentials, etc.), não só para cenários de troca de token.
- **Descoberta via RFC 8414**: campo de metadata do AS indicando suporte ao parâmetro (ex. algo equivalente a `resource_parameter_supported` em implementações que expõem esse detalhe de descoberta).
- **Não há um novo formato de resposta** — o efeito do parâmetro aparece indiretamente, na claim `aud` do token emitido (JWT, RFC 9068) ou no campo `aud` de uma resposta de introspecção (RFC 7662), nunca como um campo próprio na resposta do próprio `/token`.

5. Segurança já tratada na própria RFC 8707

- **O AS deve validar** se o client está autorizado a obter token para o `resource` pedido, com aquele `scope` específico — a RFC é explícita que o AS não deve simplesmente aceitar qualquer `resource` que o client informe e ecoar de volta sem checagem. Sem essa validação do lado do AS, o parâmetro vira só uma sugestão do client, sem reforço de segurança nenhum.
- **Recomenda fortemente que resource servers validem a audience** dos tokens que recebem — este é o ponto onde a decisão tomada no momento do pedido (`resource`) precisa efetivamente "pousar" em algum lugar verificável e concreto: a claim `aud` de um JWT access token (RFC 9068), ou o campo `aud` de uma resposta de introspecção (RFC 7662). Um AS que suporta `resource` mas cujos resource servers nunca checam `aud` está implementando só metade do mecanismo — a metade que não protege nada sozinha.
- Reforça, de forma **estrutural** (não dependente de disciplina de nomenclatura de scope, nem de qualidade de implementação de cada RS individual), exatamente a proteção que a análise da seção 1 mostrou que a segmentação de scope, sozinha, não conseguia garantir de forma confiável em toda uma organização.
- **Vazamento de topologia interna**: um efeito colateral a se ter em mente — o valor de `resource` frequentemente revela a URL real de um serviço interno (ex. `https://internal-payments.corp.example.com/api`). Em fluxos onde o parâmetro trafega pelo user-agent (a requisição de autorização, tipicamente redirecionada pelo browser), isso pode expor detalhes de topologia de rede interna que a organização preferiria não tornar públicos. Mitigação natural: combinar com **PAR (RFC 9126)**, que tira o parâmetro do browser inteiramente, movendo-o para uma chamada backend-to-backend direta entre client e AS.

6. Relação com outras RFCs

- Resolve diretamente a preocupação de "confused deputy" catalogada informalmente na **RFC 6819**, e reforçada explicitamente pela **RFC 9700** como prática recomendada moderna.
- A claim `aud` da **RFC 9068** (JWT access tokens) e o campo `aud` da resposta de introspecção da **RFC 7662** são exatamente onde a decisão de `resource` desta RFC fica registrada, de forma verificável, no token — a mesma divisão "decisão no pedido vs. efeito registrado no token" que já vimos entre a RFC 8693 (Token Exchange) e a RFC 9068.
- Companheira direta da **RFC 8693 (Token Exchange)**, que já usava esse mesmo parâmetro `resource` antes desta RFC generalizá-lo para os grants padrão do núcleo OAuth.
- Combina naturalmente com a **RFC 9126 (PAR)** para evitar o vazamento de topologia interna discutido na seção 5, tirando o parâmetro `resource` do browser.
- É o penúltimo degrau de uma escada de least privilege que atravessa o roteiro inteiro, e que o próximo capítulo (RFC 9396) fecha: escopo amplo (RFC 6749) → escopo mínimo por convenção (RFC 6750) → **audience estruturalmente restrita** (esta RFC) → detalhe no nível de uma transação específica (RFC 9396, próximo capítulo).
- Fecha o ciclo aberto desde o capítulo da RFC 6819 e revisitado tanto na RFC 8693 (o cenário de gateway como confused deputy) quanto na RFC 9068 (onde a claim `aud` foi definida como ponto de extensão).

7. O que mudou desde a publicação (2020)

- Adoção crescente, particularmente em implantações com múltiplos `resource` servers/microserviços — exatamente o cenário onde a análise da seção 1 mostra que confiar só em nomenclatura de scope se torna frágil com a escala.
- Em perfis adjacentes a FAPI — que, no seu contexto FSI, já esperam indicação explícita de recurso em vez de depender só de disciplina de nomenclatura de escopo, exatamente como a análise da seção 1 concluiu ser insuficiente sozinha.
- Continua sendo, junto com a RFC 8693, a resposta estrutural moderna ao problema que a RFC 6819 catalogou informalmente mais de uma década antes — um bom exemplo de como o ecossistema OAuth vai fechando lacunas conhecidas há muito tempo com mecanismos formalizados, um RFC de cada vez.

Nota de laboratório: nenhum lab construído ainda para esta RFC neste projeto. Um cenário natural seria estender um lab existente com múltiplos `resource` servers simulados, mostrando o mesmo `client_id` obtendo tokens com `aud` diferentes conforme o `resource` pedido, e um `resource` server rejeitando explicitamente um token cuja `aud` não bate com sua própria identidade — exercitando exatamente o mecanismo de confused deputy discutido na seção 1. Os exemplos de código abaixo são, portanto, standalone.

Exemplo em Java (JDK 21, sem frameworks)

Usamos `java.net.http.HttpClient` para montar as requisições com o parâmetro `resource`, e o utilitário `SimpleJson`, definido no Capítulo 1 (RFC 6749) deste livro, para lidar com o payload do token.

Nota didática: como nos capítulos anteriores desta parte, extrair a claim `aud` de um JWT abaixo é feito com parsing manual só para fins didáticos. Em produção, valide o JWT inteiro (assinatura, `exp`, `nbf`, etc.) com uma biblioteca JOSE antes de examinar qualquer claim de negócio como `aud`.

1. Incluindo `resource` na authorization request e no `/token`

```
import java.net.URI;
import java.net.URLEncoder;
import java.net.http.HttpClient;
import java.net.http.HttpRequest;
import java.net.http.HttpResponse;
import java.nio.charset.StandardCharsets;

public class ClienteComResourceIndicator {

    /**
     * Monta a URL de authorization request incluindo resource – a
     * declaracao explicita, feita ja no PEDIDO, de para qual resource
     * server o token resultante deve valer. O parametro pode ser
     * repetido se o client precisar de token multi-audience (o AS decide
     * se aceita ou recusa por ambiguidade – secao 3).
     */
    static String montarUrlAutorizacao(String authorizationEndpoint, String clientId,
                                       String redirectUri, String scope,
                                       String resource, String state,
                                       String codeChallenge) {

        return authorizationEndpoint
            + "?response_type=code"
            + "&client_id=" + urlEncode(clientId)
            + "&redirect_uri=" + urlEncode(redirectUri)
            + "&scope=" + urlEncode(scope)
            + "&resource=" + urlEncode(resource) // URI absoluta do RS pretendido
            + "&state=" + urlEncode(state)
            + "&code_challenge=" + urlEncode(codeChallenge)
            + "&code_challenge_method=S256";
    }

    /**
     * POST /token repetindo o MESMO resource usado na authorization
     * request – praticas recomendadas sugerem consistencia entre os dois
     * pedidos, embora a RFC nao proiba um AS de tratar o resource do
     * /token como a fonte de verdade final.
     */
    static HttpResponse<String> trocarCodePorToken(HttpClient client, String tokenEndpoint,
                                                    String clientId, String code,
                                                    String redirectUri, String codeVerifier,
                                                    String resource) throws Exception {

        String corpo = "grant_type=authorization_code"
            + "&client_id=" + urlEncode(clientId)
            + "&code=" + urlEncode(code)
            + "&redirect_uri=" + urlEncode(redirectUri)
            + "&code_verifier=" + urlEncode(codeVerifier)
            + "&resource=" + urlEncode(resource);

        HttpRequest request = HttpRequest.newBuilder()
            .uri(URI.create(tokenEndpoint))
```

```
        .header("Content-Type", "application/x-www-form-urlencoded")
        .POST(HttpRequest.BodyPublishers.ofString(corpo))
        .build();

    return client.send(request, HttpResponse.BodyHandlers.ofString());
}

private static String urlEncode(String valor) {
    return URLEncoder.encode(valor, StandardCharsets.UTF_8);
}
}
```


2. Validando `aud` no resource server (mitigação de confused deputy)

```
import java.util.Base64;
import java.util.List;
import java.util.Map;

public class ValidadorAudience {

    /**
     * Identidade PRÓPRIA deste resource server específico – normalmente
     * configurada, não descoberta dinamicamente. É o valor que este RS
     * exige encontrar na claim aud de qualquer token que aceitar.
     */
    private static final String MINHA_IDENTIDADE_DE_RECURSO =
        "https://api.exemplo.com/pagamentos";

    /**
     * Extraí o payload de um JWT compacto (parsing manual, só didático –
     * ver nota no topo do capítulo).
     */
    static String extrairPayloadJson(String jwtCompacto) {
        String[] partes = jwtCompacto.split("\\.");
        if (partes.length != 3) {
            throw new IllegalArgumentException("JWT malformado");
        }
        return new String(Base64.getUrlDecoder().decode(partes[1]),
            java.nio.charset.StandardCharsets.UTF_8);
    }

    /**
     * O passo de segurança central desta RFC, do lado do resource server:
     * antes de aceitar QUALQUER chamada, confirmar que este token foi
     * emitido pensando EXPLICITAMENTE neste resource server – não aceitar
     * só porque o scope aparenta bater (a lição da seção 1: scope sozinho
     * não é suficiente).
     *
     * aud pode ser uma única string ou uma lista (quando o AS emitiu um
     * token multi-audience) – tratamos os dois casos.
     */
    static boolean audienceValidaParaEsteRecurso(List<String> audiencesDoToken) {
        if (audiencesDoToken == null || audiencesDoToken.isEmpty()) {
            // Sem aud nenhuma: política de segurança deste RS deve decidir
            // se rejeita (recomendado, se este RS exige resource indicator)
            // ou aceita com uma política de fallback mais permissiva.
            return false;
        }
        return audiencesDoToken.contains(MINHA_IDENTIDADE_DE_RECURSO);
    }

    /**
     * Fluxo completo simplificado: parseia, extraí aud (aqui ilustrado
     * com um único valor via SimpleJson.parseFlat; um payload real com

```

```
    * aud como array exigiria um parser com suporte a listas, que o
    * SimpleJson didatico do Capitulo 1 nao cobre por simplicidade) e
    * decide se aceita a chamada.
    */
    static boolean validarChamada(String accessTokenJwt) {
        String payloadJson = extrairPayloadJson(accessTokenJwt);
        Map<String, String> payload = SimpleJson.parseFlat(payloadJson);
        String aud = payload.get("aud"); // ilustrativo - adaptar para lista se necessario
        if (aud == null) {
            return false;
        }
        return audienceValidaParaEsteRecurso(List.of(aud));
    }
}
```

RFC 9396 — OAuth 2.0 Rich Authorization Requests (RAR)

1. Contexto e motivação

O `scope` tradicional (RFC 6749) é uma lista simples de identificadores string, separados por espaço (`profile email accounts:read`). Esse modelo funciona bem para permissões grosseiras e relativamente estáveis — "este client pode ler perfil", "este client pode ler saldo de conta" — mas quebra estruturalmente quando a autorização precisa ser **final, estruturada e específica de uma transação**. O exemplo canônico, direto do seu contexto FSI: "autorizar um pagamento de exatamente EUR 50,00 para esta conta específica, só desta vez, não um pagamento de qualquer valor para qualquer conta". Não existe forma razoável de expressar isso como uma string de scope solta — mesmo tentativas criativas (`payments:initiate:eur:50.00:BR00...`) esbarram rapidamente em limites práticos: strings de scope não têm gramática, não têm tipos, não têm validação estrutural, e crescem sem limite conforme mais dimensões precisam ser codificadas (moeda, valor, conta, data-limite, finalidade).

Na prática, ecossistemas regulados de Open Banking (UK Open Banking, Berlin Group NextGenPSD2, e outros) **precisaram** resolver esse problema antes desta RFC existir — a pressão regulatória para "iniciação de pagamento com consentimento granular" não esperou o IETF. O resultado foi que cada ecossistema inventou sua própria codificação ad-hoc, tipicamente:

- Espremendo dados estruturados dentro do próprio nome do scope (frágil, difícil de parsear com segurança, limites de tamanho de string HTTP);
- Ou transportando um objeto de "detalhes do consentimento" fora do fluxo OAuth padrão, por um canal proprietário paralelo (quebrando a portabilidade entre AS de fornecedores diferentes, e forçando cada client a implementar integração específica por país/ecossistema).

Ambas as abordagens são incompatíveis entre si, difíceis de validar de forma genérica, e sem nenhum padrão comum entre implementações — exatamente o tipo de fragmentação que uma especificação IETF existe para resolver.

A RFC 9396 resolve isso introduzindo um parâmetro novo, `authorization_details`, carregando um **array JSON de objetos estruturados** — cada um descrevendo uma autorização específica, com campos arbitrários próprios do tipo de autorização em questão, mas com uma pequena moldura comum reutilizável entre tipos.

Por que um array, e não um objeto único?

O uso de um **array** (em vez de um único objeto de detalhe) permite que uma única requisição de autorização carregue múltiplas autorizações independentes ao mesmo tempo — por exemplo, "iniciar este pagamento de EUR 50" e "ler o extrato dos últimos 90 dias", como dois objetos distintos no

mesmo array, cada um com seu próprio `type` e seus próprios campos. Isso evita ter que inventar um esquema artificial de "múltiplos tipos misturados dentro de um objeto só" e mantém cada autorização estruturalmente isolada e independentemente validável.

2. Terminologia e papéis

Nenhum papel novo. Introduce:

- `authorization_details` — parâmetro novo, utilizável tanto na authorization request quanto (em certos fluxos, como Client Credentials) diretamente na token request. Valor: array JSON de "objetos de detalhe de autorização".
- **Objeto de detalhe de autorização** — cada elemento do array tem:
- `type` (obrigatório) — string que identifica a semântica daquele objeto específico (ex.: `"payment_initiation"`). Crucialmente, **a RFC 9396 não padroniza nenhum valor genérico de `type`** — ela define só a moldura (o "envelope"). Cada ecossistema, perfil regulatório ou implantação específica é quem define seus próprios tipos e o significado de cada campo associado (o Berlin Group, por exemplo, define seus próprios tipos para iniciação de pagamento europeia; um AS interno de um banco brasileiro definiria os seus, adaptados a PIX/TED/boleto).
- **Campos específicos do tipo** — arbitrários, definidos por quem especifica aquele `type` (no exemplo de pagamento: `instructedAmount`, `creditorAccount`, etc.).
- **Campos comuns opcionais, reutilizáveis por qualquer tipo** (a RFC os define para promover alguma interoperabilidade mínima entre tipos diferentes):
 - `locations` — quais resource servers este objeto de detalhe se destina; mesmo espírito do parâmetro `resource` da RFC 8707 (capítulo anterior), só que aplicado por **objeto de detalhe individual**, não pelo token inteiro.
 - `actions` — quais ações são autorizadas (ex.: `["initiate"]`, `["read"]`).
 - `datatypes` — quais tipos de dado estão em jogo (útil sobretudo para autorizações de leitura, ex.: `["balances", "transactions"]`).
 - `identifier` — um identificador de uma instância específica de recurso (ex.: o ID de uma conta específica).
 - `privileges` — um refinamento adicional de nível de privilégio dentro daquele tipo.

`authorization_details` coexiste com `scope`?

Sim — a RFC não deprecia `scope`. Uma requisição pode carregar `scope` (para autorizações grosseiras e estáveis, como "acesso ao perfil") e `authorization_details` (para a parte fina e específica de transação, como o pagamento) simultaneamente. Cabe a cada implantação decidir como as duas se combinam na tela de consentimento e na avaliação de política do AS — a RFC não define uma regra universal de precedência entre os dois.

3. Fluxo

Não é um grant novo — é um parâmetro adicional que se encaixa no fluxo já existente (tipicamente Authorization Code, mas nada impede uso em outros grants onde fizer sentido, como Client Credentials para autorizações machine-to-machine estruturadas):

```
GET /authorize?response_type=code&client_id=...
&authorization_details=[{
  "type": "payment_initiation",
  "actions": ["initiate"],
  "locations": ["https://api.banco.exemplo.com/pagamentos"],
  "instructedAmount": {"currency": "BRL", "amount": "120.00"},
  "creditorAccount": {"iban": "BR0000000000000000000000"}
}]
```

AS mostra uma tela de consentimento refletindo ESSE detalhe específico — não um scope genérico "payments", literalmente algo como "autorizar pagamento de BRL 120,00 para esta conta específica"

Usuario aprova -> AS emite o token, ecoando authorization_details (possivelmente REDUZIDO em relação ao pedido original) na resposta do /token, para o client confirmar exatamente o que foi concedido

Por que o AS pode "reduzir" o que foi pedido, e por que isso importa

Um ponto normativo central, fácil de subestimar: o AS **não é obrigado** a conceder exatamente o que foi pedido em `authorization_details`. Ele pode reduzir (por exemplo, se o usuário aprovar um valor menor numa tela de consentimento interativa, ou se uma política do AS limitar o valor máximo por transação de um certo tipo de client). A resposta do `/token` **eco o que foi efetivamente concedido**, não uma confirmação vazia de "ok, aceito o que você pediu". Isso é uma mudança de disciplina importante em relação ao `scope` tradicional, onde muitas implementações históricas simplesmente devolviam o mesmo `scope` pedido sem necessariamente confirmar explicitamente o que foi de fato concedido caso a caso.

Isso implica uma responsabilidade adicional para o client: **ele deve inspecionar o `authorization_details` devolvido na resposta do token, não assumir cegamente que recebeu exatamente o que pediu**. Um client bem implementado que pediu autorização para EUR 100 e recebeu de volta uma autorização reduzida para EUR 50 precisa tratar essa diferença explicitamente (por exemplo, informando o usuário, ou abortando a operação se a redução tornar a transação sem sentido).

Caso de borda: `authorization_details` inválido ou tipo desconhecido

Se o AS não reconhece um `type` pedido (não está em sua lista de tipos suportados — descoberta via `authorization_details_types_supported`, RFC 8414), a RFC espera que o AS rejeite a requisição com um erro apropriado, em vez de silenciosamente ignorar aquele objeto de detalhe específico e prosseguir

sem ele — silenciosamente descartar autorização pedida seria uma falha de segurança "silenciosa" (o client acredita que pediu algo e recebeu confirmação implícita, quando na verdade não recebeu nada).

Caso de borda: token de recurso único vs. múltiplos `locations`

Assim como o `resource` da RFC 8707 pode gerar um token multi-audience, um único objeto de `authorization_details` pode ter `locations` com múltiplos resource servers. A mesma tensão entre "um token poderoso falando com vários RSs" e "tokens mais restritos, um por RS" se aplica aqui, e a decisão de política cabe à implantação — a RFC não resolve essa tensão de forma universal, só fornece o vocabulário.

4. Parâmetros e endpoints normativos

Nenhum endpoint novo — estende tanto a authorization request quanto a token request/response existentes.

- `authorization_details` (pedido): array de objetos, cada um com `type` obrigatório + campos específicos do tipo + campos comuns opcionais (`locations` , `actions` , `datatypes` , `identifier` , `privileges`).
- `authorization_details` (resposta do `/token`): também devolvido, ecoando exatamente (ou de forma reduzida — ver seção 3) o que foi efetivamente concedido — não apenas um scope aproximado ou uma confirmação vazia.
- **Descoberta via RFC 8414:** `authorization_details_types_supported` , listando quais valores de `type` aquele AS especificamente entende e sabe processar. Um client que pretende usar RAR deve, em princípio, checar esse campo de descoberta antes de assumir que um `type` específico será aceito por um AS desconhecido.
- **Introspecção (RFC 7662):** uma resposta de introspecção de um token vinculado a `authorization_details` também deve expor esse campo, permitindo que um resource server (ou um AS diferente, num cenário de token exchange) veja exatamente qual autorização estruturada aquele token carrega — não só um `scope` genérico.

5. Segurança já tratada na própria RFC 9396

Telas de consentimento muito mais precisas

O ganho mais imediato e tangível: o usuário vê, na tela de consentimento, algo como "autorizar pagamento de BRL 120,00 para esta conta específica", em vez de um scope opaco tipo `payments:write` que não comunica nada sobre o valor ou destinatário real da transação. Isso conecta diretamente com a preocupação de consentimento informado (e o risco de phishing por consentimento genérico demais) já discutida no capítulo da RFC 6819 — um usuário que aprova "acesso a pagamentos" de forma genérica está, na prática, dando um cheque em branco; um usuário que aprova "pagar BRL 120 para esta conta" está aprovando algo que pode efetivamente entender e conferir.

O AS deve validar e pode reduzir — nunca aceitar cegamente

Mesma disciplina já vista com `scope` (RFC 6749/6750) e com `resource` (RFC 8707, capítulo anterior): o AS nunca deve aceitar cegamente o que o client pede em `authorization_details` — precisa validar contra a política daquele client, daquele usuário, e daquele tipo de autorização, podendo reduzir o que é efetivamente concedido (seção 3).

O último degrau de uma escada de least privilege

É importante enxergar RAR não como uma peça isolada, mas como o **último degrau de uma escada de granularidade** que este roteiro percorreu inteiro:

1. **Escopo amplo** (RFC 6749) — o mecanismo básico de "o que este client pode fazer", frequentemente grosseiro na prática (`read` , `write`).
2. **Escopo mínimo por convenção** (RFC 6750 e a prática de least privilege discutida desde então) — disciplina de nomear e conceder scopes o mais estreitos possível, mas ainda por convenção, não por garantia estrutural.
3. **Restrição de audience** (RFC 8707, capítulo anterior) — garantia estrutural de **para quem** o token vale, resolvendo confused deputy independentemente de nomenclatura de scope.
4. **Detalhe no nível de uma transação específica** (RFC 9396, este capítulo) — não apenas "o que" e "para quem", mas **exatamente qual transação**, com seus parâmetros concretos (valor, conta, data), verificável pelo próprio resource server no momento de executar a ação.

Cada degrau resolve uma limitação estrutural que o degrau anterior não conseguia cobrir sozinho — RAR é onde essa escada termina, no nível mais fino que o protocolo oferece antes de sair do domínio de "autorização" e entrar no domínio de "lógica de negócio da transação em si".

Combinação natural com PAR e JAR

Como `authorization_details` pode carregar dados sensíveis estruturados diretamente na requisição (IBAN, valor exato, possivelmente até finalidade da transação), isso reforça por que combinar com:

- **PAR (RFC 9126)** — para não expor esses dados sensíveis no browser (query string de uma requisição GET de redirecionamento), movendo o conteúdo completo para uma chamada backend-to-backend.
- **JAR (RFC 9101)** — para assinar a requisição inteira, garantindo integridade (nenhum campo, incluindo o `authorization_details`, pode ser adulterado em trânsito sem invalidar a assinatura).

A combinação PAR + JAR + RAR é, na prática, o padrão de referência para cenários de alta garantia como iniciação de pagamento — e é exatamente o conjunto que o FAPI 2.0 recomenda para esses casos.

6. Relação com outras RFCs

- Evolução direta do `scope` tradicional — mesmo papel conceitual (comunicar o que está sendo concedido), agora estruturado em JSON tipado em vez de uma string plana separada por espaços.

- Combina naturalmente com **RFC 9126 (PAR)** e **RFC 9101 (JAR)** — a trinca PAR + JAR + RAR é exatamente o padrão avançado do FAPI 2.0 para casos como iniciação de pagamento, amarrando construções vistas nas partes anteriores deste roteiro (PAR/JAR, categorias de segurança avançada da requisição) ao mecanismo de autorização fina apresentado aqui.
- `locations` ecoa diretamente o conceito de `resource` da **RFC 8707** (capítulo anterior), só que aplicado por objeto de detalhe individual, não pelo token inteiro — a mesma ideia de restrição de audience, só que com granularidade mais fina.
- Descoberto via **RFC 8414** (`authorization_details_types_supported`).
- Fecha, junto com a RFC 8707, o fio de least privilege que atravessa o roteiro inteiro: de escopo amplo (RFC 6749) a escopo mínimo por convenção (RFC 6750) a audience estruturalmente restrita (RFC 8707) a transação específica (RFC 9396) — ver a escada detalhada na seção 5.

7. O que mudou desde a publicação (2023)

- RFC recente, motivada por pressão regulatória real em ecossistemas de Open Banking, que já precisavam desse nível de granularidade havia vários anos e resolviam de formas proprietárias e mutuamente incompatíveis antes da padronização — a RFC 9396 chega, em certo sentido, formalizando uma necessidade que o mercado já tinha suprido de forma fragmentada.
- O **FAPI 2.0 trata RAR como uma ferramenta disponível** (não universalmente obrigatória, diferente do PKCE, que o FAPI 2.0 tornou praticamente onipresente) — usada sempre que o caso de uso exige esse nível de precisão, sendo iniciação de pagamento o exemplo canônico e mais citado.
- No seu contexto FSI, é provavelmente o capítulo com **aplicação profissional mais direta** de todo o roteiro: qualquer sistema de iniciação de pagamento, ou qualquer cenário de "consentimento granular por transação" em serviços financeiros regulados, converge para este mecanismo.

Nota de laboratório: nenhum lab construído ainda para esta RFC neste projeto. O cenário natural seria estender algum lab existente com um `authorization_details` de "iniciação de pagamento" fictícia, mostrando a tela de consentimento refletindo o valor/conta específicos (em vez de um scope genérico do tipo `payments:write`), e o resource server validando o `type` e os campos concretos daquele objeto antes de autorizar a execução da transação. Os exemplos de código abaixo são, portanto, standalone.

Exemplo em Java (JDK 21, sem frameworks)

Usamos `java.net.http.HttpClient` para montar a requisição e o utilitário `SimpleJson`, definido no Capítulo 1 (RFC 6749) deste livro, para as partes planas do JSON — com uma extensão mínima e local para lidar com o array/objetos aninhados que `authorization_details` exige (o `SimpleJson` do Capítulo 1 é deliberadamente simples e cobre só JSON "flat").

Nota didática: assim como nos capítulos anteriores, o parsing manual abaixo é só para expor o mecanismo do protocolo. Em produção, use uma biblioteca JSON madura (Jackson, Gson, ou equivalente) para qualquer estrutura JSON aninhada real — o `SimpleJson` deste livro é deliberadamente mínimo, para fins didáticos.

1. Montando `authorization_details` para iniciação de pagamento

```
import java.net.URLEncoder;
import java.nio.charset.StandardCharsets;

public class AuthorizationDetailsBuilder {

    /**
     * Monta manualmente o array JSON de authorization_details para um
     * unico objeto de "iniciacao de pagamento" – type e um valor definido
     * pelo ECOSSISTEMA/perfil regulatorio, NAO pela RFC 9396 em si (que so
     * define o envelope comum: type + Locations/actions/datatypes/
     * identifier/privileges).
     *
     * Isto substitui um scope plano como "payments:write" por uma
     * descricao estruturada e especifica da transacao concreta.
     */
    static String montarPagamentoUnico(String moeda, String valor,
                                       String ibanDestino, String resourceServerUrl) {

        return "[{"
            + "\"type\": \"payment_initiation\", \""
            + "\"actions\": [\"initiate\"], \""
            + "\"locations\": [\"" + resourceServerUrl + "\"], \""
            + "\"instructedAmount\": {"
            +     "\"currency\": \"" + moeda + "\", \""
            +     "\"amount\": \"" + valor + "\""
            +     "}, \""
            + "\"creditorAccount\": {"
            +     "\"iban\": \"" + ibanDestino + "\""
            +     "}"
            + "}]";
    }

    /**
     * Codifica o array JSON como valor de query string, para uso numa
     * authorization request via GET. Em cenarios reais de alta garantia
     * (dados sensiveis como IBAN e valor exato no proprio parametro),
     * prefira combinar com PAR (RFC 9126) para nao expor isso no browser –
     * ver discussao na secao 5 deste capitulo.
     */
    static String montarUrlAutorizacaoComRar(String authorizationEndpoint, String clientId,
                                             String redirectUri, String state,
                                             String authorizationDetailsJson) {

        return authorizationEndpoint
            + "?response_type=code"
            + "&client_id=" + urlEncode(clientId)
            + "&redirect_uri=" + urlEncode(redirectUri)
            + "&state=" + urlEncode(state)
            + "&authorization_details=" + urlEncode(authorizationDetailsJson);
    }

    private static String urlEncode(String valor) {
```

```
        return URLEncoder.encode(valor, StandardCharsets.UTF_8);  
    }  
}
```

2. Parseando e validando `authorization_details` no resource server

```
import java.util.List;
import java.util.regex.Matcher;
import java.util.regex.Pattern;

public class ValidadorAuthorizationDetails {

    /**
     * Parser MINIMO e ilustrativo para o unico formato que este exemplo
     * precisa entender (um objeto de payment_initiation dentro do array).
     * NAO e um parser JSON de proposito geral – para isso, use uma
     * biblioteca real em producao (ver nota didatica no topo).
     */
    record DetalhePagamento(String type, String moeda, String valor, String iban,
        List<String> locations) {}

    static DetalhePagamento parsearPrimeiroDetalhePagamento(String authorizationDetailsJson) {
        String tipo = extrairCampo(authorizationDetailsJson, "\"type\"\\s*:\\s*\"([^\"]+)\"");
        String moeda = extrairCampo(authorizationDetailsJson, "\"currency\"\\s*:\\s*\"([^\"]+)\"");
        String valor = extrairCampo(authorizationDetailsJson, "\"amount\"\\s*:\\s*\"([^\"]+)\"");
        String iban = extrairCampo(authorizationDetailsJson, "\"iban\"\\s*:\\s*\"([^\"]+)\"");
        return new DetalhePagamento(tipo, moeda, valor, iban, List.of());
    }

    private static String extrairCampo(String json, String regex) {
        Matcher m = Pattern.compile(regex).matcher(json);
        return m.find() ? m.group(1) : null;
    }

    /**
     * O passo de seguranca central deste capitulo, do lado do resource
     * server: antes de EXECUTAR a transacao, confirmar type + cada campo
     * especifico – nao basta que o token "tenha algum scope de pagamento",
     * o RS precisa validar que ESTA transacao concreta (valor, conta) e
     * exatamente o que foi autorizado, nao uma inferida a partir de um
     * scope generico.
     */
    static boolean autorizadoParaExecutarEstaTransacao(DetalhePagamento detalhe,
        String valorRequisitado,
        String moedaRequisitada,
        String ibanRequisitado) {
        if (!"payment_initiation".equals(detalhe.type())) {
            return false; // tipo errado – este token nao autoriza pagamento algum
        }
        // Comparacao EXATA de valor/moeda/conta – nada de "valor menor ou
        // igual" implicito aqui; o objeto de detalhe concedido e o teto
        // exato desta autorizacao especifica (o AS ja pode ter reduzido o
        // pedido original antes de conceder – secao 3).
        return detalhe.moeda().equals(moedaRequisitada)
            && detalhe.valor().equals(valorRequisitado)
            && detalhe.iban().equals(ibanRequisitado);
    }
}
```

```
}  
}
```

3. Comparando `scope` plano vs. `authorization_details` estruturado

```
public class ComparacaoScopeVsRar {  
  
    /**  
     * Ilustra, lado a lado, a diferenca de expressividade entre o modelo  
     * antigo (scope) e o modelo desta RFC (authorization_details) para o  
     * MESMO caso de uso: autorizar um pagamento especifico.  
     */  
    static void demonstrarDiferenca() {  
        // Modelo antigo (RFC 6749): scope plano, sem estrutura nenhuma –  
        // autoriza uma CLASSE de acao ("iniciar pagamentos"), nao uma  
        // transacao especifica. O resource server, ao ver esse scope,  
        // nao tem como saber QUAL pagamento foi de fato consentido pelo  
        // usuario – teria que confiar em algum canal fora de banda.  
        String scopeAntigo = "payments:initiate";  
  
        // Modelo desta RFC: JSON estruturado, autoriza uma TRANSACAO  
        // especifica e verificavel – o resource server pode (e deve)  
        // conferir type + campos concretos antes de executar, sem  
        // depender de nenhum canal fora de banda.  
        String authorizationDetailsNovo = AuthorizationDetailsBuilder.montarPagamentoUnico(  
            "BRL", "120.00", "BR000000000000000000000000",  
            "https://api.banco.exemplo.com/pagamentos");  
  
        System.out.println("scope (RFC 6749): " + scopeAntigo);  
        System.out.println("authorization_details (RFC 9396): " + authorizationDetailsNovo);  
    }  
}
```